# MAKI

Media Asset Keeper & Indexer

# User Manual

Version 4.0.6

2026-03-20

Media Asset Keeper & Indexer

A digital asset manager for photographers and media professionals

Apache-2.0 License

# Contents

**maki User Manual** 8
User Guide 8
Reference Guide 8
Developer Guide 9

**Overview & Concepts** 10
Core Concepts 10
Content-Addressable Storage 12
Two-Tier Storage 13
Multi-Volume Support 13
Variant Grouping 13
Bidirectional XMP Sync 14
Architecture 15
Preview Generation 16
Collections, Stacks, and Saved Searches 16
The Round-Trip Workflow 16

**Setup** 18
Installation 18
Initializing a Catalog 19
Registering Volumes 20
Configuration (maki.toml) 22

**Ingesting Assets** 24
Import Pipeline 24
Basic Import 26
Auto-Grouping 26
Recipe Handling 28
Metadata Extraction 29
Preview Generation 30
Import Options 31
Duplicate Handling 32
AI Auto-Tagging 33
VLM Image Descriptions 33

**Organizing Assets** 38
Tags 38
Editing Metadata 39
Manual Grouping 40
Auto-Grouping 40
Splitting Assets 43
Collections (Static Albums) 43

Stacks . . . 44
Saved Searches (Smart Albums) . . . 46
People Management (Face Recognition) . . . 47
Putting It All Together . . . 48

**Browsing & Searching** **50**
Searching Assets . . . 50
Search Filters Quick Reference . . . 51
Path Prefix Filter . . . 52
Output Formats . . . 52
Sort Options . . . 53
Scripting with maki . . . 54
Showing Asset Details . . . 54
Finding Duplicates . . . 55
Catalog Statistics . . . 56
Exporting Files . . . 57
Visual Similarity . . . 58
Related Topics . . . 58

**Web UI** **60**
Starting the Web UI . . . 60
Dark Mode . . . 61
Browse Page . . . 61
Navigation Between Views . . . 65
Lightbox . . . 66
Asset Detail Page . . . 67
Batch Operations . . . 70
Keyboard Navigation . . . 73
Compare View . . . 74
Tags Page . . . 75
Collections Page . . . 76
Stats Page . . . 76
People Page . . . 77
Stroll Page . . . 77
Analytics Page . . . 79
Backup Status Page . . . 79

**Maintenance** **80**
Verification . . . 80
Sync . . . 82
Refresh . . . 83
Write Back . . . 84
Sync Metadata . . . 85
Cleanup . . . 86
Relocating Assets . . . 87
Updating File Locations . . . 88
Rebuilding the Catalog . . . 89
Fix Dates . . . 89
Fix Roles . . . 90
Recommended Maintenance Workflow . . . 91

**Scripting** **93**
Bash Scripting . . . 93
Python Scripting . . . 95
Real-World Example: Fix Orphaned XMP Files . . . 96

Tips . . . 97
Related Topics . . . 97

**Interactive Shell** **98**
Starting the Shell . . . 98
Basic Usage . . . 98
Variables . . . 99
Session Defaults . . . 100
Tab Completion . . . 101
Script Files . . . 102
Session Management . . . 103
Quote Handling . . . 103
Tilde Expansion . . . 104
Built-in Commands . . . 104
Practical Examples . . . 104
Blocked Commands . . . 106
Tips . . . 106
Related Topics . . . 107

**Organizing and Culling** **108**
Three Populations . . . 108
Rating vs. Curation . . . 108
Workflow Approaches . . . 108
The Default Filter . . . 109
Practical Workflow Examples . . . 111
Tips . . . 111

**CLI Conventions** **113**
Global Flags . . . 113
Catalog Discovery . . . 114
Asset ID Matching . . . 114
Exit Codes . . . 115
Output Conventions . . . 115
Scripting Patterns . . . 115
Safe Defaults . . . 116
Command Categories . . . 117

**Setup Commands** **118**
maki init . . . 119
maki volume add . . . 121
maki volume list . . . 123
maki volume set-purpose . . . 124
maki volume remove . . . 125
maki volume combine . . . 127

**Ingest Commands** **129**
maki import . . . 130
maki delete . . . 133
maki tag . . . 135
maki edit . . . 137
maki group . . . 139
maki split . . . 140
maki auto-group . . . 141
maki auto-tag . . . 143
maki embed . . . 146

maki describe . . . 148

**Organize Commands** **151**
maki collection create . . . 152
maki collection list . . . 153
maki collection show . . . 154
maki collection add . . . 155
maki collection remove . . . 156
maki collection delete . . . 157
maki saved-search save . . . 158
maki saved-search list . . . 159
maki saved-search run . . . 160
maki saved-search delete . . . 161
maki stack create . . . 162
maki stack add . . . 163
maki stack remove . . . 164
maki stack pick . . . 165
maki stack dissolve . . . 166
maki stack list . . . 167
maki stack show . . . 168
maki stack from-tag . . . 169
maki faces detect . . . 170
maki faces cluster . . . 171
maki faces people . . . 172
maki faces name . . . 173
maki faces merge . . . 174
maki faces delete-person . . . 175
maki faces unassign . . . 176
maki faces export . . . 177
maki faces download . . . 178
maki faces status . . . 179

**Retrieve Commands** **180**
maki search . . . 181
maki show . . . 184
maki preview . . . 185
maki export . . . 186
maki contact-sheet . . . 188
maki duplicates . . . 190
maki stats . . . 192
maki backup-status . . . 194
maki serve . . . 196
maki shell . . . 198

**Maintain Commands** **201**
maki verify . . . 202
maki sync . . . 204
maki refresh . . . 206
maki sync-metadata . . . 208
maki writeback . . . 210
maki cleanup . . . 212
maki dedup . . . 214
maki relocate . . . 216
maki update-location . . . 218

maki generate-previews . . . 220
maki fix-roles . . . 222
maki fix-dates . . . 223
maki fix-recipes . . . 225
maki rebuild-catalog . . . 227
maki migrate . . . 228

**Search Filter Reference** **229**
Negation and OR Syntax . . . 229
Free Text . . . 230
type . . . 230
tag . . . 230
format . . . 231
rating . . . 231
label . . . 232
camera . . . 232
lens . . . 232
iso . . . 232
focal . . . 233
f (aperture) . . . 233
width . . . 233
height . . . 234
meta . . . 234
path . . . 234
date . . . 235
dateFrom . . . 235
dateUntil . . . 235
collection . . . 236
volume . . . 236
id . . . 237
orphan . . . 237
missing . . . 237
stale . . . 237
stacked . . . 238
copies . . . 238
variants . . . 239
scattered . . . 239
geo . . . 239
faces . . . 240
person . . . 241
similar (AI feature) . . . 241
min_sim (AI feature) . . . 241
embed (AI feature) . . . 242
text (AI feature) . . . 242
Combining Filters . . . 242
Quoted Values . . . 243
Filter Availability . . . 244
Related Topics . . . 245

**Format Templates Reference** **246**
Presets . . . 246
Custom Templates . . . 247
Placeholders (search) . . . 247
Placeholders (duplicates) . . . 247

Escape Sequences . . . 248
Examples . . . 248
Format Flag Behavior . . . 249
Related Topics . . . 249

**Configuration Reference (maki.toml) 250**
File Location . . . 250
Top-Level Fields . . . 250
preview Section . . . 250
[serve] Section . . . 252
[import] Section . . . 253
[dedup] Section . . . 255
[verify] Section . . . 255
[contact_sheet] Section . . . 255
[ai] Section . . . 256
[vlm] Section . . . 258
[browse] Section . . . 261
Full Example . . . 262
Minimal Examples . . . 264
Defaults Summary . . . 265
Related Topics . . . 266

**Data Model 267**
Entities . . . 267
Variant Roles . . . 272
Display Priority . . . 273
Storage . . . 273

**VLM Model Guide 277**
Quick Start . . . 277
Tested Models . . . 277
Backends . . . 278
Choosing a Model . . . 280
Model Comparison . . . 281
Thinking Models . . . 281
Tips . . . 282
See Also . . . 282

**REST API Reference 284**
Page Routes . . . 284
Search API . . . 286
Asset Editing . . . 286
Batch Operations . . . 291
Face & People APIs . . . 300
Data APIs . . . 303
Static Assets . . . 307
Error Handling . . . 308
Content Types Summary . . . 308

**Module Reference 309**
Module Dependency Graph . . . 309
Module Table . . . 309
Generated API Documentation . . . 317

**Building & Testing 318**

Building . . . . . 318
Testing . . . . . 318
Documentation . . . . . 319
Release Process . . . . . 320
Dependencies . . . . . 320

# maki User Manual

**maki** is a command-line digital asset manager built in Rust, designed for photographers and media professionals who manage large collections across multiple storage devices.

This manual is organized into three sections:

## User Guide

Workflow-oriented guides that walk you through common tasks.


1. Overview & Concepts — Data model, architecture, and the round-trip workflow
2. Setup — Installation, initialization, volumes, and configuration
3. Ingesting Assets — Importing files, auto-grouping, metadata extraction, and previews
4. Organizing Assets — Tags, editing, grouping, collections, and saved searches
5. Browsing & Searching — CLI search, filters, output formats, and statistics
6. Web UI — Browser interface, batch operations, and keyboard navigation
7. Maintenance — Verification, sync, refresh, cleanup, and relocation
8. Scripting — Shell and Python scripting patterns, workflow automation
9. Interactive Shell — Variables, tab completion, script files, and session management
10. Organizing & Culling — Rating vs. curation, default filters, and workflow patterns


## Reference Guide

Man-page style documentation for every command, filter, and configuration option.


- CLI Conventions — Global flags, scripting patterns, exit codes
- Setup Commands — `init`, `volume add`, `volume list`, `volume combine`, `volume remove`
- Ingest Commands — `import`, `delete`, `tag`, `edit`, `group`, `split`, `auto-group`, `auto-tag`, `embed`, `describe`
- Organize Commands — `collection`, `saved-search`, `stack`, `faces`
- Retrieve Commands — `search`, `show`, `preview`, `export`, `contact-sheet`, `duplicates`, `stats`, `backup-status`, `serve`, `shell`
- Maintain Commands — `verify`, `sync`, `refresh`, `sync-metadata`, `writeback`, `cleanup`, `dedup`, `relocate`, `update-location`, `generate-previews`, `fix-roles`, `fix-dates`, `fix-recipes`, `rebuild-catalog`, `migrate`
- Search Filters — Complete filter syntax reference
- Format Templates — Output format presets, custom templates, placeholders
- Configuration — `maki.toml` reference
- Data Model — Asset, Variant, Recipe, Volume, and FileLocation entities
- VLM Model Guide — Vision-language models for `maki describe`: tested models, backends, hardware guide

# Developer Guide

Technical documentation for integrators and contributors.

1. REST API — Complete web API documentation
2. Module Reference — Rust module overview and dependency graph
3. Building & Testing — Build commands, tests, and release process

---

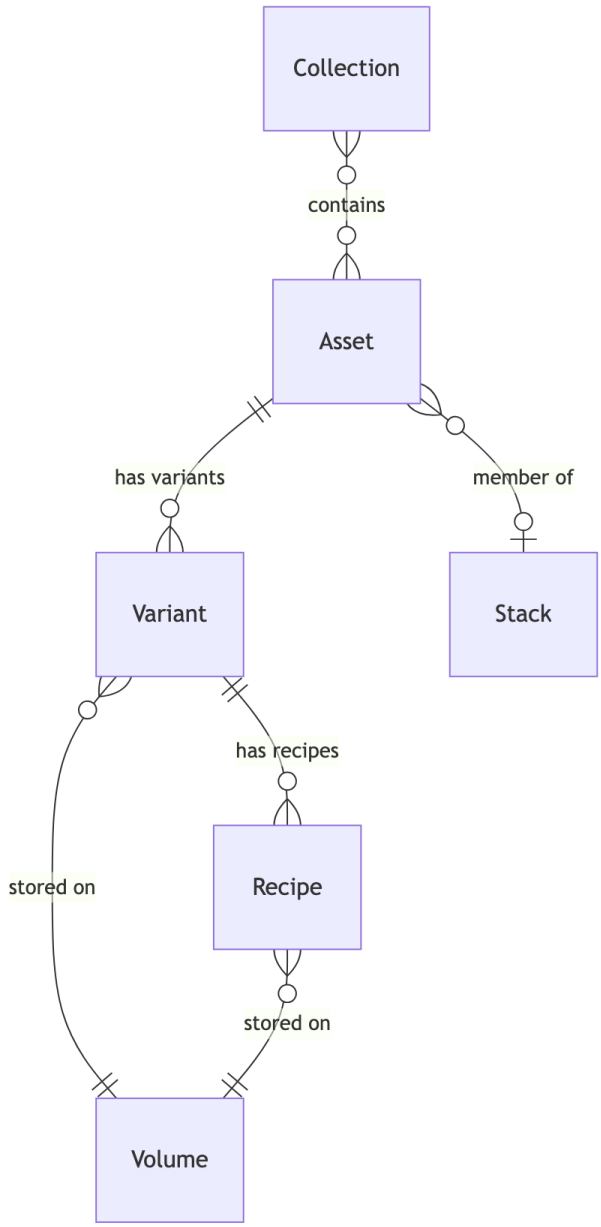
**Version**: v4.0.6 | **Source**: GitHub | **License**: Apache-2.0

# Overview & Concepts

This chapter introduces the data model, architecture, and workflow of **maki** – a CLI digital asset manager designed for photographers and media professionals managing large collections (terabytes of images, videos, and processing files) across multiple storage devices.

## Core Concepts

maki organizes your files around these core concepts.




## Asset

The central entity. An Asset represents a single logical image or media item – "photo of sunset at the beach" – regardless of how many files exist for it. An Asset has a deterministic UUID derived from the content hash of its first variant, plus optional metadata: name, description, tags (keywords), rating (1–5 stars), and color label (Red, Orange, Yellow, Green, Blue, Pink, Purple – a superset of Lightroom's palette that matches CaptureOne). Tags support hierarchy using `/` as a separator (e.g., `animals/birds/eagles`); searching for a parent tag matches all descendants.

## Variant

A concrete file belonging to an Asset. A RAW file, its JPEG conversion, and a high-res TIFF export are three Variants of the same Asset. Each Variant is identified by its **SHA-256 content hash** – the same file always produces the same hash, regardless of where it is stored.

Variants carry a **role** that describes their purpose:

| Role | Meaning |
|---|---|
| **Original** | Camera original (RAW, in-camera JPEG) |
| **Alternate** | Another original of the same shot (e.g. in-camera JPEG alongside RAW) |
| **Processed** | Intermediate edit (PSD, layered TIFF) |
| **Export** | Final deliverable (resized JPEG, web TIFF) |
| **Sidecar** | Accompanying non-media file |

Each Variant tracks one or more **file locations** – a volume plus a relative path. A single Variant can exist on multiple drives (copies, backups on a NAS), and maki tracks them all. Each location stores a `verified_at` timestamp from the last integrity check.

### Recipe

A processing sidecar file (`.xmp`, `.cos`, `.cot`, `.cop`, `.pp3`, `.dop`, `.on1`) attached to a Variant. Recipes store edits from tools like CaptureOne, Lightroom, RawTherapee, DxO, and ON1. Unlike Variants, Recipes are identified by **location** (volume + path) rather than content hash, because external tools routinely modify them in place. This means re-importing after an external edit updates the existing Recipe rather than creating a duplicate.

maki reads metadata from Recipes during import (rating, tags, description, color label) and can write changes back to XMP files via `maki writeback` or `maki sync-metadata`.

### Volume

A registered storage device: local disk, external drive, or network share. Each Volume has a human-readable label and a mount point. maki detects online/offline status at runtime, so you can browse and search assets on unmounted drives using cached metadata and preview thumbnails.

### Stack

A group of related but independent assets – burst shots, exposure brackets, or similar scenes that you want to collapse into a single entry in the browse grid. Each asset can belong to at most one stack. Members are position-ordered; position 0 is the **pick** (the representative image shown when the stack is collapsed). Unlike variants (which merge files into one asset), stacked assets keep their own ratings, tags, and descriptions.

### Collection

A manual album – a named list of assets you curate by hand. Collections are static: adding or removing assets requires an explicit action. Use collections for portfolios, client deliveries, print selections, or any grouping that doesn't follow a search pattern.

### Saved Search

A smart album – a named search query that dynamically matches assets. Saved searches appear as quick-access chips in the browse toolbar. When your catalog changes (new imports, rating changes, tag edits), the results update automatically because the query is re-evaluated each time.

## Content-Addressable Storage

Every file imported into maki is hashed with SHA-256. This hash becomes the file's identity:

- **Deduplication**: Importing the same file twice (even from different paths or drives) recognizes it as the same content and adds the new location to the existing Variant rather than creating a duplicate.
- **Integrity verification**: The `verify` command re-hashes files on disk and compares against stored hashes to detect corruption or bit rot.
- **Transparent relocation**: Moving a file to a different drive does not change its identity. The `relocate` and `update-location` commands update the catalog to reflect the new path.

Originals (RAW files, camera JPEGs) are immutable – their content never changes, so their hash is stable forever. Recipe files are the exception: they are modified by external tools, so maki tracks them by location and updates their stored hash when changes are detected.

## Two-Tier Storage

maki uses a dual-storage architecture. Neither tier alone is sufficient; together they provide both robustness and performance.

**YAML sidecar files** (source of truth): One `.yml` file per Asset, stored in the catalog's `assets/` directory. Contains the complete Asset record: metadata, all Variants, all Recipes, all FileLocations. Human-readable, diffable, and version-control friendly. If the SQLite database is ever lost or corrupted, running `rebuild-catalog` reconstructs it entirely from these files.

**SQLite catalog** (derived cache): A single `catalog.db` file providing fast indexed queries. Contains denormalized columns for efficient browse-grid rendering (best variant hash, primary format, variant count). The catalog is always rebuildable from sidecars – it is a performance optimization, not a source of truth.

This design means you never lose data from a database corruption event. The YAML sidecars are the authoritative record.

## Multi-Volume Support

Real-world photo libraries span multiple storage devices: a fast internal SSD for current projects, external drives for archives, a NAS for backups. maki handles this natively:

- **Register volumes** with `maki volume add` – give each drive a label and mount point.
- **Import from any volume** – maki auto-detects which registered volume a file path belongs to and stores a volume-relative path.
- **Offline browsing** – when a drive is unmounted, its assets remain searchable and browsable via cached metadata and preview thumbnails in the local catalog.
- **Cross-volume operations** – `relocate` copies or moves assets between volumes, `verify` checks file integrity across all online volumes, and `cleanup` detects missing files on online volumes.

See Setup for volume registration and Maintenance for cross-volume operations.

## Variant Grouping

During import, maki automatically groups related files into a single Asset using **stem-based matching**: files that share the same filename stem in the same directory are grouped together.

For example, importing a directory containing:

```
DSC_4521.NEF       --> Original variant (RAW)
DSC_4521.jpg       --> Original variant (JPEG)
DSC_4521.xmp       --> Recipe (attached to NEF variant)
```

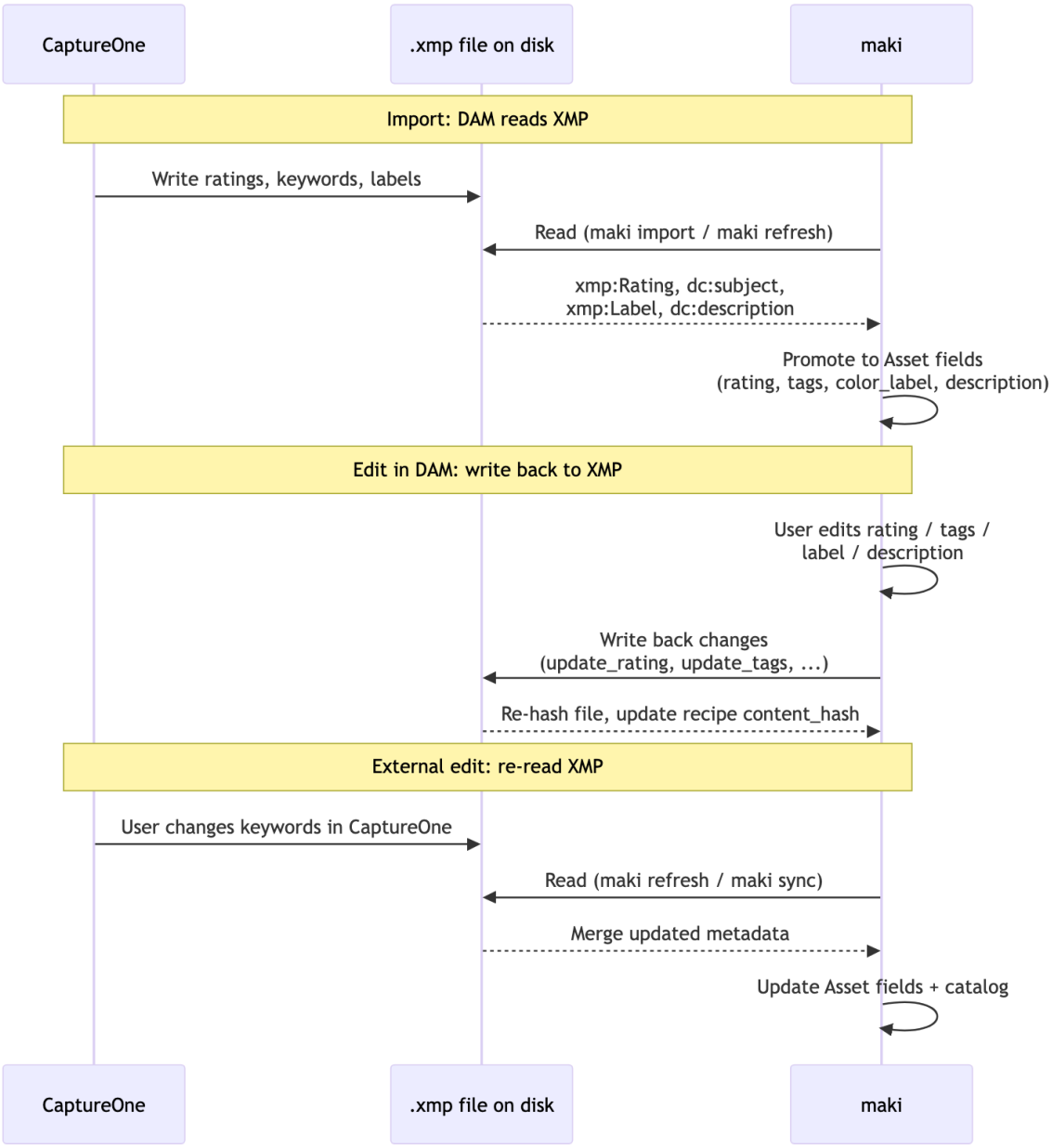
produces one Asset with two Variants and one Recipe. The RAW file takes priority as the primary variant (defining the asset's identity, EXIF metadata, and deterministic UUID).

For files that end up in different directories – common with CaptureOne exports that land in an `output/` subfolder – the `auto-group` command performs **fuzzy prefix matching** across the entire catalog. It matches stems like `DSC_4521` and `DSC_4521-1-HighRes` by checking that the shorter stem is a prefix of the longer one and the next character is a non-alphanumeric separator (`-`, `_`, space, `(`).

See Ingesting Assets for import details and Organizing Assets for manual and automatic grouping.

## Bidirectional XMP Sync

maki maintains two-way synchronization with `.xmp` sidecar files, enabling a round-trip workflow with tools like CaptureOne and Lightroom.




**DAM to disk**: When you change a rating, tag, description, or color label via the CLI (`maki edit`, `maki tag`) or the web UI, maki writes the change back to any associated `.xmp` files on disk, then re-hashes them and

updates the stored recipe hash.

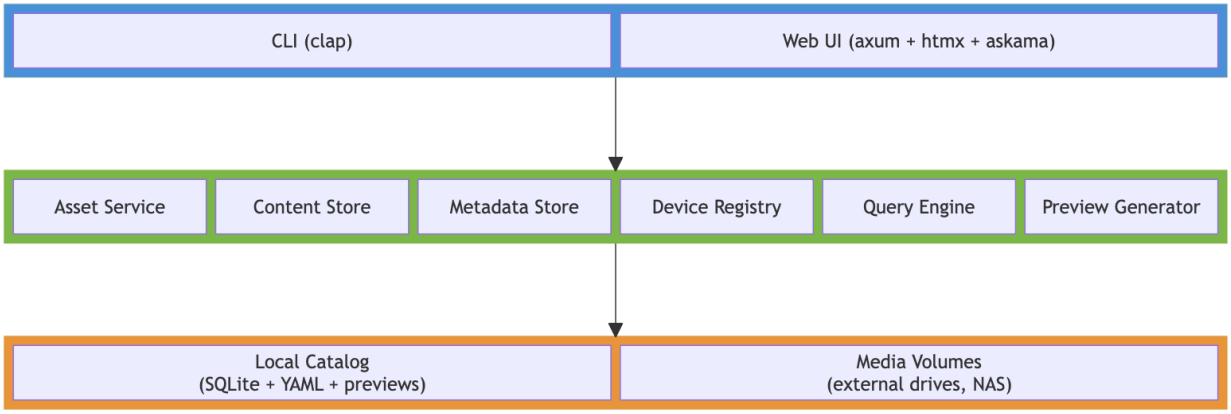
**Disk to DAM**: When CaptureOne or another tool modifies an `.xmp` file, running `maki refresh` or `maki sync` detects the changed hash, re-extracts the XMP metadata, and updates the Asset in both the catalog and sidecar YAML.

Tags added independently in CaptureOne are preserved during write-back – maki uses operation-level deltas (add/remove specific tags) rather than overwriting the entire tag list.

See Ingesting Assets for XMP extraction during import and Maintenance for the `refresh` and `sync` commands.

# Architecture

maki is structured in three layers.




## Interface Layer

**CLI** (clap): Subcommand-based interface (`maki import`, `maki search`, `maki serve`, etc.). Supports `--json` for machine-readable output, `--log` for per-file progress, `--format` for custom output templates.

**Web UI** (axum + htmx + askama): Browser-based interface started with `maki serve`. Provides a browse grid with filters, inline editing, batch operations, and keyboard navigation. Uses htmx for partial page updates without a JavaScript framework.

## Core Library

| Component | Responsibility |
|---|---|
| **Asset Service** | Orchestrates import, grouping, relocation, verification, preview generation |
| **Content Store** | SHA-256 hashing, deduplication, hash-to-location mapping |
| **Metadata Store** | YAML sidecar read/write, the authoritative data layer |
| **Device Registry** | Volume tracking, mount point detection, online/offline status |
| **Query Engine** | SQLite search, metadata editing (tags, rating, label), auto-grouping |
| **Preview Generator** | Thumbnail creation via `image` crate, `dcraw`/LibRaw (RAW), `ffmpeg` (video) |

Additional modules handle EXIF extraction, XMP reading/writing, configuration parsing (`maki.toml`), output formatting, and collection/saved-search management.

### Storage Layer

**Local Catalog** (always available on local disk): - `catalog.db` – SQLite database (derived index, rebuildable) - `assets/` – YAML sidecar files (source of truth) - `previews/` – JPEG thumbnails for offline browsing - `volumes.yaml` – registered volume definitions - `searches.toml` – saved searches - `collections.yaml` – static album definitions - `stacks.yaml` – stack definitions (member order and pick) - `maki.toml` – configuration

**Media Volumes** (may be offline): The actual asset files on external drives, NAS, or local directories. maki never moves or modifies original media files unless explicitly asked (via `relocate --remove-source`).

## Preview Generation

maki generates preview thumbnails (800px JPEG by default) for each variant during import, enabling offline browsing even when media volumes are unmounted. Different file types use different strategies:

| File type | Strategy |
|---|---|
| Standard images (JPEG, PNG, TIFF, WebP) | `image` crate (native Rust) |
| RAW files (NEF, ARW, CR3, …) | `dcraw` or `dcraw_emu` (LibRaw) |
| Video files (MP4, MOV, …) | `ffmpeg` (frame extraction) |
| Audio, documents, other | Info card (metadata summary rendered as JPEG) |

Preview failure never blocks import. If an external tool is missing, an info card is generated as a fallback.

The display logic prefers Export > Processed > Original variant previews, with standard image formats preferred over RAW within the same role. This means your best-quality deliverable is shown in the browse grid, not the camera original.

Preview settings (max edge size, format, quality) are configurable in `maki.toml`. See Setup for configuration details and Ingesting Assets for the `generate-previews` command.

## Collections, Stacks, and Saved Searches

maki provides three ways to organize assets into groups:

**Collections** (static albums): Manually curated lists of asset IDs. You explicitly add and remove assets. Backed by both SQLite (for fast queries) and a `collections.yaml` file (for persistence across catalog rebuilds). Use these for curated sets like "Portfolio" or "Client Deliverables".

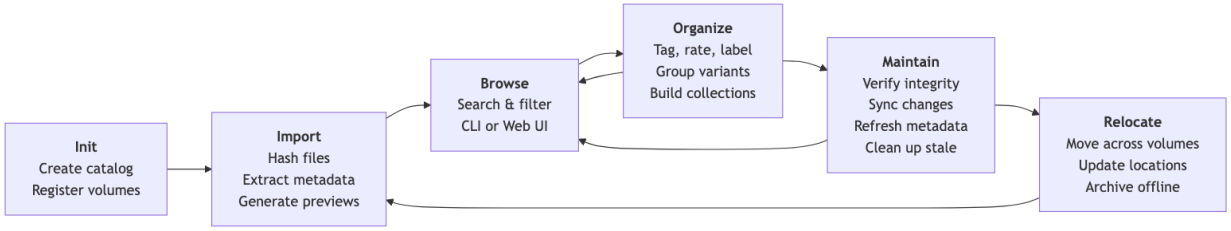
**Stacks**: Lightweight anonymous groups for burst shots, bracketing, or similar scenes. The browse grid collapses each stack to show only the pick image with a count badge, reducing visual clutter. Backed by a `stacks.yaml` file for persistence across catalog rebuilds. Use these to declutter the browse grid without permanently merging variants.

**Saved searches** (smart albums): Named queries stored in `searches.toml`. A saved search is a set of filter criteria (tags, rating, date range, volume, etc.) that is re-evaluated every time you run it. Results update automatically as your catalog changes. Use these for dynamic views like "5-star landscapes" or "untagged imports from last week".

All three are accessible from the CLI and the web UI. See Organizing Assets for detailed usage.

## The Round-Trip Workflow

A typical maki workflow follows this cycle:




1. **Init** – Create a catalog directory and register your storage volumes. See Setup.
2. **Import** – Point maki at directories of files. It hashes everything, extracts EXIF/XMP metadata, groups related files into Assets, generates preview thumbnails, and builds the catalog. See Ingesting Assets.
3. **Browse** – Search and filter your catalog by tags, rating, color label, camera, date, format, volume, path, collection, or free text. Use the CLI for scripting or the web UI for visual browsing. See Browsing & Searching and Web UI.
4. **Organize** – Add tags, set ratings and color labels, write descriptions, group variants, build collections, and save searches. Changes write back to XMP files for interoperability. See Organizing Assets.
5. **Maintain** – Verify file integrity against stored hashes, sync the catalog with disk after external tools move or rename files, refresh metadata from modified XMP sidecars, and clean up stale location records. See Maintenance.
6. **Relocate** – Copy or move assets between volumes (external drives, NAS, archive storage). Update location records after manual file moves. The cycle continues as you import new work or re-process existing assets.

Each step is covered in detail in the following chapters.

---

Next: Setup – Installation, catalog initialization, volume registration, and configuration.

# Setup

This chapter covers building maki from source, initializing a catalog, registering storage volumes, and configuring the system to match your workflow.

## Installation

### Building from source

maki is written in Rust. You need a working Rust toolchain (rustc + cargo). Install one via rustup if you have not already.

Clone the repository and build a release binary:

```
git clone https://github.com/your-org/maki.git
cd maki
cargo build --release
```

The binary is at `target/release/maki`. Copy or symlink it to a directory on your `PATH`:

```
cp target/release/maki /usr/local/bin/
```

Verify the installation:

```
maki --version
```

### Supported platforms

maki builds and runs on **macOS**, **Linux**, and **Windows**. Both x86_64 and ARM (Apple Silicon) are supported.

### External tools (highly recommended)

maki handles standard image formats (JPEG, PNG, TIFF, WebP) natively. The following external tools extend its capabilities:

| Tool | Purpose | Install |
|---|---|---|
| **dcraw** or **dcraw_emu** (LibRaw) | RAW file previews (NEF, ARW, CR2, CR3, etc.) | `brew install dcraw` or `brew install libraw` on macOS; your package manager on Linux; `winget install LibRaw.LibRaw` or `scoop install libraw` on Windows |
| **ffmpeg** | Video thumbnail extraction | `brew install ffmpeg` on macOS; your package manager on Linux; `winget install Gyan.FFmpeg` or `scoop install ffmpeg` on Windows |

| Tool | Purpose | Install |
|---|---|---|
| **curl** | AI model download and VLM image descriptions | Pre-installed on macOS and most Linux distributions; `winget install cURL.cURL` or `scoop install curl` on Windows |

When an external tool is missing, maki prints a warning on first use explaining what is needed and why. It still imports RAW and video files, but generates an info card (a placeholder JPEG showing file metadata) instead of a rendered preview. You can install the tools later and run `maki generate-previews --force` to regenerate real previews.

# Initializing a Catalog

A catalog is a directory that holds maki's database, configuration, metadata sidecars, and preview images. Create one by navigating to the directory you want as the catalog root and running `maki init`:

```
mkdir ~/Photos
cd ~/Photos
maki init
```

Output:

```
Initialized new maki catalog in /Users/you/Photos
```

This creates the following structure:

```
~/Photos/
  maki.toml          # Configuration file
  catalog.db        # SQLite database (cache/index)
  volumes.yaml      # Registered storage volumes
  metadata/         # YAML sidecar files (source of truth)
  previews/         # Generated preview images
```

## How catalog detection works

After initialization, you can run maki commands from the catalog root or any subdirectory. maki locates the catalog by walking up from your current working directory, looking for a `maki.toml` file. This means you can organize files in subdirectories and still run commands without specifying the catalog path.

```
cd ~/Photos
maki stats            # works -- maki.toml is here

cd ~/Photos/metadata
maki stats            # also works -- finds maki.toml in parent

cd /tmp
maki stats            # fails -- no maki.toml above /tmp
```

If no catalog is found, maki prints:

```
Error: No maki catalog found. Run `maki init` to create one.
```

## Reinitializing

`maki init` refuses to overwrite an existing catalog. If you need to start fresh, delete the catalog files first (`maki.toml`, `catalog.db`, `metadata/`, `previews/`, `volumes.yaml`) and run `maki init` again.

# Registering Volumes

A **volume** represents a storage location – a local directory, an external drive, or a NAS mount point. Before importing files, you must register the volume they live on.

## Adding a volume

```
maki volume add "Photos 2024" /Volumes/PhotosDrive
```

Output:

```
Registered volume 'Photos 2024' (a1b2c3d4-e5f6-7890-abcd-ef1234567890)
  Path: /Volumes/PhotosDrive
```

Each volume gets a UUID that stays constant even if the drive letter or mount point changes. The label is a human-readable name you choose.

A few more examples:

```
# External SSD
maki volume add "Travel SSD" /Volumes/TravelPhotos

# Network-attached storage
maki volume add "NAS Archive" /Volumes/nas/photos

# Local directory
maki volume add "Local Work" /Users/you/Photography
```

## Listing volumes

```
maki volume list
```

Output:

```
Photos 2024 (a1b2c3d4-e5f6-7890-abcd-ef1234567890) [online]
  Path: /Volumes/PhotosDrive
Travel SSD (b2c3d4e5-f6a7-8901-bcde-f12345678901) [offline]
  Path: /Volumes/TravelPhotos
```

## Online vs. offline

maki checks whether each volume's mount point directory exists on disk:

- **Online**: The directory exists. maki can read files, generate previews, and verify integrity.
- **Offline**: The directory does not exist (drive disconnected, NAS unreachable). maki still knows about the volume and its assets, but file operations are skipped gracefully.

This design lets you manage a photo library that spans multiple external drives. You can search, browse, and view cached previews for assets on offline volumes. When you reconnect a drive, maki picks it up automatically.

## Multiple volumes

There is no limit on the number of volumes. A typical setup might look like:

- A fast local SSD for current work
- One or more external drives for archive storage
- A NAS for backups

Assets can have files on multiple volumes simultaneously (see Maintenance for the `relocate` command).

## Volume purposes

Each volume can optionally be assigned a **purpose** that describes its role in your storage hierarchy:

| Purpose | Meaning |
|---|---|
| `working` | Active editing drive — fast SSD with current projects |
| `archive` | Long-term primary storage — the "master" copy |
| `backup` | Redundancy copy — exists purely for safety |
| `cloud` | Cloud-synced folder (Dropbox, iCloud, Google Drive) |

```
maki volume add "Laptop SSD" /Volumes/MacintoshHD --purpose working
maki volume add "Archive"    /Volumes/MediaDrive   --purpose archive
maki volume add "Backup A"   /Volumes/BackupDisk   --purpose backup
maki volume add "Dropbox"    ~/Dropbox/Photos      --purpose cloud
```

Purpose metadata drives two features:

- **Duplicate analysis** (`maki duplicates`): Distinguishes unwanted duplicates (same file twice on the same working drive) from wanted redundancy (same file on an archive and a backup).
- **Backup coverage** (`maki backup-status`): Reports which assets lack copies on archive or backup volumes and flags at-risk assets.

You can set or change a purpose at any time:

```
maki volume set-purpose "Laptop SSD" archive
maki volume set-purpose "Laptop SSD" none      # clear
```

Volumes without a purpose are treated as unclassified — they still work for import, search, and all other operations, but are excluded from purpose-based analysis.

## Symlinks and path resolution

maki resolves symlinks when registering volumes and importing files. All paths stored in the catalog are **physical (canonical) paths**, not the symlink paths you may see in your filesystem.

This matters when your directory layout uses symlinks to span multiple disks. For example, if you have:

```
/Volumes/Pictures/masters/2025/   (real directory)
/Volumes/Pictures/masters/2026 → /Volumes/Dropbox/masters/2026/  (symlink)
```

and you register `/Volumes/Pictures` as a volume, then:

- Files under `.../masters/2025/` are tracked on the "Pictures" volume as expected.
- Files under `.../masters/2026/` resolve through the symlink to `/Volumes/Dropbox/masters/2026/`, which is **outside** the Pictures volume mount point. Import will fail with "No registered volume contains path" unless the Dropbox path is also registered as a volume.

**Why maki resolves symlinks:**

- **Reliable verification.** `maki verify` re-hashes files by their catalog path. If the catalog stored a symlink path and the symlink later changed target, verification would silently check the wrong file — or fail when the link breaks.
- **Correct offline detection.** A volume is "online" when its mount point exists. A broken symlink inside an online volume would cause confusing partial failures.

- **Predictable sync behavior.** `maki sync` detects moved and missing files by comparing disk state to catalog paths. Symlink changes would cause false "moved" or "missing" reports for every file under the changed link.
- **Unambiguous volume mapping.** Each file belongs to exactly one volume (determined by longest mount-point prefix match on the physical path). Symlinks could cause the same file to appear to belong to two different volumes depending on which path you use.

**The recommended workaround** is to register each physical storage location as its own volume:

```
maki volume add "Pictures"  /Volumes/Pictures  --purpose archive
maki volume add "Dropbox"   /Volumes/Dropbox   --purpose cloud
```

Both volumes work independently. Import auto-detects the correct volume based on the resolved physical path. `backup-status` shows correct copy counts across both. You can still navigate your symlinked directory structure normally — maki simply tracks where files physically reside.

### Removing a volume

If a volume is no longer needed (drive decommissioned, cloud service cancelled), you can cleanly remove it:

```
maki volume remove "Old Drive"          # report what would be removed
maki volume remove "Old Drive" --apply  # actually remove
```

This removes all file location and recipe records on that volume from the catalog and sidecar files, deletes any assets that become orphaned (no remaining file locations), and cleans up orphaned preview files. Without `--apply`, it runs in report-only mode so you can review the impact first.

See the volume remove reference for details.

### Combining volumes

If you initially registered year-based subdirectories as separate volumes (e.g., `media_2024`, `media_2025` under `/Volumes/Media`) and now want to consolidate them into a single disk-level volume, use `volume combine`:

```
# First register the parent directory as a volume
maki volume add "Media" /Volumes/Media

# Preview what combining would do
maki volume combine "media_2024" "Media"
# Would combine 'media_2024' into 'Media': 4832 locations, 312 recipes (3210 assets, prefix 'media_2024/')

# Execute the combination
maki volume combine "media_2024" "Media" --apply
```

The source volume's mount point must be a subdirectory of the target's. All file paths are automatically rewritten with the appropriate prefix (e.g., `photo.jpg` becomes `media_2024/photo.jpg`). After combining, the source volume is removed. You can repeat this for each year-volume to consolidate into one.

See the volume combine reference for details.

## Configuration (maki.toml)

The `maki.toml` file at the catalog root controls maki's behavior. All sections are optional – an empty file (or one with only comments) uses sensible defaults.

Here is a complete example showing every available option:

```
# Default volume for import when auto-detection is ambiguous
default_volume = "a1b2c3d4-e5f6-7890-abcd-ef1234567890"

[preview]
max_edge = 800        # Maximum width/height in pixels (default: 800)
format = "jpeg"       # "jpeg" or "webp" (default: "jpeg")
quality = 85          # JPEG quality 1-100 (default: 85; ignored for webp)

[serve]
port = 8080           # Web UI port (default: 8080)
bind = "127.0.0.1"    # Bind address (default: "127.0.0.1")

[import]
exclude = [           # Glob patterns to skip during import
    ".DS_Store",
    "Thumbs.db",
    "*.tmp",
]
auto_tags = [         # Tags automatically applied to every new asset
    "inbox",
    "unreviewed",
]
```

## Section summary

**`default_volume`** – UUID of the volume to use when `maki import` cannot auto-detect the correct volume from the file path. Useful if you always import from the same drive.

**`[preview]`** – Controls preview generation. `max_edge` sets the longest side of generated thumbnails. `format` chooses between JPEG (smaller, lossy) and WebP (lossless via the `image` crate). `quality` only applies to JPEG output.

**`[serve]`** – Web UI server settings. Change `bind` to `"0.0.0.0"` to allow access from other machines on your network. CLI flags `--port` and `--bind` override these values.

**`[import]`** – `exclude` patterns are matched against filenames (not full paths) using glob syntax. Common choices: OS junk files, editor temp files, thumbnail caches. `auto_tags` are merged with any tags extracted from XMP metadata during import; useful for tagging everything in a session as "unreviewed" for later triage.

For a complete reference of every option and its behavior, see the Configuration Reference.

---

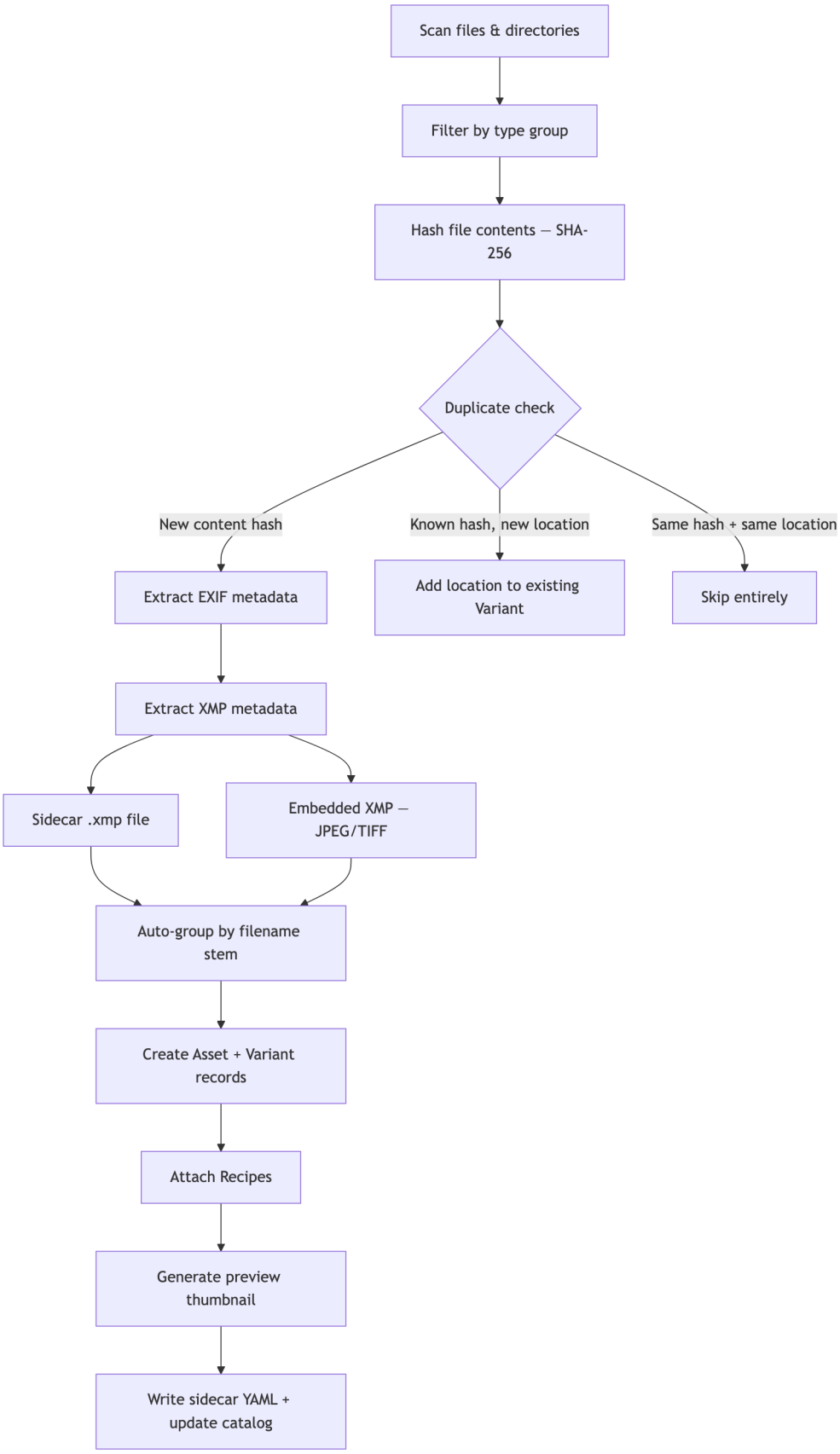
Next: Ingesting Assets – importing files into the catalog, auto-grouping, and metadata extraction.

# Ingesting Assets

Importing is how files enter the maki catalog. Unlike traditional asset managers that copy files into a managed library folder, maki catalogs files **in place** on your existing volumes. The import process hashes each file (SHA-256), extracts metadata, generates previews, and records everything in the catalog – but your originals stay exactly where they are.

## Import Pipeline

The following diagram shows the stages a file passes through during import:


Scan files & directories
Filter by type group
Hash file contents — SHA-256
Duplicate check
New content hash
Known hash, new location
Same hash + same location
Extract EXIF metadata
Add location to existing Variant
Skip entirely
Extract XMP metadata
Sidecar .xmp file
Embedded XMP — JPEG/TIFF
Auto-group by filename stem
Create Asset + Variant records
Attach Recipes
Generate preview thumbnail
Write sidecar YAML + update catalog

**Metadata precedence**: On initial import, fields are set by the first source that provides them: EXIF data is extracted first, then embedded XMP fills any remaining empty fields, then sidecar XMP fills what is still unset. Tags (keywords) are merged from all sources as a union. On subsequent recipe updates (e.g. after editing in Capture One or Lightroom), sidecar XMP takes highest precedence and overwrites rating, description, and color label. The `created_at` date is always derived from EXIF and never overwritten by XMP.

# Basic Import

Import a directory of photos:

```
maki import /Volumes/PhotosDrive/Photos/2026-02-20/
```

Import specific files:

```
maki import /Volumes/PhotosDrive/DSC_0042.nef /Volumes/PhotosDrive/DSC_0043.nef
```

The path you provide must reside on a registered volume. maki resolves the volume automatically from the file path. If auto-detection picks the wrong volume (for example, when mount points are nested), specify it explicitly:

```
maki import --volume "Photos 2024" /path/to/files
```

## Preview Before Committing

Use `--dry-run` to see what would happen without writing anything to the catalog:

```
maki import --dry-run /Volumes/PhotosDrive/Photos/2026-02-20/
```

Files are still hashed during a dry run (so duplicate detection works), but no assets, variants, recipes, or previews are created. The output shows the same counters as a real import:

```
Dry run — would import: 47 imported, 3 skipped, 2 locations added, 5 recipes attached
```

Add `--json` for structured output (includes a `dry_run: true` field) or `--log` to see per-file details.

## What Import Does NOT Do

Import does not copy or move your files. The catalog stores references to files on their volumes. This means:

- Your directory structure stays intact.
- Files are not renamed or reorganized.
- The same file can be cataloged from multiple volumes (e.g., original drive and a backup).
- If a volume is disconnected, the catalog remembers the files but marks the volume as offline.

# Auto-Grouping

When maki imports a directory, it groups files by **filename stem** – the filename without its extension. Files sharing the same stem in the same directory become a single Asset with multiple Variants.

## How It Works

Consider a directory containing:

```
DSC_0042.nef
DSC_0042.jpg
DSC_0042.xmp
```

maki creates **one Asset** with:

| File | Role | Record Type |
|---|---|---|
| `DSC_0042.nef` | Original (primary) | Variant |
| `DSC_0042.jpg` | Alternate variant | Variant |
| `DSC_0042.xmp` | Processing sidecar | Recipe |

RAW files always take priority as the primary variant because they represent the original capture. The primary variant defines the asset's identity and provides the authoritative EXIF data.

## Practical Example

A typical CaptureOne session directory might contain:

```
Z91_8561.ARW          # RAW original
Z91_8561.ARW.xmp      # XMP sidecar from CaptureOne
Z91_8561.ARW.cos      # CaptureOne settings
Z91_8561.jpg          # JPEG export
```

After import:

```
maki show <asset-id>
```

```
Asset: Z91_8561
  Variant 1 (Original): Z91_8561.ARW [abc123...]
  Variant 2 (Alternate): Z91_8561.jpg [def456...]
  Recipe 1: Z91_8561.ARW.xmp
  Recipe 2: Z91_8561.ARW.cos
```

For more advanced grouping across different directories or after the fact, see the auto-group command and group command.

## Cross-Directory Grouping with –auto-group

Standard auto-grouping only matches files within the same directory. But photo tools like CaptureOne often place RAW originals and exports in separate folders:

```
2026-02-22/
  Capture/
    DSC_0042.ARW
  Output/
    DSC_0042.JPG
    DSC_0042-1-HighRes.tif
```

Without `--auto-group`, importing both directories creates separate assets for the RAW and exports. The `--auto-group` flag runs a post-import grouping step scoped to the "neighborhood" of the imported files:

```
maki import --auto-group /Volumes/Photos/2026-02-22/Capture /Volumes/Photos/2026-02-22/Output
```

```
Import complete: 3 imported, 1 recipe(s) attached, 3 preview(s) generated
Auto-group: 1 stem group(s), 2 donor(s) merged, 2 variant(s) moved
```

**How the neighborhood is determined**: For each imported file's directory (e.g., `2026-02-22/Capture/`), maki goes up one level to find the "session root" (`2026-02-22/`). It then searches the catalog for all assets under those session roots on the same volume. Only those assets participate in grouping.

This scoping prevents false positives from restarting camera counters – `DSC_0001` from a January shoot won't be grouped with `DSC_0001` from a June shoot because they live under different session roots.

**Incremental workflow**: You can also use `--auto-group` when adding exports to a previously imported shoot:

```
# First import: RAW originals only
maki import /Volumes/Photos/2026-02-22/Capture

# Later: CaptureOne exports are ready
maki import --auto-group /Volumes/Photos/2026-02-22/Output
```

The second import detects the existing RAW assets in the sibling `Capture/` directory and groups the exports with them.

Combine with `--dry-run` to preview what would be grouped without making changes:

```
maki import --auto-group --dry-run /Volumes/Photos/2026-02-22/Capture /Volumes/Photos/2026-02-22/Output
```

# Recipe Handling

Processing sidecars are files created by RAW processors and image editors to store non-destructive edit settings. maki recognizes the following recipe formats:

| Extension | Tool |
|---|---|
| `.xmp` | Adobe Lightroom, CaptureOne, generic XMP |
| `.cos`, `.cot`, `.cop` | CaptureOne sessions/catalogs |
| `.pp3` | RawTherapee |
| `.dop` | DxO PhotoLab |
| `.on1` | ON1 Photo RAW |

## Attachment Rules

Recipes are attached to the **primary variant** of the matching asset (matched by filename stem in the same directory). They are identified by **location** (volume + path), not by content hash. This is an important distinction:

- If you edit settings in CaptureOne and re-import, maki updates the existing recipe record rather than creating a duplicate.
- The recipe's content hash is updated to reflect the new file contents.

## Standalone Recipe Resolution

You can import recipe files after their media files:

```
# First import: RAW files
maki import /Volumes/PhotosDrive/Shoot/

# Later: CaptureOne creates XMP sidecars
maki import /Volumes/PhotosDrive/Shoot/*.xmp
```

maki finds the parent variant by matching the filename stem and directory, then attaches the recipe to it.

## Enabling Non-Default Recipe Groups

By default, only `.xmp` recipe files are imported. CaptureOne, RawTherapee, DxO, and ON1 sidecars require opting in:

```
maki import --include captureone /Volumes/PhotosDrive/Shoot/
maki import --include captureone --include rawtherapee /Volumes/PhotosDrive/Shoot/
```

See Import Options below for the full list of type groups.

# Metadata Extraction

maki extracts metadata from three sources during import, each contributing different information about the asset.

## EXIF Data

Extracted directly from image files (RAW, JPEG, TIFF). Provides camera and capture information:

- Camera make and model
- Lens
- ISO, aperture, shutter speed, focal length
- Image dimensions
- Capture date and time
- GPS coordinates (if present)

EXIF data is stored in the variant's `source_metadata` and used to populate the asset's `captured_at` timestamp.

## XMP Sidecar Files

When an `.xmp` file is attached as a recipe, its contents are parsed and merged into the asset:

| XMP Field | maki Field | Notes |
|---|---|---|
| `dc:subject` | Tags | Merged with existing tags (union) |
| `dc:description` | Description | Set if not already present |
| `xmp:Rating` | Rating | Integer 1-5 |
| `xmp:Label` | Color label | Red, Orange, Yellow, Green, Blue, Pink, Purple |
| `dc:creator` | Source metadata | Stored as `creator` |
| `dc:rights` | Source metadata | Stored as `rights` |

## Embedded XMP

JPEG and TIFF files can contain XMP metadata embedded in their binary structure (APP1 marker for JPEG, IFD tag 700 for TIFF). maki extracts the same fields as from sidecar XMP. This captures keywords, ratings, and labels from tools like CaptureOne and Lightroom that embed XMP directly in exported files.

Other file formats (RAW, video, audio, etc.) are skipped for embedded XMP extraction – zero I/O overhead.

## Processing Order and Precedence

Metadata sources are processed in this order during import:

1. **EXIF** — extracted first. Sets `created_at`, camera/lens info, exposure data, GPS coordinates.
2. **Embedded XMP** (JPEG/TIFF only) — fills any fields still empty after EXIF (insert-if-absent).
3. **Sidecar XMP** — fills any fields still empty after embedded XMP (insert-if-absent).

For rating, description, and color label, the **first source to set a value wins** on initial import. Tags are always merged from all sources (union of all keywords, no removal).

When a sidecar recipe is **updated later** (e.g. after editing in Capture One or Lightroom), the behavior changes: sidecar XMP **overwrites** rating, description, and color label. This ensures that edits made in photo editing software take effect. The `created_at` date is never overwritten — it is always derived from EXIF.

# Preview Generation

maki generates preview thumbnails during import so you can browse assets without accessing the original files.

## By File Type

| File type | Method | Result |
|---|---|---|
| Standard images (JPEG, PNG, TIFF, WebP, etc.) | `image` crate | 800px JPEG thumbnail |
| RAW files (NEF, ARW, CR3, DNG, etc.) | `dcraw` or `dcraw_emu` (LibRaw) | 800px JPEG thumbnail |
| Video (MP4, MOV, MKV, etc.) | `ffmpeg` frame extraction | 800px JPEG thumbnail |
| Audio, documents, unknown | Info card renderer | 800x600 JPEG with file metadata |

The maximum edge size (default 800px) and output format (JPEG or WebP) are configurable in `maki.toml`:

```
[preview]
max_edge = 1200
format = "jpeg"
quality = 85
```

## Info Cards

When a file has no visual preview (audio files, documents) or when external tools (`dcraw`, `ffmpeg`) are not installed, maki generates an **info card** – an 800x600 JPEG displaying the file's metadata:

- Filename and format
- File size
- For audio: duration, bitrate, sample rate (extracted via the `lofty` crate)

## Storage and Failure

Previews are stored under the catalog's `previews/` directory:

```
previews/
  ab/
    ab3f7c9e2d...1a4b.jpg
  f2/
    f29a4bc81e...7d3c.jpg
```

The first two characters of the content hash serve as a subdirectory prefix to avoid having too many files in a single directory.

**Preview generation never blocks import.** If `dcraw` is missing, a RAW file still imports successfully – it just gets an info card instead of a rendered preview. You can regenerate previews later with `maki generate-previews`.

# Import Options

## File Type Groups

maki organizes recognized file extensions into groups. Some are enabled by default; others require opting in.

| Group | Extensions | Default |
|---|---|---|
| `images` | jpg, jpeg, png, gif, bmp, tiff, tif, webp, heic, heif, svg, ico, psd, xcf, raw, cr2, cr3, crw, nef, nrw, arw, sr2, srf, orf, rw2, dng, raf, pef, srw, mrw, 3fr, fff, iiq, erf, kdc, dcr, mef, mos, rwl, bay, x3f | On |
| `video` | mp4, mov, avi, mkv, wmv, flv, webm, m4v, mpg, mpeg, 3gp, mts, m2ts | On |
| `audio` | mp3, wav, flac, aac, ogg, wma, m4a, aiff, alac | On |
| `xmp` | xmp | On |
| `documents` | pdf, doc, docx, xls, xlsx, ppt, pptx, txt, md, rtf, csv, json, xml, html, htm | Off |
| `captureone` | cos, cot, cop | Off |
| `rawtherapee` | pp3 | Off |
| `dxo` | dop | Off |
| `on1` | on1 | Off |

Enable additional groups with `--include`, disable default groups with `--skip`:

```
# Import photos plus CaptureOne sidecars, but skip audio files
maki import --include captureone --skip audio /Volumes/PhotosDrive/Shoot/
```

## Exclude Patterns

Configure glob patterns in `maki.toml` to permanently exclude files by name:

```toml
[import]
exclude = ["Thumbs.db", "*.tmp", ".DS_Store"]
```

## Auto-Tags

Apply tags automatically to every newly imported asset:

```toml
[import]
auto_tags = ["inbox", "unreviewed"]
```

Auto-tags are merged with any tags extracted from XMP metadata (deduplicated).

## All Flags

| Flag | Effect |
|---|---|
| `--volume <label>` | Use a specific volume instead of auto-detecting from the file path |

| Flag | Effect |
|---|---|
| `--dry-run` | Show what would happen without writing to catalog, sidecar, or disk |
| `--auto-group` | After import, group new assets with nearby catalog assets by filename stem |
| `--include <group>` | Enable an additional file type group (repeatable) |
| `--skip <group>` | Disable a default file type group (repeatable) |
| `--smart` | Generate smart previews (2560px) for zoom/pan in the web UI |
| `--embed` | Generate image embeddings for visual similarity search (requires `--features ai`) |
| `--json` | Structured JSON output to stdout |
| `--log` | Per-file progress to stderr (`filename -- status (duration)`) |
| `--time` | Show total elapsed time |

# Duplicate Handling

maki uses content hashing (SHA-256) to detect duplicates at the file level.

## Three Outcomes

When importing a file, one of three things happens:

1. **New content hash** – The file is new to the catalog. A new Variant is created (and potentially a new Asset).
2. **Known hash, new location** – The file's content already exists in the catalog, but this is a new path (perhaps on a different volume, or the file was copied to a new directory). The new location is **added** to the existing Variant. The asset is not duplicated.
3. **Same hash and same location** – Both the content and the path are already recorded. The file is truly skipped.

## Example: Backup Drive

```
# Import from primary drive
maki import /Volumes/Photos/Shoot-2026-02-20/

# Later, import the same files from a backup
maki import /Volumes/Backup/Shoot-2026-02-20/
```

The second import adds backup locations to the existing variants. Each variant now has two file locations (one per volume), but the catalog contains only one copy of each asset.

## Finding Duplicates

Use `maki duplicates` to find files with identical content across locations:

```
maki duplicates
```

```
abc3f7c9...  DSC_0042.nef
  /Volumes/Photos/Shoot-2026-02-20/DSC_0042.nef
  /Volumes/Backup/Shoot-2026-02-20/DSC_0042.nef
```

This is useful for identifying redundant copies, verifying backups, or cleaning up after file reorganization. See the maintenance chapter for more on managing file locations.

# AI Auto-Tagging

> Requires building with `--features ai`. For GPU-accelerated inference on macOS, use `--features ai-gpu` (CoreML). See Building & Testing.

After importing, you can use AI to automatically suggest tags for your images:

```
# First time: download the default SigLIP model (~207 MB)
maki auto-tag --download

# Preview suggested tags (report-only)
maki auto-tag "type:image"

# Apply suggested tags
maki auto-tag "type:image" --apply
```

The command uses SigLIP vision-language models for zero-shot classification against ~100 built-in photography categories (landscape, portrait, architecture, animals, etc.). Tags above the confidence threshold (default 0.1) are suggested. Use `--threshold` to adjust sensitivity.

Two models are available: the default ViT-B/16-256 (~207 MB) and the larger ViT-L/16-256 (~670 MB) for higher accuracy. Switch with `--model` or set `[ai] model` in `maki.toml`:

```
# Download and use the larger model
maki auto-tag --download --model siglip-vit-l16-256
maki auto-tag "type:image" --model siglip-vit-l16-256 --apply
```

You can provide a custom label vocabulary:

```
maki auto-tag --labels my-labels.txt "" --apply
```

Image embeddings are stored per model in the catalog, enabling visual similarity search:

```
maki auto-tag --similar <asset-id>
```

## Embedding Generation During Import

To generate embeddings automatically during import (so "Find similar" works immediately), use the `--embed` flag:

```
maki import --embed /Volumes/PhotosDrive/Photos/2026-02-20/
```

Or enable it permanently in `maki.toml`:

```toml
[import]
embeddings = true
```

This runs embedding generation as a post-import phase using the preview image for each imported asset. If the AI model is not downloaded, the embedding phase is silently skipped. You can also batch-generate embeddings for existing assets with `maki embed ""`.

See the auto-tag reference for all options and the configuration reference for `[ai]` settings in `maki.toml`.

# VLM Image Descriptions

While SigLIP auto-tagging classifies images against a fixed vocabulary (~100 labels), vision-language models (VLMs) generate free-form text descriptions that capture scene context, spatial relationships, lighting, and

mood. The `maki describe` command sends preview images to a VLM server and stores the generated text as the asset's description.

Unlike auto-tagging, this feature requires **no special build flags** – it works with any maki binary because it communicates with an external server via HTTP.

## Setting Up a Local VLM Server

The recommended approach is Ollama, a one-command install that manages model downloads, quantization, and GPU memory automatically.

**1. Install Ollama:**

```
# macOS (also available as .dmg from ollama.com)
brew install ollama

# Linux
curl -fsSL https://ollama.com/install.sh | sh
```

**2. Start the server:**

```
ollama serve
```

Ollama runs in the background on `http://localhost:11434`. On macOS, the Ollama desktop app starts the server automatically.

**3. Pull a vision model:**

```
# Recommended: good balance of quality, speed, and memory
ollama pull qwen2.5vl:3b

# Faster, lighter alternative for batch processing
ollama pull moondream

# Higher quality for important assets (needs ~6 GB RAM)
ollama pull qwen2.5vl:7b
```

**4. Verify the setup:**

```
maki describe --check
# Connected to http://localhost:11434. 2 model(s) available: qwen2.5vl:3b, moondream:latest
```

## Model Recommendations

| Model | Size | RAM | Speed (M3 Pro) | Best For |
|---|---|---|---|---|
| **Moondream 2B** | 1.7 GB | ~2 GB | ~3–5s | Large batch processing |
| **Qwen2.5-VL 3B** | 2.0 GB | ~3 GB | ~8–12s | Daily use (default) |
| **Gemma 3 4B** | 3.3 GB | ~4 GB | ~10–15s | Strong reasoning |
| **Qwen2.5-VL 7B** | 4.7 GB | ~6 GB | ~20–36s | Best quality, key assets |
| **SmolVLM 2.2B** | 1.5 GB | ~2 GB | ~4–8s | Minimal resources |

All models use GPU automatically on macOS (Metal) and Linux (CUDA) when run through Ollama.

## Alternative Local Servers

Any server implementing the OpenAI-compatible `/v1/chat/completions` API works:

- **LM Studio** – GUI-based model manager with built-in server. Default endpoint: `http://localhost:1234`
- **llama.cpp server** – Lightweight C++ inference. Default endpoint: `http://localhost:8080`
- **vLLM** – High-throughput production server. Good for processing large catalogs on a dedicated GPU machine.

```
# Example: LM Studio
[vlm]
endpoint = "http://localhost:1234"
model = "qwen2.5vl-3b"
```

## Using Cloud APIs

Cloud vision APIs that implement the OpenAI chat completions format can also be used. Note that these charge per request.

**OpenAI:**

```
[vlm]
endpoint = "https://api.openai.com"
model = "gpt-4o"
```

Requires `OPENAI_API_KEY` in your environment. Cost: ~$0.01–0.03 per image depending on resolution.

**Groq (free tier available):**

```
[vlm]
endpoint = "https://api.groq.com/openai"
model = "llama-3.2-90b-vision-preview"
```

Requires `GROQ_API_KEY`. Free tier allows ~30 requests/minute.

**Together AI:**

```
[vlm]
endpoint = "https://api.together.xyz"
model = "meta-llama/Llama-3.2-90B-Vision-Instruct-Turbo"
```

Requires `TOGETHER_API_KEY`.

> **Note on authentication:** maki currently sends requests without authentication headers. For cloud APIs that require bearer tokens, you may need a local proxy that adds the header, or use a server like Ollama as a gateway. Native API key support may be added in a future version.

## Generating Descriptions

```
# Preview descriptions without saving (report-only)
maki describe "description:none type:image" --log

# Apply descriptions to undescribed assets on a volume
maki describe --volume "Photos 2024" --apply --log

# Describe a single asset
maki describe --asset a1b2c3d4 --apply

# Overwrite existing descriptions with a better model
maki describe "rating:5" --model qwen2.5vl:7b --force --apply
```

```
# Use a custom prompt for specific content
maki describe --prompt "List the key subjects and artistic techniques." "tag:art" --apply

# Generate tags instead of descriptions
maki describe --mode tags "tag:untagged" --apply

# Generate both descriptions and tags (two VLM calls per asset)
maki describe --mode both --asset a1b2c3d4 --apply

# Dry run to see what would be processed
maki describe "date:2024-06" --dry-run
```

Three modes are available: `--mode describe` (default) generates descriptions, `--mode tags` generates tag suggestions, and `--mode both` runs both — making two separate VLM calls per asset so each uses its optimal prompt. Tags are deduplicated and merged with existing asset tags.

The report-only default (no `--apply`) lets you review generated descriptions and tags before committing them. This is especially useful when tuning prompts or trying different models.

## Configuration

Persistent settings go in `maki.toml`:

```
[vlm]
endpoint = "http://localhost:11434"
model = "qwen2.5vl:3b"
max_tokens = 500
timeout = 300
temperature = 0.7
concurrency = 1  # parallel VLM requests (increase for faster batch processing)
```

To offer multiple models in the web UI (e.g., a fast model for routine work and a larger model for difficult images), add a `models` list:

```
[vlm]
model = "moondream"
models = ["moondream", "qwen3-vl:4b"]
```

A dropdown appears next to the Describe button when two or more models are configured. CLI flags (`--endpoint`, `--model`, `--prompt`, `--max-tokens`, `--timeout`, `--temperature`, `--mode`) override `maki.toml` for one-off runs.

## SigLIP vs VLM: When to Use Which

| | SigLIP Auto-Tag | VLM Describe |
|---|---|---|
| **Output** | Tags from fixed vocabulary | Free-form text, tags, or both |
| **Speed** | ~50–150 ms/image | ~3–36s/image |
| **Best for** | Categorical filtering, similarity | Documentation, open-ended tagging |
| **Requires** | `-features ai`, ONNX models | `curl`, running VLM server |
| **GPU** | CoreML via `--features ai-gpu` | Automatic via Ollama |

The two approaches are complementary. Use SigLIP for fast categorical tagging across large batches, and VLM for rich descriptions when quality matters more than speed.

See the describe reference for all options and the configuration reference for `[vlm]` settings.

---

Next: Organizing Assets – tags, ratings, labels, grouping, collections, and saved searches.

# Organizing Assets

Once your files are imported, maki provides several tools for organizing them: tags, metadata editing, variant grouping, static collections, and saved searches. All changes are persisted in both the SQLite catalog and YAML sidecar files. When `.xmp` recipe files are present, edits to ratings, tags, descriptions, and color labels are written back to disk for bidirectional sync with tools like CaptureOne and Lightroom.

---

## Tags

Tags are free-form keywords attached to an asset. They are deduplicated automatically – tags from XMP sidecar files, embedded XMP in JPEGs, and manual additions are merged into a single set.

### Hierarchical tags

Tags can contain `/` as a hierarchy separator, allowing you to organize keywords into a tree structure:

```
maki tag a1b2c3d4 animals/birds/eagles
maki tag a1b2c3d4 location/europe/germany
```

Parent tag searches match all descendants – searching for `tag:animals` will find assets tagged `animals/birds/eagles`. This works in both CLI and web UI searches.

Hierarchical tags interoperate with Lightroom's `lr:hierarchicalSubject` XMP field. When importing XMP files that contain `lr:hierarchicalSubject` entries, maki reads the hierarchy and stores the full path as a tag. Write-back preserves the hierarchy in the XMP file.

The tags page in the web UI displays hierarchical tags as a collapsible tree (see Web UI for details).

### Adding tags

Pass one or more tag names after the asset ID:

```
maki tag a1b2c3d4 landscape nature golden-hour
```

This adds three tags to the asset. If a tag already exists on the asset, it is silently skipped.

### Removing tags

Use the `--remove` flag:

```
maki tag a1b2c3d4 --remove unwanted blurry
```

Tags that are not present on the asset are silently ignored.

### XMP write-back

When an asset has `.xmp` recipe files on disk, tag changes are written back immediately. Added tags are inserted into the `dc:subject` / `rdf:Bag` block; removed tags are deleted from it. Tags that were added independently in CaptureOne or Lightroom (not managed by maki) are preserved – maki operates on deltas, not full replacement.

If the volume containing the `.xmp` file is offline, the write-back is skipped with a warning.

### Browsing tags in the web UI

The web UI provides a dedicated tags page at `/tags` with:

- **Sortable columns** – click the Name or Count header to sort ascending or descending.
- **Live text filter** – type 2 or more characters to filter the tag list in real time.
- **Multi-column layout** – adapts to the viewport width using CSS columns.

Clicking a tag name navigates to the browse page filtered to that tag.

---

## Editing Metadata

The `maki edit` command sets or clears first-class metadata fields on an asset. At least one flag is required.

### Setting fields

```
maki edit a1b2c3d4 --name "Sunset at Beach"
maki edit a1b2c3d4 --description "A beautiful sunset over the Pacific coast"
maki edit a1b2c3d4 --rating 5
maki edit a1b2c3d4 --label Red
maki edit a1b2c3d4 --date "2024-12-25"
```

Multiple flags can be combined in a single invocation:

```
maki edit a1b2c3d4 --name "Sunset at Beach" --rating 5 --label Orange
```

### Clearing fields

Each field has a corresponding `--clear-*` flag:

```
maki edit a1b2c3d4 --clear-name
maki edit a1b2c3d4 --clear-description
maki edit a1b2c3d4 --clear-rating
maki edit a1b2c3d4 --clear-label
maki edit a1b2c3d4 --clear-date
```

An empty description string is normalized to a clear operation – these two commands are equivalent:

```
maki edit a1b2c3d4 --description ""
maki edit a1b2c3d4 --clear-description
```

### Color labels

The label field accepts one of seven colors: **Red**, **Orange**, **Yellow**, **Green**, **Blue**, **Pink**, **Purple**. Input is case-insensitive and stored in canonical title-case:

```
maki edit a1b2c3d4 --label red      # stored as "Red"
maki edit a1b2c3d4 --label YELLOW   # stored as "Yellow"
```

This is a superset of Lightroom's 5-color palette, matching CaptureOne's 7 colors.

### XMP write-back for metadata

Changes to **rating**, **description**, and **color label** are automatically written back to any `.xmp` recipe files attached to the asset. This enables round-trip editing with external tools:

1. Rate an image in maki (`maki edit ... --rating 4`).
2. Open CaptureOne – the rating is already reflected in the XMP sidecar.
3. Change the rating in CaptureOne.
4. Run `maki refresh` to pick up the change.

### Structured output

Use the global `--json` flag for machine-readable output:

```
maki --json edit a1b2c3d4 --rating 5
```

---

## Manual Grouping

The `maki group` command merges multiple variants (identified by their content hashes) into a single asset. This is useful when related files were imported separately and ended up as distinct assets.

### Basic usage

```
maki group abc123def456 789fed321cba
```

You can pass any number of content hashes. maki identifies the assets that own those variants and merges them.

### Merge rules

- The **oldest asset** (by `created_at`) is chosen as the target.
- Donor variants that had the **Original** role are re-roled to **Alternate** (in both the YAML sidecar and the SQLite catalog) to avoid multiple originals on one asset.
- **Tags** from donor assets are merged into the target (union, deduplicated).
- **Recipes** from donor assets are moved to the target.
- Donor assets are **deleted** after merging.

### Finding content hashes

Use `maki show` to display an asset's variants and their content hashes:

```
maki show a1b2c3d4
```

Or use `maki search --format '{id}\t{name}'` to locate the assets first, then inspect them with `maki show`.

---

## Auto-Grouping

The `maki auto-group` command finds assets that belong together by matching their filename stems across the catalog. This handles the common case where a RAW original and its processed exports end up as separate assets – for example, when CaptureOne exports land in a different directory from the originals.

## Report mode (dry run)

Without `--apply`, auto-group runs in report-only mode. Nothing is modified:

```
maki auto-group
```

Output shows which groups would be formed, how many donors would be merged, and how many variants would be moved.

## Applying changes

```
maki auto-group --apply
```

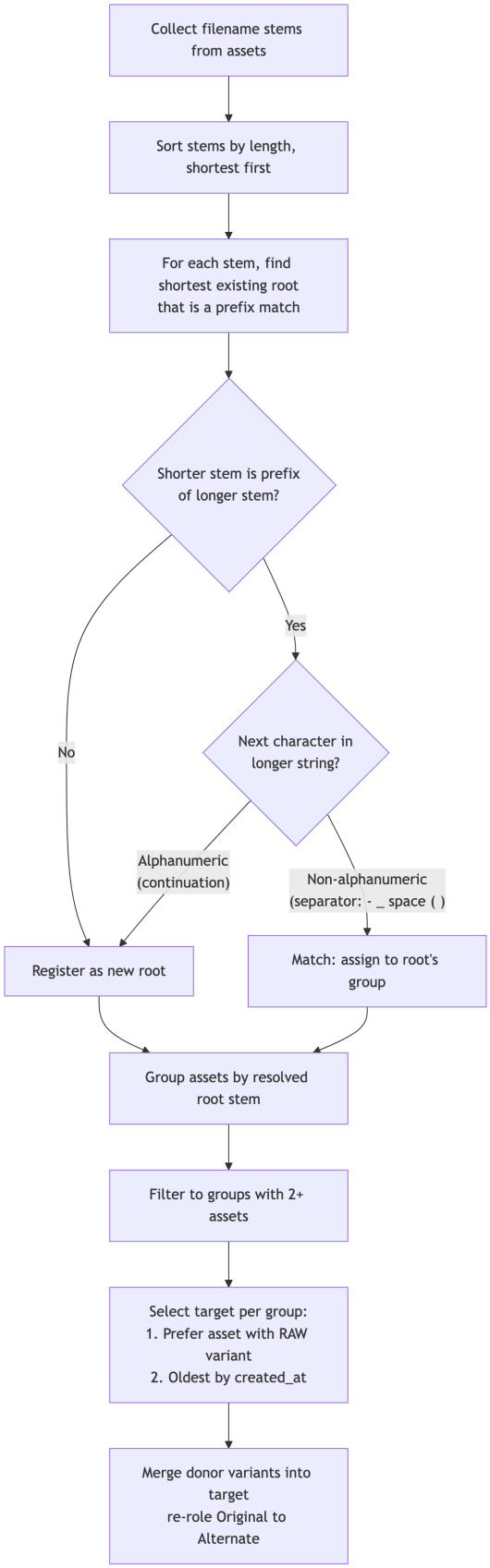
## Scoping to a search query

Pass a search query to limit the scope:

```
maki auto-group "tag:landscape" --apply
maki auto-group "path:Capture/2024" --apply
```

## The matching algorithm

Auto-group uses fuzzy prefix matching with separator-boundary detection:


Collect filename stems from assets
Sort stems by length, shortest first
For each stem, find shortest existing root that is a prefix match
Shorter stem is prefix of longer stem?
No
Yes
Next character in longer string?
Alphanumeric (continuation)
Non-alphanumeric (separator: - _ space ( )
Register as new root
Match: assign to root's group
Group assets by resolved root stem
Filter to groups with 2+ assets
Select target per group:
1. Prefer asset with RAW variant
2. Oldest by created_at
Merge donor variants into target
re-role Original to Alternate

**Matching examples:**

| Shorter stem | Longer stem | Next char | Result |
|---|---|---|---|
| `Z91_8561` | `Z91_8561-1-HighRes` | `-` (hyphen) | Match |
| `Z91_8561` | `Z91_8561_edit` | `_` (underscore) | Match |
| `DSC_001` | `DSC_0010` | `0` (digit) | No match |
| `IMG_2000` | `IMG_2000 (1)` | (space) | Match |

**Chain resolution:** When multiple stems form a chain, they all resolve to the shortest root. For example, `Z91_8561`, `Z91_8561-1`, and `Z91_8561-1-HighRes` all group under `Z91_8561`.

**Target selection:** Within each group, the target asset is the one with a RAW variant (if any), breaking ties by oldest `created_at` timestamp. Donor variants with the Original role are re-roled to Alternate, just like manual grouping.

### Output options

```
maki auto-group --json            # structured JSON result
maki auto-group --log --apply     # per-group progress on stderr
```

### Web UI

In the web UI, select multiple assets on the browse page, then click the **"Group by name"** button in the batch toolbar. A confirmation dialog shows how many groups would be formed before applying.

---

## Splitting Assets

The inverse of grouping: extract one or more variants from an asset into new standalone assets.

```
# View variants of an asset
maki show abc12345

# Extract the JPEG variant into its own asset
maki split abc12345 sha256:def456...
```

Each extracted variant becomes a new asset with: - Role set to `original` - Tags, rating, color label, and description copied from the source - Associated recipes moved with the variant

In the web UI, the asset detail page shows checkboxes next to each variant. Select the variants to extract and click "Extract as new asset(s)".

**Constraints:** At least one variant must remain in the source asset.

---

## Collections (Static Albums)

Collections are manually curated lists of assets – the equivalent of albums in a traditional photo manager. They are backed by both SQLite (for fast queries) and a `collections.yaml` file at the catalog root (which survives `rebuild-catalog`).

The alias `maki col` can be used in place of `maki collection` for brevity.

### Creating a collection

```
maki collection create "Best of 2024" --description "Top picks from the year"
```

### Listing collections

```
maki collection list
```

### Viewing collection contents

```
maki collection show "Best of 2024"
maki collection show "Best of 2024" --format full
```

### Adding assets

```
maki collection add "Best of 2024" a1b2c3d4 e5f6a7b8
```

You can pipe asset IDs from a search:

```
maki search -q "rating:5 tag:landscape" | xargs maki col add "Best of 2024"
```

This is a powerful pattern – combine any search filter to build a collection in a single command.

### Removing assets

```
maki collection remove "Best of 2024" a1b2c3d4
```

### Deleting a collection

```
maki collection delete "Best of 2024"
```

This removes the collection definition and all membership records. The assets themselves are not affected.

### Searching within a collection

Use the `collection:` filter in any search:

```
maki search "collection:\"Best of 2024\" rating:4+"
```

Note the quoted value for collection names with spaces.

### Web UI

- The **`/collections`** page lists all collections with a **"+ New Collection"** button.
- On the **asset detail page**, collection membership is shown as chips with a remove button.
- In the **batch toolbar**, a collection dropdown lets you add or remove selected assets from a collection. The dropdown includes a **"New..."** option that creates a collection inline.

---

## Stacks

Stacks are lightweight anonymous groups of assets – burst shots, exposure brackets, or similar scenes that you want to collapse into a single entry in the browse grid. Unlike grouping (which permanently merges variants into one asset), stacks keep each asset independent. You can dissolve a stack at any time to separate the assets again.

Each asset can belong to at most one stack. Members are position-ordered; position 0 is the **pick** – the representative image shown when the stack is collapsed in the browse grid. Stacks auto-dissolve when they have one or fewer members.

The alias `maki st` can be used in place of `maki stack` for brevity.

## Creating a stack

Pass two or more asset IDs to create a new stack:

```
maki stack create a1b2c3d4 e5f6a7b8 c9d0e1f2
```

The first asset becomes the pick (position 0). The remaining assets are ordered by their position in the argument list.

## Adding assets to a stack

```
maki stack add <stack-id> f3a4b5c6
```

The new asset is appended to the end of the stack.

## Setting the pick

Promote any member to the pick position:

```
maki stack pick <stack-id> e5f6a7b8
```

The previous pick moves to position 1; all other positions shift accordingly.

## Removing assets from a stack

```
maki stack remove <stack-id> c9d0e1f2
```

If the stack has one or fewer members after removal, it auto-dissolves.

## Dissolving a stack

Dissolve a stack entirely, returning all members to standalone assets:

```
maki stack dissolve <stack-id>
```

## Listing stacks

```
maki stack list
```

Shows all stacks with their member counts.

## Showing stack details

```
maki stack show <stack-id>
```

Displays the stack's members in position order, with the pick marked.

## Searching for stacked assets

Use the `stacked:` filter in any search:

```
maki search "stacked:true"      # assets that are in a stack
maki search "stacked:false"     # standalone assets
```

### Structured output

All stack commands support `--json`:

```
maki --json stack list
maki --json stack show <stack-id>
```

### Web UI

See Web UI for the stack experience in the browse grid, batch operations, and asset detail page.

---

## Saved Searches (Smart Albums)

Saved searches are named queries stored in `searches.toml` at the catalog root. Unlike collections, they are dynamic – running a saved search always reflects the current state of the catalog.

The alias `maki ss` can be used in place of `maki saved-search`.

### Saving a search

```
maki saved-search save "Landscapes" "tag:landscape rating:3+" --sort date_desc
```

This stores the query string and sort order under the name "Landscapes".

### Listing saved searches

```
maki saved-search list
```

### Running a saved search

```
maki saved-search run "Landscapes"
maki saved-search run "Landscapes" --format full
maki saved-search run "Landscapes" --format json
```

### Deleting a saved search

```
maki saved-search delete "Landscapes"
```

### Practical examples

Save commonly used filters for quick access:

```
maki ss save "Unrated"     "rating:0"
maki ss save "Five Stars"  "rating:5"
maki ss save "Red Label"   "label:Red"
maki ss save "Recent"      "path:Capture/2024" --sort date_desc
maki ss save "Orphans"     "orphan:true"
maki ss save "Theater"     "tag:\"Fools Theater\""
```

Combine saved searches with other commands:

```
maki ss run "Five Stars" --format ids | xargs maki col add "Portfolio"
```

### Web UI

- **Clickable chips** on the browse page load saved searches into the filter UI with a single click.
- The **Save button** captures the current search state (query, filters, sort order) as a new saved search.
- **Hover** over a chip to reveal rename and delete buttons.

---

## People Management (Face Recognition)

> Requires `cargo build --features ai`.

maki can detect faces in your images, group them into people, and let you search by person. This is an AI-powered feature that uses YuNet for face detection and ArcFace for face recognition.

### Setup

Download the required models first:

```
maki faces download
```

This downloads the YuNet face detection model (~230 KB) and ArcFace recognition model (~28 MB) to the model cache directory.

### Detecting faces

Run face detection on your catalog:

```
maki faces detect --query "type:image" --apply
```

This finds faces in each image's preview, stores bounding boxes and embeddings, and generates face crop thumbnails. Use `--log` for per-asset progress.

You can scope detection to a specific shoot or volume:

```
maki faces detect --query "path:Capture/2026-03" --apply
maki faces detect --volume "Photos" --apply
maki faces detect --asset a1b2c3d4 --apply
```

### Clustering faces into people

After detection, cluster similar faces into groups:

```
maki faces cluster --apply
```

This uses greedy single-linkage clustering to group faces that look similar enough (default threshold 0.5). Each group becomes an unnamed person.

Adjust the threshold for stricter or looser grouping:

```
maki faces cluster --threshold 0.6 --apply    # stricter (fewer false matches)
maki faces cluster --threshold 0.4 --apply    # looser (more grouped together)
```

### Naming people

List the discovered people and name them:

```
maki faces people
maki faces name 550e8400-... "Alice"
maki faces name 661f9511-... "Bob"
```

### Managing people

Merge duplicate person groups:

```
maki faces merge 550e8400-... 661f9511-...    # merge source into target
```

Remove a face from a person (misidentification):

```
maki faces unassign face-uuid-...
```

Delete a person entirely (faces become unassigned):

```
maki faces delete-person 550e8400-...
```

### Searching by face and person

Use the `faces:` and `person:` search filters:

```
maki search "faces:any"                # assets with detected faces
maki search "faces:none type:image"    # images without faces
maki search "faces:2+"                # group photos (2+ faces)
maki search "person:Alice"            # assets containing Alice
maki search "person:Alice rating:4+"  # Alice's best photos
```

### Web UI

The web UI provides a complete face management experience:

- **People page** (`/people`) — gallery grid of person cards with thumbnails, names, and face counts. Inline rename, merge, and delete. "Cluster" button for on-demand clustering.
- **Asset detail page** — faces section shows detected faces as chips with crop thumbnails and confidence scores. "Detect faces" button for on-demand detection. Assign/unassign faces to people via dropdown.
- **Browse filters** — person dropdown in the filter row, and `faces:` filter in the query input.
- **Batch toolbar** — "Detect faces" button processes all selected assets.
- **Browse cards** — face count badge alongside variant count.

### Configuration

In `maki.toml`:

```toml
[ai]
face_cluster_threshold = 0.5    # clustering similarity threshold
face_min_confidence = 0.5       # minimum detection confidence
```

---

## Putting It All Together

A typical organizing workflow after import might look like this:

1. **Auto-group** to merge related variants:

   ```
   maki auto-group --apply --log
   ```

2. **Tag** a batch of assets from the CLI:

   ```
   maki search -q "path:Capture/2024-08-15" | \
     xargs -I{} maki tag {} vacation beach
   ```

3. **Rate and label** standouts in the web UI using keyboard shortcuts (1-5 for ratings, r/o/y/g/b/p/u for labels).

4. **Build a collection** from your top picks:

   ```
   maki search -q "rating:5 tag:vacation" | xargs maki col add "Summer 2024"
   ```

5. **Save a smart search** for ongoing curation:

   ```
   maki ss save "Vacation Selects" "tag:vacation rating:4+" --sort date_desc
   ```

---

Next: Browsing & Searching | Previous: Ingesting Assets | Back to Manual

# Browsing & Searching

This chapter covers how to find, inspect, and analyze assets in your catalog using the CLI. For the web-based browser interface, see Web UI.

---

## Searching Assets

The `maki search` command is the primary way to find assets. It accepts free-text keywords and structured filters in a single query string.

**Free-text search** matches against asset names, filenames, descriptions, and source metadata:

```
maki search "sunset"
maki search "beach vacation"
```

**Structured filters** use a `key:value` syntax and can be combined freely with free-text:

```
maki search "tag:landscape type:image rating:3+"
maki search "sunset tag:landscape camera:fuji iso:100-800"
maki search "format:nef focal:35-70 f:1.4-2.8"
```

Tokens that don't match a known filter prefix are treated as free-text, so you can mix them in any order:

```
maki search "type:image sunset rating:4+ golden hour"
```

**Values with spaces** are supported using double quotes inside the query:

```
maki search 'tag:"Fools Theater"'
maki search 'camera:"Canon EOS R5" lens:"RF 50mm f/1.2"'
maki search 'collection:"My Favorites"'
maki search 'path:"Photos/Family Trip"'
```

Note the outer single quotes to prevent your shell from stripping the inner double quotes.

**Negation** excludes matching assets by prefixing any filter or free-text term with `-`:

```
maki search "-tag:rejected"              # exclude rejected assets
maki search "landscape -tag:processed"   # landscapes not yet processed
maki search "-format:xmp -type:other"    # exclude XMP files and "other" types
maki search "-sunset"                    # exclude free-text match on "sunset"
```

**OR within a filter** matches any of several values using commas:

```
maki search "tag:alice,bob"              # tagged alice OR bob (or both)
maki search "format:nef,cr3"             # NEF or CR3 format
maki search "type:image,video"           # images or videos
maki search "label:Red,Orange"           # Red or Orange labeled
```

**Repeated filters = AND**. To require multiple tags, repeat the filter:

```
maki search "tag:landscape tag:sunset"    # BOTH landscape AND sunset tags
```

---

## Search Filters Quick Reference

All filters can be combined in a single query. Remaining tokens become free-text search terms.

| Filter | Syntax | Example |
|---|---|---|
| Asset type | `type:<type>` | `type:image`, `type:video` |
| Tag | `tag:<name>` | `tag:landscape`, `tag:"Fools Theater"` |
| Format | `format:<ext>` | `format:nef`, `format:jpg` |
| Rating (exact) | `rating:<N>` | `rating:5` |
| Rating (minimum) | `rating:<N>+` | `rating:3+` |
| Rating (range) | `rating:<N>-<M>` | `rating:3-5` |
| Rating (OR) | `rating:<N>,<M>` | `rating:2,4`, `rating:2,4+` |
| Color label | `label:<color>` | `label:Red`, `label:Blue` |
| Camera | `camera:<text>` | `camera:fuji`, `camera:"Canon EOS R5"` |
| Lens | `lens:<text>` | `lens:56mm`, `lens:"RF 50mm f/1.2"` |
| ISO (exact or range) | `iso:<N>` or `iso:<min>-<max>` | `iso:100`, `iso:100-800` |
| Focal length | `focal:<N>` or `focal:<min>-<max>` | `focal:50`, `focal:35-70` |
| Aperture | `f:<N>` or `f:<min>-<max>` | `f:2.8`, `f:1.4-2.8` |
| Width (minimum) | `width:<N>+` | `width:4000+` |
| Height (minimum) | `height:<N>+` | `height:2000+` |
| Source metadata | `meta:<key>=<value>` | `meta:software=CaptureOne` |
| Path prefix | `path:<prefix>` | `path:Capture/2026-02-22` |
| Collection | `collection:<name>` | `collection:Favorites` |
| Date (prefix match) | `date:<prefix>` | `date:2026-02-25`, `date:2026-02`, `date:2026` |
| Date (from) | `dateFrom:<date>` | `dateFrom:2026-01-01` |
| Date (until) | `dateUntil:<date>` | `dateUntil:2026-12-31` |
| Volume | `volume:<label>`, `volume:none` | `volume:Archive`, `volume:none` |
| Asset ID | `id:<prefix>` | `id:72a0bb4b` |
| Copies | `copies:<N>`, `copies:<N>+` | `copies:1`, `copies:2+` |
| Variants | `variants:<N>`, `variants:<N>+` | `variants:2+` |
| Scattered | `scattered:<N>+` | `scattered:2+` |
| GPS | `geo:<south>,<west>,<north>,<east>` | `geo:47.5,11.0,48.5,13.0` |
| Orphan assets | `orphan:true` | `orphan:true` |
| Missing files | `missing:true` | `missing:true` |
| Stale verification | `stale:<days>` | `stale:30` |
| Stacked | `stacked:true` or `stacked:false` | `stacked:true` |
| Face count | `faces:any`, `faces:none`, `faces:N`, `faces:N+` | `faces:2+` |
| Person | `person:<name>` | `person:Alice`, `person:"John Smith"` |
| Visual similarity (ai) | `similar:<id>` or `similar:<id>:<limit>` | `similar:72a0bb4b`, `similar:72a0bb4b:50` |
| Similarity threshold (ai) | `min_sim:<percent>` | `min_sim:90` |
| Text search (ai) | `text:<query>` | `text:sunset beach` |
| Embedding status (ai) | `embed:any`, `embed:none` | `embed:none type:image` |

**Hierarchical tag matching**: The `tag:` filter matches hierarchically. Searching for `tag:animals` finds assets tagged `animals`, `animals/birds`, `animals/birds/eagles`, and any other descendant of `animals`. To match only the exact tag, use the full path (e.g., `tag:animals/birds/eagles`).

For the complete filter reference with detailed syntax and behavior, see Search Filters.

---

# Path Prefix Filter

The `path:` filter restricts results to assets whose file locations start with a given prefix. This is useful for finding everything imported from a particular directory:

```
maki search "path:Capture/2026-02-22"
maki search "path:Events/Wedding"
```

**Automatic path normalization** in the CLI handles several convenience cases:

- `~` expands to your home directory
- `./` and `../` resolve relative to your current working directory
- Absolute paths that match a registered volume's mount point are automatically stripped to volume-relative, and the corresponding volume filter is applied

For example, if you have a volume mounted at `/Volumes/Photos`:

```
maki search "path:/Volumes/Photos/Capture/2026"
# Equivalent to: maki search "path:Capture/2026" on the Photos volume
```

An explicit `volume:` filter in the same query takes precedence over the auto-detected volume.

---

# Output Formats

Control how search results are displayed with `--format` or `-q`.

## Preset Formats

**Default (short)** – one line per result with a result count:

```
$ maki search "sunset"
a1b2c3d4  IMG_1234.jpg [image] (JPEG) — 2026-01-15T10:30:00
e5f6a7b8  DSC_5678.nef [image] (NEF) — 2026-01-14T16:45:00

2 result(s)
```

**IDs only** (`--format ids` or `-q`) – one UUID per line, ideal for scripting:

```
$ maki search -q "tag:landscape"
a1b2c3d4-5678-9abc-def0-123456789abc
e5f6a7b8-1234-5678-9abc-def012345678
```

**Full** (`--format full`) – includes tags and description:

```
$ maki search "sunset" --format full
a1b2c3d4  IMG_1234.jpg [image] (JPEG) — 2026-01-15T10:30:00 tags:sunset,landscape A golden sunset over the mountains
```

**JSON** (`--format json` or `--json`) – structured JSON array:

```
$ maki search "sunset" --json
[
  {
    "asset_id": "a1b2c3d4-5678-9abc-def0-123456789abc",
    "original_filename": "IMG_1234.jpg",
```

```
    "asset_type": "image",
    ...
  }
]
```

### Custom Templates

Build your own output format using `{placeholder}` syntax:

```
maki search "sunset" --format '{id}\t{name}\t{tags}'
maki search "type:image" --format '{short_id} {filename} [{format}] {label}'
maki search "rating:5" --format '{name}\n  {description}\n'
```

**Available placeholders:**

| Placeholder | Content |
|---|---|
| `{id}` | Full asset UUID |
| `{short_id}` | First 8 characters of the UUID |
| `{name}` | Asset name (falls back to filename) |
| `{filename}` | Original filename |
| `{type}` | Asset type (image, video, audio, etc.) |
| `{format}` | File format (JPEG, NEF, MOV, etc.) |
| `{date}` | Creation date |
| `{tags}` | Comma-separated tags |
| `{description}` | Asset description |
| `{hash}` | Content hash of the best variant |
| `{label}` | Color label (Red, Blue, etc.) |

**Escape sequences:** `\t` (tab), `\n` (newline), `\\` (literal backslash).

When `--format` is explicitly provided, the result count line at the end is suppressed, keeping the output clean for piping and parsing.

For the complete format reference, see Format Templates.

---

## Sort Options

The web UI provides inline sort toggle buttons (Name, Date, Size) with ascending/descending direction indicators. In the CLI, sort order is available when saving searches:

```
maki saved-search save "landscapes" "tag:landscape" --sort name_asc
```

Available sort values:

| Sort | Description |
|---|---|
| `date_desc` | Newest first (default) |
| `date_asc` | Oldest first |
| `name_asc` | Alphabetical A-Z (by name, falling back to filename) |
| `name_desc` | Alphabetical Z-A |
| `size_asc` | Smallest first |
| `size_desc` | Largest first |

The default sort for `maki search` is `date_desc` (newest first).

---

## Scripting with maki

The `-q` flag and format options make `maki` composable with standard Unix tools.

**Loop over search results:**

```
for id in $(maki search -q "tag:landscape"); do
    maki show "$id"
done
```

**Pipe results into collection commands:**

```
maki search -q "rating:5" | xargs maki col add "Best"
```

**Extract specific fields with jq:**

```
maki search "sunset" --json | jq '.[].asset_id'
maki search "type:video" --json | jq '.[] | {name: .original_filename, date: .created_at}'
```

**Count results:**

```
maki search -q "format:nef" | wc -l
```

**Tab-separated output for spreadsheets:**

```
maki search "type:image" --format '{name}\t{format}\t{date}\t{tags}'
```

**Combine with other maki commands:**

```
# Tag all 5-star images as "portfolio"
maki search -q "rating:5 type:image" | xargs -I{} maki tag {} portfolio

# Generate previews for a specific path
maki search -q "path:Capture/2026-02" | xargs -I{} maki generate-previews --asset {}
```

---

## Showing Asset Details

The `maki show` command displays comprehensive information about a single asset.

```
maki show a1b2c3d4
```

Asset ID prefix matching is supported – you only need to type enough characters to uniquely identify the asset:

```
maki show a1b2     # Works if "a1b2" is a unique prefix
```

### Output

The human-readable output includes:

```
Asset: a1b2c3d4-5678-9abc-def0-123456789abc
Name:  Golden Sunset
Type:  image
Date:  2026-01-15T10:30:00
Tags:  sunset, landscape, mountains
Rating: ★★★★★ (5/5)
Label: Red
```

```
Description: A golden sunset over the mountain range
Preview: /path/to/catalog/previews/a1/a1b2c3d4...jpg

Variants:
  [original] DSC_1234.NEF (NEF, 48.2 MB)
    Hash: abc123def456...
    Location: Photos → Capture/2026-01-15/DSC_1234.NEF
    camera_make: NIKON CORPORATION
    camera_model: NIKON Z 9
    focal_length: 70.0
    f_number: 8.0
    iso: 200
  [export] DSC_1234-Edit.tif (TIFF, 120.5 MB)
    Hash: def789abc012...
    Location: Photos → Export/2026-01-15/DSC_1234-Edit.tif

Recipes:
  [xmp] CaptureOne (e4f5a6b7c8...)
    Path: Capture/2026-01-15/DSC_1234.NEF.xmp
```

For structured output, use the `--json` flag:

```
maki show a1b2c3d4 --json
```

This returns a full `AssetDetails` JSON object with all variants, locations, recipes, and source metadata.

---

# Finding Duplicates

The `maki duplicates` command identifies files with identical content hashes that exist in multiple locations.

```
$ maki duplicates
DSC_1234.NEF (NEF, 48.2 MB)
  Hash: abc123def456...
    Photos → Capture/2026-01-15/DSC_1234.NEF
    Backup → Capture/2026-01-15/DSC_1234.NEF

1 file(s) with duplicate locations
```

This is useful for identifying files that have been copied to multiple volumes (e.g., when migrating data or creating backups).

## Duplicate Output Formats

The `--format` flag supports the same presets as search, plus an additional `{locations}` placeholder for templates:

```
maki duplicates --format ids
maki duplicates --format json
maki duplicates --format '{filename}\t{format}\t{locations}'
```

The `{locations}` placeholder expands to a comma-separated list of all locations where the duplicate file exists.

---

# Catalog Statistics

The `maki stats` command provides an overview of your catalog's contents and health.

## Overview (Default)

```
$ maki stats
Catalog Overview
  Assets:    12,847
  Variants:  18,203
  Recipes:   9,614
  Volumes:   3 (2 online, 1 offline)
  Total size: 1.8 TB
```

## Asset Types and Formats

```
$ maki stats --types
Catalog Overview
  ...

Asset Types
  image         11,204  (87.2%)
  video          1,412  (11.0%)
  audio            231  (1.8%)

Variant Formats
  NEF            8,100
  JPEG           6,543
  TIFF           2,104
  MOV            1,412

Recipe Formats
  xmp            7,200
  cos            2,414
```

## Per-Volume Details

```
$ maki stats --volumes
Catalog Overview
  ...

Volumes
  Photos [online]
    Assets: 10,234  Variants: 14,567  Recipes: 7,890
    Size: 1.2 TB  Directories: 342
    Formats: NEF, JPEG, TIFF, MOV
    Verified: 14,200/14,567 (97.5%)
    Oldest verification: 2026-01-01T08:00:00
  Backup [offline]
    Assets: 8,412  Variants: 12,100  Recipes: 5,200
    Size: 980.4 GB  Directories: 298
    Formats: NEF, JPEG, TIFF
    Verified: 12,100/12,100 (100.0%)
    Oldest verification: 2025-12-15T14:30:00
```

### Tag Statistics

```
$ maki stats --tags
Tags
  Unique tags:     145
  Tagged assets:   9,876
  Untagged assets: 2,971

  Top Tags
    landscape               1,234
    portrait                  987
    street                    654
    ...
```

### Verification Health

```
$ maki stats --verified
Verification
  Total locations:    26,667
  Verified:           26,300
  Unverified:            367
  Coverage:           98.6%
  Oldest verified:    2025-12-15T14:30:00
  Newest verified:    2026-02-22T19:00:00

  Per Volume
    Photos [online]: 14,200/14,567 (97.5%)
    Backup [offline]: 12,100/12,100 (100.0%)
```

### Combined and Tuned Output

Use `--all` to show every section at once:

```
maki stats --all
```

Control the number of entries in top-N lists (default is 20):

```
maki stats --tags --limit 50
```

For structured output:

```
maki stats --all --json
```

---

## Exporting Files

The `maki export` command copies files matching a search query to a target directory — useful for client deliveries, sharing, or copying to external media.

### Basic Export

Export the best variant of each matching asset:

```
maki export "rating:5 tag:portfolio" /tmp/delivery/
```

### Layout Modes

**Flat** (default) — all files in one directory:

```
maki export "collection:Selects" /tmp/flat/
```

Filename collisions from different assets are resolved by appending a hash suffix (e.g., `DSC_001_a1b2c3d4.jpg`).

**Mirror** — preserves source directory structure:

```
maki export "tag:landscape" /Volumes/USB/export --layout mirror
```

When assets span multiple volumes, each volume's files are placed under a `<volume-label>/` prefix.

### Options

Export all variants (not just the best):

```
maki export "tag:portfolio" /tmp/all/ --all-variants
```

Include sidecars (`.xmp`, `.cos`, etc.):

```
maki export "collection:Print" /tmp/handoff/ --include-sidecars
```

Create symlinks instead of copies:

```
maki export "type:image" ~/links/ --symlink
```

Preview without writing files:

```
maki export "rating:4+" /tmp/test/ --dry-run
```

Re-running an export skips files that already exist with matching content. Use `--overwrite` to force re-copy.

For the full command reference, see export.

---

## Visual Similarity

With the `ai` feature and image embeddings generated (via `maki embed` or `maki import --embed`), you can find visually similar assets:

```
maki search "similar:72a0bb4b"              # top 20 similar assets
maki search "similar:72a0bb4b:50"           # top 50 similar
maki search "similar:72a0bb4b rating:4+"    # similar AND 4+ stars
```

The `similar:` filter uses SigLIP image embeddings for fast dot-product similarity search. It composes with all other filters.

For an interactive visual exploration experience, the web UI offers a **Stroll page** (`/stroll`) where you can navigate through your collection by clicking visually similar neighbors arranged around a center image. A fan-out slider (0–10) controls L2 transitive neighbor discovery: when fan-out is greater than 0, hovering or arrow-keying to a satellite reveals its own nearest neighbors fanning out behind it – useful for exploring broader visual themes. See Web UI – Stroll Page for details.

---

## Related Topics

- Organizing Assets – tags, editing, collections, and saved searches
- Web UI – browser-based search with interactive filters, sort controls, and batch operations
- Search Filters Reference – complete filter syntax documentation

- Format Templates Reference – all placeholders and template syntax
- CLI Conventions – global flags (`--json`, `--log`, `--time`)

# Web UI

maki includes a browser-based interface for browsing, searching, and editing your catalog. It runs as a local web server and provides a responsive grid view of your assets, inline metadata editing, batch operations, and keyboard-driven navigation. All changes made in the web UI are written to both the SQLite catalog and YAML sidecar files, and any editable fields (rating, tags, description, color label) are automatically synced back to XMP recipe files on disk.

## Starting the Web UI

Launch the server with `maki serve`:

```
maki serve
```

Output:

```
Listening on http://127.0.0.1:8080
```

Open that URL in your browser to start browsing. The server runs in the foreground; press Ctrl+C to stop it.

### Custom port and bind address

Use `--port` and `--bind` to change the listening address:

```
maki serve --port 9090 --bind 0.0.0.0
```

Binding to `0.0.0.0` makes the UI accessible from other devices on your local network (use with caution on untrusted networks).

### Request logging

Add the `--log` flag to print each HTTP request to stderr:

```
maki serve --log
```

Output on stderr:

```
GET / -> 200 (12ms)
GET /static/style.css -> 200 (1ms)
GET /previews/ab/ab3f...jpg -> 200 (2ms)
```

This is useful for debugging slow requests or understanding access patterns.

### Configuration via maki.toml

You can set default values for port and bind address in the `[serve]` section of `maki.toml`:

```toml
[serve]
port = 9090
bind = "127.0.0.1"
```

Command-line flags always override `maki.toml` settings. See the Configuration Reference for details.

# Dark Mode

The web UI supports dark mode. A toggle button (sun/moon icon) in the navigation bar switches between light and dark themes. Your preference is saved in the browser and applied instantly on every page load.

If you haven't chosen a theme explicitly, the UI follows your operating system preference automatically (via `prefers-color-scheme`). Dark mode covers all pages: browse, asset detail, tags, collections, stats, and backup status.

# Browse Page

The browse page is the main entry point for the web UI. It shows a searchable, filterable grid of asset thumbnails.

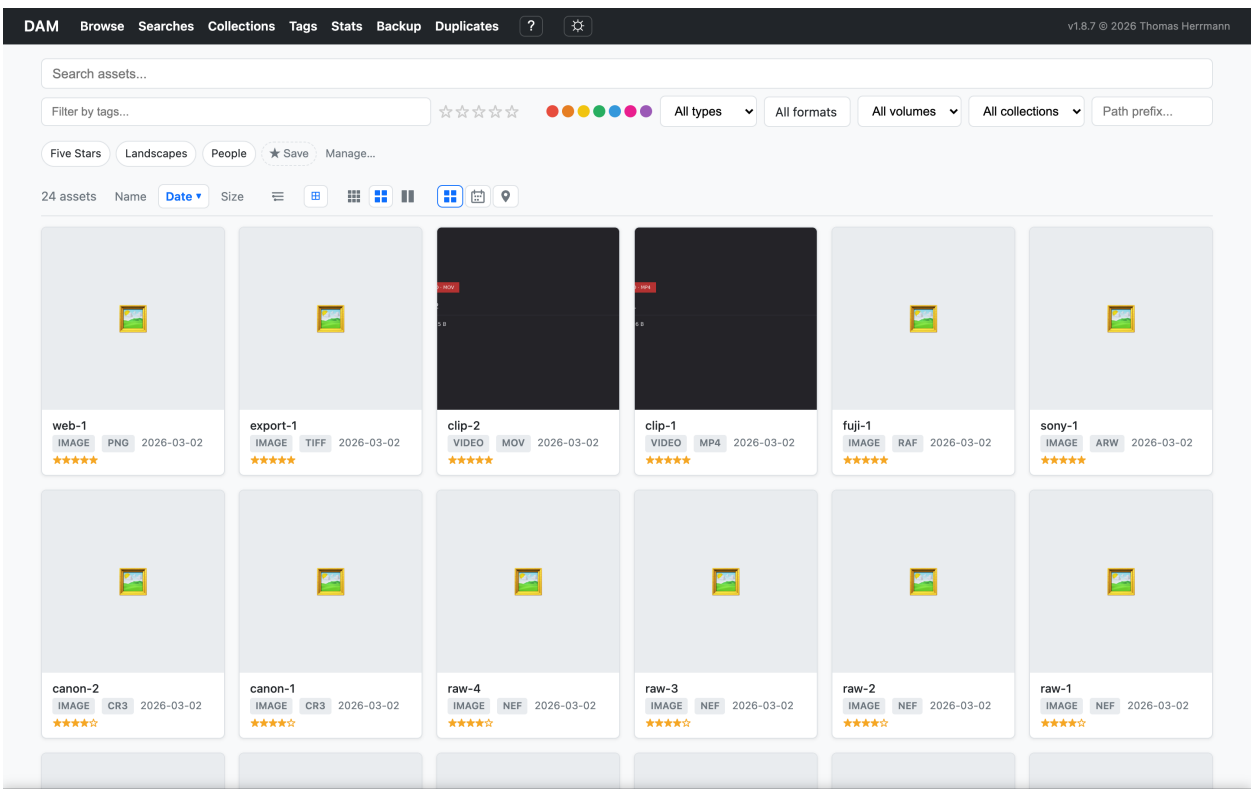



Figure 1: Browse page with search bar, filter controls, and results grid

## Search bar

The search bar has two rows:

**Row 1** – a full-width text input for free-text search. Type any keyword, filename, or structured filter (like `camera:"Canon EOS R5"`) and results update as you type with a 300ms debounce. Press Enter to search

immediately without waiting for the debounce.

The filter bar is collapsible. Press **Shift+F** to toggle its visibility on the browse page. Your preference is saved in the browser. This is useful for maximizing screen space when you want to focus on the grid without changing filters.

**Row 2** – a row of filter controls, left to right:

- **Tag filter**: a chip-based input with autocomplete. Type to see tag suggestions, click or press Enter to add a tag chip. Multiple tags narrow the results (AND logic). Remove a tag by clicking the x on its chip, or press Backspace in an empty input to remove the last chip. Adding or removing a tag triggers an immediate search.
- **Star rating filter**: five clickable stars with a three-state cycle. First click sets an exact match (e.g. "3"), second click switches to minimum match ("3+"), third click clears the filter. Star 5 is two-state (exact and clear, since 5 and 5+ are identical). Triggers an immediate search.
- **Color label filter**: seven colored dots (Red, Orange, Yellow, Green, Blue, Pink, Purple). Click a dot to filter by that label. Click the active dot again to clear. Triggers an immediate search.
- **Type dropdown**: filter by asset type (Image, Video, Audio, Document). Triggers immediately on change.
- **Format filter**: a grouped multi-select dropdown. Click the trigger button to open a panel showing all formats in your catalog organized by category (RAW, Image, Video, Audio, Other). Each format shows its variant count. Check individual formats or use the group-level toggles ("All RAW", "All Image", etc.) to select entire categories. Multiple selections combine with OR logic (e.g., selecting NEF and CR3 shows assets in either format). The trigger button shows: "All formats" when nothing is selected, the format name for a single selection, the group name when a full group is selected (e.g., "All RAW"), or a compact summary like "nef +3…" for mixed selections. Hover the button to see the full list in a tooltip. Triggers an immediate search on every checkbox change.
- **Volume dropdown**: filter by storage volume. Only shown when you have registered volumes. Triggers immediately on change.
- **Collection dropdown**: filter by collection membership. Only shown when you have collections. Triggers immediately on change.
- **Path prefix input**: filter by file location path prefix. Type a directory path to see only assets stored under that path. Debounces at 300ms, Enter fires immediately.

All filters compose with each other. You can combine a text search with a tag, a minimum rating, a color label, and a volume restriction at the same time.

## Results grid

Below the search bar, results appear as a grid of thumbnail cards. Each card shows:

- A preview thumbnail (lazy-loaded for performance)
- The asset name (or filename as fallback)
- Badges for type and primary format (e.g., "image" and "NEF")
- A variant count badge when the asset has multiple variants (e.g., "3v")
- A stack badge when the asset is in a stack (e.g., a small grid icon with the member count)
- Star rating (filled stars)
- Color label dot

Cards that belong to a stack have a colored left border for quick visual identification.

**Stack collapsing**: By default, stacks are collapsed – only the pick image (position 0) is shown in the grid, with a stack count badge indicating how many assets are hidden. A collapse toggle in the results bar lets you expand all stacks so every member is visible. When expanded, all stack members appear in position order with the colored border indicating membership.

**Per-stack expand/collapse**: Click the stack badge ( N) on any browse card to expand or collapse just that individual stack, regardless of the global collapse setting. When stacks are globally collapsed, clicking

a badge fetches and displays that stack's members inline. When stacks are globally expanded, clicking a badge collapses only that stack while others remain expanded. Re-clicking restores the previous state.

Click a card to open the asset detail page.

## View mode toggle

Next to the density controls, three view mode buttons switch between **Grid**, **Calendar**, and **Map** views:

- **Grid** (default): the standard thumbnail grid described above
- **Calendar**: a year-at-a-glance heatmap showing when assets were created
- **Map**: a map showing geotagged assets as clustered markers

Your view mode preference is saved in the browser and preserved across reloads.

## Calendar heatmap view

The calendar view displays a GitHub-style heatmap with 12 months laid out in a responsive grid (4 columns on wide screens, adapting down to 2 on narrow ones). Each day cell is colored by the number of assets created on that date, using a quartile-based 5-level color scale from empty (no assets) through increasingly intense shades.

**Year navigation**: Arrow buttons and clickable year chips above the calendar let you browse between years. Only years that contain assets appear as chips.

**Filter integration**: All browse filters (text search, tag, rating, label, type, format, volume, collection, path) apply to the calendar aggregation. For example, adding `tag:landscape` shows only landscape photos in the heatmap, letting you see which dates had landscape shots.

**Day click**: Click any day cell to filter the browse grid to that date. The view switches back to grid mode with `date:YYYY-MM-DD` added to the search query, showing only assets from that day. You can then clear the date filter from the search input to return to unfiltered browsing.

**Legend**: A color scale legend at the bottom of the calendar shows the meaning of each intensity level.

## Map view

The map view displays geotagged assets on an interactive OpenStreetMap map powered by Leaflet.js. Assets with GPS coordinates appear as markers, automatically clustered at wider zoom levels for performance.

**Markers and popups**: Click a marker (or cluster) to see a popup with a thumbnail preview, asset name, rating stars, and color label dot. Click the thumbnail to open the asset in the lightbox with full prev/next navigation through visible markers. Click the name or metadata area to navigate to the asset detail page.

**Filter integration**: All browse filters (text search, tag, rating, label, type, format, volume, collection, path, date) apply to the map. Adding `tag:landscape` shows only landscape photos on the map. The `geo:` search filter also works: `geo:any` shows only geotagged assets, `geo:none` shows assets without GPS data, and `geo:52.5,13.4,10` restricts to a 10km radius around a point.

**Dark mode**: Map tiles are automatically inverted for the dark theme to maintain visual consistency. Popups and controls adapt to the current color scheme.

**Keyboard shortcut**: Press `m` to switch to map view from any other view mode.

**Embedded libraries**: Leaflet.js and MarkerCluster are embedded as static assets — no external CDN or internet connection is required. The map tiles themselves are loaded from OpenStreetMap servers, so an internet connection is needed for the tile layer (but not for the application code).

## Grid density

Next to the sort controls, three grid density buttons let you adjust the thumbnail size:

- **Compact** (3×3 grid icon): smaller thumbnails, metadata and star ratings hidden, tighter spacing. Good for scanning many images quickly.
- **Normal** (2×2 grid icon): the default layout with full metadata.
- **Large** (1×2 grid icon): bigger thumbnails with room for two-line titles. Good for detailed visual comparison.

Your density preference is saved in the browser and preserved across page reloads and navigation.

## Faceted sidebar

The faceted sidebar provides a read-only statistical breakdown of your current results by rating, color label, format, volume, tag, year, and geolocation. It helps you understand the distribution of values in your collection at a glance. All filtering is done through the filter bar at the top of the page.

**Toggle**: Click the facet icon button in the results bar (next to the stack collapse button) or press `f`. The sidebar appears on the left side of the browse page. Your preference is saved in the browser.

**Facet sections**: Each section is collapsible. Ratings and years include a proportional bar chart. Labels show color dots. Formats, volumes, and tags show counts sorted by frequency. The geotagged facet shows a single count of assets with GPS data. Counts automatically update when the current search filters change.

**Collapse state**: Each section remembers its open/closed state in the browser across page loads.

**Responsive**: The sidebar is hidden on viewports narrower than 768px.

## Sorting

Above the results grid, sort toggle buttons let you order results by **Name**, **Date**, or **Size**. Each button toggles between ascending and descending. The active sort shows a direction arrow. Clicking a sort button updates results immediately.

## Pagination

When results span multiple pages, pagination controls appear both above and below the grid:

- First page, previous page, numbered page links, next page, last page
- A "Page X of Y" indicator

Page numbers with ellipsis keep the controls compact for large result sets. The number of results per page defaults to 60 and can be configured via `[serve] per_page` in `maki.toml` or the `--per-page` CLI flag.

Use **Shift+Left/Right arrow** keys to quickly turn pages from the keyboard. The results grid fades during loading to provide visual feedback while the new page loads. In the lightbox, regular arrow keys at page boundaries automatically navigate to the next/previous page with a loading spinner overlay.

## Saved search chips

Below the search bar, a row of saved search chips provides quick access to your named searches (smart albums). Each chip loads its stored query and filters into the search bar when clicked.

- **Save the current search**: click the "Save" button at the end of the chip row. A prompt asks for a name, and the current search state (text query, filters, sort order) is saved.
- **Rename**: hover over a chip to reveal the rename button. Click it and enter a new name.
- **Delete**: hover over a chip to reveal the delete button. Click it and confirm.

Saved searches are stored in `searches.toml` at the catalog root and work identically to CLI saved searches (see Organizing Assets for the `maki saved-search` command).

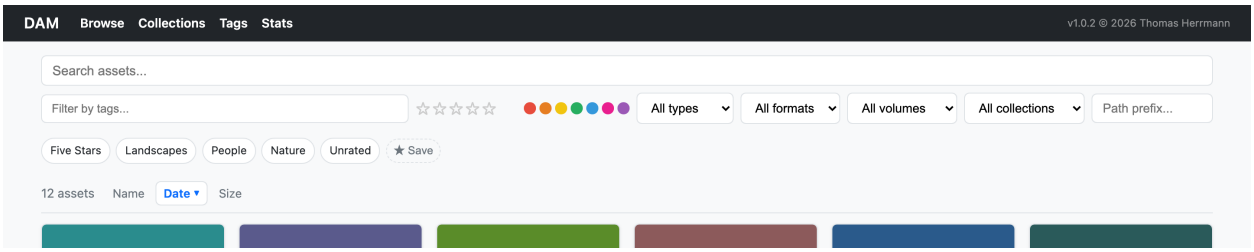



Figure 2: Saved search chips row on the browse page

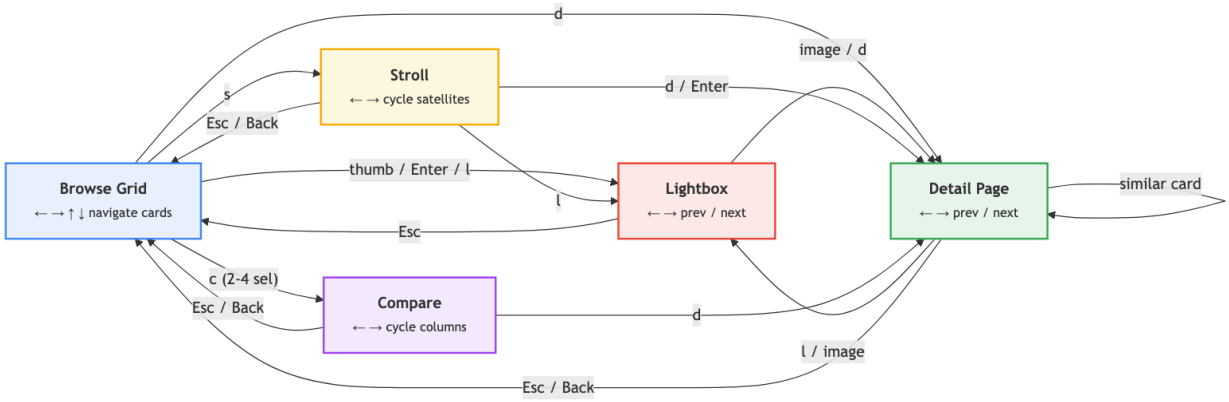
### How page updates work

The web UI uses htmx for partial page updates. When you search, filter, sort, or paginate, only the results area reloads – the search bar and its state remain intact. This makes interactions fast and fluid.

Because the URL updates with every search (via `hx-push-url`), the browser back button, forward button, reload, and bookmarks all work correctly. Navigating back from an asset detail page also refreshes results to reflect any edits you made.

## Navigation Between Views

The browse grid, lightbox, detail page, stroll, and compare form a connected navigation graph. You can switch between views with keyboard shortcuts, clicks, and the browser back button. Focus, scroll position, and selection are preserved when you return to browse.




| From | Action | To |
|---|---|---|
| Browse | Thumbnail click / Enter / `l` | Lightbox |
| Browse | `d` | Detail page |
| Browse | `s` | Stroll (from focused card) |
| Browse | `c` (2–4 selected) | Compare |
| Lightbox | Click image / `d` | Detail page |
| Lightbox | `s` | Stroll (from current asset) |
| Lightbox | Escape | Close overlay (return to host page) |
| Lightbox | Left / Right arrows | Prev / next asset |
| Detail | Click image / `l` | Lightbox |
| Detail | Left / Right arrows | Prev / next detail page |
| Detail | Similar card click | Detail (chained navigation) |
| Detail | `s` | Stroll (from current asset) |
| Detail | Escape / Back | Previous page (`history.back()`) |

| From | Action | To |
|---|---|---|
| Stroll | `l` | Lightbox (center or focused satellite) |
| Stroll | `d` / Enter | Detail page |
| Stroll | Escape / Back | Previous page (`history.back()`) |
| Compare | `d` | Detail page (focused column) |
| Compare | Escape / Back | Previous page (`history.back()`) |
| Any page | Shift+B | Jump directly to browse grid |

### Back navigation

There are two distinct "go back" actions:

- **Escape** (or browser Back button) — step back one level. On a page, this returns to wherever you came from via `history.back()`. In the lightbox overlay, it closes the overlay. On the browse grid, it clears the current selection or focus.
- **Shift+B** — jump directly to the browse grid, regardless of how deep you are. Useful when you've followed a chain of similar images or navigated through stroll and want to return to your search results.

The lightbox is available on browse, detail, and stroll pages. It uses the same shared component with page-specific callbacks for rating/label changes and navigation. Rating and color label editing (stars, color dots, 0–5 / Alt+1–7 keys) works everywhere the lightbox is open.

Focus and scroll position are preserved when navigating back to the browse page, even when the browser restores the page from its back/forward cache. Arrow key navigation in the lightbox and detail page updates which card will be focused when you return to browse.

## Lightbox

Clicking a thumbnail opens a full-screen lightbox overlay. The lightbox is available on the browse grid, detail page, and stroll page.

### Navigation

- **Arrow buttons**: click the left (‹) or right (›) side panels to move to the previous or next asset
- **Keyboard**: Left and Right arrow keys navigate between assets
- **Counter**: the top bar shows your position (e.g., "3 / 156")
- **Close**: click the × button or press Escape to close the overlay
- **Detail page**: click the image, press `d`, or click "  Detail" to open the full detail page for the current asset
- **Stroll**: press `s` to start strolling from the current asset

### Info panel

Click the  button in the top bar (or press `i`) to toggle a side panel showing:

- **Metadata**: asset type, format, date, and variant count
- **Rating**: interactive stars — click to set a rating (1-5), click × to clear
- **Color label**: interactive color dots — click to set a label, click × to clear

Rating and label changes made in the lightbox are saved immediately via the API and reflected on the host page. The top bar also shows interactive rating stars and color label dots for quick editing without opening the info panel.

The info panel has its own × close button.

### Rating and label

The lightbox top bar shows interactive rating stars and color label dots, always visible without opening the info panel. Click a star to set rating, click × to clear. Click a color dot to set a label, click the active dot again to clear.

These controls are also available in the info panel with more detail, but the top bar provides quick access during rapid browsing.

### Smart preview and zoom

When a smart preview (high-resolution, 2560px) is available, the lightbox loads the regular preview instantly for display, then background-loads the smart preview. A pulsing **HD** badge in the top-right corner provides visual feedback during loading. Once the smart preview loads, the badge briefly shows with solid opacity before fading out.

With a smart preview loaded, you can zoom and pan:

- **Mouse wheel**: zoom in/out centered on cursor position
- **Drag**: pan the image when zoomed in (cursor changes to grab/grabbing)
- **Keyboard**: `,` fit to screen, `.` 100% zoom, `+` zoom in, `-` zoom out

Smart previews can be generated via `maki import --smart`, `[import] smart_previews = true` in config, the "Regenerate previews" button on the detail page, or on-demand via `[preview] generate_on_demand = true`.

### Keyboard shortcuts

All browse keyboard shortcuts for rating and label work inside the lightbox:

| Key | Action |
|---|---|
| Left / Right arrow | Previous / next asset |
| Escape | Close lightbox |
| Click image | Open detail page |
| i | Toggle info panel |
| d | Open detail page |
| 1-5 | Set rating |
| 0 | Clear rating |
| Alt+1 through Alt+7 | Set color label |
| Alt+0, x | Clear color label |
| r/o/y/g/b/p/u | Set label by color initial |
| , (comma) | Fit to screen (zoom) |
| . (period) | 100% zoom |
| + | Zoom in |
| - | Zoom out |

When the lightbox is open, browse keyboard navigation (arrow keys for card focus, Enter, Space) is suppressed.

## Asset Detail Page

Click the "  Detail" link in the lightbox top bar, or press `d` in the lightbox, to open the asset detail page.

### Navigation

A navigation bar at the top of the page provides Prev, Back, and Next buttons:

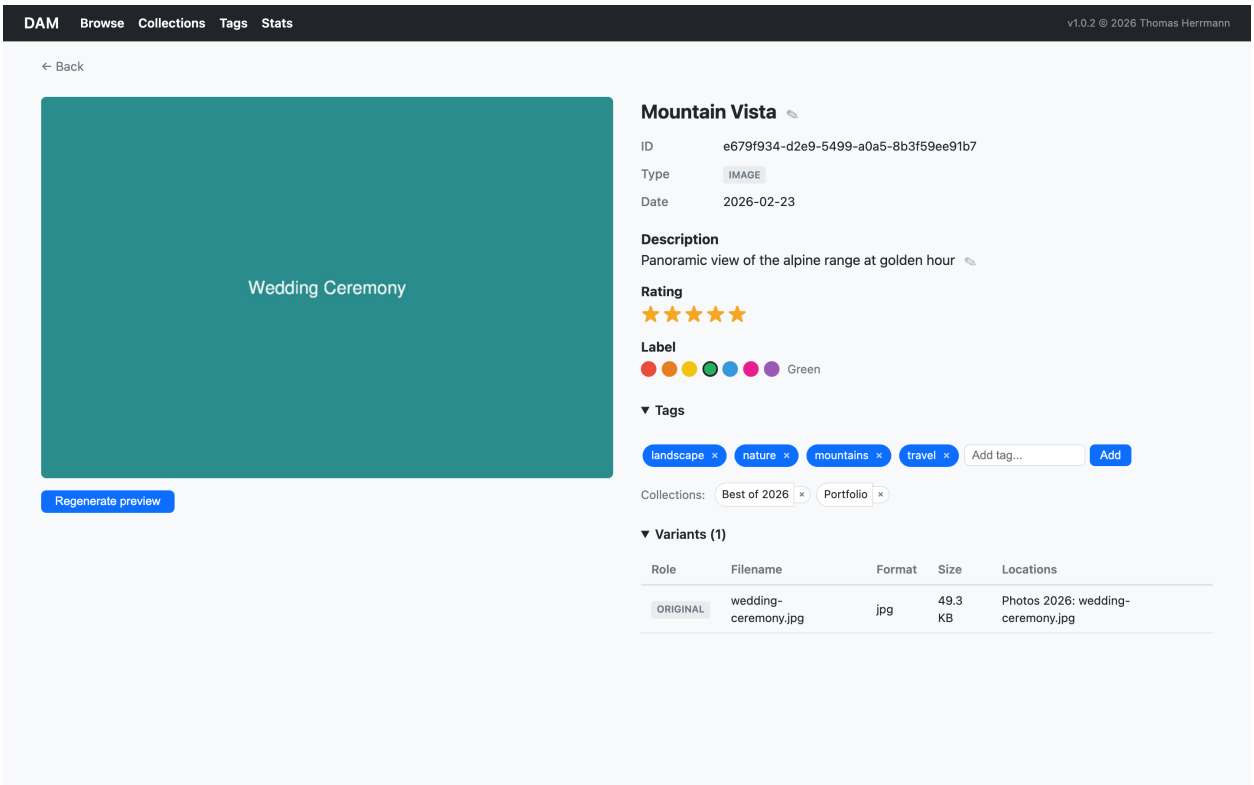



Figure 3: Asset detail page with preview, metadata editing, and variants

- **Prev / Next**: navigate to adjacent assets in the browse results. These buttons use the browse page's card list (stored in sessionStorage) for unlimited multi-hop navigation — you can press Next repeatedly to step through the entire result set. When accessed via a direct URL without browse context, these buttons are hidden.
- **Back**: returns to the browse page, preserving your search and filters.

Keyboard navigation:

| Key | Action |
|---|---|
| Left arrow | Previous asset |
| Right arrow | Next asset |
| Click image | Open in lightbox |
| l | Open in lightbox |
| Escape | Return to browse page |

Rating, label, and zoom keyboard shortcuts also work on the detail page:

| Key | Action |
|---|---|
| 1-5 | Set rating |
| 0 | Clear rating |
| Alt+1 through Alt+7 | Set color label |
| Alt+0, x | Clear color label |
| r/o/y/g/b/p/u | Set label by color initial |
| , (comma) | Fit to screen (zoom) |

| Key | Action |
|---|---|
| . (period) | 100% zoom |
| + | Zoom in |
| - | Zoom out |

## Preview

The left side shows a large preview image. This is the best available preview for the asset, preferring export variants over processed variants over originals, and standard image formats over RAW.

When a smart preview is available, it loads in the background with a pulsing "HD" badge. Once loaded, the image supports zoom and pan via mouse wheel, drag, and keyboard shortcuts (`, . + -`). Clicking the preview image opens the asset in the lightbox for full-screen viewing. Buttons below the preview let you regenerate the regular preview, generate/regenerate the smart preview, or rotate the image 90° clockwise. Rotation cycles through 0° → 90° → 180° → 270° → 0° and is persisted per asset. Previews are automatically oriented using EXIF tags during generation; manual rotation is applied on top.

## Editable metadata

The right side contains the asset's metadata, all editable inline:

**Name** – displayed as a heading. Click the pencil icon to switch to an inline text input with Save and Cancel buttons. Saving an empty name clears it, and the display falls back to the original filename in muted italic. The name is stored on the asset.

**Description** – click the pencil icon to switch to a textarea with Save and Cancel buttons. Saving an empty description clears it. Changes are written back to XMP recipe files on disk. When a VLM server is available, a "Describe" button appears next to the heading. Click it to generate a description using the configured vision-language model — the button shows "Generating…" while waiting for the VLM response, then updates the description in-place. When multiple VLM models are configured via `[vlm] models` in `maki.toml`, a model selector dropdown appears next to the Describe button, letting you choose which model to use per request (e.g., a fast model like Moondream for routine descriptions and a larger model like Qwen3-VL for difficult images).

**Rating** – five clickable stars. Click a star to set that rating. Click the same star again to clear the rating. Changes are written back to XMP recipe files on disk.

**Color label** – seven colored dots (Red, Orange, Yellow, Green, Blue, Pink, Purple) with a label name shown below the active dot. Click a dot to set that label; click the active dot again to clear it. Changes are written back to XMP recipe files on disk.

**Tags** – displayed as removable chips. Click the x on a chip to remove that tag. Use the text input below to add new tags – it offers autocomplete suggestions from your catalog's tag list as you type. Changes are written back to XMP recipe files on disk with operation-level deltas (tags added independently in CaptureOne or Lightroom are preserved).

**Suggest tags** (requires `--features ai`) – a "Suggest tags" button appears below the tag input when the server is compiled with AI support. Click it to analyze the asset image with the SigLIP vision model. The button shows "Analyzing…" while the model processes (the first request may take a few seconds while the model loads). Results appear as suggestion chips, each showing the tag name and a confidence percentage. Tags already on the asset appear dimmed with an "already applied" label. Click  to accept a new tag (it is added immediately), click × to dismiss it, or click "Accept new" to apply all unapplied suggestions at once.

**Re-import metadata** – a "Re-import metadata" button appears below the tag section. Click it to clear the asset's tags, description, rating, and color label, then re-extract metadata from variant source files (XMP recipe sidecars and embedded XMP in JPEG/TIFF files). A confirmation dialog asks before proceeding since the operation is destructive. This is useful for cleaning up metadata after splitting mis-grouped assets, where

tags from multiple unrelated images may have been merged together. The page reloads after completion to reflect all changes. Offline volumes are silently skipped during re-extraction.

### Asset information

Below the editable fields:

- **ID**: the asset's UUID
- **Type**: image, video, audio, or document
- **Date**: the asset's creation date (from EXIF or import time). Click the pencil icon to edit it inline with a date input and Save/Cancel buttons.

### Collections

If the asset belongs to any collections, they appear as clickable chips. Click a chip to browse that collection. Click the x button on a chip to remove the asset from that collection.

### Stack members

If the asset belongs to a stack, a section shows all members in position order with thumbnail previews. The pick is indicated. You can click any member to navigate to its detail page.

### Variants

An expandable section lists all variants of the asset in a table with columns for role, filename, format, size, and file locations (volume and path). This gives you a complete picture of where the asset's files live across your storage volumes.

For assets with multiple variants, the **role column** is an inline dropdown. Click it to change a variant's role (original, alternate, processed, export, sidecar). Changes are saved immediately — a green flash confirms success. This is useful for correcting roles after import or `fix-roles` assigns an incorrect role (e.g. marking a re-exported JPEG as "alternate" when it should be "export").

For locations on online volumes, two action buttons appear next to each path: - **Reveal** — opens the system file manager with the file selected (Finder on macOS, Explorer on Windows, file manager on Linux). - **>_ Terminal** — opens a terminal window in the file's parent directory (Terminal.app on macOS, cmd on Windows, system terminal emulator on Linux).

### Recipes

An expandable section lists attached recipe files (XMP sidecars, CaptureOne settings, etc.) with columns for recipe type, software, and file path. Recipes with pending write-back changes (edits made while the volume was offline) show an orange sync indicator. When any recipe has pending changes, a "Write back to XMP" button appears at the top of the section — clicking it replays the queued edits to the recipe files on disk (equivalent to `maki writeback --asset`).

### Source metadata

If the primary variant has EXIF or XMP metadata, an expandable section displays it as key-value pairs (camera model, lens, exposure settings, GPS coordinates, etc.).

## Batch Operations

The web UI supports applying changes to multiple assets at once. Select assets in the browse grid and use the batch toolbar to tag, rate, label, group, or organize them into collections.

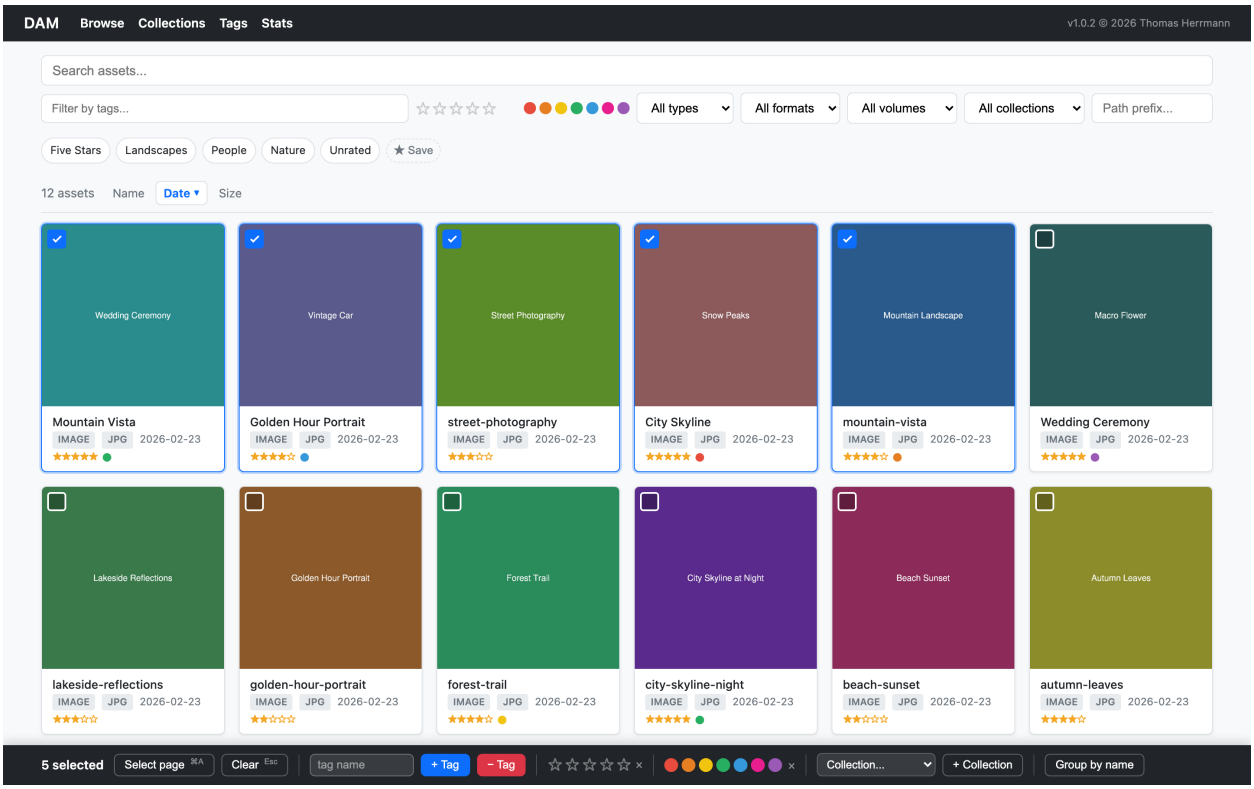



Figure 4: Batch toolbar with selection count, tag/rating/label controls

## Selecting assets

Each browse card has a checkbox that appears on hover. Once any card is selected, all checkboxes become permanently visible (until the selection is cleared) so you can see what is selected at a glance. Selected cards receive a visible selection border.

- **Click a checkbox** to toggle selection of individual cards
- **"Select page"** button selects all cards on the current page
- **"Clear"** button deselects everything

## Batch toolbar

A fixed toolbar appears at the bottom of the screen whenever one or more assets are selected. It shows the selection count and provides these controls:

**Tags**: a text input with autocomplete, plus "+ Tag" and "- Tag" buttons. Type a partial tag name to see suggestions from your catalog's tag list; navigate with arrow keys, Enter or click to select a suggestion. Click "+ Tag" to add the tag to all selected assets, or "- Tag" to remove it. Press Enter in the input to add. Newly created tags are immediately available in the autocomplete.

**Rating**: five clickable stars and a clear button (x). Click a star to set that rating on all selected assets. Click x to clear rating.

**Color label**: seven colored dots and a clear button (x). Click a dot to set that label on all selected assets. Click x to clear label.

**Collection**: a dropdown listing your collections (plus a "New…" option to create one inline). The buttons next to it are context-sensitive: - When you are **not** browsing a collection, a "+ Collection" button adds the selected assets to the chosen collection. - When you **are** browsing a collection (the collection filter is

active), a "- Collection" button removes the selected assets from that collection. The dropdown auto-selects the current collection.

**Stack**: creates a stack from the selected assets (or adds them to an existing stack if one of the selected assets is already stacked). The first selected asset becomes the pick. This is a lightweight, reversible grouping – assets remain independent.

**Unstack**: removes the selected assets from their stacks. If a stack has one or fewer members after removal, it auto-dissolves.

**Group by name**: merges the selected assets by filename stem. A confirmation dialog explains the action. Assets whose filenames share a common prefix (e.g., `DSC_001.nef` and `DSC_001.jpg`) are merged into a single asset with multiple variants. This cannot be undone.

**Compare**: opens a side-by-side compare view with the selected assets (2–4). See Compare View below.

**Describe** (requires VLM server): sends each selected asset's preview image to the configured VLM server and generates a natural language description. Assets that already have a description are skipped. A confirmation dialog shows the count and warns about per-asset timing. After processing, a summary reports how many descriptions were set. This button only appears when a VLM server is reachable at startup (see VLM Setup).

**Auto-tag** (requires `--features ai`): analyzes each selected asset with the SigLIP vision model and applies suggested tags above the configured confidence threshold. A confirmation dialog shows the count of selected assets. After processing, a summary reports how many tags were applied. This button only appears when the server is compiled with the `ai` feature.

**Detect faces** (requires `--features ai`): runs face detection on each selected asset's preview image using YuNet and ArcFace. Detected faces are stored with bounding boxes, embeddings, and crop thumbnails. A summary reports how many faces were found. This button only appears when the server is compiled with the `ai` feature.

**Export**: downloads the selected assets as a ZIP archive. A modal dialog lets you choose layout (flat — all files in root with hash-suffix collision resolution, or mirror — preserves original directory structure), whether to include all variants or just the best, and whether to include sidecar/recipe files. The ZIP is streamed to the browser as a download.

## Export all results

The results bar includes an "Export all" link that exports all assets matching the current search and filter state — not just the current page. Clicking it opens the same export modal as the batch Export button. The full set of active browse filters (type, tag, format, volume, rating, label, collection, path, person) is sent to the server, which resolves the matching assets using the same search pipeline as the browse page.

After every batch operation, the selection clears and the results grid refreshes to reflect the changes. All toolbar buttons are disabled during the operation to prevent double submissions.

## Drag-and-drop

The web UI supports drag-and-drop for two common operations:

**Adding to collections**: Drag one or more browse cards onto the collection dropdown in the batch toolbar. If cards are selected, dragging any selected card moves the entire selection. A toast notification confirms how many assets were added. The collection dropdown highlights as a valid drop target during the drag.

**Reordering stack members**: On the asset detail page, stack members can be dragged to reorder. Dropping a member into the first position sets it as the stack pick.

## Keyboard shortcuts for selection

| Key | Action |
|---|---|
| Cmd+A (Mac) / Ctrl+A | Select all cards on the current page |
| Escape | Clear selection (if any), otherwise clear keyboard focus |

These shortcuts are suppressed when focus is in a text input, textarea, or dropdown.

# Keyboard Navigation

The browse page supports full keyboard navigation for efficient photo culling and rating workflows. No mouse required.

## Keyboard help

Press `?` on any page (or click the "?" button in the navigation bar) to open a keyboard shortcuts overlay. The overlay shows all shortcuts available on the current page, organized by category. Press Escape, click ×, or click outside the panel to close it.

## Movement

| Key | Action |
|---|---|
| Arrow Left | Move focus to the previous card |
| Arrow Right | Move focus to the next card |
| Arrow Up | Move focus up one row (column-aware) |
| Arrow Down | Move focus down one row (column-aware) |

The focused card has a blue outline, visually distinct from the selection highlight. If no card is focused, the first arrow key press focuses the first card.

## Actions on the focused card

| Key | Action |
|---|---|
| Shift+Left | Previous page |
| Shift+Right | Next page |
| Enter | Open the focused card in the lightbox |
| d | Open the focused card's detail page |
| Space | Toggle selection of the focused card |
| 1-5 | Set rating (applies to focused card, or to all selected if a batch selection is active) |
| 0 | Clear rating |
| Alt+1 through Alt+7 | Set color label (1=Red, 2=Orange, 3=Yellow, 4=Green, 5=Blue, 6=Pink, 7=Purple). On macOS, use Option+number. |
| Alt+0 | Clear color label |
| r | Set Red label |
| o | Set Orange label |
| y | Set Yellow label |
| g | Set Green label |
| b | Set Blue label |
| p | Set Pink label |
| u | Set Purple label |

| Key | Action |
|---|---|
| x | Clear label |
| Shift+F | Toggle filter bar visibility |
| s | Open the stroll page for the focused card (visual similarity exploration) |

Single-letter label shortcuts and number keys for rating operate on the focused card when no batch selection is active. When assets are selected (selection count > 0), rating keys apply to the entire batch.

### Focus persistence

Focus position is preserved across pagination and sort changes. If you are focused on card 5 and sort by name, card 5 (by position) remains focused after the grid reloads.

### Input suppression

All keyboard shortcuts are suppressed when focus is in a text input, textarea, or select dropdown. This prevents accidental rating or label changes while typing a search query or tag name.

## Compare View

Select 2–4 assets in the browse grid and click the "Compare" button in the batch toolbar to open a side-by-side comparison view.

### Layout

Each asset appears in its own column with a preview image on top and metadata below. The metadata section shows the display name (clickable link to detail page), creation date, interactive rating stars, color label dots, and EXIF data (camera, lens, focal length, aperture, shutter speed, ISO).

### Zoom and pan

All columns support zoom and pan:

- **Mouse wheel**: zoom in/out centered on cursor position (0.5× to 20×)
- **Drag**: pan the image when zoomed
- **Double-click**: reset zoom to 1×

**Sync zoom**: when enabled (on by default), zooming or panning one column applies the same transform to all columns. This lets you inspect the same area across all images simultaneously. Toggle sync with the checkbox in the top bar or press `s`.

Smart previews are loaded automatically when available, with a pulsing HD badge during loading.

### Keyboard shortcuts

| Key | Action |
|---|---|
| Left / Right arrow | Move focus between columns |
| d | Open detail page for focused asset |
| s | Toggle sync zoom |
| Escape | Return to browse |
| 1-5 | Set rating on focused asset |
| 0 | Clear rating |

| Key | Action |
|---|---|
| Alt+1 through Alt+7 | Set color label |
| Alt+0, x | Clear color label |
| r/o/y/g/b/p/u | Set label by color initial |
| , (comma) | Fit to view (zoom) |
| . (period) | Toggle 3× zoom |
| + | Zoom in |
| - | Zoom out |

The focused column has a blue border. Rating and label changes are saved immediately.

# Tags Page

The tags page shows all tags in your catalog with their usage counts.

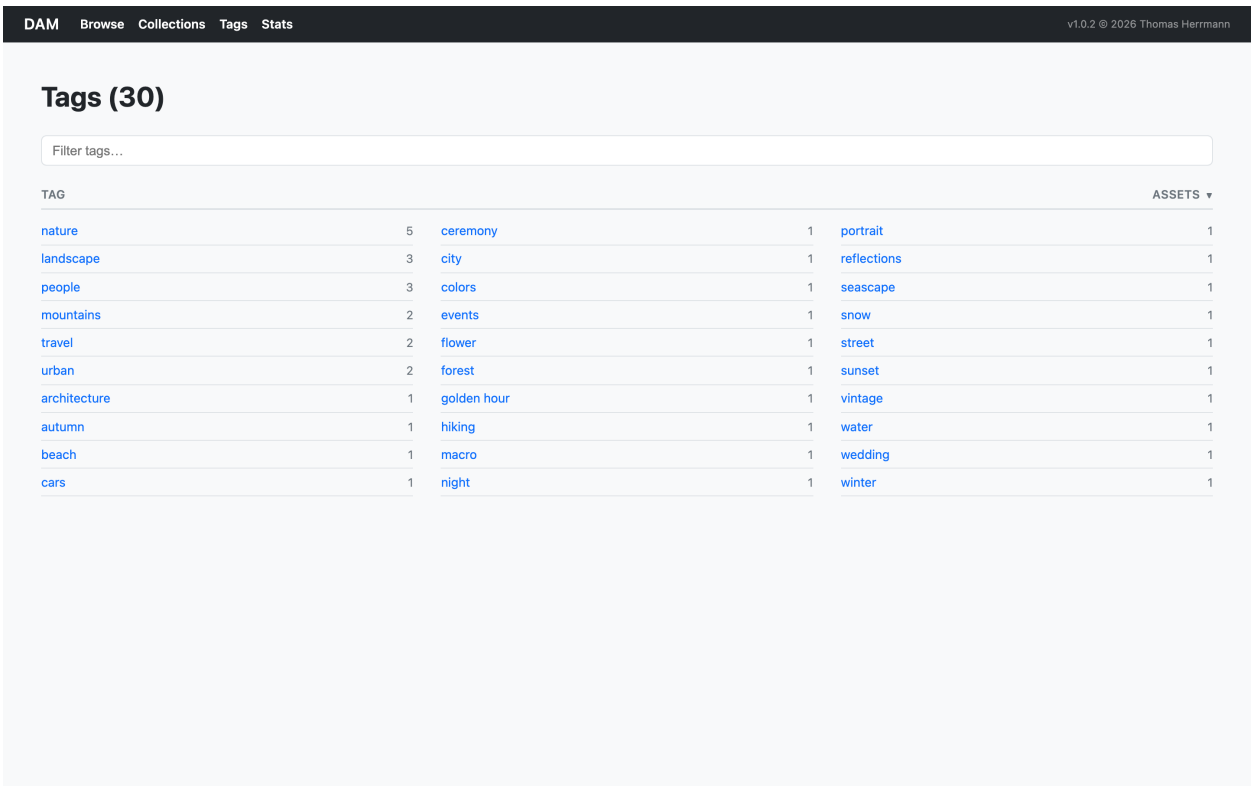



Figure 5: Tags page with sortable columns and text filter

Navigate to `/tags` or click "Tags" in the navigation bar.

## Features

- **Sortable columns**: click the "Tag" header to sort alphabetically, or the "Assets" header to sort by usage count. Click again to reverse direction. The active sort shows a direction arrow.
- **Live text filter**: type in the filter input to narrow the tag list. Filtering begins at 2 characters. The count display updates to show "X of Y" tags.
- **Multi-column layout**: tags flow into multiple columns automatically, adapting to the viewport width.
- **Clickable tags**: click any tag name to jump to the browse page filtered by that tag.

- **Hierarchical tree view**: tags containing `/` separators are displayed as a collapsible tree. Each node shows its own count (assets tagged with that exact tag) and a total count (including all descendants). Click the disclosure triangle to expand or collapse a branch. Clicking a parent tag searches for all descendants.

# Collections Page

The collections page lists all your static collections (manually curated albums).

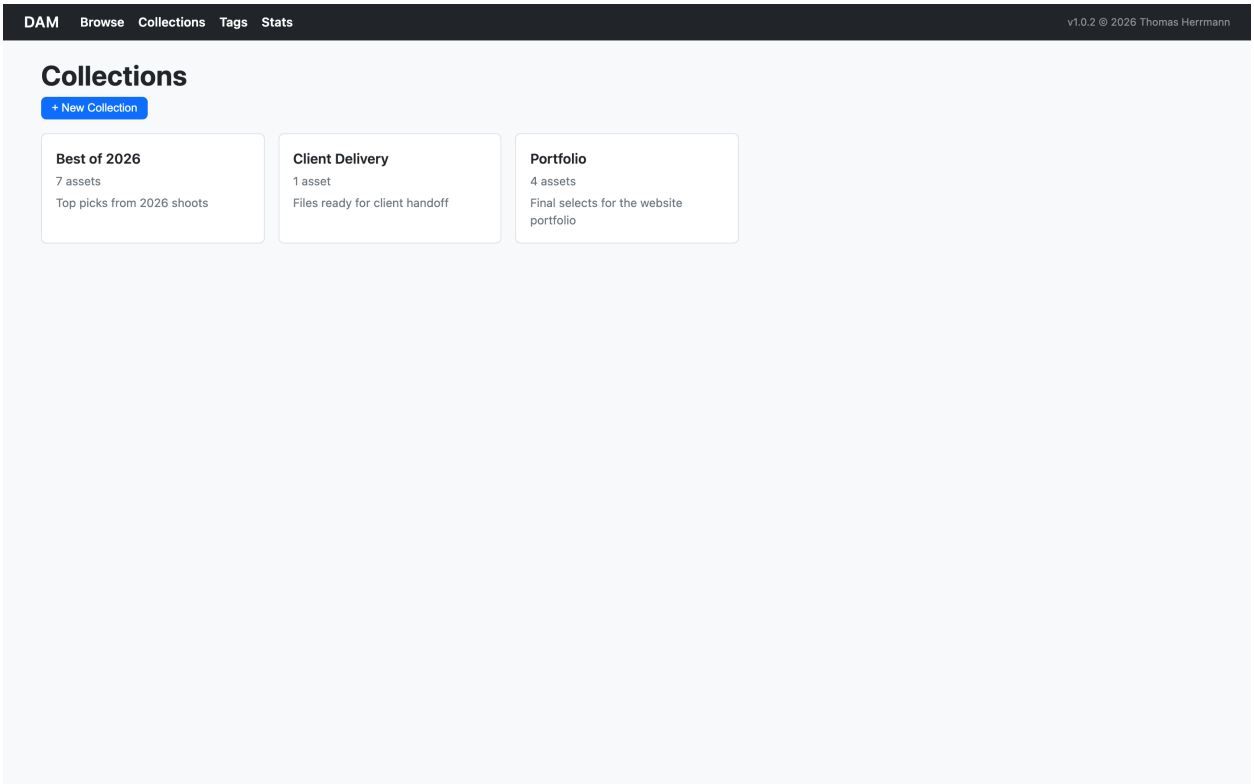



Figure 6: Collections page with collection cards and create button

Navigate to `/collections` or click "Collections" in the navigation bar.

## Features

- **Collection cards**: each collection is shown as a card with its name, asset count, and description (if set). Click a card to browse its assets on the main browse page.
- **"+ New Collection" button**: prompts for a name and optional description, then creates the collection immediately. The page reloads to show the new card.

Collections created here are the same as those created via the CLI `maki collection create` command. See Organizing Assets for full details on managing collections.

# Stats Page

Navigate to `/stats` or click "Stats" in the navigation bar to see a visual overview of your catalog.

The stats page displays:

- **Overview cards**: total assets, variants, recipes, online/total volumes, and total storage size
- **Asset types**: bar chart showing the distribution of images, videos, audio files, and documents
- **Variant and recipe formats**: bar charts showing format breakdowns (NEF, ARW, JPEG, XMP, etc.)
- **Volumes**: table with per-volume details including label, online/offline status, asset count, variant count, recipe count, total size, formats present, and verification coverage
- **Tags**: summary of unique tags, tagged assets, and untagged assets, plus a weighted tag cloud of the most-used tags
- **Verification health**: overall coverage bar, plus a per-volume breakdown of verification status

This is the web equivalent of `maki stats --all` on the command line. See Browsing & Searching for the CLI stats command.

# People Page

> Requires `--features ai` compilation.

Navigate to `/people` or click "People" in the navigation bar to manage detected faces and named people.

The people page displays:

- **Person cards**: each person is shown as a card with a representative face crop thumbnail, name (or "Unknown" for unnamed clusters), and the number of detected faces. Click a card to browse assets containing that person.
- **Inline rename**: click the name on a person card to edit it in place.
- **Merge**: drag a person card onto another to merge them, or use the merge controls.
- **Delete**: click the delete button on a person card to remove the person (faces become unassigned).
- **Cluster button**: runs face auto-clustering from the UI, grouping unassigned faces into new person groups.

Faces are detected via `maki faces detect` on the CLI, the "Detect faces" button on the asset detail page, or the batch "Detect faces" button on the browse toolbar. After detection, use clustering (CLI or web UI) to group faces into people, then name them.

## Asset detail faces section

On the asset detail page, a "Faces" section appears when faces have been detected for the asset:

- **Face chips**: each detected face is shown as a small chip with the crop thumbnail and confidence score.
- **Detect faces button**: triggers on-demand face detection for the asset.
- **Person assignment**: a dropdown on each face chip lets you assign or unassign the face to a person.

## Browse face filters

- **Person dropdown**: in the browse filter row, a person dropdown (when people exist) lets you filter to assets containing a specific person.
- **Face count badge**: browse cards show a face count badge next to the variant count badge.
- **Query filters**: `faces:any`, `faces:none`, `faces:N`, `faces:N+`, `person:<name>` work in the query input.

# Stroll Page

> Requires `--features ai` compilation and image embeddings (generated via `maki embed` or `maki import --embed`).

The stroll page provides a visual similarity exploration experience. Instead of browsing a flat grid, you start with a center image and see its most visually similar neighbors arranged in a radial layout around it.

Click any neighbor to make it the new center, and the neighbors update – letting you "stroll" through your collection by visual similarity.

Navigate to `/stroll` or click "Stroll" in the navigation bar. You can also press `s` on the browse page (when a card is focused) or click the "Stroll" button on the asset detail page to start strolling from a specific asset.

## Layout

The center image is displayed prominently in the middle of the page. Surrounding it, satellite thumbnails show the most similar assets based on SigLIP embedding similarity (dot-product distance), arranged in an elliptical layout that adapts to the viewport aspect ratio for optimal use of screen space. Each neighbor shows a preview thumbnail, and clicking it navigates to that asset as the new center.

## Stroll modes

Three mode buttons in the control panel determine how neighbors are selected:

- **Nearest** (default): shows the top N most similar assets by embedding distance. Results are deterministic – the same center always produces the same neighbors.
- **Discover**: picks N random assets from a wider candidate pool (configurable via `[serve] stroll_discover_pool` in `maki.toml`, default 80). Each visit produces a different set of neighbors, encouraging serendipitous exploration. Useful for breaking out of tight visual clusters.
- **Explore**: skips past the K nearest neighbors to find more distant visual connections. A skip slider (0–200) appears in Explore mode, letting you control how far to reach. Higher skip values surface increasingly surprising matches.

## "Other shoots" filter

A checkbox toggle labeled "Other shoots" excludes assets from the same directory or session as the center image. When enabled, only similar images from different shoots are shown as neighbors. This is useful for finding visual connections across your collection rather than seeing near-duplicates from the same session.

## Neighbor count

A slider control (5–25, default 12, configurable via `[serve] stroll_neighbors` and `stroll_neighbors_max` in `maki.toml`) lets you adjust how many neighbor thumbnails are shown around the center image. Fewer neighbors give a cleaner view; more neighbors let you see a wider range of similar assets.

## Filter bar

The stroll page includes the same collapsible filter bar as the browse page. When filters are active, only assets matching the filters are considered as neighbors. Press **Shift+F** to toggle the filter bar visibility.

## Fan-Out slider (transitive neighbors)

A fan-out slider in the bottom-left corner (range 0–10, configurable via `[serve] stroll_fanout` and `stroll_fanout_max` in `maki.toml`) controls level-2 neighbor exploration. When fan-out is greater than 0 and you hover over or arrow-key to a satellite thumbnail, smaller thumbnails fan out from it showing that satellite's own nearest visual neighbors (excluding assets already visible on the page). The L2 thumbnails are arranged in a direction-dependent arc – the fan-out radius adapts based on each satellite's position, and satellites with L2 neighbors are pulled slightly toward the center to keep the layout balanced. L2 neighbor thumbnails show name, rating, and color label consistently with L1 satellites. Click any level-2 thumbnail to navigate to it as the new center. Results are cached per satellite, so moving focus back to a previously explored neighbor re-displays its fan instantly without another query.

### Keyboard shortcuts

| Key | Action |
|---|---|
| Arrow Left / Right | Navigate focus between neighbor thumbnails at the current level |
| Arrow Up | Move focus from an L2 thumbnail back to its parent L1 satellite |
| Arrow Down | Move focus from an L1 satellite into its L2 fan-out thumbnails (when fan-out > 0) |
| Enter | Make the focused neighbor the new center |
| Shift+F | Toggle filter bar visibility |
| Escape | Return to the browse page |
| ? | Open keyboard help |

## Analytics Page

Navigate to `/analytics` or click "Analytics" in the navigation bar to see shooting and storage analytics.

The analytics page displays:

- **Yearly shooting counts**: bar chart of assets imported per year
- **Monthly sparkline**: monthly import volume for the most recent years
- **Camera usage**: top camera bodies by asset count
- **Lens usage**: top lenses by asset count
- **Rating distribution**: vertical bar chart showing how many assets have each star rating (1–5) plus unrated
- **Format distribution**: bar chart of file formats across the catalog
- **Storage by volume**: bar chart showing total size per volume

All charts auto-scale to their data. This complements the Stats Page with a more visual, photography-oriented perspective.

## Backup Status Page

Navigate to `/backup` or click "Backup" in the navigation bar to see backup health at a glance.

The backup status page displays:

- **Summary cards**: total assets, at-risk count (highlighted in red when > 0), and minimum copies threshold (default 2)
- **At-risk link**: when at-risk assets exist, a prominent link navigates to the browse page filtered to `copies:1` for immediate review
- **Volume distribution**: horizontal bar chart showing how many assets exist on 0, 1, 2, or 3+ volumes, with red/amber/green coloring and "AT RISK" badges on under-backed-up buckets
- **Coverage by purpose**: table showing each volume purpose (working, archive, backup) with the number of volumes, asset count, and a coverage bar
- **Volume gaps**: table listing volumes with missing assets, showing the volume label, purpose, and missing count

This is the web equivalent of `maki backup-status` on the command line. See Maintenance for the CLI backup-status command.

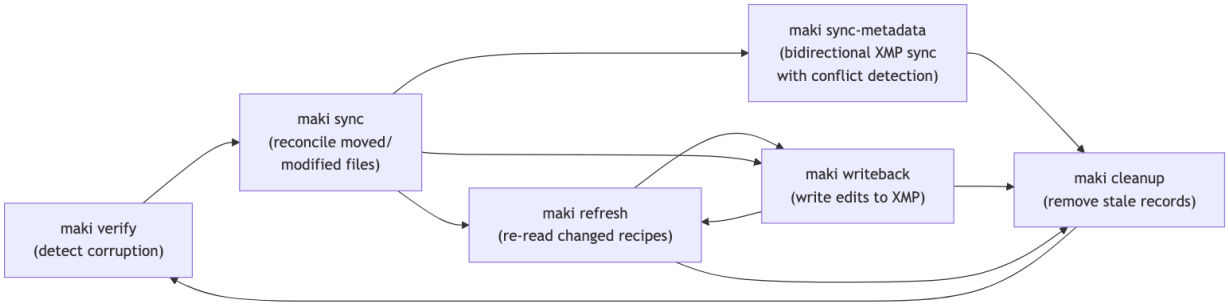
---

Next: Maintenance – verification, sync, refresh, cleanup, and file relocation.

# Maintenance

Over time, files move, drives get swapped, external tools edit recipes, and storage devices accumulate stale references. This chapter covers the commands that keep your catalog accurate and your files healthy.

The core maintenance commands form a cycle:




After `sync`, there are two paths: `sync-metadata` handles both directions (reading recipe changes and writing back edits) in a single command with conflict detection. Alternatively, `refresh` and `writeback` can be run separately for finer control.

Each command is safe by default – destructive operations require an explicit `--apply` flag, and most commands support `--dry-run` or report-only mode.

## Verification

`maki verify` re-hashes every file on disk and compares the result to the content hash stored in the catalog. This detects silent corruption, bit rot, and accidental modifications.

### Verify everything

```
maki verify
```

This walks all file locations on all online volumes. Offline volumes are skipped automatically.

Sample output:

```
Verify complete: 1847 verified, 2 modified, 0 FAILED, 3 skipped
```

### Verify specific files or directories

```
maki verify /Volumes/PhotosDrive/Capture/2026-02-01
```

Only files under the given path are checked.

### Limit to a volume

```
maki verify --volume "Photos 2024"
```

Useful when you reconnect a drive and want to spot-check it before trusting its contents.

### Verify a single asset

```
maki verify --asset a1b2c3d4
```

Asset IDs can be abbreviated to a unique prefix.

### Filter by file type

```
# Only verify RAW files
maki verify --include raw

# Skip audio files
maki verify --skip audio
```

### Two verification modes

**Catalog mode** (no paths): `maki verify` walks all file locations known to the catalog on online volumes. This checks whether your cataloged files are intact.

**Path mode** (with paths): `maki verify /some/path` scans files on disk and looks them up in the catalog. This also detects files that are not in the catalog at all.

### What the results mean

- **verified** – file hash matches the catalog. The `verified_at` timestamp is updated.
- **modified** – a recipe file (`.xmp`, `.cos`, etc.) was changed externally. maki updates the stored hash and reports it as "modified" rather than "FAILED". This is expected when CaptureOne or Lightroom edits a sidecar.
- **FAILED** – a media file's hash does not match. This indicates corruption, accidental overwrite, or a file that was replaced. Investigate immediately.
- **MISSING** – a file referenced in the catalog no longer exists on disk (catalog mode only). The file's location record exists but the file is gone from an online volume.
- **UNTRACKED** – a file was found on disk but is not in the catalog (path mode only). The file's content hash does not match any known variant or recipe. Run `maki import` to bring it into the catalog, or ignore it if it is not a media file.
- **skipped** – the file is on an offline volume, or the path could not be read.

If any files fail verification (FAILED status), maki exits with code 1. Scripts can check `$?` to detect problems:

```
maki verify --volume "Archive" || echo "Integrity check failed!"
```

### Monitoring flags

```
# Per-file progress to stderr
maki verify --log

# Machine-readable output
maki verify --json
```

```
# Show elapsed time
maki verify --time

# Combine them
maki verify --volume "Photos 2024" --log --time
```

### Checking verification age

Use the `stale:N` search filter to find assets that have not been verified recently:

```
# Assets not verified in the last 30 days
maki search "stale:30"

# Assets never verified
maki search "stale:0"
```

See Browsing & Searching for more on search filters.

## Sync

`maki sync` reconciles the catalog with what is actually on disk. Run it after moving, renaming, or deleting files with external tools (Finder, `mv`, CaptureOne's "move to folder", etc.).

Unlike `verify` (which only checks hashes), `sync` scans the filesystem for structural changes: files that moved, new files that appeared, and files that disappeared.

### Report mode (safe default)

```
maki sync /Volumes/PhotosDrive/Photos/
```

Without `--apply`, sync scans the directory and reports what it finds, but changes nothing:

```
Sync complete: 1200 unchanged, 3 moved, 2 new, 1 modified, 4 missing
  Tip: run 'maki import' to import new files.
```

### Apply changes

```
maki sync /Volumes/PhotosDrive/Photos/ --apply
```

With `--apply`, sync updates the catalog and sidecar files:

- **Moved files**: The catalog path is updated to the new location (same content hash found at a different path, old path gone).
- **Modified recipes**: Recipe hash is updated, and if it is an XMP file, metadata is re-extracted.
- **Missing files**: Reported but not removed (use `--remove-stale`).
- **New files**: Reported but not imported. Run `maki import` separately.

### Removing stale records

```
maki sync /Volumes/PhotosDrive/Photos/ --apply --remove-stale
```

`--remove-stale` (which requires `--apply`) removes catalog location records for files that are confirmed missing. Use this when you intentionally deleted files and want the catalog to reflect that.

### Scoping to a volume

```
maki sync /Volumes/PhotosDrive/ --volume "Photos 2024"
```

Explicitly sets the volume context when auto-detection picks the wrong one.

### Detection categories

| Status | Meaning | Action with `--apply` |
|---|---|---|
| unchanged | Hash matches at expected path | None |
| moved | Known hash found at new path, old path gone | Path updated in catalog |
| new | Unknown hash, not in catalog | Reported (run `maki import`) |
| modified | Same path, different hash (recipe files) | Hash updated, XMP re-extracted |
| missing | Catalog location exists but file is gone | Reported, or removed with `--remove-stale` |

### Monitoring flags

```
maki sync /Volumes/PhotosDrive/ --apply --log --time --json
```

`--log` shows per-file status, `--time` shows elapsed time, `--json` outputs structured results.

## Refresh

`maki refresh` re-reads metadata from recipe and media files without scanning the full filesystem. It is lighter than `sync` – it only checks files the catalog already knows about, comparing their on-disk hash to the stored hash.

This is the right tool after editing in CaptureOne, Lightroom, or any other tool that modifies XMP sidecars.

### Refresh all recipes

```
maki refresh
```

Checks every recipe file location on all online volumes. For each recipe whose hash has changed:

- **XMP recipes** (`.xmp`): re-extracts keywords, rating, description, and color label, then updates the catalog and sidecar YAML.
- **Non-XMP recipes** (`.cos`, `.pp3`, `.dop`, etc.): hash updated, but no metadata extraction (these formats are opaque to maki).

### Refresh specific paths

```
maki refresh /Volumes/PhotosDrive/Capture/2026-02-01/
```

Only checks recipe files under the given path.

### Limit to a volume or asset

```
# All recipes on a specific volume
maki refresh --volume "Photos 2024"

# Only recipes for a specific asset
maki refresh --asset a1b2c3d4
```

### Re-extract embedded XMP from media files

```
maki refresh --media
```

The `--media` flag also scans JPEG and TIFF variant files, re-extracting embedded XMP metadata. This is useful in two scenarios:

1. **Retroactive extraction**: You imported files before the embedded XMP feature existed and want to pick up keywords/ratings/labels that were embedded all along.
2. **External edits**: A tool like CaptureOne or Lightroom modified the embedded XMP in a JPEG/TIFF file.

### Dry run

```
maki refresh --dry-run
```

Shows what would change without applying anything. Combine with `--log` for detailed output:

```
maki refresh --dry-run --log
```

Sample output:

```
  DSC_001.xmp — changed (12ms)
  DSC_002.xmp — unchanged (3ms)
  DSC_003.xmp — changed (11ms)
Dry run — Refresh complete: 2 refreshed, 1 unchanged, 0 missing, 0 skipped (offline)
```

### Monitoring flags

```
maki refresh --log --time --json
```

## Write Back

`maki writeback` replays pending metadata writes to XMP recipe files. When you edit metadata (rating, label, tags, description) while a volume is offline, the XMP write-back is skipped and the recipe is marked with a `pending_writeback` flag. The edits are safe in the YAML sidecar and SQLite catalog, but the `.xmp` files on disk still have old values. This command pushes those pending changes to XMP when the volume comes back online.

### Process pending write-backs

```
maki writeback
```

Without flags, only recipes with `pending_writeback=1` are processed. Each recipe's XMP file is updated with the current asset metadata (rating, label, tags, description), then re-hashed and the pending flag is cleared.

### Write back all XMP recipes

```
maki writeback --all
```

The `--all` flag writes current metadata to every XMP recipe, not just pending ones. Useful for an initial sync or to force-push all DAM metadata to XMP files.

### Scope to a volume or asset

```
# Only write back to recipes on a specific volume
maki writeback --volume "Photos 2024"

# Only write back for a specific asset
maki writeback --asset a1b2c3d4
```

### Dry run

```
maki writeback --dry-run
```

Shows what would be written without modifying any files. Combine with `--log` for detailed output.

### How pending tracking works

When any metadata edit (CLI `maki edit`, `maki tag`, web UI stars/labels/tags/description) triggers XMP write-back, each recipe is checked:

- **Volume online, file exists**: XMP is written, `pending_writeback` stays 0.
- **Volume offline or file missing**: `pending_writeback` is set to 1. The flag persists until `maki writeback` clears it.

The flag records the *intent* to write back, not *what* changed. When writeback runs, it reads the current asset metadata and writes all four fields (rating, label, tags, description) to the XMP file.

### Recommended workflow: volume comes back online

```
# 1. Push DAM edits to XMP (DAM wins for fields edited while offline)
maki writeback --volume "Archive 2025"

# 2. Pull any CaptureOne/Lightroom edits from XMP
maki refresh --volume "Archive 2025"
```

Order matters: writeback first ensures DAM edits land in the XMP files. Then refresh picks up anything the external tool changed independently.

### Monitoring flags

```
maki writeback --log --time --json
```

## Sync Metadata

`maki sync-metadata` performs bidirectional XMP metadata sync in a single command — combining the inbound (refresh) and outbound (writeback) steps with conflict detection.

### Basic usage

```
maki sync-metadata
```

This runs three phases:

1. **Inbound**: Detects externally modified XMP recipe files and re-reads their metadata (keywords, rating, description, color label).
2. **Outbound**: Finds recipes marked `pending_writeback` and writes current DAM metadata back to the XMP file.
3. **Conflict detection**: When both the XMP file changed on disk AND the recipe has pending DAM edits, the recipe is reported as a conflict and skipped.

### Scope to a volume or asset

```
maki sync-metadata --volume "Photos 2024"
maki sync-metadata --asset a1b2c3d4
```

### Include embedded XMP

```
maki sync-metadata --media
```

The `--media` flag adds a third phase that re-extracts embedded XMP from JPEG/TIFF variant files — useful after external tools modify embedded metadata.

### Dry run

```
maki sync-metadata --dry-run --log
```

Shows what would change without modifying any files.

### When to use sync-metadata vs. writeback + refresh

- **`sync-metadata`**: The recommended single command for most workflows. Handles both directions and detects conflicts.
- **`writeback` + `refresh`**: Use separately when you want explicit control over direction (e.g., force DAM edits to win with `writeback --all`, then pull external changes with `refresh`).

## Cleanup

`maki cleanup` scans all file locations and recipes across online volumes, checking whether the referenced files still exist on disk. It removes stale records, locationless variants, and orphaned derived files in eight passes.

### Report mode (safe default)

```
maki cleanup
```

Without `--apply`, cleanup reports what it finds:

```
Cleanup complete: 1500 checked, 12 stale, 3 orphaned assets, 5 orphaned previews, 2 orphaned embeddings
  Run with --apply to remove stale records and orphaned files.
```

### Apply cleanup

```
maki cleanup --apply
```

The eight passes:

1. **Stale location and recipe records**: Removes catalog entries for files that no longer exist on disk (variant file locations and recipe file locations). Updates sidecar YAML files accordingly.
2. **Locationless variants**: Removes variants with zero remaining file locations from assets that still have other located variants. Prevents ghost variants from accumulating after file moves or reimports.
3. **Orphaned assets**: Deletes assets where all variants have zero file locations remaining. Their recipes, variants, faces, embeddings, previews, smart previews, face crops, embedding binaries, catalog rows, and sidecar YAML files are removed.
4. **Orphaned previews**: Removes preview JPEG files whose content hash no longer matches any variant in the catalog.
5. **Orphaned smart previews**: Same for the `smart_previews/` directory.
6. **Orphaned embeddings**: Removes SigLIP embedding binaries whose asset ID no longer exists.
7. **Orphaned face crops**: Removes face crop thumbnails whose face ID no longer exists.
8. **Orphaned ArcFace embeddings**: Removes face embedding binaries whose face ID no longer exists.

### Limit to a specific volume

```
maki cleanup --volume "Photos 2024" --apply
```

Only scans file locations on the specified volume. Useful after removing a drive's contents intentionally.

### Limit to a specific path

```
maki cleanup --path "Capture/2026-02" --apply
maki cleanup --volume "Photos" --path "Archive/Old" --apply --log
```

Scopes stale-location scanning to files under a path prefix. Absolute paths are auto-detected to extract the volume and relative prefix.

### List stale entries

```
maki cleanup --list
```

Prints stale entries to stderr (similar to `--log`, but only shows stale entries rather than every file checked).

### Offline volumes

Offline volumes are skipped automatically with a note. maki never removes records for files on an offline volume – it cannot know whether the file is truly gone or just disconnected.

### Monitoring flags

```
maki cleanup --apply --log --time --json
```

## Relocating Assets

`maki relocate` copies (or moves) all of an asset's files – variants and recipes – to another volume. Use this to migrate assets between drives, create backups on a second volume, or consolidate scattered files.

### Copy to another volume

```
maki relocate a1b2c3d4 "Archive Drive"
```

This copies all files for the asset to the target volume, preserving their relative paths. The asset now has files on both volumes. After the copy, each file is verified via SHA-256 to ensure integrity.

### Move (copy + delete source)

```
maki relocate a1b2c3d4 "Archive Drive" --remove-source
```

With `--remove-source`, the source files are deleted after successful copy and verification. The asset's locations are updated to point to the new volume only.

### Dry run

```
maki relocate a1b2c3d4 "Archive Drive" --dry-run
```

Shows what would be copied or moved without making any changes:

```
Dry run — no changes made:
  Copy Capture/2026-02-01/DSC_001.nef → Archive Drive:Capture/2026-02-01/DSC_001.nef
  Copy Capture/2026-02-01/DSC_001.xmp → Archive Drive:Capture/2026-02-01/DSC_001.xmp
```

## Updating File Locations

`maki update-location` fixes the catalog after you manually moved a single file on disk. Unlike `sync` (which scans a directory), this command targets one specific file.

### Basic usage

```
maki update-location a1b2c3d4 \
    --from Capture/2026-02-01/DSC_001.nef \
    --to /Volumes/PhotosDrive/Archive/2026/DSC_001.nef
```

- `--to` must be an absolute path to the file's current location on disk.
- `--from` can be absolute or volume-relative (the path as it appears in the catalog).

### How it works

1. maki resolves the asset by ID (or unique prefix).
2. The volume is auto-detected from `--to` by matching against registered volume mount points. You can override with `--volume`.
3. maki hashes the file at `--to` and compares it to the stored content hash. If they do not match, the command fails (safety check – you may have pointed to the wrong file).
4. The catalog and sidecar YAML are updated with the new path.

### Specifying the volume explicitly

```
maki update-location a1b2c3d4 \
    --from Capture/old/DSC_001.nef \
    --to /Volumes/NewDrive/Capture/new/DSC_001.nef \
    --volume "New Drive"
```

This also handles recipe file locations, not just variant files.

# Rebuilding the Catalog

`maki rebuild-catalog` wipes the SQLite database and reconstructs it from the YAML sidecar files, which are the source of truth.

```
maki rebuild-catalog
```

Sample output:

```
Rebuild complete: 1847 assets, 2914 variants, 312 recipes, 3 collections, 45 stacks
```

## When to use it

- The SQLite database is corrupted or deleted.
- You manually edited sidecar YAML files and want the catalog to reflect those changes.
- Something seems out of sync and you want a clean slate.

## What is preserved

- **Sidecar YAML files**: Untouched (they are the source, not the target).
- **Collections**: Restored from `collections.yaml` at the catalog root.
- **Stacks**: Restored from `stacks.yaml` at the catalog root (member order and pick assignments are preserved).
- **Faces and people** *(ai feature)*: Restored from `faces.yaml` and `people.yaml`. ArcFace face embeddings restored from binary files in `embeddings/arcface/`.
- **Image embeddings** *(ai feature)*: SigLIP embeddings restored from binary files in `embeddings/<model>/`.
- **Preview files**: Untouched (they are content-addressed by hash).
- **Volumes**: Re-registered from `volumes.yaml`.

## What is regenerated

- All SQLite tables (assets, variants, recipes, file locations, tags, stacks).
- Denormalized columns (`best_variant_hash`, `primary_variant_format`, `variant_count`).
- Collection membership records (from `collections.yaml`).
- Stack membership records (from `stacks.yaml`).

This is a safe operation – only the derived cache (SQLite) is rebuilt. No files on disk are modified or deleted.

# Fix Dates

`maki fix-dates` corrects asset dates that were set incorrectly during import. This commonly happens when files lack EXIF metadata (e.g., old JPEGs without DateTimeOriginal) — the import timestamp was used instead of the capture date.

## How it works

For each asset, `fix-dates` collects candidate dates from all variants:

1. **EXIF DateTimeOriginal** from stored `source_metadata` (for assets imported since v1.3.1)
2. **Re-extracted EXIF** from the file on disk (for older assets without stored date)
3. **File modification time** on disk

The oldest date across all variants becomes the corrected `created_at`.

### Report mode (safe default)

```
maki fix-dates
```

Shows what would be changed:

```
Warning: volume 'Archive' is offline — cannot read files for date extraction
Fix-dates: 5000 checked, 3200 fixed, 800 already correct, 1000 skipped (volume offline)
  Run with --apply to make changes.
  Mount offline volumes and re-run to fix remaining assets.
```

### Apply fixes

```
maki fix-dates --apply --log
```

Updates both the SQLite catalog and sidecar YAML files. Also backfills EXIF dates into variant metadata so future runs work without needing the volume online.

### Offline volumes

`fix-dates` needs file access for EXIF re-extraction and file modification times. Assets on offline volumes are skipped with a clear message. Mount the volume and re-run to fix them.

### Scope to a volume or asset

```
# Fix dates for assets on a specific volume
maki fix-dates --volume "Photos 2024" --apply --log

# Fix a single asset
maki fix-dates --asset a1b2c3d4 --apply
```

## Fix Roles

`maki fix-roles` corrects variant roles in assets that have both RAW and non-RAW variants. In these groups, the RAW variant should be the Original and non-RAW variants should be Exports. If roles are incorrect (e.g., both marked as Original), this command fixes them.

### Report mode (safe default)

```
maki fix-roles
```

Shows what would be changed:

```
Dry run — Fix-roles: 150 checked, 3 fixed (5 variant(s)), 147 already correct
  Run with --apply to make changes.
```

### Apply fixes

```
maki fix-roles --apply
```

Updates both the SQLite catalog and sidecar YAML files.

### Scope to a volume or asset

```
# Only check assets on a specific volume
maki fix-roles --volume "Photos 2024" --apply

# Fix a single asset
maki fix-roles --asset a1b2c3d4 --apply
```

## Recommended Maintenance Workflow

A practical schedule for keeping your catalog healthy:

### 1. Periodic integrity checks

Run `maki verify` on a regular schedule – weekly for active volumes, monthly for archives:

```
# Verify the active working drive
maki verify --volume "Work SSD"

# Find assets that haven't been verified in 90 days
maki search "stale:90"
```

### 2. After editing in external tools

When you finish a session in CaptureOne, Lightroom, or another tool that edits XMP sidecars, run `maki refresh` to pick up the changes:

```
maki refresh --volume "Work SSD" --log
```

If the external tool also modified embedded metadata in JPEG/TIFF exports:

```
maki refresh --media --volume "Work SSD"
```

### 3. After moving or reorganizing files

If you moved files on disk (renamed directories, reorganized folder structure), run `maki sync` to reconcile:

```
# First, see what changed
maki sync /Volumes/PhotosDrive/Photos/

# Then apply
maki sync /Volumes/PhotosDrive/Photos/ --apply
```

If new files appeared (e.g., CaptureOne exported new TIFFs), import them:

```
maki import /Volumes/PhotosDrive/Photos/Exports/
```

### 4. Periodic cleanup

Run `maki cleanup` periodically to remove stale records for files that no longer exist:

```
# See what would be cleaned up
maki cleanup --list

# Apply
maki cleanup --apply
```

### 5. Nuclear option: rebuild

If the catalog seems fundamentally out of sync – searches return wrong results, show displays stale data – rebuild from the sidecar files:

```
maki rebuild-catalog
```

This is safe and fast. The sidecar YAML files are the source of truth; the SQLite database is just a derived cache.

## Putting it all together

A typical maintenance session after reconnecting an archive drive:

```
# 1. Check file integrity
maki verify --volume "Archive 2025" --time

# 2. Push any DAM edits made while the drive was offline
maki writeback --volume "Archive 2025"

# 3. Pick up any recipe changes made while the drive was connected elsewhere
maki refresh --volume "Archive 2025"

# 4. Reconcile any moved/renamed files
maki sync /Volumes/Archive2025/ --apply

# 5. Clean up any stale records
maki cleanup --volume "Archive 2025" --apply

# 6. Confirm everything looks good
maki stats --volumes
```

---

Next: Scripting – shell and Python scripting patterns for workflow automation.

For complete flag and option details on every maintenance command, see the Maintain Commands Reference.

# Scripting

maki's CLI is designed for composability with shell scripts and external programs. Every command supports `--json` for structured output, `-q` for piping asset IDs, and stderr/stdout separation so human messages never pollute machine-readable output.

This chapter covers practical scripting patterns using bash and Python. For interactive multi-step workflows with variables and tab completion, see the Interactive Shell chapter — `maki shell` provides a REPL that eliminates much of the piping and quoting complexity shown here.

---

## Bash Scripting

### Batch Tagging by Path

Tag all assets from a specific session directory:

```
maki search -q "path:Capture/2026-02-15" | xargs -I{} maki tag {} session/february
```

### Conditional Edits

Set a color label on all unrated images:

```
for id in $(maki search -q "type:image rating:0"); do
    maki edit "$id" --label Yellow
done
```

Rate all videos from a specific camera:

```
for id in $(maki search -q 'type:video camera:"NIKON Z 9"'); do
    maki edit "$id" --rating 3
done
```

### Multi-Step Workflows

Import, auto-group, and generate smart previews for a new shoot:

```
#!/usr/bin/env bash
set -euo pipefail

SESSION="/Volumes/Photos/Capture/2026-02-28"

echo "Importing..."
maki import --smart --auto-group "$SESSION" --log --time

echo "Fixing roles..."
```

```
maki fix-roles --apply --log

echo "Done."
maki stats
```

## Exporting for Delivery

Export rated picks to a client folder with sidecars:

```
maki export "rating:4+ collection:ClientProject" /tmp/delivery/ --include-sidecars --log
```

Mirror directory structure to a USB drive:

```
maki export "tag:portfolio" /Volumes/USB/export/ --layout mirror
```

Dry-run first to check what would be exported:

```
maki export "collection:Print" /tmp/test/ --dry-run --json | jq '{files: .files_exported, size: .total_bytes}'
```

## Reporting with jq

Generate a tab-separated report of all 5-star assets with their tags:

```
maki search "rating:5" --json \
  | jq -r '.[] | [.original_filename, .created_at, (.tags // [] | join(", "))] | @tsv'
```

List volumes and their online status:

```
maki volume list --json | jq -r '.[] | "\(.label)\t\(if .is_online then "online" else "offline" end)\t\(.path // "
```

Count assets per format:

```
maki stats --types --json | jq -r '.types.variant_formats[] | "\(.format)\t\(.count)"'
```

## Building Collections from Filters

Create a "Best of 2026" collection from highly-rated images:

```
maki col create "Best of 2026"
maki search -q "rating:4+ dateFrom:2026-01-01 dateUntil:2026-12-31" \
  | xargs maki col add "Best of 2026"
echo "Added $(maki col show 'Best of 2026' -q | wc -l | tr -d ' ') assets"
```

## Verification and Health Checks

Weekly integrity check with notification:

```
#!/usr/bin/env bash
set -euo pipefail

LOG="/tmp/maki-verify-$(date +%Y%m%d).log"
if maki verify --log 2>"$LOG"; then
    echo "Verification passed" >> "$LOG"
else
    echo "VERIFICATION FAILURES DETECTED" >> "$LOG"
    # Add notification here (e.g., mail, Slack webhook)
fi
```

Find assets at risk (only one copy):

```
maki search -q "copies:1" | wc -l
```

### Sync After External Edits

After editing files in CaptureOne or Lightroom:

```
# Detect what changed
maki sync /Volumes/Photos

# Apply catalog updates
maki sync /Volumes/Photos --apply --log

# Re-read modified XMP metadata
maki refresh /Volumes/Photos --log
```

---

## Python Scripting

Python scripts can call `maki` as a subprocess and parse the `--json` output. This is useful for more complex logic that would be unwieldy in bash.

### Helper Functions

A minimal helper for calling maki from Python:

```
import json
import subprocess
import sys

def dam_json(*args):
    """Run a maki command with --json and return parsed output."""
    cmd = ["maki", "--json"] + list(args)
    result = subprocess.run(cmd, capture_output=True, text=True)
    if result.returncode != 0:
        print(f"Error: maki {' '.join(args)}: {result.stderr.strip()}", file=sys.stderr)
        return None
    return json.loads(result.stdout)

def dam_ids(*args):
    """Run a maki search with -q and return a list of asset IDs."""
    cmd = ["maki", "search", "-q"] + list(args)
    result = subprocess.run(cmd, capture_output=True, text=True)
    if result.returncode != 0:
        return []
    return [line.strip() for line in result.stdout.strip().splitlines() if line.strip()]
```

### Example: Tag Analysis Report

Find tags that are only used once (potential typos or inconsistencies):

```
stats = dam_json("stats", "--tags")
if stats and stats.get("tags"):
    for tag in stats["tags"].get("top_tags", []):
```

```
        if tag["count"] == 1:
            print(f"  Singleton tag: {tag['tag']}")
```

### Example: Cross-Volume Backup Audit

Check which assets exist on only one volume:

```
at_risk = dam_ids("copies:1")
print(f"Assets with only 1 copy: {len(at_risk)}")

for asset_id in at_risk[:10]:  # show first 10
    details = dam_json("show", asset_id)
    if details:
        filename = details["variants"][0]["original_filename"]
        print(f"  {filename}")
```

### Example: Batch Operations with Progress

Apply ratings based on an external CSV file:

```
import csv

with open("ratings.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        asset_id = row["asset_id"]
        rating = row["rating"]
        result = subprocess.run(
            ["maki", "edit", asset_id, "--rating", rating],
            capture_output=True, text=True
        )
        if result.returncode == 0:
            print(f"  {asset_id[:8]}: rated {rating}")
        else:
            print(f"  {asset_id[:8]}: FAILED - {result.stderr.strip()}")
```

---

## Real-World Example: Fix Orphaned XMP Files

The repository includes a complete Python script at `scripts/fix-orphaned-xmp.py` that demonstrates a real-world workflow automation.

**Problem**: XMP sidecar files that ended up in a different directory than their parent RAW file (e.g., XMP stayed in `Capture/` while the RAW was moved to `Selects/`). During import, these became standalone assets of type "other" instead of being attached as recipes.

**Solution**: The script uses maki's CLI to find orphaned XMPs, locates matching RAW files by filename stem, and moves the XMP files to the correct directory. It follows the standard two-phase pattern: make file changes, then let maki reconcile.

```
# Dry run - see what would be moved
python3 scripts/fix-orphaned-xmp.py --path 2026-02

# Apply changes
python3 scripts/fix-orphaned-xmp.py --path 2026-02 --apply
```

```
# Reconcile catalog with file moves
maki sync /Volumes/Photos --apply
maki fix-recipes --apply
```

Key patterns demonstrated in the script:

- **`maki search -q "type:other format:xmp"`** — find assets matching specific criteria
- **`maki show <id> --json`** — get full asset details including file locations
- **`maki volume list --json`** — enumerate volumes and their mount points
- **Dry-run by default** — `--apply` flag opts in to changes (following maki's convention)
- **Path scoping** — `--path` parameter limits the search to a subset of the catalog

---

## Tips

**Avoid quoting pitfalls.** When search queries contain inner quotes (e.g., `tag:"Fools Theater"`), use single quotes for the outer shell argument:

```
maki search -q 'tag:"Fools Theater"'
```

**Use `--json` for reliable parsing.** Human-readable output is designed for readability, not stability. Field positions, formatting, and labels may change between versions. JSON output is versioned and stable.

**Stderr is for humans, stdout is for machines.** Progress messages, warnings, and `--log` output go to stderr. When piping maki's output, only structured results appear on stdout.

**Check exit codes.** maki returns 0 on success and 1 on failure. Commands like `verify` return 1 if any mismatches are found, making them suitable for CI/cron health checks:

```
maki verify --volume "Photos" || echo "Integrity check failed!"
```

**Dry-run first.** Most destructive commands (sync, cleanup, auto-group, fix-roles) default to report-only mode. Always review the dry-run output before adding `--apply`.

---

## Related Topics

- Interactive Shell – `maki shell` REPL with variables, tab completion, script files, and session management
- CLI Conventions – global flags, exit codes, and basic scripting patterns
- Browsing & Searching – search syntax and output format options
- Format Templates Reference – custom output templates for `--format`
- Search Filters Reference – all available filters
- REST API – programmatic access via HTTP

# Interactive Shell

maki includes an interactive shell that keeps catalog state alive between commands, giving near-instant response times and enabling multi-step workflows with named variables, session defaults, tab completion, and script files. Instead of paying the startup cost of locating the catalog, loading configuration, and opening SQLite on every invocation, the shell does this once and reuses it for the entire session.

This chapter covers interactive use, variables, scripting, and practical workflows. For one-shot CLI scripting with bash or Python, see Scripting.

---

## Starting the Shell

### Interactive mode

Launch the shell from within a catalog directory:

```
maki shell
```

Output:

```
maki shell v3.2.2 — type 'help' or 'quit' to exit
photos>
```

The prompt shows the catalog name (the directory basename). Type any maki command without the `maki` prefix.

### Run a script file

Execute a `.maki` script file and exit:

```
maki shell post-import.maki
```

### Run a single command

Run one command and exit, useful in shell scripts or aliases:

```
maki shell -c 'search "rating:5 tag:landscape"'
```

The `-c` mode shares the same parsing and variable support as interactive mode, but exits after processing the command string.

---

## Basic Usage

Inside the shell, type any maki subcommand directly. There is no need to type `maki` before each command:

```
photos> stats
photos> search "tag:portrait rating:4+"
  42 assets found
photos> show abc12345
photos> edit --rating 5 abc12345
```

### Blank lines and comments

Blank lines are silently skipped. Lines starting with `#` are treated as comments:

```
photos> # This is a comment
photos>
photos> stats
```

### History

The shell saves command history to `.maki/shell_history` inside your catalog directory. Use the up/down arrow keys to recall previous commands. History persists across sessions.

### Keyboard shortcuts

| Key | Action |
|---|---|
| Up/Down | Navigate command history |
| Ctrl-C | Cancel the current line (does not exit) |
| Ctrl-D | Exit the shell (same as `quit`) |
| Ctrl-A | Move cursor to start of line |
| Ctrl-E | Move cursor to end of line |
| Ctrl-K | Delete from cursor to end of line |
| Ctrl-W | Delete word before cursor |
| Tab | Trigger tab completion |

---

## Variables

The shell supports named variables that hold lists of asset IDs. Variables let you capture command results and reuse them in subsequent commands without re-running queries.

### The implicit _ variable

Every command that produces asset IDs (such as `search`, `duplicates`, or `show`) automatically stores its results in the implicit _ variable. Use _ in the next command to refer to those results:

```
photos> search "tag:landscape rating:4+"
  12 assets found
photos> edit --rating 5 _
  12 assets edited
photos> tag --add "portfolio" _
  12 assets tagged
```

The _ variable is updated after every command that returns assets. It only expands when it appears as a standalone token – underscores inside words like `my_tag` or `_foo` are left alone.

### Named variables

Store results in a named variable with the `$name = command` syntax:

```
photos> $picks = search "rating:5 date:2024"
  38 assets --> $picks
photos> $untagged = search "tags:none type:image"
  142 assets --> $untagged
```

Use variables in any command by referencing `$name`:

```
photos> tag --add "needs-review" $untagged
  142 assets tagged
photos> export $picks /tmp/best
  38 assets exported
```

You can also copy the last result into a named variable:

```
photos> search "rating:5"
  38 assets found
photos> $best = _
  38 assets --> $best
```

### Variable rules

- Names start with `$`, followed by letters, digits, or underscores (`$picks`, `$session_2024`, `$batch1`)
- Variables hold asset ID lists – they are not re-evaluated queries
- Variables persist for the session and are cleared on `reload`
- Expanding an undefined variable leaves it as-is in the command
- **Position does not matter.** When a variable expands, its asset IDs are always placed at the end of the argument list (the trailing positional slot where commands expect asset IDs). This means `tag --add portfolio $picks` and `tag $picks --add portfolio` produce the same result. You never need to worry about where you place the variable in the command.

### Listing and removing variables

Use `vars` to see all defined variables and their counts:

```
photos> vars
  Session defaults: --log
  _ = 38 assets
  $picks = 38 assets
  $untagged = 142 assets
```

Remove a variable with `unset`:

```
photos> unset $untagged
  Removed $untagged
```

---

## Session Defaults

Use `set` to add flags that are automatically injected into every subsequent command. This is useful when you want `--log` or `--json` output for an entire session without typing it each time.

### Setting defaults

```
photos> set --log
  Session default: --log
  Active defaults: --log
photos> set --json
  Session default: --json
  Active defaults: --json --log
```

The settable flags are: `--json`, `--log`, `--debug`, and `--time`.

After setting `--log`, every command will include progress output automatically:

```
photos> set --log
photos> generate-previews "rating:5"
  [1/38] abc12345 -- generated preview
  [2/38] def67890 -- generated preview
  ...
```

### Removing defaults

```
photos> unset --log
  Removed session default: --log
```

Session defaults are also cleared by `reload`.

**Tip:** If a session default conflicts with what a particular command needs, you do not need to unset it first. The shell will not inject a flag that already appears in the command line.

---

## Tab Completion

The shell provides context-aware tab completion powered by rustyline. Press Tab at any point to see suggestions.

### What gets completed

| Context | Completes | Example |
|---|---|---|
| First word | Subcommand names and built-in commands | `sea` -> `search` |
| `--` prefix | Common flags | `--j` -> `--json` |
| `$` prefix | Defined variable names | `$pi` -> `$picks` |
| `tag:` prefix | Tag names from the catalog | `tag:land` -> `tag:landscape` |
| `volume:` prefix | Volume labels from the catalog | `volume:Ph` -> `volume:Photos` |
| After `--volume` | Volume labels | `--volume Ph` -> `--volume Photos` |

Volume labels containing spaces are automatically quoted in completions:

```
photos> search volume:<Tab>
volume:Photos  volume:"External SSD"  volume:Archive
```

Completion data is loaded from the catalog when the shell starts. Use `reload` to refresh it after adding new tags or volumes.

---

# Script Files

A `.maki` script file is a sequence of commands, one per line. Scripts support the same syntax as interactive mode: variables, comments, blank lines, and `source`.

## Basic script format

```
# post-import.maki -- run after each shoot day
import /Volumes/Cards/DCIM --log
auto-group
generate-previews --log
auto-tag --apply --log
describe "description:none" --apply --log
stats
```

Execute it:

```
maki shell post-import.maki
```

## Variables in scripts

Variables work across lines, making multi-step workflows clean:

```
# curate-and-export.maki
$new = search "imported:today type:image"
auto-group $new
generate-previews $new --log
$picks = search "imported:today rating:4+"
tag --add "selects" $picks
export $picks /tmp/selects
stats
```

## The `--strict` flag

By default, scripts continue after errors. Use `--strict` to stop on the first failure:

```
maki shell --strict critical-workflow.maki
```

In `--strict` mode, the shell exits with code 1 on any error. Without `--strict`, errors are printed with file and line number but execution continues:

```
critical-workflow.maki:3: Error: volume "Offline" is not mounted
```

## The `source` command

Use `source` inside the shell (or another script) to run a script file inline, sharing the current session state:

```
photos> source post-import.maki
```

Variables defined in the sourced script are available after it finishes. Paths are resolved relative to the catalog root directory, or you can use absolute paths:

```
photos> source /path/to/shared/cleanup.maki
```

---

## Session Management

### reload

Re-reads `maki.toml` configuration, refreshes tab completion data (tags and volumes), and clears all variables and session defaults:

```
photos> reload
  Reloaded config, cleared variables and session defaults.
```

Use this after making external changes – editing `maki.toml`, importing from another terminal, or adding volumes.

### help

Prints a quick reference of shell syntax, variable usage, session defaults, and built-in commands:

```
photos> help
```

### quit / exit

End the session. Ctrl-D also exits. Command history is saved automatically on exit.

---

## Quote Handling

The shell uses smart quote handling that lets search filter syntax pass through naturally. The rule is simple:

- **Quotes at the start of a token** are grouping quotes – they are stripped, and their content is treated as a single argument (standard shell behavior).
- **Quotes that appear mid-token** are preserved as part of the value (syntax quotes).

This means search filters with quoted values work without extra escaping:

```
photos> search text:"woman with glasses" tag:portrait
```

The shell splits this into two tokens: `text:"woman with glasses"` and `tag:portrait`. The quotes around `woman with glasses` are preserved because they start mid-token (after `text:`), so the search engine receives them intact.

Compare with grouping quotes, which wrap the entire token from the start:

```
photos> search "tag:landscape rating:4+"
```

Here the outer quotes are stripped (they start at the beginning of the token), producing two arguments: `tag:landscape` and `rating:4+`. This is equivalent to typing them without quotes.

You can mix both styles:

```
photos> search "type:image" camera:"NIKON Z 9" rating:4+
```

This produces three tokens: `type:image` (grouping quotes stripped), `camera:"NIKON Z 9"` (mid-token quotes preserved), and `rating:4+`.

**Tip:** If you are used to the standard CLI where you type `maki search 'text:"query"'`, inside the shell you can drop the outer quotes entirely and just type `search text:"query"`.

---

# Tilde Expansion

The shell expands ~ to your home directory (`$HOME` on Unix, `%USERPROFILE%` on Windows) in any token:

```
photos> export $picks ~/Desktop/delivery
  38 assets exported to /Users/alice/Desktop/delivery
photos> export "rating:5" ~/exports/portfolio --zip
  Export complete: 12 files archived
```

Only ~ at the start of a token is expanded — ~ appearing elsewhere (e.g. inside a search query) is left as-is.

---

# Built-in Commands

Most commands in the shell are maki subcommands run directly. A few commands are **built-ins** handled specially by the shell:

| Command | Purpose |
|---|---|
| `export` | Export assets to a directory or ZIP archive (supports `$var` expansion for multi-ID export) |
| `preview` | Show asset previews in the terminal (supports `$var` and `_`) |
| `help` | Shell help and syntax reference |
| `reload` | Re-read config, clear variables and defaults |
| `set` / `unset` | Manage session defaults and variables |
| `source` | Run a script file inline |
| `vars` | List defined variables |
| `quit` / `exit` | End the session |

## Shell `export`

The `export` command is a built-in that supports variable expansion and all standard export flags:

```
photos> export <query|$var> <target> [--layout flat|mirror] [--all-variants]
        [--include-sidecars] [--dry-run] [--overwrite] [--symlink] [--zip]
```

When a variable expands to multiple asset IDs, all assets are exported in a single operation:

```
photos> $picks = search "rating:5 tag:landscape"
  24 assets --> $picks
photos> export $picks ~/Desktop/landscapes
  24 assets exported
photos> export $picks ~/Desktop/landscapes.zip --zip
  Export complete: 24 files archived
```

---

# Practical Examples

## Curate and export workflow

Browse a recent shoot, refine selections, and export:

```
photos> $candidates = search "date:2024-06 type:image rating:3+"
  287 assets --> $candidates
```

```
photos> $portraits = search "date:2024-06 tag:portrait rating:4+"
  42 assets --> $portraits
photos> tag --add "june-selects" $portraits
photos> export $portraits /tmp/june-portraits
  42 assets exported
```

## Post-import script

Save this as `post-import.maki` and run after every import:

```
# post-import.maki
$new = search "imported:today"
auto-group $new
generate-previews $new --log
auto-tag "tags:none type:image" --apply --log
describe "description:none" --apply --log
stats
```

Run it:

```
maki shell post-import.maki
```

Or source it during an interactive session:

```
photos> import /Volumes/Cards/DCIM --log
photos> source post-import.maki
```

## Audit workflow

Investigate and fix grouping issues:

```
photos> set --log
photos> $scattered = search "scattered:2 variants:3+"
  15 assets --> $scattered
photos> show $scattered
  ... review each asset ...
photos> split abc12345 --variants def678,ghi901
photos> auto-group $scattered
```

## Batch tagging with variables

Tag assets from multiple searches without re-querying:

```
photos> $landscapes = search "tag:landscape rating:4+"
  85 assets --> $landscapes
photos> $portraits = search "tag:portrait rating:4+"
  42 assets --> $portraits
photos> tag --add "portfolio" $landscapes
photos> tag --add "portfolio" $portraits
photos> $portfolio = search "tag:portfolio"
  127 assets --> $portfolio
photos> export $portfolio /tmp/portfolio
```

## Quick one-liners with `-c`

Use `-c` mode for cron jobs or shell aliases:

```
# Daily stats check
maki shell -c 'stats'

# Export today's picks
maki shell -c '$picks = search "imported:today rating:4+"; export $picks /tmp/daily'

# Verify with logging
maki shell -c 'set --log; verify'
```

---

## Blocked Commands

Four commands are blocked inside the shell because they conflict with the shell session:

| Command | Reason |
|---|---|
| `init` | Creates a new catalog; the shell is already inside one |
| `migrate` | Schema migration should be run standalone |
| `serve` | Starts a long-running web server that conflicts with the shell loop |
| `shell` | No nesting; use `source` to run script files instead |

Attempting to run a blocked command prints a clear message:

```
photos> serve
'serve' cannot be used inside the shell.
```

---

## Tips

**The shell prompt tells you what is in memory.** When named variables are defined, the prompt shows their counts in brackets:

```
photos [picks=38 untagged=142]> tag --add "review" $untagged
```

**Variables are snapshots, not live queries.** If you run `$picks = search "rating:5"` and then rate more assets, `$picks` still holds the original results. Run the assignment again to refresh.

**Use `set --time` to profile commands.** This is handy when exploring performance differences between queries during an interactive session.

**Scripts are composable.** A master script can `source` other scripts, building complex workflows from reusable pieces:

```
# nightly.maki
source post-import.maki
source cleanup.maki
source verify.maki
```

**Ctrl-C is safe.** It cancels the current input line but does not exit the shell or lose your variables. Use Ctrl-D or `quit` to exit.

---

## Related Topics

- Scripting – bash and Python scripting with `maki --json` and `jq`
- Browsing & Searching – search syntax and output format options
- CLI Conventions – global flags, exit codes, and output conventions
- Search Filters Reference – all available search filters
- Configuration Reference – `maki.toml` settings including `[import]` and `[serve]` sections

# Organizing and Culling

You shoot thousands of images but only show dozens. Every growing collection eventually faces the same challenge: separating the work you want to see from the work you don't – without losing anything permanently. This chapter covers practical strategies for culling and the default filter feature that ties them together.

---

## Three Populations

In any large catalog, your assets fall into three groups:

1. **Picks** – curated keepers you actively browse, rate, and share.
2. **Backlog** – unreviewed imports you haven't looked at yet.
3. **Set aside** – images you've consciously decided to exclude from everyday browsing (rejects, duplicates, test shots, outtakes).

The goal of a culling workflow is to move assets between these populations efficiently, without deleting anything prematurely.

---

## Rating vs. Curation

Star ratings and inclusion/exclusion serve different purposes. Conflating them leads to awkward workarounds.

**Rating** answers "how good is this image among my keepers?" It is a quality scale within your working set. A 3-star landscape and a 5-star portrait are both part of your active catalog – they just differ in quality or prominence.

**Curation** answers "should this image appear in my browsing views at all?" It is a visibility gate. A blurry test shot and a duplicate JPEG are not bad images that deserve 1 star – they are images you don't want to see while browsing.

Keep these concerns separate. Use ratings for quality within your keepers. Use tags, labels, or the absence of a rating for curation.

---

## Workflow Approaches

There is no single correct approach. Choose one that matches how you work, and stay consistent within a project.

### Tag-based culling

Add a `rest` tag to images you want to hide. Set a default filter of `-tag:rest` so they disappear from browsing. Everything else – rated or not – stays visible.

```
[browse]
default_filter = "-tag:rest"
```

**Strengths:** Simple to understand. Unreviewed imports remain visible by default, so you naturally encounter them. Opt-in exclusion means you only have to act on what you want to hide.

**When to use it:** Most users, especially if you import sporadically and want new work to show up immediately.

### Rating-based culling

Rate your keepers 1–5. Leave rejects and backlog unrated. Set a default filter of `rating:1+` so only rated images appear.

```
[browse]
default_filter = "rating:1+"
```

**Strengths:** Clean browsing experience – only reviewed work appears. Ratings carry meaning across your entire catalog.

**Trade-off:** Unreviewed imports disappear until you rate them. You must do an explicit review pass before new work shows up in the browse grid.

### Color-label workflow

Use color labels for workflow state rather than aesthetic preference. For example: Red = reject, Green = approved, Yellow = needs review.

```
[browse]
default_filter = "-label:Red"
```

**Strengths:** Visual and quick in the web UI (keyboard shortcuts r/o/y/g/b for labels). Leaves ratings free for quality scoring. Works well in teams where labels signal handoff state.

### Combined approaches

You can combine filters. For example, show everything that is either rated or not yet tagged as `rest`:

```
[browse]
default_filter = "rating:1+ OR -tag:rest"
```

Or hide Red-labeled rejects and rest-tagged shots:

```
[browse]
default_filter = "-label:Red -tag:rest"
```

---

## The Default Filter

The `[browse] default_filter` option in `maki.toml` applies a search filter automatically to all browse, search, and stroll views.

## What it does

When set, every browse page load, search query, and stroll session starts with this filter pre-applied. Assets that don't match the filter are hidden from view – but they remain in the catalog and are fully accessible to CLI commands.

## How to configure it

Add a `[browse]` section to your `maki.toml`:

```
[browse]
default_filter = "-tag:rest"
```

The value is any valid search filter string – the same syntax you use with `maki search`. See the Search Filters Reference for the full syntax.

## Toggling in the web UI

The web UI shows a checkbox in the filter bar when a default filter is configured. Unchecking it temporarily disables the filter for the current session, letting you see everything. Re-checking it restores the filter. This makes it easy to peek at hidden assets without editing `maki.toml`.

## Scope

The default filter applies to **browsing views only**: the browse grid, search results, and the stroll page. It does **not** affect operational commands like `maki export`, `maki describe`, `maki verify`, or `maki search` on the CLI. Those commands always operate on the full catalog unless you explicitly pass a query.

## Examples for common workflows

```
# Hide images tagged "rest" (recommended starting point)
[browse]
default_filter = "-tag:rest"

# Show only rated images
[browse]
default_filter = "rating:1+"

# Hide Red-labeled rejects
[browse]
default_filter = "-label:Red"

# Show only images, hide videos and documents
[browse]
default_filter = "type:image"

# Combine: hide rest-tagged and Red-labeled
[browse]
default_filter = "-tag:rest -label:Red"
```

---

# Practical Workflow Examples

## Wedding photographer

1. Import the shoot: `maki import /Volumes/Cards/DCIM --log`
2. Quick scan in the web UI. Tag obvious rejects (out of focus, closed eyes, test flashes) as `rest` using keyboard shortcuts or batch select.
3. Rate keepers: 3 = good, 4 = strong, 5 = hero shots.
4. Deliver 4–5 star images to client: `maki export "rating:4+" /Volumes/Delivery/`
5. Build a portfolio collection from the best: `maki search -q "rating:5" | xargs maki col add "Wedding Portfolio"`

Default filter: `-tag:rest`

## Travel and personal

1. Import after a trip: `maki import ~/Photos/Iceland/ --log`
2. Browse at leisure. Rate favorites as you go – no pressure to review everything immediately.
3. Revisit unrated images when you feel like it. Tag anything truly unwanted as `rest`.
4. Over time, the unrated backlog shrinks naturally.

Default filter: `-tag:rest`

## Stock and archive

1. Import with descriptive tags: `maki import /Volumes/Stock/ --add-tag stock --log`
2. Use `maki describe` to generate AI descriptions for searchability.
3. Organize into collections by theme: "Architecture", "Food", "Travel".
4. Rate for commercial potential: 5 = lead image, 4 = strong, 3 = filler.
5. Default filter hides low-rated filler from the browse grid.

Default filter: `rating:3+`

## Art and portfolio

1. Import selectively – only work you consider keeping.
2. Rate carefully for portfolio tiers:
   - 5 = hero image (exhibition, cover)
   - 4 = portfolio (website gallery)
   - 3 = extended portfolio (social media)
   - 2 = decent (personal archive)
   - 1 = keeper (not for sharing)
3. Build saved searches for each tier: `maki ss save "Portfolio" "rating:4+" --sort date_desc`
4. Use collections for specific series or exhibitions.

Default filter: `rating:2+` (hide the bottom tier from daily browsing)

---

# Tips

**Start simple.** A single `rest` tag and `-tag:rest` default filter covers most needs. You can add ratings, labels, and more structure later without changing what you've already done.

**The `rest` tag approach is recommended for most users** because unreviewed images stay visible by default. You only act on what you want to hide, which is usually the smaller set.

**Bulk-tag the remainder after a culling session.** After rating your keepers from a shoot, tag everything unrated as `rest` in one command:

```
maki search -q "path:Capture/2026-03-15 -rating:1+" | xargs maki tag rest
```

This clears the backlog for that shoot without deleting anything.

**Ratings are personal and subjective.** Be consistent within a project, but don't worry about cross-project consistency. A 4-star wedding photo and a 4-star landscape don't need to be comparable.

**Use saved searches for recurring views.** Rather than remembering filter syntax, save your common queries:

```
maki ss save "Portfolio candidates" "rating:4+ -tag:rest" --sort date_desc
maki ss save "Unreviewed this month" "date:2026-03 -rating:1+ -tag:rest" --sort date_desc
maki ss save "Needs keywords" "-tag:rest rating:1+ tag:untagged" --sort date_desc
```

**Don't delete – set aside.** The `rest` tag is reversible. Deletion is not. You can always reconsider an image later by toggling off the default filter in the web UI.

---

Previous: Interactive Shell | Back to Manual

# CLI Conventions

This page documents the global flags, output behavior, exit codes, and scripting patterns that apply across all maki commands.

## Global Flags

Five flags are available on every command. They can appear before or after the subcommand name.

### `--json`

Switches output to machine-readable JSON on stdout. Human-readable status messages are sent to stderr so they do not interfere with JSON parsing.

```
maki search "rating:5" --json
maki import /path/to/files --json
maki stats --json
```

All data types (`SearchRow`, `AssetDetails`, `ImportResult`, `VerifyResult`, `SyncResult`, `CleanupResult`, `RelocateResult`, `DuplicateEntry`, etc.) implement `serde::Serialize` and produce well-formed JSON.

### `-l` / `--log`

Enables per-file progress logging to stderr. Applies to all multi-file commands: import, verify, sync, sync-metadata, refresh, cleanup, generate-previews, relocate, writeback, dedup, export, delete, fix-roles, fix-dates, fix-recipes, auto-group, describe, and `maki serve` (logs HTTP requests).

Each file prints a status line to stderr in the format `filename -- status (duration)`.

```
maki import /Volumes/Photos/2026 --log
maki verify --volume "Archive" --log
maki serve --log
```

### `-v` / `--verbose`

Shows operational decisions and program flow to stderr. Useful for understanding what a command is doing without the low-level detail of `--debug`. Examples of verbose output:

- **import**: number of resolved files, detected volume, exclude patterns, auto-tags
- **describe**: VLM endpoint, model, mode, concurrency, candidate count
- **search**: parsed query, result count
- **preview generation**: source format, generation method (image/RAW/video/info card), max edge size

`--debug` implies `--verbose` — you never need to pass both.

```
maki import /Volumes/Photos/2026 --verbose
maki describe "rating:5" --verbose
maki search "tag:landscape" --verbose
```

### -d / --debug

Shows stderr output from external tools (ffmpeg, dcraw, dcraw_emu) and implies `--verbose`. Useful for diagnosing preview generation issues. Prints both the command line and the tool's stderr via `eprintln`.

```
maki generate-previews --force --debug
maki import /path/to/raw-files --debug
```

### -t / --time

Prints total elapsed wall-clock time after command execution.

```
maki import /Volumes/Photos/2026 --time
# ...
# Elapsed: 12.34s
```

Flags can be combined freely:

```
maki import /path --json --log --time
```

## Catalog Discovery

maki locates the active catalog by searching for a `maki.toml` file:

1. Check the current working directory.
2. Walk up through parent directories until one containing `maki.toml` is found.
3. If the filesystem root is reached without finding one, exit with an error.

```
cd ~/Photos
maki stats            # works -- maki.toml is here

cd ~/Photos/metadata
maki stats            # works -- finds maki.toml in parent

cd /tmp
maki stats            # fails -- no maki.toml above /tmp
```

Error message when no catalog is found:

```
Error: No maki catalog found. Run `maki init` to create one.
```

## Asset ID Matching

Most commands that accept an asset ID (e.g. `show`, `edit`, `relocate`, `generate-previews --asset`) support **unique prefix matching**. You do not need to type the full UUID – any unambiguous prefix is enough.

```
# Full UUID
maki show a1b2c3d4-e5f6-7890-abcd-ef1234567890

# Unique prefix (works if only one asset ID starts with "a1b2c")
maki show a1b2c
```

```
# Ambiguous prefix (multiple matches) -- maki reports an error
maki show a1
```

This applies to both command arguments and filter values where asset IDs are expected.

## Exit Codes

| Code | Meaning |
|---|---|
| **0** | Success. Command completed without errors. |
| **1** | Failure. Examples: `verify` found hash mismatches, a referenced asset/volume was not found, a required argument was missing, or any other command error. |

Standard Rust/clap error handling applies for invalid arguments, missing subcommands, and unknown flags – these also exit with a non-zero code and print usage help to stderr.

## Output Conventions

maki separates machine output from human messages:

| Stream | Content |
|---|---|
| **stdout** | Command results: search results, asset details, JSON output, format-template output |
| **stderr** | Progress messages, per-file logs (`--log`), debug output (`--debug`), timing (`--time`), warnings, and errors |

This separation means you can safely pipe or redirect stdout without capturing status noise:

```
maki search -q "tag:landscape" > asset-ids.txt
maki stats --json | jq '.total_size'
```

### Search result count

`maki search` prints a result count header by default (e.g. `Found 42 assets`). This header is suppressed when an explicit `--format` is given (including `-q`, which is shorthand for `--format=ids`), keeping output clean for scripting.

## Scripting Patterns

### Get just IDs for piping

```
maki search -q "tag:landscape"
```

`-q` is shorthand for `--format=ids` and prints one UUID per line with no header.

### JSON processing with jq

```
maki search "rating:5" --json | jq '.[].id'
maki show abc123 --json | jq '.variants[].filename'
maki stats --json | jq '{assets: .assets, size: .total_size}'
```

### Pipe search results into a collection

```
maki search -q "rating:5 tag:travel" | xargs maki col add "Travel Best"
```

### Batch operations via shell loop

```
for id in $(maki search -q "type:video"); do
  maki edit "$id" --label Blue
done
```

### Dry-run before applying

Several commands default to report-only mode and require `--apply` to make changes:

```
# Preview what would be imported
maki import --dry-run /path/to/files

# See what auto-group would merge (no --apply = dry-run)
maki auto-group "tag:landscape"
maki auto-group "tag:landscape" --apply    # actually merge

# See what sync would update (no --apply = report-only)
maki sync /Volumes/Photos
maki sync /Volumes/Photos --apply          # actually update

# See what cleanup would remove (no --apply = report-only)
maki cleanup
maki cleanup --apply                       # actually remove stale records
```

### Combining flags for verbose dry-runs

```
maki import --dry-run /path/to/files --log --time --json
maki sync /Volumes/Photos --log --json
```

## Safe Defaults

Commands that modify or delete data use conservative defaults:

| Command | Default behavior | Flag to commit changes |
|---|---|---|
| `sync` | Report-only (no changes) | `--apply` |
| `cleanup` | Report-only (no changes) | `--apply` |
| `auto-group` | Report-only (no changes) | `--apply` |
| `fix-roles` | Report-only (no changes) | `--apply` |
| `fix-dates` | Report-only (no changes) | `--apply` |
| `volume remove` | Report-only (no changes) | `--apply` |
| `volume combine` | Report-only (no changes) | `--apply` |
| `import` | Imports immediately | `--dry-run` to preview |
| `relocate` | Copies immediately | `--dry-run` to preview |

The pattern is consistent: commands that scan and potentially alter many records default to showing you what they *would* do. You opt in to changes with `--apply`. Commands that operate on explicitly named files (import, relocate) run immediately but offer `--dry-run` for previewing.

# Command Categories

Quick reference of all maki commands, organized by workflow stage:

| Category | Commands | Purpose |
|---|---|---|
| **Setup** | `init`, `volume add`, `volume list`, `volume combine`, `volume remove` | Create catalog, register and manage storage volumes |
| **Ingest** | `import`, `tag`, `edit`, `group`, `auto-group` | Bring files in, apply metadata, merge variants |
| **Organize** | `collection` (`col`), `saved-search` (`ss`) | Curate static and smart albums |
| **Retrieve** | `search`, `show`, `export`, `duplicates`, `stats`, `serve` | Find assets, inspect details, export files, browse in web UI |
| **Maintain** | `verify`, `sync`, `refresh`, `cleanup`, `relocate`, `update-location`, `generate-previews`, `fix-roles`, `fix-dates`, `rebuild-catalog` | Integrity checks, disk reconciliation, housekeeping |

Commands with aliases are shown with the alias in parentheses. For example, `maki col add` is equivalent to `maki collection add`, and `maki ss run` is equivalent to `maki saved-search run`.

---

Next: Setup Commands – `init`, `volume add`, `volume list`, `volume combine`, `volume remove`.

# Setup Commands

Commands for initializing a catalog and registering storage volumes.

---

# maki init

## NAME

maki-init – initialize a new catalog in the current directory

## SYNOPSIS

```
maki [GLOBAL FLAGS] init
```

## DESCRIPTION

Creates a new maki catalog rooted in the current working directory. This sets up the directory structure, configuration file, SQLite database, and volume registry needed to begin managing assets.

The following files and directories are created:

- `maki.toml` – catalog configuration file (preview settings, serve settings, import exclusions)
- `metadata/` – directory for YAML sidecar files (source of truth for asset metadata)
- `previews/` – directory for generated preview thumbnails
- `catalog.db` – SQLite database (derived cache for fast queries)
- `volumes.yaml` – storage volume registry

If `maki.toml` already exists in the current directory, the command fails with an error to prevent accidental re-initialization.

## ARGUMENTS

None.

## OPTIONS

This command only accepts global flags.

`--json` outputs `{"status": "initialized", "path": "<catalog_root>"}`.

## EXAMPLES

Initialize a catalog in a new directory:

```
mkdir ~/Photos && cd ~/Photos
maki init
```

Initialize and verify with JSON output:

```
cd /Volumes/Archive/PhotoLibrary
maki init --json
# {"status": "initialized", "path": "/Volumes/Archive/PhotoLibrary"}
```

Attempt to re-initialize (fails safely):

```
cd ~/Photos
maki init
# Error: A maki catalog already exists in this directory.
```

## SEE ALSO

volume add – register a storage volume after initialization. CLI Conventions – catalog discovery, global flags, exit codes.

# maki volume add

## NAME

maki-volume-add – register a new storage volume with the catalog

## SYNOPSIS

```
maki [GLOBAL FLAGS] volume add <LABEL> <PATH> [--purpose <PURPOSE>]
```

## DESCRIPTION

Registers a storage volume (a directory tree containing media files) with the catalog. Each volume is assigned a UUID and tracked by its label and mount point path. Volumes allow maki to manage files spread across multiple disks, external drives, and network mounts.

The label is a human-readable name for the volume (e.g., "Photos2026", "Archive", "ExternalSSD"). The path is the mount point or root directory of the volume. The path must exist at the time of registration.

An optional `--purpose` flag assigns a logical role to the volume in the storage hierarchy. This metadata is used by duplicate analysis and backup coverage commands to distinguish between unwanted duplicates (same file on the same working volume) and wanted backups (same file on an archive or backup volume). The purpose can be changed later with `maki volume set-purpose`.

After registration, files under the volume's path can be imported and tracked. If the volume becomes unavailable (e.g., an external drive is disconnected), it is reported as "offline" in `maki volume list`, and commands that need to access its files will skip it gracefully.

## ARGUMENTS

**LABEL (required)** Human-readable name for the volume. Used in `--volume` flags across commands.
**PATH (required)** Absolute path to the volume's mount point or root directory.

## OPTIONS

**`--purpose <PURPOSE>`** Logical role of the volume. Valid values: `working` (active editing drive), `archive` (long-term primary storage), `backup` (redundancy copy), `cloud` (cloud sync folder). Optional — volumes without a purpose are treated as unclassified.

`--json` outputs `{"id": "<uuid>", "label": "<label>", "path": "<path>", "purpose": "<purpose>"}`.

## EXAMPLES

Register an external drive:

```
maki volume add "Photos" /Volumes/PhotoDrive
# Registered volume 'Photos' (a1b2c3d4-e5f6-7890-abcd-ef1234567890)
#   Path: /Volumes/PhotoDrive
```

Register a volume with a purpose:

```
maki volume add "Master Media" /Volumes/MediaDrive --purpose archive
# Registered volume 'Master Media' (e5f67890-...)
#   Path: /Volumes/MediaDrive
#   Purpose: archive
```

Register multiple volumes for a multi-disk workflow:

```
maki volume add "Laptop" /Volumes/MacintoshHD --purpose working
maki volume add "Master Media" /Volumes/MediaDrive --purpose archive
maki volume add "Backup A" /Volumes/BackupDisk --purpose backup
maki volume add "Dropbox" ~/Dropbox/Photos --purpose cloud
```

## SEE ALSO

volume list – list registered volumes and their status. import – import files from a volume. relocate – copy or move asset files between volumes.

---

# maki volume list

## NAME

maki-volume-list – list all registered volumes and their online/offline status

## SYNOPSIS

`maki [GLOBAL FLAGS] volume list`

## DESCRIPTION

Displays all storage volumes registered with the catalog, along with their UUIDs, labels, mount point paths, and current status.

A volume is reported as **online** if its mount point path exists on disk, and **offline** if the path is not accessible (e.g., the drive is disconnected or the network share is unmounted). Offline volumes are silently skipped by commands that access files on disk (import, verify, sync, cleanup, etc.).

## ARGUMENTS

None.

## OPTIONS

This command only accepts global flags.

`--json` outputs an array of `{"id", "label", "path", "volume_type", "purpose", "is_online"}` objects.

## EXAMPLES

List all volumes:

```
maki volume list
# Laptop (a1b2c3d4-...) [online] [working]
#   Path: /Volumes/MacintoshHD
# Master Media (e5f67890-...) [online] [archive]
#   Path: /Volumes/MediaDrive
# Backup A (f1234567-...) [offline] [backup]
#   Path: /Volumes/BackupDisk
```

List volumes as JSON for scripting:

```
maki volume list --json | jq '.[] | select(.is_online) | .label'
```

Check if a specific volume is online:

```
maki volume list --json | jq '.[] | select(.label == "Archive") | .is_online'
```

## SEE ALSO

volume add – register a new volume. volume set-purpose – change a volume's purpose. stats – `--volumes` flag shows per-volume asset counts and sizes. CLI Conventions – catalog discovery rules.

---

# maki volume set-purpose

## NAME

maki-volume-set-purpose – set or clear the logical purpose of a volume

## SYNOPSIS

```
maki [GLOBAL FLAGS] volume set-purpose <VOLUME> <PURPOSE>
```

## DESCRIPTION

Changes the purpose of an existing volume. The purpose describes the volume's role in the storage hierarchy and is used by duplicate analysis and backup coverage commands to distinguish between working copies, archives, and backups.

## ARGUMENTS

**VOLUME (required)** Volume label or UUID.
**PURPOSE (required)** One of `working`, `archive`, `backup`, `cloud`, or `none` (to clear the purpose).

## OPTIONS

This command only accepts global flags.

`--json` outputs `{"id": "<uuid>", "label": "<label>", "purpose": "<purpose>"}`.

## EXAMPLES

Set a volume's purpose:

```
maki volume set-purpose "Photos" archive
# Volume 'Photos' purpose set to: archive
```

Clear a volume's purpose:

```
maki volume set-purpose "Photos" none
# Volume 'Photos' purpose cleared.
```

## SEE ALSO

volume add – register a new volume (with optional `--purpose`). volume list – list volumes and their purposes.

---

# maki volume remove

## NAME

maki-volume-remove – remove a volume and all its associated catalog data

## SYNOPSIS

```
maki [GLOBAL FLAGS] volume remove <VOLUME> [--apply]
```

## DESCRIPTION

Removes a volume and all data associated with it: file location records, recipe records, assets that become orphaned (no remaining file locations on any volume), and preview files for those orphaned assets. Also removes the volume from `volumes.yaml` and the SQLite catalog.

By default, runs in **report-only mode** – shows what would be removed without making changes. Use `--apply` to execute the removal. This is consistent with `cleanup`, `sync`, and `dedup`.

The removal proceeds in phases:

1. **Locations**: Removes all file location records on the volume from the catalog and sidecar YAML files.
2. **Recipes**: Removes all recipe records on the volume from the catalog and sidecar YAML files.
3. **Orphaned assets**: Detects assets where all variants now have zero file locations. In apply mode, deletes these assets along with their variants, remaining recipes, catalog rows, and sidecar YAML files.
4. **Orphaned previews**: Detects preview files whose content hash no longer matches any variant in the catalog. In apply mode, deletes these files.
5. **Volume**: Removes the volume entry from `volumes.yaml` and the SQLite `volumes` table.

In report-only mode, orphaned assets and previews are predicted (what *would* become orphaned if the locations were removed).

## ARGUMENTS

**VOLUME (required)** Volume label or UUID.

## OPTIONS

**`--apply`** Execute the removal. Without this flag, only a report is printed.

This command also accepts global flags. `--json` outputs a `VolumeRemoveResult` object with fields: `volume_label`, `volume_id`, `locations`, `locations_removed`, `recipes`, `recipes_removed`, `orphaned_assets`, `removed_assets`, `orphaned_previews`, `removed_previews`, `apply`, `errors`. `--log` prints per-file progress to stderr.

## EXAMPLES

Preview what removing a volume would do:

```
maki volume remove "Old Drive"
# Volume 'Old Drive' would remove: 1523 locations, 87 recipes, 412 orphaned assets, 412 orphaned previews
#   Run with --apply to remove.
```

Remove the volume:

```
maki volume remove "Old Drive" --apply
# Volume 'Old Drive' removed: 1523 locations removed, 87 recipes removed, 412 orphaned assets removed, 412 orphane
```

JSON output for scripting:

```
maki volume remove "Old Drive" --json
```

## SEE ALSO

volume list – list volumes and their status. cleanup – remove stale records for missing files (works across volumes).

---

# maki volume combine

## NAME

maki-volume-combine – merge a source volume into a target volume by rewriting paths

## SYNOPSIS

```
maki [GLOBAL FLAGS] volume combine <SOURCE> <TARGET> [--apply]
```

## DESCRIPTION

Combines a source volume into a target volume. All file location and recipe records on the source volume are moved to the target volume, with their relative paths rewritten to include the directory prefix that differentiates the two mount points.

The source volume's mount point must be a subdirectory of the target volume's mount point. For example, if the target is `/Volumes/Media` and the source is `/Volumes/Media/media_2024`, the computed prefix is `media_2024/` and all paths are prepended accordingly.

This is useful when you have multiple volumes registered as subdirectories of the same physical disk (e.g., year-based volumes `media_2024`, `media_2025` under `/Volumes/Media`). Since they all go online and offline together, maintaining separate volumes adds complexity without benefit. Combining them into a single disk-level volume simplifies `volume list`, web UI dropdowns, and `backup-status` reporting.

By default, runs in **report-only mode** – shows what would be combined without making changes. Use `--apply` to execute the combination.

In apply mode, the operation proceeds in phases:

1. **Sidecars**: Loads each affected asset's YAML sidecar, rewrites matching variant locations and recipe locations (changes volume ID and prepends path prefix), and saves.
2. **Catalog**: Ensures the target volume exists in the SQLite catalog, then bulk-updates all `file_locations` and `recipes` rows in a single SQL statement each.
3. **Cleanup**: Removes the source volume from `volumes.yaml` and the SQLite `volumes` table.

Sidecars are updated first (source of truth). If the process crashes between sidecar and catalog updates, `maki rebuild-catalog` reconstructs the correct state from sidecars.

## ARGUMENTS

**SOURCE (required)** Label or UUID of the volume to merge away. This volume is removed after combining.

**TARGET (required)** Label or UUID of the volume that receives all locations and recipes.

## OPTIONS

**`--apply`** Execute the combination. Without this flag, only a report is printed.

This command also accepts global flags. `--json` outputs a `VolumeCombineResult` object with fields: `source_label`, `source_id`, `target_label`, `target_id`, `path_prefix`, `locations`, `locations_moved`, `recipes`, `recipes_moved`, `assets_affected`, `apply`, `errors`. `--log` prints per-asset progress to stderr.

## EXAMPLES

Preview what combining would do:

```
maki volume combine "media_2024" "Media"
# Would combine 'media_2024' into 'Media': 4832 locations, 312 recipes (3210 assets, prefix 'media_2024/')
#   Run with --apply to combine.
```

Execute the combination:

```
maki volume combine "media_2024" "Media" --apply
# Volume 'media_2024' combined into 'Media': 4832 locations moved, 312 recipes moved (3210 assets, prefix 'media_2
```

Combine multiple year-volumes into a single disk volume:

```
for year in media_2010 media_2011 media_2012 media_2013 media_2014 media_2015; do
  maki volume combine "$year" "Media" --apply
done
```

## SEE ALSO

volume list – list volumes and verify the result. volume remove – remove a volume and delete its data (rather than merging). rebuild-catalog – reconstruct SQLite from sidecars if needed after a crash.

---

Next: Ingest Commands – `import`, `tag`, `edit`, `group`, `auto-group`.

# Ingest Commands

Commands for importing files, applying metadata, and merging asset variants.

---

# maki import

## NAME

maki-import – import files into the catalog

## SYNOPSIS

```
maki [GLOBAL FLAGS] import [OPTIONS] <PATHS...>
```

## DESCRIPTION

Imports files and directories into the catalog. Each file is hashed (SHA-256), and its metadata is extracted from EXIF data and XMP sidecars. Preview thumbnails are generated during import.

**Stem-based auto-grouping**: Files sharing the same filename stem in the same directory are grouped into one asset. RAW files take priority as the primary variant (defining the asset's identity and EXIF data). Additional media files become extra variants on the same asset.

**Recipe handling**: Processing sidecar files (`.xmp`, `.cos`, `.cot`, `.cop`, `.pp3`, `.dop`, `.on1`) are attached as recipe records to the primary variant rather than imported as standalone assets. Re-importing after external edits updates recipes in place.

**XMP metadata extraction**: When an `.xmp` sidecar is attached, its contents are parsed. Keywords become asset tags, `dc:description` sets the asset description, `xmp:Rating` becomes the asset rating, and `xmp:Label` becomes the color label. EXIF data takes precedence over XMP for overlapping fields.

**Embedded XMP**: JPEG and TIFF files have their embedded XMP metadata extracted automatically (keywords, ratings, descriptions, labels).

**Duplicate handling**: When a file's content hash already exists, the new file location is added to the existing variant. Only truly skips when the exact same location (volume + path) is already recorded.

**Preview generation**: Standard images get 800px JPEG thumbnails via the `image` crate. RAW files use `dcraw`/`dcraw_emu` (LibRaw). Videos use `ffmpeg`. Non-visual formats get an info card. Preview failure never blocks import.

The volume is auto-detected from the first path by matching against registered volume mount points. Use `--volume` to override.

## ARGUMENTS

**PATHS (required)** One or more file paths or directory paths to import. Directories are scanned recursively.

## OPTIONS

**–volume <LABEL>** Use a specific volume instead of auto-detecting from the path. Useful when auto-detection picks the wrong volume.

**–include <GROUP>** Include additional file type groups that are not imported by default. Can be specified multiple times. Example groups: `captureone`, `documents`.

**–skip <GROUP>** Skip file type groups that would normally be imported. Can be specified multiple times. Example groups: `audio`, `xmp`.

**–add-tag <TAG>** Add a tag to every imported asset. Can be specified multiple times. Merged with `[import] auto_tags` from `maki.toml` and XMP-extracted tags, deduplicated.

**–smart** Generate smart previews (high-resolution, 2560px) alongside regular thumbnails during import. Smart previews enable zoom and pan in the web UI. Can be enabled permanently via `[import] smart_previews = true` in `maki.toml`. Smart preview dimensions are controlled by `[preview] smart_max_edge`.

–**embed** ***(feature-gated: requires `--features ai`)*** Generate SigLIP image embeddings for visual similarity search during import. Embeddings enable `maki auto-tag --similar` and the web UI "Find similar" button. Runs as a post-import phase using preview images (smart preview preferred, then regular preview). Silently skips if the AI model is not downloaded – run `maki auto-tag --download` first. Can be enabled permanently via `[import] embeddings = true` in `maki.toml`. Non-image assets are skipped. Uses the model configured in `[ai] model`.

–**describe** Generate VLM descriptions for newly imported assets as a post-import phase. Requires a running Ollama instance (or compatible VLM endpoint configured in `[vlm]`). Runs after the embed phase if both are enabled. Uses the VLM model, prompt, and parameters from `[vlm]` config (including per-model overrides). Concurrency is controlled by `[vlm] concurrency`. Can be enabled permanently via `[import] descriptions = true` in `maki.toml`. Non-image assets are skipped.

–**dry-run** Show what would be imported without writing to catalog, sidecar, or disk. Files are still hashed to detect duplicates. Reports the same counters as a real import (imported, skipped, locations added, recipes attached/updated).

–**auto-group** After importing, automatically group newly imported assets with nearby catalog assets by filename stem. "Nearby" means assets on the same volume whose files are under sibling directories of the imported files (one level up from each import directory). This handles the common Capture-One/Lightroom pattern where RAW originals live in `Capture/` and exports in `Output/` under a shared session folder. Uses the same fuzzy prefix matching as `maki auto-group`. When combined with `--dry-run`, the auto-group phase also runs in dry-run mode.

`--json` outputs an `ImportResult` object with `imported`, `skipped`, `locations_added`, `recipes_attached`, `recipes_updated` counters and a `dry_run` boolean. When `--auto-group` produces matches, an `auto_group` key is added with the full `AutoGroupResult`. When `--embed` generates embeddings, `embeddings_generated` and `embeddings_skipped` keys are added. When `--describe` generates descriptions, `descriptions_generated` and `descriptions_skipped` keys are added.

## EXAMPLES

Import a directory of photos:

```
maki import /Volumes/Photos/Capture/2026-02-22
```

Import with explicit volume and progress logging:

```
maki import --volume "Archive" /Volumes/NAS/Photos/2025 --log --time
```

Preview what would be imported without making changes:

```
maki import --dry-run /Volumes/SD-Card/DCIM
```

Import only image files, skipping audio and XMP sidecars:

```
maki import --skip audio --skip xmp /Volumes/Photos/Mixed
```

Import with smart previews for high-resolution browsing:

```
maki import --smart /Volumes/Photos/Capture/2026-02-22
```

Import with embedding generation for visual similarity search (requires `--features ai`):

```
maki import --embed /Volumes/Photos/Capture/2026-02-22
```

Import with both smart previews and embeddings:

```
maki import --smart --embed /Volumes/Photos/Capture/2026-02-22
```

Tag assets during import:

```
maki import --add-tag landscape --add-tag "2026" /Volumes/Photos/Landscapes
```

Import a CaptureOne session and auto-group RAW+exports:

```
maki import --auto-group /Volumes/Photos/2026-02-22/Capture /Volumes/Photos/2026-02-22/Output
```

Import with JSON output for scripting:

```
maki import /Volumes/Photos/NewShoot --json | jq '.imported'
```

## SEE ALSO

tag – add or remove tags after import. edit – set name, description, rating, or label. auto-group – group related assets by filename stem. generate-previews – regenerate or upgrade previews. CLI Conventions – `--log`, `--json`, `--time` behavior.

---

# maki delete

## NAME

maki-delete – remove assets from the catalog

## SYNOPSIS

`maki [GLOBAL FLAGS] delete [OPTIONS] [ASSET_IDS...]`

## DESCRIPTION

Removes assets from the catalog. By default runs in **report-only mode** – shows what would be deleted without making changes. Use `--apply` to execute the deletion.

When `--apply` is set, the following data is removed for each asset:

- The asset row from the SQLite catalog
- All variants belonging to the asset
- All file location records for those variants
- All recipe records attached to those variants
- All preview and smart preview files
- The YAML sidecar file
- Collection memberships (the asset is removed from all collections)
- Stack membership (the stack auto-dissolves if only one member remains)

With `--remove-files` (which requires `--apply`), physical media files and recipe files are also deleted from disk on online volumes. Files on offline volumes are skipped with a warning.

Asset IDs support unique prefix matching (see CLI Conventions).

When no asset IDs are given on the command line, IDs are read from stdin (one per line). This enables piping from `maki search -q`:

```
maki search -q "orphan:true" | maki delete --apply
```

## ARGUMENTS

**ASSET_IDS (optional)** One or more asset IDs or unique prefixes. If omitted, reads from stdin.

## OPTIONS

–**apply** Execute the deletion. Without this flag, the command only reports what it would do.

–**remove-files** Also delete physical files (variant media and recipe files) from disk. Requires `--apply`. Skips files on offline volumes with a warning.

`--json` outputs a `DeleteResult` with `deleted`, `not_found`, `files_removed`, `previews_removed`, `dry_run`, and `errors`.

`--log` prints per-asset status to stderr.

## EXAMPLES

Preview what would be deleted (report-only):

```
maki delete a1b2c3d4
```

Delete an asset:

```
maki delete --apply a1b2c3d4
```

Delete multiple assets at once:

```
maki delete --apply a1b2c3d4 e5f6a7b8
```

Delete an asset and its files from disk:

```
maki delete --apply --remove-files a1b2c3d4
```

Delete all orphaned assets (no file locations):

```
maki search -q "orphan:true" | maki delete --apply
```

Delete with JSON output for scripting:

```
maki delete --apply a1b2c3d4 --json | jq '.deleted'
```

Use a short ID prefix:

```
maki delete --apply a1b2
```

## SEE ALSO

cleanup – remove stale location records and orphaned assets automatically. search – find assets to delete (`orphan:true`, `missing:true`). show – inspect an asset before deleting it.

---

# maki tag

## NAME

maki-tag – add or remove tags on an asset

## SYNOPSIS

```
maki [GLOBAL FLAGS] tag <ASSET_ID> [--remove] <TAGS...>
```

## DESCRIPTION

Adds or removes tags on an asset. Tags are free-form text strings stored on the asset record. They are persisted in both the YAML sidecar file and the SQLite catalog.

When tags are changed, maki automatically writes the changes back to any `.xmp` recipe files associated with the asset. Tag write-back uses operation-level deltas: added tags are inserted into the XMP `dc:subject` / `rdf:Bag` block, and removed tags are deleted – tags added independently in external tools like CaptureOne are preserved.

Tags are deduplicated: adding a tag that already exists is a no-op.

Asset IDs support unique prefix matching (see CLI Conventions).

## ARGUMENTS

**ASSET_ID (required)** The asset ID or a unique prefix of it.
**TAGS (required)** One or more tags to add or remove.

## OPTIONS

**–remove** Remove the specified tags instead of adding them.

`--json` outputs a `TagResult` with `asset_id`, `tags_added` or `tags_removed`, and the full `tags` list after the operation.

## EXAMPLES

Add tags to an asset:

```
maki tag a1b2c3d4 landscape mountains "golden hour"
```

Remove a tag:

```
maki tag a1b2c3d4 --remove landscape
```

Add a multi-word tag:

```
maki tag a1b2c "Fools Theater"
```

Tag assets in bulk via search:

```
for id in $(maki search -q "path:Capture/2026-02-22"); do
  maki tag "$id" "February 2026"
done
```

## SEE ALSO

edit – set asset name, description, rating, or label. search – `tag:` filter for finding tagged assets. stats – `--tags` shows tag usage frequencies.

# maki edit

## NAME

maki-edit – edit asset metadata (name, description, rating, color label, date, variant role)

## SYNOPSIS

```
maki [GLOBAL FLAGS] edit <ASSET_ID> [OPTIONS]
```

## DESCRIPTION

Sets or clears an asset's name, description, rating, color label, and creation date from the CLI. Can also change the role of a specific variant. At least one option must be provided.

Changes are written to both the YAML sidecar file (source of truth) and the SQLite catalog. Rating, description, and color label changes also trigger XMP write-back to any associated `.xmp` recipe files.

**Rating** is an integer from 1 to 5. Clearing it removes the rating entirely.

**Color label** accepts one of seven colors (case-insensitive): Red, Orange, Yellow, Green, Blue, Pink, Purple. The value is stored in canonical title-case. Clearing it removes the label.

**Name** is a custom display name for the asset. When set, it appears instead of the original filename in search results and the web UI.

**Description** is free-form text. Passing an empty string (`--description ""`) is equivalent to `--clear-description`.

**Date** accepts an ISO date string (e.g. `2024-12-25` or `2024-12-25T14:30:00`). This overrides the asset's `created_at` timestamp. Clearing it is not recommended since the field is always populated at import time.

**Variant role** (`--role` + `--variant`) changes the role of a specific variant within a multi-variant asset. Valid roles: `original`, `alternate`, `processed`, `export`, `sidecar`. This is useful when import or `fix-roles` assigns an incorrect role (e.g. marking a re-exported JPEG as "alternate" when it should be "export"). Role changes update both sidecar YAML and SQLite catalog, and recompute denormalized columns (best preview variant, primary format).

Asset IDs support unique prefix matching (see CLI Conventions).

## ARGUMENTS

**ASSET_ID (required)** The asset ID or a unique prefix of it.

## OPTIONS

–**name <TEXT>** Set the asset's display name.
–**clear-name** Remove the asset's display name (reverts to showing the original filename).
–**description <TEXT>** Set the asset's description. An empty string clears it.
–**clear-description** Remove the asset's description.
–**rating <1-5>** Set the asset's star rating (1 through 5).
–**clear-rating** Remove the asset's star rating.
–**label <COLOR>** Set the asset's color label. Accepts: Red, Orange, Yellow, Green, Blue, Pink, Purple (case-insensitive).
–**clear-label** Remove the asset's color label.
–**date <YYYY-MM-DD>** Set the asset's creation date (accepts ISO date or datetime strings).
–**clear-date** Remove the asset's creation date.
–**role <ROLE>** Change a variant's role. Must be used with `--variant`. Accepts: original, alternate, processed, export, sidecar.

**–variant <HASH>** The content hash of the variant whose role should be changed. Must be used with `--role`. Use `maki show <id>` to find variant hashes.

`--json` outputs an `EditResult` with the fields that were changed and their new values.

## EXAMPLES

Set a rating and description:

```
maki edit a1b2c3d4 --rating 5 --description "Best shot from the wedding ceremony"
```

Set a color label:

```
maki edit a1b2c --label Red
```

Give an asset a display name:

```
maki edit a1b2c3d4 --name "Sunset over Lake Constance"
```

Clear the rating and label:

```
maki edit a1b2c3d4 --clear-rating --clear-label
```

Correct an asset's date:

```
maki edit a1b2c3d4 --date "2024-12-25"
```

Clear the description (two equivalent forms):

```
maki edit a1b2c3d4 --clear-description
maki edit a1b2c3d4 --description ""
```

Change a variant's role from alternate to export:

```
maki edit a1b2c3d4 --role export --variant sha256:abcdef1234567890...
```

## SEE ALSO

tag – add or remove tags. show – display full asset details including edited fields. search – `rating:`, `label:` filters for finding assets. fix-roles – batch re-role non-RAW variants in RAW+non-RAW groups.

---

# maki group

## NAME

maki-group – merge variants into a single asset

## SYNOPSIS

```
maki [GLOBAL FLAGS] group <VARIANT_HASHES...>
```

## DESCRIPTION

Merges multiple variants (identified by their content hashes) into a single asset. This is used to combine files that belong together but were imported as separate assets – for example, a RAW file and its exported TIFF, or multiple processing versions of the same shot.

The target asset is the oldest by creation date among the variants' current assets. All other ("donor") assets have their variants, tags, and recipes merged into the target.

Donor variants that have the role `original` are re-roled to `alternate` to avoid multiple originals on the same asset. This role change is applied in both the YAML sidecar and the SQLite catalog.

After merging, donor assets are deleted (their sidecar YAML files and catalog rows are removed). The target asset's denormalized columns (best variant hash, primary format, variant count) are updated.

## ARGUMENTS

**VARIANT_HASHES (required)** Two or more content hashes (SHA-256 hex strings) of variants to group. Each hash must correspond to an existing variant in the catalog.

## OPTIONS

This command only accepts global flags.

## EXAMPLES

Group a RAW file and its export:

```
maki group abc123def456... 789012fed345...
```

Find variant hashes from asset details and group them:

```
maki show a1b2c --json | jq -r '.variants[].content_hash'
# Use the hashes from two different assets:
maki group <hash1> <hash2>
```

Group three variants (RAW, processed TIFF, final JPEG):

```
maki group abc123... def456... 789abc...
```

## SEE ALSO

split – the inverse operation: extract variants into standalone assets. auto-group – automatically group assets by filename stem. show – display variant hashes for an asset. fix-roles – fix variant roles after grouping.

---

# maki split

## NAME

maki-split – extract variants from an asset into new standalone assets

## SYNOPSIS

```
maki [GLOBAL FLAGS] split <ASSET_ID> <VARIANT_HASHES...>
```

## DESCRIPTION

Splits one or more variants out of an existing asset, creating a new standalone asset for each extracted variant. This is the inverse of `maki group`.

Each extracted variant:

- Becomes the sole variant of a new asset with role `original`.
- Inherits tags, rating, color label, and description from the source asset.
- Takes associated recipes with it.
- Gets a deterministic UUID derived from its content hash.

The source asset retains all non-extracted variants. At least one variant must remain in the source asset.

## ARGUMENTS

**ASSET_ID (required)** Asset ID (or unique prefix) to split.
**VARIANT_HASHES (required)** Content hashes of variants to extract.

## OPTIONS

Standard global flags (`--json`, `--log`, `--time`).

## EXAMPLES

Show variants of an asset:

```
maki show abc12345
```

Extract a specific variant into its own asset:

```
maki split abc12345 sha256:def456...
```

Extract multiple variants (each becomes a separate asset):

```
maki split abc12345 sha256:aaa... sha256:bbb...
```

JSON output for scripting:

```
maki --json split abc12345 sha256:def456...
```

## SEE ALSO

group – the inverse operation: merge variants into a single asset. auto-group – automatically group assets by filename stem. show – display variant hashes for an asset.

---

# maki auto-group

## NAME

maki-auto-group – automatically group assets by filename stem

## SYNOPSIS

`maki [GLOBAL FLAGS] auto-group [QUERY] [--apply]`

## DESCRIPTION

Groups assets by filename stem using fuzzy prefix matching. This handles the common case where export tools (CaptureOne, Lightroom, Photoshop) append suffixes to the original filename: `Z91_8561.ARW` matches `Z91_8561-1-HighRes.tif` because `Z91_8561` is a prefix of `Z91_8561-1-HighRes` and the next character (`-`) is non-alphanumeric (a separator).

**Fuzzy prefix matching rules**: - Two stems match if the shorter is a prefix of the longer and the character immediately after the prefix in the longer string is non-alphanumeric (e.g., `-`, `_`, , `(`). - This prevents false positives: `DSC_001` does not match `DSC_0010` because `0` is alphanumeric. - Stems are compared case-insensitively.

**Chain resolution**: When stems form a chain (e.g., `Z91_8561`, `Z91_8561-1`, `Z91_8561-1-HighRes`), all resolve to the shortest root (`Z91_8561`).

**Target selection** within each group: (1) prefer the asset that has a RAW variant, then (2) the oldest asset by creation date.

Without `--apply`, runs in report-only mode (dry run) and shows what would be grouped. With `--apply`, performs the merging: donor variants are moved to the target asset with their role changed from `original` to `alternate`, tags and recipes are merged, and donor assets are deleted.

An optional search query scopes which assets are considered. Only assets matching the query participate in grouping.

## ARGUMENTS

**QUERY (optional)** A search query (same syntax as `maki search`) to limit which assets participate in grouping.

## OPTIONS

**–apply** Actually perform the grouping. Without this flag, the command only reports what it would do.

`--json` outputs an `AutoGroupResult` with `groups` (array of group details), `total_donors_merged`, `total_variants_moved`, and `dry_run` boolean.

`--log` prints per-group details to stderr.

## EXAMPLES

Preview what auto-group would merge across the entire catalog:

```
maki auto-group
```

Auto-group only assets from a specific import path:

```
maki auto-group "path:Capture/2026-02-22"
```

Apply grouping after reviewing the dry-run report:

```
maki auto-group --apply
```

Auto-group assets with a specific tag:

```
maki auto-group "tag:wedding" --apply --log
```

Auto-group with JSON output for scripting:

```
maki auto-group --apply --json | jq '{merged: .total_donors_merged, moved: .total_variants_moved}'
```

## SEE ALSO

group – manually group specific variants by content hash. fix-roles – fix variant roles after grouping. search – query syntax for scoping auto-group.

---

# maki auto-tag

> **Feature-gated**: requires building with `--features ai`. Not available in default builds. Build with `--features ai-gpu` for CoreML GPU acceleration on macOS (see `[ai] execution_provider` in Configuration).

## NAME

maki-auto-tag – suggest or apply tags to images using AI (SigLIP zero-shot classification, multi-model)

## SYNOPSIS

```
maki [GLOBAL FLAGS] auto-tag [QUERY] [OPTIONS]
maki [GLOBAL FLAGS] auto-tag [OPTIONS] --asset <ID>
maki [GLOBAL FLAGS] auto-tag [OPTIONS] --volume <LABEL>
maki [GLOBAL FLAGS] auto-tag --download [--model <ID>]
maki [GLOBAL FLAGS] auto-tag --remove-model [--model <ID>]
maki [GLOBAL FLAGS] auto-tag --list-models
maki [GLOBAL FLAGS] auto-tag --similar <ASSET_ID>
```

## DESCRIPTION

Uses SigLIP vision-language models to classify images against a configurable tag vocabulary via zero-shot classification. Each image is encoded into an embedding, scored against all label embeddings using sigmoid scoring, and labels above the confidence threshold are suggested as tags.

Two models are available:

| Model ID | Name | Embedding | Size | Notes |
|---|---|---|---|---|
| `siglip-vit-b16-256` | SigLIP ViT-B/16-256 | 768-dim | ~207 MB | Default, good balance |
| `siglip-vit-l16-256` | SigLIP ViT-L/16-256 | 1024-dim | ~670 MB | Higher accuracy |

Select with `--model <id>` or set `[ai] model` in `maki.toml`. The default is `siglip-vit-b16-256`.

By default runs in **report-only mode** – shows suggested tags without applying them. Use `--apply` to write suggested tags to assets.

**Scope required**: at least one scope (positional query, `--asset`, or `--volume`) must be specified to prevent accidental full-catalog processing. Use `""` (empty query) to process all assets.

Model files are downloaded from HuggingFace on first use. Use `--download` to pre-download, `--remove-model` to delete cached files, and `--list-models` to show all known models with download status, size, and active indicator.

**Image selection**: For each asset, the command looks for the best available image in this order: smart preview (2560px) → regular preview (800px) → original file on an online volume. Non-image assets (video, audio, documents) are skipped.

**Embedding storage**: Image embeddings are stored in the SQLite catalog (`embeddings` table) per model. Switching models does not corrupt existing embeddings; each model's embeddings are stored separately. The `--similar` flag uses stored embeddings from the active model to find visually similar assets via an in-memory index (dot product on L2-normalized vectors). Embeddings are also stored opportunistically by the web UI "Suggest tags" and batch "Auto-tag" endpoints. Use `maki embed` to batch-generate embeddings without tagging.

**Default labels**: ~100 photography categories are built in (landscape, portrait, architecture, animals, food, etc.). Override with a custom labels file via `--labels`.

**Prompt template**: Each label is wrapped with a prompt template (default: `"a photograph of {}"`) before text encoding. Configurable via `[ai] prompt` in `maki.toml`.

## OPTIONS

**<QUERY> (positional, optional)** Filter which assets to process using the same search syntax as `maki search`.

**–asset <ID>** Process a single asset by ID (supports prefix matching).

**–volume <LABEL>** Process only assets on a specific volume.

**–model <ID>** Select which SigLIP model to use. Available: `siglip-vit-b16-256` (default), `siglip-vit-l16-256`. Overrides `[ai] model` in `maki.toml`. Also applies to `--download` and `--remove-model`.

**–threshold <FLOAT>** Minimum confidence score to suggest a tag (default: 0.1). Range: 0.0 to 1.0. Higher values produce fewer but more confident suggestions.

**–labels <FILE>** Path to a custom labels file (one label per line). Overrides the built-in default labels.

**–apply** Write suggested tags to assets. Without this flag, tags are only reported.

**–download** Download model files from HuggingFace if not already present. Returns early after download.

**–remove-model** Delete cached model files from disk. Returns early.

**–list-models** Show model download status and file sizes. Returns early.

**–list-labels** Print the active label list (one per line) and exit. Shows the built-in defaults, or the labels from `--labels` / `[ai] labels` config if set. Works without a catalog when using defaults. Pipe to a file to create a custom labels blueprint: `maki auto-tag --list-labels > my-labels.txt`. Supports `--json` (outputs a JSON array).

**–similar <ASSET_ID>** Find the 20 most visually similar assets to the given asset, using stored embeddings. If the target asset has no stored embedding, it is encoded on-the-fly. Other assets must have been previously processed by `auto-tag` or `maki embed`.

`--json` outputs an `AutoTagResult` with `assets_processed`, `assets_skipped`, `tags_suggested`, `tags_applied`, `errors`, `dry_run`, and per-asset `suggestions`.

`--log` prints per-asset status to stderr.

## EXAMPLES

Download the default model (first-time setup):

```
maki auto-tag --download
```

Download the larger model for higher accuracy:

```
maki auto-tag --download --model siglip-vit-l16-256
```

Preview suggested tags for all images (report-only):

```
maki auto-tag ""
```

Auto-tag a specific asset and apply the tags:

```
maki auto-tag --asset a1b2c3d4 --apply
```

Auto-tag images matching a search query with a higher threshold:

```
maki auto-tag "type:image rating:4+" --threshold 0.4 --apply
```

Use the larger model for a specific query:

```
maki auto-tag "tag:unreviewed" --model siglip-vit-l16-256 --apply
```

Use a custom labels file:

```
maki auto-tag --labels my-labels.txt "" --apply
```

Find visually similar images:

```
maki auto-tag --similar a1b2c3d4
```

Export default labels as a blueprint for customization:

```
maki auto-tag --list-labels > my-labels.txt
# Edit the file, then use it:
maki auto-tag --labels my-labels.txt --apply
```

Show model status:

```
maki auto-tag --list-models
```

Auto-tag with JSON output for scripting:

```
maki auto-tag "tag:unreviewed" --json | jq '.suggestions[] | {asset: .asset_id, tags: [.suggested_tags[].tag]}'
```

## SEE ALSO

embed – batch-generate embeddings for similarity search. tag – manually add or remove tags. search – `tag:` filter for finding tagged assets. Configuration – `[ai]` section for default threshold, labels, and prompt template.

---

# maki embed

> **Feature-gated**: requires building with `--features ai`. Not available in default builds. Build with `--features ai-gpu` for CoreML GPU acceleration on macOS (see `[ai] execution_provider` in Configuration).

## NAME

maki-embed – batch-generate image embeddings for visual similarity search

## SYNOPSIS

```
maki [GLOBAL FLAGS] embed [QUERY] [OPTIONS]
maki [GLOBAL FLAGS] embed --asset <ID> [OPTIONS]
maki [GLOBAL FLAGS] embed --volume <LABEL> [OPTIONS]
maki [GLOBAL FLAGS] embed --export
```

## DESCRIPTION

Pre-computes image embeddings for visual similarity search (`maki auto-tag --similar` and the web UI "Find similar" button) without applying any tags. This is useful for building up the similarity search index across your catalog.

For each matching asset, the command finds the best available image (smart preview → regular preview → original file on an online volume), encodes it with SigLIP, and stores the embedding in the SQLite catalog's `embeddings` table and as a binary file under `embeddings/<model>/<prefix>/<asset_id>.bin`.

By default, assets that already have a stored embedding for the active model are skipped. Use `--force` to re-generate embeddings (e.g., after switching to a higher-resolution preview).

**Scope required**: at least one scope (positional query, `--asset`, or `--volume`) must be specified to prevent accidental full-catalog processing. Use `""` (empty query) to process all assets.

**Non-destructive**: embedding generation does not modify any asset metadata, tags, or files. It only writes to the `embeddings` table.

## OPTIONS

**<QUERY> (positional, optional)** Filter which assets to process using the same search syntax as `maki search`.

**–asset <ID>** Process a single asset by ID (supports prefix matching).

**–volume <LABEL>** Process only assets on a specific volume.

**–model <ID>** Select which SigLIP model to use. Available: `siglip-vit-b16-256` (default), `siglip-vit-l16-256`. Overrides `[ai] model` in `maki.toml`.

**–force** Re-generate embeddings even if they already exist for the active model.

**–export** Export all existing embeddings from SQLite to binary files. No scope filter required. Useful as a one-time migration to populate the file-based persistence layer from existing data.

`--json` outputs `{embedded, skipped, errors, model, force}`.

`--log` prints per-asset status to stderr.

## EXAMPLES

Generate embeddings for all assets:

```
maki embed ""
```

Generate embeddings for a single asset:

```
maki embed --asset a1b2c3d4
```

Generate embeddings for a specific volume:

```
maki embed --volume "Photos 2024"
```

Force re-generation with a different model:

```
maki embed "" --model siglip-vit-l16-256 --force
```

Generate embeddings for untagged images only:

```
maki embed "tag:none type:image"
```

Check progress with JSON output:

```
maki embed "" --json | jq '{embedded, skipped}'
```

Export all embeddings to binary files (one-time migration):

```
maki embed --export
```

## SEE ALSO

auto-tag – AI tag suggestion and visual similarity search. Configuration – `[ai]` section for model and model directory settings.

---

# maki describe

## NAME

maki-describe – generate image descriptions using a vision-language model (VLM)

## SYNOPSIS

`maki [GLOBAL FLAGS] describe [QUERY] [OPTIONS]`

## DESCRIPTION

Sends preview images to a VLM server and generates natural language descriptions and/or tags. The command uses the OpenAI-compatible chat completions API, which is implemented by Ollama, LM Studio, vLLM, and most local inference servers.

Three modes are available via `--mode`:

- **describe** (default) — generates a natural language description for each asset.
- **tags** — asks the VLM to suggest tags, returned as a JSON array. Tags are deduplicated (case-insensitive) and existing asset tags are preserved.
- **both** — runs describe and tags as two separate VLM calls per asset, combining the results. This is equivalent to running `--mode describe` and `--mode tags` independently, so each call uses its optimal prompt.

By default, the command runs in **report-only mode**: results are generated and displayed but not saved. Use `--apply` to write descriptions/tags to assets. Use `--dry-run` to see what would be processed without calling the VLM at all.

The command requires at least one scope (positional query, `--asset`, or `--volume`) to prevent accidental processing of the entire catalog.

For each asset, the command: 1. Checks if a description already exists (skips unless `--force` is set; tags mode always runs) 2. Finds the best available image (smart preview > regular preview > original on an online volume) 3. Base64-encodes the image and sends it to the VLM endpoint 4. Parses the response and optionally saves descriptions/tags to the asset

## OPTIONS

**<QUERY> (positional, optional)** Search query to scope which assets are described. Same syntax as `maki search`.

**–asset <ID>** Process a single asset (ID or unique prefix).

**–volume <LABEL>** Limit to assets on a specific volume.

**–model <NAME>** VLM model name. Default from `[vlm] model` in `maki.toml`, or `qwen2.5vl:3b`.

**–endpoint <URL>** VLM server base URL. Default from `[vlm] endpoint` in `maki.toml`, or `http://localhost:11434`.

**–prompt <TEXT>** Custom prompt sent to the VLM. Default from `[vlm] prompt` in `maki.toml`, or a built-in photography-focused prompt. In `--mode both`, custom prompts are ignored because each call uses its specialized built-in prompt.

**–max-tokens <N>** Maximum tokens in the VLM response. Default from `[vlm] max_tokens` in `maki.toml`, or `500`.

**–timeout <SECONDS>** Timeout for each VLM request. Default from `[vlm] timeout` in `maki.toml`, or `300`. Increase for larger models or first-time model loading.

**–mode <MODE>** Output mode: `describe` (default), `tags`, or `both`. In `both` mode, two VLM calls are made per asset — one for description, one for tags.

**–temperature <FLOAT>** Sampling temperature controlling randomness. `0.0` = deterministic (always picks the most likely token), `0.7` = balanced (default), `1.0+` = more creative. Lower values produce more consistent but potentially blander output. Default from `[vlm] temperature` in `maki.toml`, or `0.7`.

–**num-ctx <N>** Context window size passed to the VLM server (Ollama `num_ctx`). When non-zero, overrides the model's default context length. Useful for models that benefit from a larger context (e.g., `--num-ctx 4096`). Default: `0` (server default).
–**top-p <FLOAT>** Nucleus sampling threshold. Only tokens whose cumulative probability exceeds this value are considered. Lower values produce more focused output. Default: `0.0` (server default).
–**top-k <N>** Top-k sampling: limit token selection to the k most likely candidates. Lower values produce more deterministic output. Default: `0` (server default).
–**repeat-penalty <FLOAT>** Repetition penalty factor. Values above `1.0` discourage the model from repeating tokens. Default: `0.0` (server default).
–**apply** Write descriptions and/or tags to assets. Without this flag, results are generated and displayed but not saved.
–**force** Overwrite existing descriptions. By default, assets that already have a description are skipped.
–**dry-run** Show what would be processed without calling the VLM. No network requests are made.
–**check** Test connectivity to the VLM endpoint. Prints the server status and available models, then exits. Does not process any assets.

## EXAMPLES

Check that Ollama is running and see available models:

```
maki describe --check
```

Preview descriptions for undescribed assets (report-only):

```
maki describe "description:none type:image"
```

Generate and apply descriptions to a specific volume:

```
maki describe --volume "Photos 2024" --apply
```

Describe a single asset:

```
maki describe --asset a1b2c3d4 --apply
```

Use a faster model for batch processing:

```
maki describe "date:2024-06" --model moondream --apply
```

Use a remote server or larger model:

```
maki describe --endpoint http://gpu-server:11434 --model qwen2.5vl:7b --apply
```

Generate tags only:

```
maki describe --mode tags "tag:untagged" --apply
```

Generate both descriptions and tags in one pass (two VLM calls per asset):

```
maki describe --mode both --asset a1b2c3d4 --apply
```

Custom prompt for architectural photography:

```
maki describe --prompt "Describe the architectural style, materials, and features." "tag:architecture" --apply
```

Deterministic output (temperature 0) for reproducible batch tagging:

```
maki describe --mode tags --temperature 0 "date:2024-06" --apply
```

Increase timeout for a large model's first load:

```
maki describe --model qwen2.5vl:7b --timeout 300 --asset a1b2c3d4
```

Use a larger context window and nucleus sampling with a specific model:

```
maki describe --model qwen3-vl:4b --num-ctx 4096 --top-p 0.9 --apply
```

Overwrite existing descriptions with a better model:

```
maki describe "rating:4+" --model qwen2.5vl:7b --force --apply
```

Dry run with JSON output:

```
maki describe "rating:4+" --dry-run --json
```

Use a cloud API (OpenAI-compatible endpoint):

```
maki describe ""
```

## SEE ALSO

edit – manually set or clear an asset's description. VLM Model Guide – tested models, backends, hardware guide, and quality comparison. Configuration – `[vlm]` section for endpoint, model, and prompt settings. VLM Setup (User Guide) – how to set up a local VLM server.

---

Previous: Setup Commands – `init`, `volume add`, `volume list`. Next: Organize Commands – `collection`, `saved-search`.

# Organize Commands

Commands for curating static collections, managing saved searches (smart albums), and grouping assets into stacks.

---

# maki collection create

## NAME

maki-collection-create – create a new collection

## SYNOPSIS

`maki [GLOBAL FLAGS] collection create <NAME> [--description <TEXT>]`

Alias: `maki col create`

## DESCRIPTION

Creates a new empty collection. Collections are manually curated lists of asset IDs, similar to static albums in photo management tools. They are backed by SQLite tables for fast queries and a `collections.yaml` file at the catalog root for persistence across `rebuild-catalog`.

Collection names must be unique. Attempting to create a collection with an existing name produces an error.

## ARGUMENTS

**NAME (required)** The name for the new collection.

## OPTIONS

**–description <TEXT>** An optional description for the collection.

`--json` outputs the created collection's details.

## EXAMPLES

Create a simple collection:

```
maki collection create "Best of 2026"
```

Create a collection with a description:

```
maki col create "Wedding Portfolio" --description "Final selects for client delivery"
```

Create with JSON output:

```
maki col create "Travel" --json
```

## SEE ALSO

collection add – add assets to a collection. collection show – view collection contents. search – `collection:` filter for searching within a collection.

---

# maki collection list

## NAME

maki-collection-list – list all collections

## SYNOPSIS

`maki [GLOBAL FLAGS] collection list`

Alias: `maki col list`

## DESCRIPTION

Lists all collections in the catalog, showing each collection's name, description (if any), and the number of assets it contains.

## ARGUMENTS

None.

## OPTIONS

This command only accepts global flags.

`--json` outputs an array of collection objects.

## EXAMPLES

List all collections:

```
maki collection list
```

List collections as JSON and extract names:

```
maki col list --json | jq '.[].name'
```

Count total collections:

```
maki col list --json | jq 'length'
```

## SEE ALSO

collection create – create a new collection. collection show – view a specific collection's contents.

---

# maki collection show

## NAME

maki-collection-show – show the contents of a collection

## SYNOPSIS

`maki [GLOBAL FLAGS] collection show <NAME> [--format <FMT>]`

Alias: `maki col show`

## DESCRIPTION

Displays the assets belonging to a collection. Output format can be customized using the same format presets and template syntax as `maki search`.

## ARGUMENTS

**NAME (required)** The name of the collection to display.

## OPTIONS

**–format <FMT>** Output format. Presets: `ids`, `short` (default), `full`, `json`. Custom templates use `{placeholder}` syntax (e.g., `'{id}\t{name}'`).

## EXAMPLES

Show a collection's contents:

```
maki collection show "Best of 2026"
```

Get just the asset IDs for piping:

```
maki col show "Wedding Portfolio" --format ids
```

Show full details including tags:

```
maki col show "Travel" --format full
```

Export collection as JSON:

```
maki col show "Favorites" --format json | jq '.[].id'
```

## SEE ALSO

collection add – add assets to the collection. collection remove – remove assets from the collection. search – `collection:` filter for searching within collections.

---

# maki collection add

## NAME

maki-collection-add – add assets to a collection

## SYNOPSIS

```
maki [GLOBAL FLAGS] collection add <NAME> <ASSET_IDS...>
```

Alias: `maki col add`

## DESCRIPTION

Adds one or more assets to an existing collection. Asset IDs that are already in the collection are silently ignored (no duplicates are created).

Supports stdin piping for integration with `maki search -q` and shell scripting.

## ARGUMENTS

**NAME (required)** The name of the collection to add assets to.
**ASSET_IDS (required)** One or more asset IDs to add. Also accepts IDs from stdin.

## OPTIONS

This command only accepts global flags.

## EXAMPLES

Add specific assets to a collection:

```
maki collection add "Favorites" a1b2c3d4-... e5f67890-...
```

Pipe search results into a collection:

```
maki search -q "rating:5 tag:travel" | xargs maki col add "Travel Best"
```

Add all 5-star landscape photos to a collection:

```
maki search -q "rating:5 tag:landscape" | xargs maki col add "Portfolio"
```

Add assets from a saved search:

```
maki ss run "Recent Imports" --format ids | xargs maki col add "Review Queue"
```

## SEE ALSO

collection remove – remove assets from a collection. collection show – view collection contents. search – find assets to add.

---

# maki collection remove

## NAME

maki-collection-remove – remove assets from a collection

## SYNOPSIS

`maki [GLOBAL FLAGS] collection remove <NAME> <ASSET_IDS...>`

Alias: `maki col remove`

## DESCRIPTION

Removes one or more assets from a collection. The assets themselves are not deleted – only their membership in the collection is removed. Asset IDs not present in the collection are silently ignored.

## ARGUMENTS

**NAME (required)** The name of the collection to remove assets from.
**ASSET_IDS (required)** One or more asset IDs to remove.

## OPTIONS

This command only accepts global flags.

## EXAMPLES

Remove a single asset from a collection:

```
maki collection remove "Favorites" a1b2c3d4-e5f6-7890-abcd-ef1234567890
```

Remove multiple assets:

```
maki col remove "Review Queue" a1b2c3d4-... e5f67890-...
```

Remove all assets with a certain label from a collection:

```
maki search -q "collection:Portfolio label:Red" --format ids | xargs maki col remove "Portfolio"
```

## SEE ALSO

collection add – add assets to a collection. collection delete – delete the entire collection.

---

# maki collection delete

## NAME

maki-collection-delete – delete a collection

## SYNOPSIS

`maki [GLOBAL FLAGS] collection delete <NAME>`

Alias: `maki col delete`

## DESCRIPTION

Deletes a collection entirely. This removes the collection record and all its membership entries. The assets themselves are not affected – only the collection is removed.

## ARGUMENTS

**NAME (required)** The name of the collection to delete.

## OPTIONS

This command only accepts global flags.

## EXAMPLES

Delete a collection:

```
maki collection delete "Old Review Queue"
```

Delete using the alias:

```
maki col delete "Temporary"
```

Delete with JSON confirmation:

```
maki col delete "Drafts" --json
```

## SEE ALSO

collection create – create a new collection. collection list – list all collections.

---

# maki saved-search save

## NAME

maki-saved-search-save – save a search query with a name

## SYNOPSIS

`maki [GLOBAL FLAGS] saved-search save <NAME> <QUERY> [--sort <SORT>] [--favorite]`

Alias: `maki ss save`

## DESCRIPTION

Saves a search query under a name for later re-use. Saved searches are stored in `searches.toml` at the catalog root and function as smart albums – the results update dynamically as the catalog changes.

If a saved search with the same name already exists, it is replaced.

Saved searches appear as clickable chips in the web UI browse page and can be executed from the CLI with `maki saved-search run`.

## ARGUMENTS

**NAME (required)** A name for the saved search.
**QUERY (required)** The search query string, using the same syntax as `maki search`.

## OPTIONS

–**sort <SORT>** Sort order for results. Values: `date_desc` (default), `date_asc`, `name_asc`, `name_desc`, `size_asc`, `size_desc`.
–**favorite** Mark the saved search as a favorite. Favorite searches are shown prominently as chips on the web UI browse page.

`--json` outputs the saved search entry.

## EXAMPLES

Save a search for highly-rated landscapes:

```
maki saved-search save "Best Landscapes" "tag:landscape rating:4+"
```

Save with a custom sort order:

```
maki ss save "Recent Videos" "type:video" --sort date_desc
```

Save a search using quoted filter values:

```
maki ss save "Canon Portraits" 'camera:"Canon EOS R5" tag:portrait'
```

Save a path-scoped search:

```
maki ss save "February Shoot" "path:Capture/2026-02"
```

## SEE ALSO

saved-search run – execute a saved search. saved-search list – list all saved searches. saved-search delete – delete a saved search. search – query syntax reference.

---

# maki saved-search list

## NAME

maki-saved-search-list – list all saved searches

## SYNOPSIS

`maki [GLOBAL FLAGS] saved-search list`

Alias: `maki ss list`

## DESCRIPTION

Lists all saved searches stored in the catalog, showing each search's name, query, and sort order.

## ARGUMENTS

None.

## OPTIONS

This command only accepts global flags.

`--json` outputs an array of saved search objects.

## EXAMPLES

List all saved searches:

```
maki saved-search list
```

List as JSON:

```
maki ss list --json
```

Count saved searches:

```
maki ss list --json | jq 'length'
```

## SEE ALSO

saved-search save – create or update a saved search. saved-search run – execute a saved search.

---

# maki saved-search run

## NAME

maki-saved-search-run – execute a saved search and display results

## SYNOPSIS

`maki [GLOBAL FLAGS] saved-search run <NAME> [--format <FMT>]`

Alias: `maki ss run`

## DESCRIPTION

Executes a previously saved search by name and displays the results. The stored query is run against the current state of the catalog, so results reflect any changes since the search was saved.

The sort order saved with the search is applied. Output format can be overridden with `--format`.

## ARGUMENTS

**NAME (required)** The name of the saved search to execute.

## OPTIONS

–**format <FMT>** Output format. Presets: `ids`, `short` (default), `full`, `json`. Custom templates use `{placeholder}` syntax.

## EXAMPLES

Run a saved search:

```
maki saved-search run "Best Landscapes"
```

Run and get just IDs for piping:

```
maki ss run "Recent Videos" --format ids
```

Run a saved search and add results to a collection:

```
maki ss run "Best Landscapes" --format ids | xargs maki col add "Portfolio"
```

Run with JSON output:

```
maki ss run "Canon Portraits" --format json | jq '.[].id'
```

## SEE ALSO

saved-search save – create or update a saved search. collection add – add search results to a collection.

---

# maki saved-search delete

## NAME

maki-saved-search-delete – delete a saved search

## SYNOPSIS

`maki [GLOBAL FLAGS] saved-search delete <NAME>`

Alias: `maki ss delete`

## DESCRIPTION

Deletes a saved search by name. The search is removed from `searches.toml`. This does not affect any assets or collections.

## ARGUMENTS

**NAME (required)** The name of the saved search to delete.

## OPTIONS

This command only accepts global flags.

## EXAMPLES

Delete a saved search:

```
maki saved-search delete "Old Query"
```

Delete using the alias:

```
maki ss delete "Temporary Search"
```

Delete with JSON confirmation:

```
maki ss delete "Drafts" --json
```

## SEE ALSO

saved-search save – create a new saved search. saved-search list – list all saved searches.

# maki stack create

## NAME

maki-stack-create – create a new stack from the given assets

## SYNOPSIS

`maki [GLOBAL FLAGS] stack create <ASSET_IDS...>`

Alias: `maki st create`

## DESCRIPTION

Creates a new stack from two or more assets. Stacks are lightweight anonymous groups for visually related images – burst shots, bracketing sequences, similar scenes. The first asset ID becomes the stack's "pick" (displayed in the browse grid).

Each asset can belong to at most one stack. Attempting to stack an asset that is already in a stack produces an error.

Stacks are persisted in `stacks.yaml` at the catalog root and restored during `rebuild-catalog`.

## ARGUMENTS

**ASSET_IDS (required)** Two or more asset IDs. The first ID becomes the pick (position 0).

## OPTIONS

This command only accepts global flags.

`--json` outputs the created stack's details (id, member_count, asset_ids).

## EXAMPLES

Create a stack from three burst shots (first becomes pick):

```
maki stack create a1b2c3d4-... e5f67890-... f1a2b3c4-...
```

Create using the alias with JSON output:

```
maki st create a1b2c3d4-... e5f67890-... --json
```

## SEE ALSO

stack add – add assets to an existing stack. stack pick – change which asset is the pick. stack dissolve – dissolve a stack entirely.

# maki stack add

## NAME

maki-stack-add – add assets to an existing stack

## SYNOPSIS

`maki [GLOBAL FLAGS] stack add <REFERENCE> <ASSET_IDS...>`

Alias: `maki st add`

## DESCRIPTION

Adds one or more assets to an existing stack. The reference asset identifies which stack to add to – it must already be a member of a stack. New assets are appended to the end of the stack's member list.

Assets that are already in a stack (any stack) cannot be added and will produce an error.

## ARGUMENTS

**REFERENCE (required)** Any asset ID that is already in the target stack.
**ASSET_IDS (required)** One or more asset IDs to add to the stack.

## OPTIONS

This command only accepts global flags.

`--json` outputs `{"added": N}` with the count of assets added.

## EXAMPLES

Add an asset to the stack containing a1b2c3d4:

```
maki stack add a1b2c3d4-... new-asset-id-...
```

Add multiple assets:

```
maki st add a1b2c3d4-... b2c3d4e5-... c3d4e5f6-...
```

## SEE ALSO

stack create – create a new stack. stack remove – remove assets from a stack.

# maki stack remove

## NAME

maki-stack-remove – remove assets from their stack

## SYNOPSIS

`maki [GLOBAL FLAGS] stack remove <ASSET_IDS...>`

Alias: `maki st remove`

## DESCRIPTION

Removes one or more assets from their stacks. Each asset is removed from whatever stack it belongs to. If removing an asset causes a stack to have fewer than 2 members, the stack is automatically dissolved.

Assets not currently in a stack are silently skipped.

## ARGUMENTS

**ASSET_IDS (required)** One or more asset IDs to remove from their stacks.

## OPTIONS

This command only accepts global flags.

`--json` outputs `{"removed": N}` with the count of assets removed.

## EXAMPLES

Remove a single asset from its stack:

```
maki stack remove a1b2c3d4-e5f6-7890-abcd-ef1234567890
```

Remove multiple assets:

```
maki st remove a1b2c3d4-... e5f67890-...
```

## SEE ALSO

stack add – add assets to a stack. stack dissolve – dissolve an entire stack at once.

---

# maki stack pick

## NAME

maki-stack-pick – set the pick (top) of a stack

## SYNOPSIS

`maki [GLOBAL FLAGS] stack pick <ASSET_ID>`

Alias: `maki st pick`

## DESCRIPTION

Promotes an asset to position 0 (the "pick") of its stack. The pick is the asset shown in the browse grid when stacks are collapsed. The asset must already be a member of a stack.

## ARGUMENTS

**ASSET_ID (required)** The asset ID to make the pick. Must be in a stack.

## OPTIONS

This command only accepts global flags.

`--json` outputs `{"pick": "<asset_id>"}`.

## EXAMPLES

Set a specific asset as the stack pick:

```
maki stack pick a1b2c3d4-e5f6-7890-abcd-ef1234567890
```

Set pick using the alias:

```
maki st pick a1b2c3d4-...
```

## SEE ALSO

stack show – view current stack members and pick. stack create – the first asset in create becomes the initial pick.

# maki stack dissolve

## NAME

maki-stack-dissolve – dissolve an entire stack

## SYNOPSIS

`maki [GLOBAL FLAGS] stack dissolve <ASSET_ID>`

Alias: `maki st dissolve`

## DESCRIPTION

Dissolves the stack that contains the given asset. All member assets become unstacked. The assets themselves are not affected – only the stack grouping is removed.

## ARGUMENTS

**ASSET_ID (required)** Any asset ID in the stack to dissolve.

## OPTIONS

This command only accepts global flags.

`--json` outputs `{"status": "dissolved"}`.

## EXAMPLES

Dissolve a stack:

```
maki stack dissolve a1b2c3d4-e5f6-7890-abcd-ef1234567890
```

Dissolve using the alias:

```
maki st dissolve a1b2c3d4-...
```

## SEE ALSO

stack remove – remove individual assets instead of dissolving the whole stack. stack list – list all stacks.

---

# maki stack list

## NAME

maki-stack-list – list all stacks

## SYNOPSIS

`maki [GLOBAL FLAGS] stack list`

Alias: `maki st list`

## DESCRIPTION

Lists all stacks in the catalog, showing each stack's ID, member count, creation date, and pick asset ID.

## ARGUMENTS

None.

## OPTIONS

This command only accepts global flags.

`--json` outputs an array of `StackSummary` objects.

## EXAMPLES

List all stacks:

```
maki stack list
```

List as JSON and count stacks:

```
maki st list --json | jq 'length'
```

## SEE ALSO

stack show – view members of a specific stack. stack create – create a new stack.

---

# maki stack show

## NAME

maki-stack-show – show members of a stack

## SYNOPSIS

`maki [GLOBAL FLAGS] stack show <ASSET_ID> [--format <FMT>]`

Alias: `maki st show`

## DESCRIPTION

Displays the ordered member list of the stack containing the given asset. The pick (position 0) is shown first. Output format can be customized using the same format presets and template syntax as `maki search`.

## ARGUMENTS

**ASSET_ID (required)** Any asset ID that belongs to a stack.

## OPTIONS

**–format <FMT>** Output format. Presets: `ids`, `short` (default), `full`, `json`. Custom templates use `{placeholder}` syntax.

## EXAMPLES

Show members of a stack:

```
maki stack show a1b2c3d4-...
```

Get just the member IDs:

```
maki st show a1b2c3d4-... --format ids
```

Show as JSON:

```
maki st show a1b2c3d4-... --json
```

## SEE ALSO

stack pick – change the pick. stack list – list all stacks. search – `stacked:true` filter finds all stacked assets.

---

# maki stack from-tag

## NAME

maki-stack-from-tag – convert matching tags into stacks

## SYNOPSIS

`maki [GLOBAL FLAGS] stack from-tag <PATTERN> [--remove-tags] [--apply]`

Alias: `maki st from-tag`

## DESCRIPTION

Finds assets whose tags match a pattern containing a `{}` wildcard, groups them by the wildcard value, and creates a stack from each group. This automates the common workflow of converting tag-based groupings (e.g., from CaptureOne or Lightroom) into maki stacks.

Without `--apply`, runs in **report-only mode** showing what stacks would be created. With `--apply`, creates the stacks. With `--remove-tags`, the matched tag is removed from each asset after stacking.

## ARGUMENTS

**PATTERN (required)** A tag pattern with `{}` as a wildcard placeholder. For example, `"Aperture Stack {}"` matches tags like "Aperture Stack 1", "Aperture Stack 2", etc.

## OPTIONS

–**remove-tags** Remove the matched tag from each asset after stack creation. Only effective with `--apply`.
–**apply** Actually create stacks. Without this flag, the command only reports what it would do.

## EXAMPLES

Preview what stacks would be created from aperture stack tags:

```
maki stack from-tag "Aperture Stack {}"
```

Create stacks and remove the grouping tags:

```
maki st from-tag "Aperture Stack {}" --remove-tags --apply
```

Create stacks from bracket sequence tags:

```
maki stack from-tag "Bracket {}" --apply
```

## SEE ALSO

stack create – manually create a stack from specific assets. stack dissolve – dissolve stacks if the result is unwanted. tag – manage asset tags.

# maki faces detect

## NAME

maki-faces-detect – detect faces in asset images

## SYNOPSIS

`maki [GLOBAL FLAGS] faces detect [--query <Q>] [--asset <id>] [--volume <label>] [--apply]`

## DESCRIPTION

Detects faces in asset preview images using the YuNet ONNX model. For each detected face, computes a 512-dimensional ArcFace embedding and generates a 150×150 JPEG crop thumbnail. Requires the `ai` feature (`cargo build --features ai`).

Without `--apply`, runs in report-only mode showing how many faces would be detected. With `--apply`, stores face records in the catalog and generates crop thumbnails.

Requires at least one scope filter (`--query`, `--asset`, or `--volume`) to prevent accidental full-catalog processing. Models must be downloaded first with `maki faces download`.

## OPTIONS

–**query <Q>** Search query to scope which assets are processed.
–**asset <id>** Process a single asset by ID.
–**volume <label>** Process assets on a specific volume.
–**min-confidence <FLOAT>** Minimum detection confidence threshold (0.0–1.0, default 0.5). Faces below this confidence are discarded.
–**apply** Actually store detected faces (default is dry run).
–**force** Re-detect faces even if faces already exist for an asset. Without this flag, assets with existing face records are skipped.

`--json`, `--log`, `--time` for output control.

## EXAMPLES

Detect faces in all images (dry run):

```
maki faces detect --query "type:image"
```

Detect and store faces for a single asset:

```
maki faces detect --asset a1b2c3d4 --apply
```

Detect faces on a specific volume with logging:

```
maki faces detect --volume "Photos" --apply --log
```

## SEE ALSO

faces cluster – group detected faces into people. faces download – download required models.

# maki faces cluster

## NAME

maki-faces-cluster – group similar faces into people

## SYNOPSIS

`maki [GLOBAL FLAGS] faces cluster [--query <Q>] [--asset <id>] [--volume <label>] [--threshold <F>] [--apply]`

## DESCRIPTION

Groups similar face embeddings into unnamed person groups using greedy single-linkage clustering. Faces that have already been assigned to a person are skipped.

The threshold controls how similar two faces must be to be grouped together (0.0–1.0, higher = stricter). Default is 0.5, configurable via `[ai] face_cluster_threshold` in `maki.toml`.

Without `--apply`, shows a dry-run report of cluster sizes. With `--apply`, creates person records and assigns faces.

Scope filters (`--query`, `--asset`, `--volume`) limit which faces are considered for clustering.

## OPTIONS

**–query <Q>** Scope clustering to faces on assets matching this query.
**–asset <id>** Scope clustering to faces on a single asset.
**–volume <label>** Scope clustering to faces on assets on a specific volume.
**–threshold <F>** Similarity threshold for clustering (default 0.5).
**–apply** Actually create person groups (default is dry run).

`--json`, `--log`, `--time` for output control.

## EXAMPLES

Preview clustering results:

```
maki faces cluster
```

Apply clustering with a stricter threshold:

```
maki faces cluster --threshold 0.6 --apply
```

Cluster only faces from a specific shoot:

```
maki faces cluster --query "path:Capture/2026-03" --apply
```

## SEE ALSO

faces detect – detect faces first. faces name – name the resulting person groups.

---

# maki faces people

## NAME

maki-faces-people – list all people

## SYNOPSIS

```
maki [GLOBAL FLAGS] faces people
```

## DESCRIPTION

Lists all people in the catalog with their names (if assigned) and face counts.

## EXAMPLES

```
maki faces people
maki faces people --json
```

## SEE ALSO

faces name – name a person.

---

# maki faces name

## NAME

maki-faces-name – name a person

## SYNOPSIS

```
maki [GLOBAL FLAGS] faces name <PERSON_ID> <NAME>
```

## DESCRIPTION

Assigns a human-readable name to a person. Person IDs are shown by `maki faces people`.

## ARGUMENTS

**PERSON_ID (required)** The person's UUID.
**NAME (required)** The name to assign.

## EXAMPLES

```
maki faces name 550e8400-... "Alice"
```

---

# maki faces merge

## NAME

maki-faces-merge – merge two people

## SYNOPSIS

```
maki [GLOBAL FLAGS] faces merge <TARGET_ID> <SOURCE_ID>
```

## DESCRIPTION

Moves all faces from the source person to the target person, then deletes the source person. Useful for combining duplicate person groups after clustering.

## ARGUMENTS

**TARGET_ID (required)** The person to keep.
**SOURCE_ID (required)** The person to merge into the target (deleted after merge).

## EXAMPLES

```
maki faces merge 550e8400-... 661f9511-...
```

---

# maki faces delete-person

## NAME

maki-faces-delete-person – delete a person

## SYNOPSIS

```
maki [GLOBAL FLAGS] faces delete-person <PERSON_ID>
```

## DESCRIPTION

Deletes a person record. All faces assigned to this person become unassigned (they are not deleted).

## ARGUMENTS

**PERSON_ID (required)** The person's UUID.

## EXAMPLES

```
maki faces delete-person 550e8400-...
```

---

# maki faces unassign

## NAME

maki-faces-unassign – remove a face from its person

## SYNOPSIS

```
maki [GLOBAL FLAGS] faces unassign <FACE_ID>
```

## DESCRIPTION

Removes the person assignment from a single face. The face record is preserved; only the person link is cleared.

## ARGUMENTS

**FACE_ID (required)** The face's UUID.

## EXAMPLES

```
maki faces unassign a1b2c3d4-...
```

---

# maki faces export

## NAME

maki-faces-export – export faces and people to YAML and binary files

## SYNOPSIS

```
maki [GLOBAL FLAGS] faces export
```

## DESCRIPTION

Exports all face and people data from the SQLite catalog to file-based storage:

- `faces.yaml` — face records (bounding boxes, confidence, person assignments)
- `people.yaml` — people records (names, representative faces)
- `embeddings/arcface/<prefix>/<face_id>.bin` — ArcFace face recognition embeddings as raw binary files

This is a one-time migration command for catalogs that have existing face data in SQLite but no corresponding YAML/binary files (i.e., data created before v2.2.1). After running this command, `rebuild-catalog` will be able to restore face and people data.

Going forward, all face/people write operations automatically persist to both SQLite and files.

## EXAMPLES

Export all face data:

```
maki faces export
```

Export with JSON output:

```
maki faces export --json
```

## SEE ALSO

faces detect – detect faces in images. rebuild-catalog – rebuilds catalog from files.

---

# maki faces download

## NAME

maki-faces-download – download face detection models

## SYNOPSIS

`maki [GLOBAL FLAGS] faces download`

## DESCRIPTION

Downloads the YuNet face detection model and ArcFace face recognition model from HuggingFace. Models are cached in the model directory (default `~/.cache/maki/models`, configurable via `[ai] model_dir` in `maki.toml`).

## EXAMPLES

```
maki faces download
```

---

# maki faces status

## NAME

maki-faces-status – show face detection model status

## SYNOPSIS

```
maki [GLOBAL FLAGS] faces status
```

## DESCRIPTION

Shows the download status of face detection and recognition models. Reports whether the YuNet face detection model and ArcFace face recognition model are downloaded and ready to use.

## EXAMPLES

```
maki faces status
```

## SEE ALSO

faces download – download face models. faces detect – detect faces in images.

---

Previous: Ingest Commands – `import`, `tag`, `edit`, `group`, `auto-group`. Next: Retrieve Commands – `search`, `show`, `duplicates`, `stats`, `serve`.

# Retrieve Commands

Commands for finding assets, inspecting details, and browsing the catalog.

---

# maki search

## NAME

maki-search – search for assets using filters and free-text keywords

## SYNOPSIS

```
maki [GLOBAL FLAGS] search <QUERY> [--format <FMT>] [-q]
```

## DESCRIPTION

Searches the catalog for assets matching the given query. The query string supports both free-text keywords (matched against filenames and metadata) and structured filter prefixes.

**Filter syntax**: `prefix:value`. Values containing spaces must be double-quoted: `tag:"Fools Theater"`.

**Available filters**:

| Filter | Description | Examples |
|---|---|---|
| `type:` | Asset type | `type:image`, `type:video`, `type:audio` |
| `tag:` | Tag name (hierarchical: matches descendants) | `tag:landscape`, `tag:"golden hour"`, `tag:animals/birds` |
| `format:` | File format | `format:jpg`, `format:nef`, `format:mp4` |
| `rating:` | Star rating (exact, range, OR) | `rating:5`, `rating:3+`, `rating:3-5`, `rating:2,4` |
| `label:` | Color label | `label:Red`, `label:Green` |
| `camera:` | Camera model (partial match) | `camera:fuji`, `camera:"Canon EOS R5"` |
| `lens:` | Lens model (partial match) | `lens:56mm`, `lens:"RF 50mm f/1.2"` |
| `iso:` | ISO value or range | `iso:100`, `iso:100-800` |
| `focal:` | Focal length or range (mm) | `focal:50`, `focal:35-70` |
| `f:` | Aperture or range | `f:1.4`, `f:1.4-2.8` |
| `width:` | Minimum width (pixels) | `width:4000`, `width:4000+` |
| `height:` | Minimum height (pixels) | `height:2000`, `height:2000+` |
| `meta:` | Source metadata key=value | `meta:Copyright=2026` |
| `path:` | File location path prefix | `path:Capture/2026-02`, `path:/Volumes/Photos/2026` |
| `collection:` | Collection membership | `collection:Favorites`, `collection:"My Picks"` |
| `date:` | Creation date (prefix match) | `date:2026-02-25`, `date:2026-02`, `date:2026` |
| `dateFrom:` | Creation date lower bound (inclusive) | `dateFrom:2026-01-01` |
| `dateUntil:` | Creation date upper bound (inclusive) | `dateUntil:2026-12-31` |
| `copies:` | File location count (exact) | `copies:1`, `copies:2` |
| `copies:N+` | File location count (minimum) | `copies:2+`, `copies:3+` |
| `id:` | Asset ID (prefix match) | `id:72a0bb4b` |
| `variants:` | Variant count (exact or minimum) | `variants:2`, `variants:2+` |
| `scattered:` | Variants on N+ different volumes | `scattered:2+` |
| `orphan:true` | Assets with zero file locations | `orphan:true` |
| `orphan:false` | Assets with at least one file location | `orphan:false` |
| `missing:true` | Assets with files missing on disk | `missing:true` |
| `stale:N` | Not verified in N days | `stale:30`, `stale:90` |

| Filter | Description | Examples |
|---|---|---|
| `volume:` | Assets on a specific volume | `volume:Archive`, `volume:none` |
| `stacked:true` | Assets in a stack | `stacked:true` |
| `stacked:false` | Assets not in any stack | `stacked:false` |
| `geo:` | GPS geolocation | `geo:any`, `geo:none`, `geo:52.5,13.4,10` |
| `faces:` | Face count (ai feature) | `faces:any`, `faces:none`, `faces:2+` |
| `person:` | Named person (ai feature) | `person:Alice`, `person:"John Smith"` |
| `similar:` | Visual similarity (ai feature) | `similar:72a0bb4b`, `similar:72a0bb4b:50` |
| `min_sim:` | Minimum similarity threshold (ai) | `min_sim:90` |
| `text:` | Semantic text-to-image search (ai) | `text:sunset`, `text:"woman reading"` |
| `embed:` | Embedding status (ai feature) | `embed:any`, `embed:none` |

Filters can be freely combined. Free-text tokens that do not match a filter prefix are joined as a text search against filenames and metadata.

**Path normalization**: The `path:` filter automatically normalizes paths. `~` expands to `$HOME`, `./` and `../` resolve relative to the current working directory, and absolute paths matching a registered volume's mount point are stripped to volume-relative form with the volume filter implicitly applied (e.g., `path:/Volumes/Photos/Capture/2026` becomes `path:Capture/2026` scoped to the Photos volume).

**Output formats**: The `--format` flag controls output. Presets: `ids` (one UUID per line), `short` (default: abbreviated ID, name, type, format, date), `full` (adds tags, description, rating, label), `json` (JSON array). Custom templates use `{placeholder}` substitution with escape sequences (`\t`, `\n`).

Available placeholders: `{id}`, `{name}`, `{filename}`, `{type}`, `{format}`, `{tags}`, `{description}`, `{rating}`, `{label}`, `{date}`, `{size}`.

The result count header (e.g., "Found 42 assets") is suppressed when an explicit `--format` is given.

## ARGUMENTS

**QUERY (required)** Search query string with optional filter prefixes and free-text keywords.

## OPTIONS

**–format <FMT>** Output format preset or custom template.
**-q / –quiet** Shorthand for `--format=ids`. Prints one asset ID per line with no header, ideal for piping.

`--json` (global flag) is equivalent to `--format json`.

## EXAMPLES

Search by tag and minimum rating:

```
maki search "tag:landscape rating:4+"
```

Find all videos:

```
maki search "type:video"
```

Search with camera and aperture filters:

```
maki search 'camera:"Canon EOS R5" f:1.4-2.8'
```

Get IDs for scripting:

```
maki search -q "tag:travel label:Green"
```

Custom format template:

```
maki search "rating:5" --format '{id}\t{name}\t{label}'
```

Search within a path and pipe to a collection:

```
maki search -q "path:Capture/2026-02-22 rating:4+" | xargs maki col add "Feb Selects"
```

Find assets with files missing on disk:

```
maki search "missing:true"
```

Find assets with no backup (only one copy):

```
maki search "copies:1"
```

Find highly-rated assets with at least two copies:

```
maki search "copies:2+ rating:4+"
```

Find orphaned assets (no file locations):

```
maki search "orphan:true"
```

Find visually similar assets (requires ai feature + embeddings):

```
maki search "similar:72a0bb4b"
maki search "similar:72a0bb4b:50"
maki search "similar:72a0bb4b rating:4+ tag:landscape"
```

## SEE ALSO

show – display full details for a specific asset. saved-search – save and re-run searches. CLI Conventions – output conventions, scripting patterns.

---

# maki show

## NAME

maki-show – display full details for an asset

## SYNOPSIS

```
maki [GLOBAL FLAGS] show <ASSET_ID>
```

## DESCRIPTION

Displays comprehensive details for a single asset, including:

- Asset metadata: ID, name, description, type, tags, rating (as stars), color label, creation date.
- Variants: content hash, role (original/export/processed/sidecar), format, file size, original filename, file locations (volume + path), source metadata (camera, lens, ISO, etc.), and preview status.
- Recipes: content hash, format (XMP, COS, etc.), original filename, and file locations.

Asset IDs support unique prefix matching (see CLI Conventions).

The display logic for previews prefers Export > Processed > Original variant previews (skipping Sidecar). Within the same role, standard image formats are preferred over RAW, with file size as a tiebreaker.

## ARGUMENTS

**ASSET_ID (required)** The asset ID or a unique prefix of it.

## OPTIONS

This command only accepts global flags.

`--json` outputs an `AssetDetails` object with full asset, variant, and recipe information.

## EXAMPLES

Show an asset by full ID:

```
maki show a1b2c3d4-e5f6-7890-abcd-ef1234567890
```

Show an asset by prefix:

```
maki show a1b2c
```

Show as JSON and extract variant filenames:

```
maki show a1b2c --json | jq '.variants[].original_filename'
```

Show as JSON and list file locations:

```
maki show a1b2c --json | jq '.variants[].file_locations[]'
```

## SEE ALSO

search – find assets to inspect. edit – modify the fields shown here. tag – add or remove tags.

---

# maki preview

## NAME

`maki preview` – display an asset's preview image in the terminal

## SYNOPSIS

```
maki [GLOBAL FLAGS] preview <ASSET_ID> [--open]
```

## DESCRIPTION

Renders the asset's best preview image directly in the terminal using the viuer library. Auto-detects the terminal's graphics protocol (iTerm2, Kitty, Sixel) and falls back to Unicode half-block characters.

With `--open`, launches the preview file in the OS default image viewer instead of rendering in the terminal.

Also available as a shell built-in: `preview <id>`, `preview $var`, `preview _ --open`.

## OPTIONS

- `<ASSET_ID>` – Asset ID or prefix (required)
- `--open` – Open in the OS default viewer instead of terminal display

## EXAMPLES

Display a preview in the terminal:

```
maki preview a1b2c3d4
```

Open in the default viewer:

```
maki preview a1b2c --open
```

In the shell with variable expansion:

```
photos> $picks = search "rating:5 date:2024"
photos [picks=12]> preview $picks
```

## SEE ALSO

show – display full asset metadata. search – find assets to preview.

---

# maki export

## NAME

maki-export – copy files matching a search query to a directory or ZIP archive

## SYNOPSIS

```
maki [GLOBAL FLAGS] export <QUERY> <TARGET> [OPTIONS]
```

## DESCRIPTION

Exports files from the catalog to a target directory or ZIP archive. Searches for assets matching the query, resolves their file locations on online volumes, and copies (or symlinks) files to the target. With `--zip`, writes a single ZIP archive instead of individual files.

By default, only the **best variant** per asset is exported (Export > Processed > Original, image formats preferred, file size tiebreaker). Use `--all-variants` to export every variant.

**Layout modes**:

- **`flat`** (default): All files placed in the target root. Filename collisions (different content, same filename) are resolved by appending `_<8-char-hash>` before the extension. Files with identical content hashes reuse the same target path.
- **`mirror`**: Preserves the source volume-relative directory structure. When files span multiple volumes, each volume's files are placed under a `<volume-label>/` prefix to avoid path collisions.

Files are copied with SHA-256 integrity verification. Existing files at the target path are skipped if their content hash matches (use `--overwrite` to force re-copy).

## ARGUMENTS

**QUERY (required)** Search query string (same syntax as `maki search`).

**TARGET (required)** Target directory path (or ZIP file path with `--zip`). Created automatically if it does not exist (except in `--dry-run` mode).

## OPTIONS

**–layout <MODE>** File layout mode: `flat` (default) or `mirror`.

**–symlink** Create symbolic links instead of copying files. On Unix, uses `std::os::unix::fs::symlink`; on Windows, uses `std::os::windows::fs::symlink_file`.

**–all-variants** Export every variant of each matching asset. Default is best variant only.

**–include-sidecars** Also copy recipe/sidecar files (`.xmp`, `.cos`, `.cot`, `.cop`, `.pp3`, `.dop`, `.on1`) alongside the exported variants.

**–dry-run** Report the export plan without writing any files or creating directories.

**–overwrite** Re-copy files even if the target already contains a file with a matching content hash. Default behavior skips matching files.

**–zip** Write a ZIP archive to the target path instead of copying files to a directory. The archive uses stored (uncompressed) entries since media files are already compressed. The `.zip` extension is appended automatically if not present. Cannot be combined with `--symlink`. Layout and sidecar options work the same as directory export.

`--json` outputs an `ExportResult` object with fields: `dry_run`, `assets_matched`, `files_exported`, `files_skipped`, `sidecars_exported`, `total_bytes`, `errors`.

## EXAMPLES

Export best-of picks to a delivery folder:

```
maki export "rating:5 tag:portfolio" /tmp/delivery/
```

Export with directory structure preserved:

```
maki export "collection:Print" /Volumes/USB/export --layout mirror
```

Include sidecars for another workstation:

```
maki export "tag:client" /tmp/handoff/ --include-sidecars
```

Create symlinks instead of copies:

```
maki export "type:image rating:4+" ~/links/ --symlink
```

Export all variants (RAW + processed):

```
maki export "tag:portfolio" /tmp/all/ --all-variants
```

Export as a ZIP archive:

```
maki export "tag:client" ~/Desktop/delivery --zip
```

Dry run to see what would be exported:

```
maki export "collection:Best" /tmp/test/ --dry-run
```

JSON output for scripting:

```
maki --json export "rating:5" /tmp/out/ | jq '.files_exported'
```

## SEE ALSO

search – find assets to export. contact-sheet – generate a PDF contact sheet from search results. relocate – move/copy asset files between volumes with catalog updates. CLI Conventions – global flags, scripting patterns.

---

# maki contact-sheet

## NAME

maki-contact-sheet – generate a PDF contact sheet from search results

## SYNOPSIS

```
maki [GLOBAL FLAGS] contact-sheet <QUERY> <OUTPUT> [OPTIONS]
```

## DESCRIPTION

Generates a printable PDF contact sheet with thumbnail grids from assets matching a search query. Pages are composed as images at 300 DPI and wrapped in PDF. Smart previews (2560px) are preferred with automatic fallback to regular previews (800px).

Layout presets control the grid density:

- **dense** (6x8): Maximum thumbnails per page.
- **standard** (4x5): Balanced view (default).
- **large** (3x3): Detailed thumbnails.

Color labels can be rendered as colored borders around cells, small dots next to filenames, or hidden entirely.

## ARGUMENTS

**QUERY (required)** Search query (same syntax as `maki search`).
**OUTPUT (required)** Output PDF file path.

## OPTIONS

**–layout <PRESET>** Layout preset: `dense`, `standard`, `large` (default: `standard`).
**–columns <N>** Number of columns (overrides layout preset).
**–rows <N>** Number of rows per page (overrides layout preset).
**–paper <SIZE>** Paper size: `a4`, `letter`, `a3` (default: `a4`).
**–landscape** Use landscape orientation.
**–title <TEXT>** Title printed in page headers.
**–fields <FIELDS>** Comma-separated metadata fields: `filename`, `date`, `rating`, `label`, `format`, `size`, `dimensions`.
**–sort <ORDER>** Sort order: `date`, `name`, `rating`, `filename`.
**–no-smart** Use regular previews instead of smart previews.
**–group-by <FIELD>** Group by field with section headers: `date`, `volume`, `collection`, `label`.
**–margin <MM>** Page margin in millimeters.
**–label-style <STYLE>** Color label display: `border`, `dot`, `none` (default: `border`).
**–quality <N>** JPEG quality for page images, 1-100.
**–copyright <TEXT>** Copyright text in center of page footer.
**–dry-run** Report page/asset count without generating.

## EXAMPLES

Basic contact sheet of all 5-star images:

```
maki contact-sheet "rating:5" stars.pdf
```

Dense layout on A3 landscape with title:

```
maki contact-sheet "tag:landscape" landscapes.pdf --layout dense --paper a3 --landscape --title "Landscapes 2026"
```

Group by date with copyright:

```
maki contact-sheet "" all.pdf --group-by date --copyright "© 2026 Thomas Herrmann"
```

Dry run to check page count:

```
maki contact-sheet "format:nef" raw.pdf --dry-run
```

## SEE ALSO

search – find assets matching a query. export – copy files to a directory. generate-previews – generate or upgrade previews.

---

# maki duplicates

## NAME

maki-duplicates – find files with identical content at multiple locations

## SYNOPSIS

`maki [GLOBAL FLAGS] duplicates [--format <FMT>] [--same-volume] [--cross-volume] [--volume <LABEL>] [--filter-format <FMT>] [--path <PREFIX>]`

## DESCRIPTION

Finds variants whose content hash appears at more than one file location. This detects files that exist on multiple volumes or at multiple paths, helping identify redundant copies or verify backup coverage.

Each result shows the content hash, filename, volume count, and all locations where the identical file exists. The `short` and `full` formats annotate locations with the volume's purpose (if set) and flag same-volume duplicate groups. The `full` format additionally shows the last verification timestamp for each location.

**Duplicate modes**:

- **Default** (no flag): All variants with 2+ file locations, regardless of volume layout.
- **`--same-volume`**: Only variants with 2+ locations on the **same** volume. These are likely unwanted copies (e.g., accidentally imported twice from different paths on the same drive).
- **`--cross-volume`**: Only variants with locations on 2+ **different** volumes. These represent intentional backups or copies across drives.

The `--volume`, `--filter-format`, and `--path` flags narrow results via SQL filtering. All filters compose with any mode.

## ARGUMENTS

None.

## OPTIONS

–**format <FMT>** Output format. Same presets as `maki search`: `ids`, `short` (default), `full`, `json`. Custom templates support all search placeholders plus `{locations}` and `{volumes}` (distinct volume count). Location strings include the volume purpose in brackets (e.g., `Photos[working]:Capture/photo.jpg`).
–**same-volume** Show only same-volume duplicates. Mutually exclusive with `--cross-volume`.
–**cross-volume** Show only cross-volume copies. Mutually exclusive with `--same-volume`.
–**volume <LABEL>** Filter results to entries involving this volume. Combines with any mode.
–**filter-format <FORMAT>** Filter to entries matching this file format (e.g. `nef`, `jpg`).
–**path <PREFIX>** Filter to entries with a location under this path prefix.

`--json` outputs an array of `DuplicateEntry` objects (includes `volume_count` and `same_volume_groups` fields).

## EXAMPLES

Find all duplicates:

```
maki duplicates
```

Find likely unwanted same-volume duplicates:

```
maki duplicates --same-volume
```

Show cross-volume copies (backup verification):

```
maki duplicates --cross-volume
```

Filter to duplicates involving a specific volume:

```
maki duplicates --volume "Backup Drive"
```

Show full details with verification timestamps:

```
maki duplicates --format full
```

List duplicates as JSON:

```
maki duplicates --json | jq '.[].locations'
```

Cross-volume copies as JSON for a specific volume:

```
maki duplicates --cross-volume --volume Photos --json
```

Custom format showing hash and locations:

```
maki duplicates --format '{hash}\t{filename}\t{volumes} volumes\t{locations}'
```

## SEE ALSO

verify – verify file integrity on disk. cleanup – remove stale location records. search – use `copies:` filter for location-count-based queries.

---

# maki stats

## NAME

maki-stats – show catalog statistics

## SYNOPSIS

`maki [GLOBAL FLAGS] stats [OPTIONS]`

## DESCRIPTION

Displays summary statistics about the catalog. Without any section flags, shows a compact overview: total assets, variants, recipes, volumes, and total file size.

Additional sections can be enabled with flags to show breakdowns by type, format, volume, tag usage, and verification health. `--all` enables all sections.

## ARGUMENTS

None.

## OPTIONS

–**types** Show asset type breakdown (image, video, audio, etc.) and format distribution.
–**volumes** Show per-volume details: asset count, total size, directory count, and verification status.
–**tags** Show tag usage frequencies (most-used tags first).
–**verified** Show verification health: how many files have been verified, when, and how many are overdue.
–**all** Enable all sections (equivalent to `--types --volumes --tags --verified`).
–**limit <N>** Maximum number of entries for top-N lists (default: 20). Applies to format breakdown, tag list, etc.

`--json` outputs a structured JSON object with all requested sections.

## EXAMPLES

Quick overview:

```
maki stats
```

Full statistics:

```
maki stats --all
```

Show only tag frequencies:

```
maki stats --tags --limit 50
```

Show volume details as JSON:

```
maki stats --volumes --json | jq '.volumes[] | {label, assets, size}'
```

Show verification health:

```
maki stats --verified
```

## SEE ALSO

search – find specific assets matching criteria. verify – run verification checks. volume list – list volumes with online/offline status.

# maki backup-status

## NAME

maki-backup-status – check backup coverage and find under-backed-up assets

## SYNOPSIS

`maki [GLOBAL FLAGS] backup-status [QUERY] [OPTIONS]`

## DESCRIPTION

Answers the question: "Are my important assets safely backed up?"

In **overview mode** (default), displays aggregate statistics about backup coverage:

- **Totals**: asset count, variant count, file location count.
- **Coverage by volume purpose**: how many assets exist on volumes of each purpose (Working, Archive, Backup, Cloud), with percentages.
- **Volume distribution**: histogram of assets by number of distinct volumes they exist on (0, 1, 2, 3+), with "AT RISK" markers for assets below the threshold. Multiple variants or locations on the same volume count as one — what matters for backup safety is how many distinct volumes hold the asset.
- **At-risk summary**: count of assets with fewer than `--min-copies` locations, with hints for listing them.
- **Volume gaps**: per-volume count of missing assets (assets in scope but not on that volume).

In **at-risk listing mode** (`--at-risk`, `-q`, or `--format`), outputs a list of under-backed-up assets using the same output formats as `maki search`. When `--volume` is specified, lists assets missing from that specific volume instead of those with fewer than `--min-copies` locations overall.

An optional positional `QUERY` argument scopes the analysis to matching assets (same syntax as `maki search`).

## ARGUMENTS

**QUERY (optional)** Search query to scope the asset universe. Same syntax as `maki search`. When omitted, all catalog assets are analyzed.

## OPTIONS

**–at-risk** Switch to listing mode. Output under-backed-up assets instead of the overview.

**–min-copies <N>** Threshold for "adequately backed up" (default: 2). Assets on fewer than N distinct volumes are considered at-risk.

**–volume <LABEL>** In overview mode, adds a detailed volume coverage section for this volume. In at-risk listing mode, lists assets missing from this specific volume.

**–format <FMT>** Output format for at-risk listings. Same presets as `maki search`: `ids`, `short`, `full`, `json`, or a custom template.

**-q / –quiet** Shorthand for `--format=ids`. Prints one asset ID per line, ideal for piping to other commands.

`--json` (global flag) outputs a structured `BackupStatusResult` object in overview mode, or a JSON array of `SearchRow` objects in at-risk listing mode.

## EXAMPLES

Quick overview of backup coverage:

```
maki backup-status
```

Scope to highly-rated images:

```
maki backup-status "rating:3+ type:image"
```

Require 3 copies and check coverage:

```
maki backup-status --min-copies 3
```

List at-risk asset IDs for scripting:

```
maki backup-status --at-risk -q
```

Find assets missing from a specific volume:

```
maki backup-status --volume "Master Media" --at-risk -q
```

Pipe at-risk assets to relocate:

```
maki backup-status --volume "Master Media" --at-risk -q "rating:3+" \
  | xargs -I{} maki relocate {} "Master Media"
```

Add at-risk assets to a collection for review:

```
maki backup-status --at-risk -q | xargs maki collection add "Needs Backup"
```

JSON output for scripting:

```
maki --json backup-status | jq '.at_risk_count'
```

## SEE ALSO

search – use `copies:` filter for location-count-based queries. duplicates – find duplicate files across volumes. stats – general catalog statistics. verify – verify file integrity.

---

# maki serve

## NAME

maki-serve – start the web UI server

## SYNOPSIS

```
maki [GLOBAL FLAGS] serve [--port <PORT>] [--bind <ADDR>]
```

## DESCRIPTION

Starts a local web server that provides a browser-based interface for browsing, searching, and editing assets. The web UI offers a rich interactive experience including:

- **Browse page**: Grid of asset thumbnails with a two-row search bar. Row 1: full-width text input. Row 2: tag filter, star rating filter, color label dots, type/format/volume/collection dropdowns, and path prefix input. All filters auto-search with 300ms debounce on text inputs and immediate trigger on dropdowns, stars, labels, and tags.
- **Asset detail page**: Full preview, editable metadata (inline name, description, star rating, color label, tags), variant list, recipe list, and collection membership chips.
- **Tags page** (`/tags`): Sortable tag list with counts, live text filter, and multi-column layout.
- **Collections page** (`/collections`): List of all collections with creation button.
- **Stacks**: When stack collapsing is enabled (default), stacked assets are collapsed in the browse grid to show only the pick image with a stack count badge. A toggle button in the results bar switches between collapsed and expanded views. The `stacked:true` and `stacked:false` search filters are available in the query input.
- **Batch operations**: Checkbox selection on browse cards with a fixed bottom toolbar for batch tagging, rating, labeling, auto-grouping, stacking, and unstacking. "Stack" creates a new stack from selected assets; "Unstack" removes selected assets from their stacks.
- **Keyboard navigation**: Arrow keys navigate between cards, Enter opens details, Space toggles selection, 1-5 sets rating, 0 clears rating, Alt+1-7 sets label, single letters (r/o/y/g/b/p/u) set label by color initial, x clears label.
- **Saved search chips**: Clickable chips on the browse page load saved searches into the filter UI.
- **Tags page**: Shows a collapsible tree view for hierarchical tags (tags containing `/` as a hierarchy separator). Non-hierarchical tags continue to display in the flat multi-column layout.
- **Analytics page** (`/analytics`): Shooting frequency, camera/lens usage, rating distribution, format breakdown, monthly import volume, and storage per volume charts.
- **Drag-and-drop**: Drag browse cards onto the collection dropdown to add to a collection. Drag stack members on the detail page to reorder.
- **Per-stack expand/collapse**: Click the stack badge ( N) on a browse card to expand/collapse just that stack, independent of the global toggle.

The server defaults to `127.0.0.1:8080`. These can be overridden by CLI flags or the `[serve]` section in `maki.toml`. CLI flags take precedence over configuration.

SQLite connections are opened per-request. Previews are served as static files. Static assets (htmx.min.js, style.css) are embedded at compile time.

## ARGUMENTS

None.

## OPTIONS

**–port <PORT>** Port to listen on. Default: 8080, or the value from `maki.toml [serve]` section.

**–bind <ADDR>** Address to bind to. Default: `127.0.0.1`, or the value from `maki.toml [serve]` section.

`--log` (global flag) enables per-request logging to stderr in the format `METHOD /path -> STATUS (duration)`.

## EXAMPLES

Start the web UI with defaults:

```
maki serve
# Listening on http://127.0.0.1:8080
```

Start on a custom port:

```
maki serve --port 9090
```

Bind to all interfaces (for LAN access):

```
maki serve --bind 0.0.0.0 --port 8080
```

Start with request logging:

```
maki serve --log
```

Start with all diagnostics:

```
maki serve --log --time
```

## SEE ALSO

search – CLI equivalent of the web UI browse page. show – CLI equivalent of the web UI asset detail page. CLI Conventions – `maki.toml` configuration reference.

---

# maki shell

## NAME

maki-shell – interactive asset management shell

## SYNOPSIS

```
maki [GLOBAL FLAGS] shell [SCRIPT]
maki [GLOBAL FLAGS] shell -c <COMMAND>
maki [GLOBAL FLAGS] shell [--strict] [SCRIPT | -c <COMMAND>]
```

## DESCRIPTION

Starts an interactive shell that keeps catalog state cached across commands. Commands are entered without the `maki` prefix. The shell provides:

- **Readline editing** with persistent history (stored in `.maki/shell_history`) and tab completion for subcommand names, `--flags`, `$variables`, `tag:` names, and `volume:` labels.
- **Named variables** (`$name`) that store asset ID sets from command results, enabling multi-step workflows.
- **Implicit last result** (`_`) that expands to the asset IDs produced by the most recent command.
- **Session defaults** (`set --flag`) that inject flags into every command for the remainder of the session.
- **Script files** for repeatable workflows — plain text files with one command per line, comments with `#`.
- **Source command** that executes a script file within the current session, sharing variables and defaults.
- **Reload** that re-reads configuration, refreshes tab-completion data, and clears all variables and defaults.

In interactive mode, the prompt shows the catalog directory name and any active variable counts:

```
photos [picks=42 best=5]>
```

Certain commands are blocked inside the shell: `init`, `migrate`, `serve`, and `shell`.

## ARGUMENTS

**SCRIPT** Path to a script file to execute. Each non-empty, non-comment line is run as a shell command. The shell exits after the script completes.

## OPTIONS

**-c, –command <COMMAND>** Execute a single command string and exit. Supports multiple lines (separated by newlines) and variable assignments, just like a script.

**–strict** Exit with a non-zero status on the first error. Applies to script and `-c` modes. Without `--strict`, errors are printed but execution continues with the next line.

## SHELL-ONLY COMMANDS

These commands are available only inside the shell (interactive, script, and `-c` modes):

| Command | Description |
|---|---|
| `help` | Show a summary of shell features and syntax |
| `quit` / `exit` | End the session (also Ctrl-D) |
| `vars` | List all named variables with asset counts, plus active session defaults |
| `unset $name` | Remove a named variable |
| `unset --flag` | Remove a session default flag |

| Command | Description |
|---|---|
| `set --flag` | Add a session default flag. Settable flags: `--json`, `--log`, `--debug`, `--time` |
| `reload` | Re-read config, refresh completions, clear all variables and defaults |
| `source <file>` | Execute a script file in the current session, sharing variables and defaults. Paths are resolved relative to the catalog root |

## VARIABLE SYNTAX

**Assignment:**

```
$name = <command>
```

Runs `<command>` and stores the resulting asset IDs in `$name`. The count is printed to stderr. Variable names may contain letters, digits, and underscores.

**Expansion:**

`$name` and `_` expand to asset IDs. The IDs are always placed at the **end** of the argument list, regardless of where the variable appears in the command. This means `tag $picks --add portfolio` and `tag --add portfolio $picks` are equivalent — the IDs always land in the trailing positional slot where commands expect asset IDs. Unknown variables are left as-is.

`_` (standalone, not inside a word like `_foo` or `foo_bar`) expands to the asset IDs from the most recent command that produced results.

## QUOTE HANDLING

The shell uses a two-rule quoting model:

- **Token-start quotes** (quotes at the beginning of a token) are stripped. They act as grouping quotes, like in a POSIX shell: `"tag:landscape rating:4+"` becomes a single token `tag:landscape rating:4+`.
- **Mid-token quotes** (quotes appearing after other characters) are preserved. This keeps search filter syntax intact: `text:"woman with glasses"` passes through unchanged as a single token.

## EXAMPLES

Interactive session with variables:

```
$ maki shell
photos> $picks = search "rating:5 date:2024"
  42 assets → $picks
photos [picks=42]> tag --add portfolio $picks
photos [picks=42]> export --target /tmp/best $picks
photos [picks=42]> quit
```

Run a script file:

```
maki shell workflow.maki
```

One-liner with `-c`:

```
maki shell -c 'search "tag:landscape rating:4+" --format ids'
```

Session defaults:

```
photos> set --log
  Session default: --log
```

```
  Active defaults: --log
photos> search tag:landscape
# (output includes per-file logging)
photos> unset --log
```

Source a file into the current session:

```
photos> source post-import.maki
```

Strict mode for CI scripts:

```
maki shell --strict batch-updates.maki
```

## SEE ALSO

search – primary command for finding assets inside the shell. CLI Conventions – global flags (`--json`, `--log`, `--debug`, `--time`) usable as session defaults.

---

Previous: Organize Commands – `collection`, `saved-search`, `stack`. Next: Maintain Commands – `verify`, `sync`, `refresh`, `cleanup`, `relocate`, `update-location`, `generate-previews`, `fix-roles`, `fix-dates`, `rebuild-catalog`.

# Maintain Commands

Commands for integrity checks, disk reconciliation, file relocation, preview generation, and catalog housekeeping.

---

# maki verify

## NAME

maki-verify – re-hash files on disk and compare against stored content hashes

## SYNOPSIS

```
maki [GLOBAL FLAGS] verify [PATHS...] [--max-age DAYS] [--force] [OPTIONS]
```

## DESCRIPTION

Verifies file integrity by re-hashing files on disk and comparing the computed SHA-256 hash against the stored content hash in the catalog. Detects corruption, bit rot, or unauthorized modification.

**Catalog mode** (no paths): Verifies all file locations on all online volumes. Checks whether cataloged files are intact, detecting OK, FAILED, MODIFIED, MISSING, and SKIPPED statuses. `--volume` limits to a specific volume; `--asset` limits to a specific asset.

**Path mode** (with paths): Scans files at the given paths on disk and looks them up in the catalog by content hash. In addition to the catalog mode statuses, can report UNTRACKED files — files on disk whose content hash does not match any known variant or recipe.

On successful verification, updates the `verified_at` timestamp on each file location record (persisted to both SQLite catalog and sidecar YAML for variant and recipe locations).

**Incremental verify**: Use `--max-age` to skip files that were verified recently. Only files whose `verified_at` timestamp is older than the given number of days (or never verified) are re-hashed. This enables fast periodic checks on large catalogs. `--force` overrides the skip and re-verifies everything. A default `max_age_days` can be set in `maki.toml` under `[verify]`.

**Result statuses:**

| Status | Description | Mode |
|---|---|---|
| **OK** | File hash matches the catalog record. | Both |
| **FAILED** | Media file hash does not match (corruption or replacement). | Both |
| **MODIFIED** | Recipe file changed externally; hash updated in catalog. | Both |
| **MISSING** | Catalog location exists but file is gone from disk. | Catalog |
| **UNTRACKED** | File on disk not found in catalog. | Path |
| **SKIPPED** | Volume offline, path unreadable, or other error. | Both |

**Exit codes**: Exits with code 1 if any hash mismatches (FAILED) are found. Exits with code 0 for all other statuses, including MODIFIED.

**Recipe handling**: Recipe files (XMP, COS, etc.) that have been modified externally are reported as "modified" rather than "FAILED" and do not trigger exit code 1. Their stored hash is updated to reflect the new content.

Offline volumes are silently skipped.

## ARGUMENTS

**PATHS (optional)** One or more file paths or directories to verify. When omitted, verifies all file locations on online volumes.

## OPTIONS

**–volume <LABEL>** Limit verification to a specific volume.
**–asset <ID>** Verify only the file locations of a specific asset. Supports prefix matching.
**–max-age <DAYS>** Skip files verified within the given number of days. Enables incremental verification.
**–force** Override `--max-age` and re-verify all files.
**–include <GROUP>** Include additional file type groups. Can be specified multiple times.
**–skip <GROUP>** Skip file type groups. Can be specified multiple times.

`--json` outputs a `VerifyResult` with `verified`, `failed`, `modified`, `skipped`, `skipped_recent`, `missing` counters and detail arrays.

`--log` prints per-file verification status and timing to stderr.

## EXAMPLES

Verify the entire catalog:

```
maki verify
```

Verify a specific volume with progress logging:

```
maki verify --volume "Photos" --log --time
```

Verify a single asset:

```
maki verify --asset a1b2c3d4
```

Verify a specific directory:

```
maki verify /Volumes/Photos/Capture/2026-02-22
```

Incremental verify — skip files checked in the last 30 days:

```
maki verify --max-age 30
```

Force re-verify everything, ignoring recent timestamps:

```
maki verify --force --max-age 30
```

Verify and check for failures in a script:

```
if ! maki verify --volume "Archive"; then
  echo "Integrity check failed!"
fi
```

## SEE ALSO

sync – reconcile catalog with disk after files are moved or modified. cleanup – remove stale location records for missing files. stats – `--verified` shows verification health overview.

---

# maki sync

## NAME

maki-sync – reconcile catalog with disk changes

## SYNOPSIS

```
maki [GLOBAL FLAGS] sync <PATHS...> [OPTIONS]
```

## DESCRIPTION

Scans paths on disk and reconciles the catalog with the current disk state. Detects files that have been moved, renamed, modified, or deleted by external tools since the last import.

**Detected states**:

| State | Description |
|---|---|
| **Unchanged** | File at expected path with matching hash. |
| **Moved** | Known hash found at a new path; old path is gone. |
| **New** | Unknown hash at a new path (not yet imported). |
| **Modified recipe** | Recipe file at same path but with a different hash. |
| **Missing** | Catalog location points to a file that no longer exists. |

Without `--apply`, runs in **report-only mode** (safe default) and shows what it found without making changes. With `--apply`, updates catalog and sidecar files for moved files and modified recipes. `--remove-stale` (requires `--apply`) removes catalog location records for confirmed-missing files.

New files are reported but not auto-imported – run `maki import` separately to bring them into the catalog.

## ARGUMENTS

**PATHS (required)** One or more file paths or directories to scan.

## OPTIONS

–**volume <LABEL>** Use a specific volume instead of auto-detecting from the path.
–**apply** Apply changes to catalog and sidecar files. Without this flag, only reports what it found.
–**remove-stale** Remove catalog location records for missing files. Requires `--apply`.

`--json` outputs a `SyncResult` with counts and detail arrays for each state.

`--log` prints per-file status to stderr.

`--time` shows elapsed wall-clock time.

## EXAMPLES

Preview what sync would find (report-only):

```
maki sync /Volumes/Photos/Capture
```

Apply changes for moved and modified files:

```
maki sync /Volumes/Photos --apply --log
```

Apply changes and remove stale location records:

```
maki sync /Volumes/Photos --apply --remove-stale
```

Sync a specific volume:

```
maki sync /Volumes/Archive --volume "Archive" --apply
```

Sync with full diagnostics:

```
maki sync /Volumes/Photos --apply --log --time --json
```

## SEE ALSO

refresh – re-read metadata from changed recipe files. cleanup – remove stale records across all volumes. verify – check file integrity without reconciliation. import – import new files discovered by sync.

---

# maki refresh

## NAME

maki-refresh – re-read metadata from changed sidecar and recipe files

## SYNOPSIS

`maki [GLOBAL FLAGS] refresh [PATHS...] [OPTIONS]`

## DESCRIPTION

Checks recipe files for changes and re-extracts metadata when modifications are detected. For each recipe, compares the on-disk hash to the stored hash. If the file has changed, re-extracts XMP metadata (keywords, rating, description, color label) and updates catalog and sidecar.

This is useful for picking up changes made by external tools like CaptureOne or Lightroom that modify XMP sidecars outside of maki.

Without arguments, checks all recipe locations on all online volumes. With paths, scans recipe files under given paths. `--volume` limits to a specific volume; `--asset` limits to a specific asset's recipes.

**–media** additionally scans JPEG and TIFF variant files and re-extracts their embedded XMP metadata (keywords, rating, description, label, creator, rights). This is useful for retroactively extracting embedded XMP from files imported before the feature existed, or after external tools edit the embedded metadata.

Non-XMP recipes (COS, pp3, etc.) get their hash updated but no metadata extraction.

## ARGUMENTS

**PATHS (optional)** One or more file paths or directories to scan for recipe files. When omitted, checks all recipe locations on online volumes.

## OPTIONS

**–volume <LABEL>** Limit to a specific volume.
**–asset <ID>** Refresh only a specific asset's recipes. Supports prefix matching.
**–dry-run** Report what would change without applying updates.
**–media** Also re-extract embedded XMP from JPEG/TIFF media files (not just recipe files).

`--json` outputs a refresh result with changed/unchanged counts and detail arrays.

`--log` prints per-file status to stderr.

`--time` shows elapsed wall-clock time.

## EXAMPLES

Check all recipes for changes (report-only by default):

```
maki refresh --dry-run
```

Refresh all recipes on a specific volume:

```
maki refresh --volume "Photos" --log
```

Refresh recipes for a single asset:

```
maki refresh --asset a1b2c3d4
```

Refresh recipes and also re-extract embedded XMP from media files:

```
maki refresh --media --log --time
```

Refresh a specific directory after editing in CaptureOne:

```
maki refresh /Volumes/Photos/Capture/2026-02-22 --log
```

## SEE ALSO

sync – full reconciliation of catalog with disk. sync-metadata – bidirectional XMP metadata sync. verify – integrity checking without metadata re-extraction. import – initial import with XMP extraction.

---

# maki sync-metadata

## NAME

maki-sync-metadata – bidirectional XMP metadata sync between DAM and recipe files

## SYNOPSIS

`maki [GLOBAL FLAGS] sync-metadata [QUERY] [OPTIONS]`

## DESCRIPTION

Performs bidirectional metadata synchronization between the DAM catalog and XMP recipe files on disk. Scope can be narrowed with a positional search query, `--asset`, or `--volume`. Runs in three phases:

1. **Inbound (read external changes)**: Detects XMP recipe files that have been modified externally (e.g., by CaptureOne or Lightroom). Re-reads keywords, rating, description, and color label from the changed XMP file and updates the catalog and sidecar YAML.
2. **Outbound (write pending edits)**: Finds recipes marked `pending_writeback` (edits made while the volume was offline) and writes the current DAM metadata back to the XMP file on disk.
3. **Media (with `--media`)**: Re-extracts embedded XMP metadata from JPEG/TIFF variant files.

When both the disk file and the DAM have changed (phase 1 detects a change AND `pending_writeback` is set), the recipe is reported as a **conflict**. Conflicts are skipped — resolve manually or re-run with `refresh` or `writeback` as appropriate.

Without `--dry-run`, changes are applied immediately.

## OPTIONS

**<QUERY> (positional, optional)** Search query to scope which assets are synced (same syntax as `maki search`).
**–volume <LABEL>** Limit to recipes on a specific volume.
**–asset <ID>** Limit to recipes belonging to a specific asset (prefix match).
**–dry-run** Report what would change without applying.
**–media** Also re-extract embedded XMP from JPEG/TIFF variant files (phase 3).

`--json` outputs a `SyncMetadataResult` with per-status counts and conflict details.

`--log` prints per-recipe status to stderr.

`--time` shows elapsed wall-clock time.

## EXAMPLES

Sync only assets matching a query:

```
maki sync-metadata "volume:Photos date:2024"
```

Preview what sync-metadata would do:

```
maki sync-metadata --dry-run
```

Sync a specific volume after reconnecting:

```
maki sync-metadata --volume "Photos 2024" --log
```

Full sync including embedded XMP:

```
maki sync-metadata --media --log --time
```

## SEE ALSO

refresh – one-way inbound metadata re-read. writeback – one-way outbound metadata write. sync – file-level reconciliation (moves, renames, missing files).

---

# maki writeback

## NAME

maki-writeback – write pending metadata changes to XMP recipe files

## SYNOPSIS

`maki [GLOBAL FLAGS] writeback [QUERY] [OPTIONS]`

## DESCRIPTION

Replays pending metadata writes to XMP recipe files on disk. When metadata is edited (rating, label, tags, description) while a volume is offline, XMP write-back is skipped and the recipe is marked with `pending_writeback`. This command pushes those pending changes to XMP when the volume comes back online.

Without `--all`, only recipes with `pending_writeback=1` are processed. Each recipe's XMP file is updated with the current asset metadata (rating, label, tags, description), then re-hashed and the pending flag is cleared.

Scope can be narrowed with a positional search query, `--asset`, or `--volume`. Without scope options, all pending recipes across all online volumes are processed.

## OPTIONS

**<QUERY> (positional, optional)** Search query to scope which assets are written back (same syntax as `maki search`).

**–volume <LABEL>** Limit to recipes on a specific volume.

**–asset <ID>** Limit to recipes belonging to a specific asset (prefix match).

**–all** Write current metadata to every XMP recipe, not just pending ones. Useful for an initial sync or to force-push all DAM metadata to XMP files.

**–dry-run** Report what would be written without modifying any files.

`--json` outputs a result object with counts for written, skipped, failed, and cleared recipes.

`--log` prints per-recipe status to stderr.

`--time` shows elapsed wall-clock time.

## EXAMPLES

Process all pending write-backs:

```
maki writeback
```

Write back only assets matching a query:

```
maki writeback "rating:4+ tag:landscape"
```

Write back to a specific volume after reconnecting:

```
maki writeback --volume "Photos 2024" --log
```

Force-write all metadata to XMP (initial sync):

```
maki writeback --all --log --time
```

Preview what would be written:

```
maki writeback --dry-run
```

Shell variable expansion:

```
# In maki shell
writeback $picks
writeback _
```

## SEE ALSO

sync-metadata – bidirectional metadata sync (read + write). refresh – one-way inbound metadata re-read.

---

# maki cleanup

## NAME

maki-cleanup – remove stale location records, orphaned assets, and orphaned derived files

## SYNOPSIS

```
maki [GLOBAL FLAGS] cleanup [OPTIONS]
```

## DESCRIPTION

Scans all file locations and recipes across online volumes, checking for files that no longer exist on disk. Performs eight passes:

1. **Stale locations and recipes**: Removes catalog and sidecar records for files that are missing from disk.
2. **Locationless variants**: Removes variants with zero remaining file locations from assets that still have other located variants. Prevents ghost variants from accumulating after file moves or reimports. Cleans up derived files (previews, smart previews) and updates denormalized columns.
3. **Orphaned assets**: Deletes assets where all variants have zero remaining file locations, along with their recipes, variants, faces, embeddings, previews, smart previews, face crops, embedding binaries, catalog rows, and sidecar YAML files.
4. **Orphaned previews**: Removes preview files whose content hash no longer matches any variant in the catalog.
5. **Orphaned smart previews**: Same as above for the `smart_previews/` directory.
6. **Orphaned SigLIP embeddings**: Removes embedding binary files (`embeddings/<model>/`) whose asset ID no longer exists in the catalog.
7. **Orphaned face crops**: Removes face crop thumbnails (`faces/`) whose face ID no longer exists in the catalog.
8. **Orphaned ArcFace embeddings**: Removes face embedding binaries (`embeddings/arcface/`) whose face ID no longer exists in the catalog.

Without `--apply`, runs in **report-only mode** (safe default) and predicts what would be removed, including orphaned assets and derived files that would result from removing stale locations.

Offline volumes are skipped with a note.

## ARGUMENTS

None.

## OPTIONS

–**volume <LABEL>** Limit stale-location scanning to a specific volume. When omitted, checks all online volumes.

–**path <PREFIX>** Limit stale-location scanning to files under this path prefix. Accepts either a relative path on the volume or an absolute path (auto-detects the volume and converts to a relative prefix). Useful for scoping cleanup to a specific subdirectory without scanning the full volume.

–**list** Print stale entries to stderr (shows only stale items, unlike `--log` which prints all entries including OK ones).

–**apply** Apply changes: remove stale records, delete orphaned assets, and remove all orphaned derived files.

`--json` outputs a `CleanupResult` with counts for stale locations, locationless variants, orphaned assets, orphaned previews, orphaned smart previews, orphaned embeddings, and orphaned face files.

`--log` prints per-file status to stderr (both OK and stale entries).

`--time` shows elapsed wall-clock time.

## EXAMPLES

Preview what cleanup would remove:

```
maki cleanup
```

List only the stale entries:

```
maki cleanup --list
```

Apply cleanup across all volumes:

```
maki cleanup --apply --log
```

Cleanup a specific volume:

```
maki cleanup --volume "Photos" --apply
```

Cleanup a specific directory:

```
maki cleanup --path "Capture/2026-02" --list
maki cleanup --volume "Photos" --path "Archive/Old" --apply --log
```

Cleanup with JSON output for scripting:

```
maki cleanup --apply --json | jq '{stale: .stale_locations, orphans: .orphaned_assets, previews: .orphaned_preview
```

## SEE ALSO

sync – reconcile individual paths (more targeted than cleanup). verify – check integrity without removing records. search – `orphan:true` filter finds assets with no file locations.

---

# maki dedup

## NAME

maki-dedup – remove same-volume duplicate file locations

## SYNOPSIS

```
maki [GLOBAL FLAGS] dedup [OPTIONS]
```

## DESCRIPTION

Identifies variants with 2+ file locations on the **same** volume and removes the redundant copies. This targets accidental duplicates (e.g. files copied into multiple directories on the same drive) while leaving cross-volume copies untouched (those are intentional backups).

For each set of same-volume duplicate locations, a resolution heuristic selects which copy to **keep**:

1. If `--prefer` is given (or set in `[dedup] prefer` config), prefer locations whose relative path **contains** the specified string (substring match, not prefix-only).
2. Prefer more recently verified files (by `verified_at` timestamp; never-verified sorts oldest).
3. Prefer shorter relative paths (closer to the volume root).
4. Tiebreak: first alphabetically (deterministic).

Before removing a location, the command checks that the variant's total location count across **all** volumes won't drop below `--min-copies`.

When a file location is removed, co-located recipe files (XMP sidecars etc.) in the same directory are automatically cleaned up from disk, catalog, and sidecar YAML.

Without `--apply`, runs in **report-only mode** (safe default): shows what would be removed without making any changes. Recipe file counts are included in the dry-run report.

## ARGUMENTS

None.

## OPTIONS

–**volume <LABEL>** Limit deduplication to a specific volume. When omitted, processes same-volume duplicates on all volumes.

–**prefer <STRING>** Prefer keeping locations whose relative path contains this string (substring match). Useful for keeping files in a curated directory (e.g. `--prefer Selects`) while removing copies elsewhere. Falls back to the `[dedup] prefer` value in `maki.toml` when not given on the command line.

–**filter-format <FORMAT>** Filter to a specific file format (e.g. `nef`, `jpg`). Only processes duplicate groups matching this format.

–**path <PREFIX>** Filter to locations under this path prefix. Only processes duplicates with locations matching the prefix.

–**min-copies <N> (default: 1)** Minimum total locations to preserve per variant across all volumes. Prevents removing a location if it would leave fewer than N copies total. Set to 2 to ensure at least one backup copy survives.

–**apply** Apply changes: delete physical files and co-located recipe files from disk, remove location and recipe records from catalog and sidecar YAML.

`--json` outputs a `DedupResult` with `duplicates_found`, `locations_to_remove`, `locations_removed`, `files_deleted`, `recipes_removed`, `bytes_freed`, `dry_run`, and `errors`.

`--log` prints per-location status to stderr (keep, remove, skipped).

`--time` shows elapsed wall-clock time.

## EXAMPLES

Preview what dedup would remove:

```
maki dedup
```

Remove same-volume duplicates across all volumes:

```
maki dedup --apply --log
```

Dedup a specific volume, preferring files under `Selects/`:

```
maki dedup --volume "Photos" --prefer "Selects" --apply
```

Ensure at least 2 copies survive per variant:

```
maki dedup --min-copies 2 --apply
```

JSON output for scripting:

```
maki --json dedup --apply | jq '{groups: .duplicates_found, removed: .locations_removed, freed: .bytes_freed}'
```

## SEE ALSO

duplicates – find duplicates without removing them. cleanup – remove stale locations for files that no longer exist on disk. verify – update `verified_at` timestamps used by the dedup heuristic.

---

# maki relocate

## NAME

maki-relocate – copy or move asset files to another volume

## SYNOPSIS

```
maki [GLOBAL FLAGS] relocate <ASSET_ID> <VOLUME> [OPTIONS]
maki [GLOBAL FLAGS] relocate <ASSET_IDS...> --target <VOLUME> [OPTIONS]
maki [GLOBAL FLAGS] relocate --query <QUERY> --target <VOLUME> [OPTIONS]
```

## DESCRIPTION

Copies all files belonging to one or more assets (variants and recipes) to a target volume. After copying, verifies file integrity via SHA-256 comparison. Preserves the relative path structure on the target volume.

Without `--remove-source`, files are copied and the asset gains additional file locations on the target volume. With `--remove-source`, source files are deleted after verified copy, effectively moving the asset.

**Single-asset mode**: `maki relocate <ID> <VOLUME>` — backward-compatible form with two positional arguments.

**Batch mode**: Use `--target` with one of: - Multiple positional IDs: `maki relocate <ID1> <ID2> ... --target <VOL>` - Search query: `maki relocate --query "tag:landscape" --target <VOL>` - Stdin: `maki search -q "rating:5" | maki relocate --target <VOL>`

Asset IDs support unique prefix matching. Batch mode reports per-asset progress with `--log` and a summary at the end.

## ARGUMENTS

**ASSET_IDS (positional)** One or more asset IDs or unique prefixes. Reads from stdin when empty and no `--query` is given.

## OPTIONS

**–target <VOLUME>** Target volume label or UUID. Required for batch mode. In single-asset mode, the volume can be given as the second positional argument instead.

**–query <QUERY>** Search query to select assets for batch relocate. Uses the same query syntax as `maki search`.

**–remove-source** Delete source files after successful copy and SHA-256 verification.

**–dry-run** Show what would happen without making any changes.

`--json` outputs a JSON result with `assets`, `copied`, `skipped`, `removed`, `errors`, and `dry_run` fields.

## EXAMPLES

Copy an asset to an archive volume:

```
maki relocate a1b2c3d4 "Archive"
```

Move an asset (copy + delete source):

```
maki relocate a1b2c3d4 "Archive" --remove-source
```

Batch relocate by search query:

```
maki relocate --query "tag:landscape rating:4+" --target "Archive" --dry-run
maki relocate --query "date:2024" --target "Archive 2024" --remove-source --log
```

Batch relocate via stdin:

```
maki search -q "volume:Working date:2023" | maki relocate --target "Cold Storage" --remove-source
```

Multiple IDs:

```
maki relocate a1b2c3d4 e5f6g7h8 --target "Backup" --dry-run
```

Relocate with full diagnostics:

```
maki relocate a1b2c3d4 "Archive" --remove-source --log --time
```

## SEE ALSO

update-location – update path after a manual move. verify – verify file integrity after relocation. volume list – see available volumes.

---

# maki update-location

## NAME

maki-update-location – update a file's catalog path after it was manually moved on disk

## SYNOPSIS

```
maki [GLOBAL FLAGS] update-location <ASSET_ID> --from <OLD_PATH> --to <NEW_PATH> [--volume <LABEL>]
```

## DESCRIPTION

Updates the catalog to reflect a file that was manually moved on disk (outside of maki). The file at the new path is verified to have the same content hash as the catalog record (safety check against accidental mismatches).

`--from` specifies the old path as recorded in the catalog (absolute or volume-relative). `--to` must be an absolute path to the file's current location on disk.

The volume is auto-detected from `--to` by matching against registered volume mount points, or can be specified explicitly with `--volume`.

Handles both variant file locations and recipe file locations.

Asset IDs support unique prefix matching.

## ARGUMENTS

**ASSET_ID (required)** The asset ID or a unique prefix of it.

## OPTIONS

**–from <OLD_PATH> (required)** The old path where the file was before (absolute or volume-relative).
**–to <NEW_PATH> (required)** The new absolute path where the file is now.
**–volume <LABEL>** Volume label or UUID. Auto-detected from `--to` if omitted.

`--json` outputs the updated location details.

## EXAMPLES

Update a file that was moved to a new directory:

```
maki update-location a1b2c3d4 \
  --from "Capture/2026-02-22/DSC_001.nef" \
  --to /Volumes/Photos/Processed/2026/DSC_001.nef
```

Update with explicit volume:

```
maki update-location a1b2c3d4 \
  --from /Volumes/OldDrive/Photos/IMG_001.jpg \
  --to /Volumes/NewDrive/Photos/IMG_001.jpg \
  --volume "NewDrive"
```

Update a recipe file location:

```
maki update-location a1b2c3d4 \
  --from "Capture/2026-02-22/DSC_001.xmp" \
  --to /Volumes/Photos/Processed/2026/DSC_001.xmp
```

## SEE ALSO

relocate – copy or move files with maki managing the transfer. sync – automatic detection of moved files.

---

# maki generate-previews

## NAME

maki-generate-previews – generate or regenerate preview thumbnails

## SYNOPSIS

`maki [GLOBAL FLAGS] generate-previews [PATHS...] [OPTIONS]`

## DESCRIPTION

Generates preview thumbnails for assets. Standard image formats produce 800px JPEG thumbnails via the `image` crate. RAW files use `dcraw` or `dcraw_emu` (LibRaw). Videos use `ffmpeg`. Non-visual formats (audio, documents) get an info card showing file metadata.

**Without PATHS**: Iterates all catalog assets and generates previews for the best variant of each asset (Export > Processed > Original, standard image formats preferred over RAW, file size tiebreak). Can be filtered with `--asset` or `--volume`.

**With PATHS**: Resolves files on disk and looks up their variants in the catalog, generating previews only for those specific files.

Previews are stored in `previews/<hash-prefix>/<hash>.jpg`. Preview settings (max edge size, format, quality) are configured in the `[preview]` section of `maki.toml`.

## ARGUMENTS

**PATHS (optional)** One or more files or directories to generate previews for.

## OPTIONS

–**volume <LABEL>** Limit to variants on a specific volume.
–**asset <ID>** Generate preview only for a specific asset. Supports prefix matching.
–**include <GROUP>** Include additional file type groups. Can be specified multiple times.
–**skip <GROUP>** Skip file type groups. Can be specified multiple times.
–**force** Regenerate previews even if they already exist.
–**upgrade** Regenerate previews for assets where a better variant (export or processed) exists than what the current preview was generated from. Skips assets where the best variant is already the source of the preview.
–**smart** Also generate smart previews (high-resolution, 2560px) alongside regular thumbnails. Smart previews enable zoom and pan in the web UI lightbox. Dimensions controlled by `[preview] smart_max_edge` in `maki.toml`. Combines with `--force` to regenerate both preview types.

`--json` outputs a result with generated/skipped/failed counts.

`--log` prints per-file generation status to stderr.

`--debug` shows stderr output from external tools (dcraw, ffmpeg) for diagnosing failures.

## EXAMPLES

Generate all missing previews:

```
maki generate-previews
```

Regenerate all previews (force):

```
maki generate-previews --force --log --time
```

Regenerate all previews including smart previews:

```
maki generate-previews --smart --force --log --time
```

Upgrade previews to use better variants (e.g., after grouping exports with originals):

```
maki generate-previews --upgrade --log
```

Generate preview for a single asset:

```
maki generate-previews --asset a1b2c3d4
```

Generate previews for files in a specific directory:

```
maki generate-previews /Volumes/Photos/Capture/2026-02-22
```

Generate previews with debug output for troubleshooting RAW files:

```
maki generate-previews --force --debug --asset a1b2c3d4
```

## SEE ALSO

import – previews are generated automatically during import. serve – web UI displays preview thumbnails.

---

# maki fix-roles

## NAME

maki-fix-roles – fix variant roles in multi-variant assets

## SYNOPSIS

`maki [GLOBAL FLAGS] fix-roles [PATHS...] [OPTIONS]`

## DESCRIPTION

Scans assets that contain both RAW and non-RAW variants and re-roles non-RAW variants from `original` to `export`. This corrects role assignments that may have been missed during import or grouping.

In a properly organized asset, only the RAW file should have the `original` role. Non-RAW files (JPEG, TIFF, etc.) in the same asset should be `export` or `processed`.

Without `--apply`, runs in **report-only mode** and shows what roles would change. With `--apply`, updates both YAML sidecar files and the SQLite catalog.

## ARGUMENTS

**PATHS (optional)** Files or directories to scope the fix. When omitted, checks all assets.

## OPTIONS

**–volume <LABEL>** Limit to a specific volume.
**–asset <ID>** Fix only a specific asset. Supports prefix matching.
**–apply** Apply the role changes. Without this flag, only reports what would change.

## EXAMPLES

Preview what roles would change:

```
maki fix-roles
```

Apply role fixes:

```
maki fix-roles --apply --log
```

Fix roles for a single asset:

```
maki fix-roles --asset a1b2c3d4 --apply
```

Fix roles on a specific volume:

```
maki fix-roles --volume "Photos" --apply --log --time
```

## SEE ALSO

group – grouping merges variants and adjusts roles. auto-group – automatic grouping with role adjustment. show – inspect current variant roles.

---

# maki fix-dates

## NAME

maki-fix-dates – fix asset dates from variant EXIF metadata and file modification times

## SYNOPSIS

`maki [GLOBAL FLAGS] fix-dates [QUERY] [OPTIONS]`

## DESCRIPTION

Scans assets and corrects their `created_at` date by examining the EXIF DateTimeOriginal metadata and file modification times of their variants. This fixes assets that were imported with the wrong date (e.g., import timestamp instead of capture date). Scope can be narrowed with a positional search query, `--asset`, or `--volume`.

For each asset, the command collects candidate dates from all variants:

1. **Stored EXIF date**: The `date_taken` field in the variant's `source_metadata` (stored since v1.3.1).
2. **Re-extracted EXIF**: For variants imported before `date_taken` was stored, re-reads the file on disk and extracts EXIF DateTimeOriginal. Requires the volume to be online.
3. **File modification time**: The filesystem mtime of the variant file on disk. Requires the volume to be online.

The oldest date found across all variants is used as the corrected `created_at`. A 1-second tolerance is applied when comparing to the current date (to handle rounding).

Without `--apply`, runs in **report-only mode** and shows what dates would change. With `--apply`, updates both YAML sidecar files and the SQLite catalog. When applying, also backfills `date_taken` into variant `source_metadata` so future runs work from metadata alone without needing the volume online.

**Offline volume handling**: Assets whose only file locations are on offline volumes cannot be fixed (no file access for EXIF re-extraction or mtime). These are counted separately as "skipped (volume offline)" and a warning is printed for each offline volume. Mount the volume and re-run to fix these assets.

## OPTIONS

**<QUERY> (positional, optional)** Search query to scope which assets are checked (same syntax as `maki search`).
**–volume <LABEL>** Limit to assets with locations on a specific volume.
**–asset <ID>** Fix only a specific asset. Supports prefix matching.
**–apply** Apply date corrections. Without this flag, only reports what would change.

`--json` outputs a `FixDatesResult` with `checked`, `fixed`, `already_correct`, `no_date`, `skipped_offline` counters and `offline_volumes` list.

`--log` prints per-asset status to stderr.

`--time` shows elapsed wall-clock time.

## EXAMPLES

Preview what dates would be corrected:

```
maki fix-dates
```

Preview with per-asset detail:

```
maki fix-dates --log
```

Apply date corrections:

```
maki fix-dates --apply --log
```

Fix dates for a specific volume:

```
maki fix-dates --volume "Photos 2024" --apply --log
```

Fix a single asset:

```
maki fix-dates --asset a1b2c3d4 --apply
```

## SEE ALSO

fix-roles – fix variant roles in multi-variant assets. fix-recipes – re-attach recipe files imported as standalone assets. refresh – re-read metadata from changed recipe files. import – import now uses EXIF date → file mtime → current time fallback chain.

---

# maki fix-recipes

## NAME

maki-fix-recipes – re-attach recipe files that were imported as standalone assets

## SYNOPSIS

`maki [GLOBAL FLAGS] fix-recipes [QUERY] [OPTIONS]`

## DESCRIPTION

Finds assets that consist of a single variant with a recipe-type extension (xmp, cos, cot, cop, pp3, dop, on1) and `asset_type = other`, then tries to re-attach them as recipe records on a matching parent variant. Scope can be narrowed with a positional search query, `--asset`, or `--volume`.

This fixes a situation where recipe files were imported before their corresponding media files, or when a media format wasn't recognized at import time (e.g., NRW before extension support was added). In both cases, the recipe file ends up as a standalone asset instead of being attached to the media asset.

**Matching logic**: For each standalone recipe asset, the command extracts the filename stem and directory from the recipe's file location, then searches for a media variant with the same stem in the same directory on the same volume. Compound extensions are handled: if `DSC_001.NRW.xmp` doesn't match directly (stem = `DSC_001.NRW`), the last extension is stripped and `DSC_001` is tried as the stem.

When a parent is found and `--apply` is specified:

1. A `Recipe` record is created on the parent variant.
2. For XMP files, metadata (keywords, rating, description, color label) is extracted and applied to the parent asset.
3. The parent asset's denormalized columns are updated.
4. The standalone recipe asset is fully deleted (recipes, file locations, variants, asset row, and sidecar YAML).

Without `--apply`, runs in **report-only mode** and shows what would change.

## OPTIONS

**<QUERY> (positional, optional)** Search query to scope which assets are checked (same syntax as `maki search`).
**–volume <LABEL>** Limit to assets with locations on a specific volume.
**–asset <ID>** Fix only a specific asset. Supports prefix matching.
**–apply** Apply the changes. Without this flag, only reports what would change.

`--json` outputs a `FixRecipesResult` with `checked`, `reattached`, `no_parent`, `skipped` counters.

`--log` prints per-asset status to stderr.

`--time` shows elapsed wall-clock time.

## EXAMPLES

Preview what would be reattached:

```
maki fix-recipes
```

Preview with per-asset detail:

```
maki fix-recipes --log
```

Apply the fixes:

```
maki fix-recipes --apply --log
```

Fix a specific asset:

```
maki fix-recipes --asset a1b2c3d4 --apply
```

Fix recipes on a specific volume:

```
maki fix-recipes --volume "Photos" --apply --log --time
```

## SEE ALSO

fix-roles – fix variant roles in multi-variant assets. fix-dates – fix asset dates from EXIF metadata. refresh – re-read metadata from changed recipe files. import – import with recipe attachment logic.

---

# maki rebuild-catalog

## NAME

maki-rebuild-catalog – rebuild the SQLite catalog from YAML sidecar files

## SYNOPSIS

```
maki [GLOBAL FLAGS] rebuild-catalog
```

## DESCRIPTION

Rebuilds the SQLite catalog database entirely from the YAML sidecar files in the `metadata/` directory. The sidecar files are the source of truth for all asset metadata; the SQLite database is a derived cache for fast queries.

This command is useful when:

- The SQLite database is corrupted or deleted.
- Schema changes require a full rebuild.
- The catalog needs to be verified against sidecar files.

After rebuilding, all denormalized columns (best variant hash, primary format, variant count) are recomputed. Collections are preserved via `collections.yaml`. Stacks are restored from `stacks.yaml`, re-populating the `stacks` table and the `stack_id`/`stack_position` columns on the `assets` table. When built with `--features ai`: faces are restored from `faces.yaml`, people from `people.yaml`, ArcFace face embeddings from binary files in `embeddings/arcface/`, and SigLIP image embeddings from binary files in `embeddings/<model>/`. The `face_count` denormalized column is recomputed.

## ARGUMENTS

None.

## OPTIONS

This command only accepts global flags.

## EXAMPLES

Rebuild the catalog:

```
maki rebuild-catalog
```

Rebuild with timing:

```
maki rebuild-catalog --time
```

Rebuild with progress logging:

```
maki rebuild-catalog --log --time
```

## SEE ALSO

init – initial catalog creation. stats – verify catalog statistics after rebuild. verify – verify file integrity after rebuild.

---

# maki migrate

## NAME

maki-migrate – run database schema migrations

## SYNOPSIS

`maki [GLOBAL FLAGS] migrate`

## DESCRIPTION

Runs all pending database schema migrations. Migrations are idempotent (safe to run repeatedly) and add new columns, indexes, and tables needed by newer versions of maki.

A `schema_version` table tracks the current schema version. On startup, maki performs a single-query fast-check against this version number and skips migration logic entirely when the schema is already up to date, avoiding the overhead of running dozens of idempotent `ALTER TABLE` statements on every launch.

Migrations also run automatically once at program startup for all commands, so this command is primarily useful for:

- Explicit migration after a version upgrade
- Scripting and automation
- Verifying the schema is up to date

**Rating repair**: The migration includes a `MicrosoftPhoto:Rating` normalization pass. Windows Photo Gallery and some other tools write a percentage-based rating scale (1/13/25/50/75/99) into `MicrosoftPhoto:Rating` instead of the standard `xmp:Rating` (1–5). The `normalize_rating()` function maps these values to the 1–5 scale. The migration fixes affected ratings in both the SQLite catalog and YAML sidecar files.

## EXAMPLES

Run migrations:

```
maki migrate
```

Verify with JSON:

```
maki migrate --json
```

## SEE ALSO

rebuild-catalog – full catalog rebuild from sidecar files.

---

Previous: Retrieve Commands – `search`, `show`, `preview`, `export`, `contact-sheet`, `duplicates`, `stats`, `backup-status`, `serve`, `shell`.

# Search Filter Reference

Complete reference for all search filters available in `maki search`, the web UI browse page, and saved searches. Filters use a `key:value` syntax and can be mixed freely with free-text terms in a single query string.

All filters combine with AND – every filter must match for an asset to appear in results.

---

## Negation and OR Syntax

### Negation (Excluding Results)

Prefix any filter or free-text term with `-` to exclude matches:

```
maki search "-tag:rejected"              # exclude assets with the "rejected" tag
maki search "-format:xmp"                # exclude XMP files
maki search "-type:other"                # exclude "other" type assets
maki search "-label:Red"                 # exclude Red labeled assets
maki search "-camera:phone"              # exclude phone camera shots
maki search "landscape -tag:processed"   # landscapes that aren't tagged processed
maki search "-sunset"                    # exclude free-text match on "sunset"
```

Negation works with all filter types and free-text terms.

### OR Within a Filter (Comma Operator)

Use commas within a single filter value to match any of the given options (OR logic):

```
maki search "tag:alice,bob"              # assets tagged alice OR bob (or both)
maki search "format:nef,cr3"             # NEF or CR3 format
maki search "type:image,video"           # images or videos
maki search "label:Red,Orange"           # Red or Orange labeled assets
maki search "rating:4,5"                 # 4-star or 5-star assets
```

The comma operator works within a single filter value. To require multiple tags, repeat the filter:

```
maki search "tag:landscape tag:sunset"   # assets with BOTH landscape AND sunset tags
```

### Combining Negation and OR

Negation and OR can be combined in complex queries:

```
maki search "type:image,video -format:xmp"            # images or videos, but not XMP files
maki search "tag:landscape,portrait -label:Red"       # landscape or portrait, excluding Red labeled
maki search "format:nef,cr3 -tag:rejected rating:3+"  # RAW files (NEF or CR3), not rejected, 3+ stars
```

---

## Free Text

**Syntax:** any token that does not match a recognized `prefix:value` pattern

**Description:** Remaining tokens after all prefix filters are extracted are joined into a single free-text string. This text is matched against asset names, original filenames, descriptions, and source metadata values (partial, case-insensitive).

**Examples:**

```
maki search "sunset"
maki search "golden hour mountains"
maki search "sunset tag:landscape"       # "sunset" is free text, tag:landscape is a filter
```

**SQL behavior:** `WHERE (a.name LIKE '%text%' OR a.original_filename LIKE '%text%' OR a.description LIKE '%text%' OR v.source_metadata LIKE '%text%')`. Triggers a JOIN to the variants table for metadata matching.

---

## type

**Syntax:** `type:<value>`

**Values:** `image`, `video`, `audio`, `document`, `other`

**Description:** Filters by asset type. Exact match, case-sensitive.

**Examples:**

```
maki search "type:image"
maki search "type:video rating:3+"
```

**SQL behavior:** `WHERE a.asset_type = ?`. Pure assets-table filter, no JOIN required.

---

## tag

**Syntax:** `tag:<name>` or `tag:"<multi-word name>"`

**Description:** Filters to assets that have a specific tag. Supports quoted values for multi-word tags.

**Hierarchical matching:** Tags can be organized hierarchically using `/` as a separator (e.g., `animals/birds/eagles`). Searching for a parent tag matches all descendants: `tag:animals` finds assets tagged `animals`, `animals/birds`, and `animals/birds/eagles`. Searching for an intermediate level also works: `tag:animals/birds` matches both `animals/birds` and `animals/birds/eagles`.

Internally, hierarchical tags are stored with `|` as the separator (to avoid conflicts with literal slashes in tag names like `f/1.4`). The search system handles the conversion transparently – users always type `/` in queries.

**Examples:**

```
maki search "tag:landscape"
maki search 'tag:"Fools Theater"'
maki search 'tag:"Black and White" rating:4+'
maki search "tag:animals"                  # matches animals, animals/birds, animals/birds/eagles
maki search "tag:animals/birds"            # matches animals/birds and animals/birds/eagles
```

**SQL behavior:** `WHERE (a.tags LIKE '%"stored"%' OR a.tags LIKE '%"stored|%')`. The second LIKE clause enables parent-matches-children semantics. Tags containing literal `/` get an additional raw fallback clause.

---

## format

**Syntax:** `format:<extension>`

**Description:** Filters by file format (extension) of any variant on the asset. Case-insensitive match against the variant's format field.

**Examples:**

```
maki search "format:nef"
maki search "format:jpg"
maki search "format:mp4 type:video"
maki search "format:tif tag:processed"
```

**SQL behavior:** `WHERE v.format = ?`. Triggers a JOIN to the variants table.

---

## rating

**Syntax:** `rating:<N>` | `rating:<N>+` | `rating:<N>-<M>` | `rating:<N>,<M>` | `rating:<N>,<M>+`

**Values:** 0 through 5 (0 = unrated)

**Description:** Filters by the asset's star rating (1–5 scale, 0 = unrated). Supports the full numeric filter syntax:

| Syntax | Meaning | Example |
|---|---|---|
| `N` | exactly N | `rating:5` |
| `N+` | N or more | `rating:3+` |
| `N-M` | from N to M | `rating:3-5` |
| `N,M` | exactly N OR M | `rating:2,4` |
| `N,M+` | exactly N OR M+ | `rating:2,4+` |

Ratings are extracted from XMP during import. Microsoft Photo ratings (percentage values 1–99) are automatically normalized to the 1–5 scale (1=1, 25=2, 50=3, 75=4, 99=5).

**Examples:**

```
maki search "rating:5"          # exactly 5 stars
maki search "rating:3+"         # 3 stars or more
maki search "rating:3-5"        # 3, 4, or 5 stars
maki search "rating:2,4"        # exactly 2 or 4 stars
maki search "rating:2,4+"       # exactly 2, or 4 stars and above
maki search "rating:0"          # unrated assets
```

**SQL behavior:** - Exact: `WHERE a.rating = ?` - Minimum: `WHERE a.rating >= ?` - Range: `WHERE a.rating >= ? AND a.rating <= ?` - Values: `WHERE a.rating IN (?, ?)` - Combined: `WHERE (a.rating IN (?) OR a.rating >= ?)`

Pure assets-table filter, no JOIN required.

---

## label

**Syntax:** `label:<color>`

**Values:** `Red`, `Orange`, `Yellow`, `Green`, `Blue`, `Pink`, `Purple` (case-insensitive input, stored as title-case)

**Description:** Filters by color label. The 7-color set is a superset of Lightroom's 5 colors, matching CaptureOne's label palette.

**Examples:**

```
maki search "label:Red"
maki search "label:blue"
maki search "label:Green rating:4+"
```

**SQL behavior:** `WHERE a.color_label = ?`. Pure assets-table filter, no JOIN required.

---

## camera

**Syntax:** `camera:<text>` or `camera:"<multi-word text>"`

**Description:** Partial match against the `camera_model` field in variant source metadata. Case-insensitive substring match.

**Examples:**

```
maki search "camera:fuji"
maki search 'camera:"Canon EOS R5"'
maki search 'camera:"NIKON Z 9" rating:4+'
```

**SQL behavior:** `WHERE v.source_metadata LIKE '%camera_model%' AND v.source_metadata LIKE '%value%'`. Triggers a JOIN to the variants table.

---

## lens

**Syntax:** `lens:<text>` or `lens:"<multi-word text>"`

**Description:** Partial match against the `lens` field in variant source metadata. Case-insensitive substring match.

**Examples:**

```
maki search "lens:56mm"
maki search 'lens:"RF 50mm f/1.2"'
maki search 'lens:"24-70" camera:sony'
```

**SQL behavior:** `WHERE v.source_metadata LIKE '%lens%' AND v.source_metadata LIKE '%value%'`. Triggers a JOIN to the variants table.

---

## iso

**Syntax:** `iso:<N>` (exact) or `iso:<N>+` (minimum) or `iso:<min>-<max>` (range)

**Description:** Filters by ISO sensitivity from variant source metadata. Supports exact match, minimum threshold, and inclusive range.

**Examples:**

```
maki search "iso:3200"          # exactly ISO 3200
maki search "iso:3200+"         # ISO 3200 or higher
maki search "iso:100-800"       # ISO between 100 and 800 inclusive
```

**SQL behavior:** Extracts the `iso` value from `v.source_metadata` JSON and applies numeric comparison. Triggers a JOIN to the variants table.

- Exact: sets both min and max to the same value
- Minimum (`+` suffix): sets min only, no upper bound
- Range: sets both min and max

---

## focal

**Syntax:** `focal:<N>` (exact) or `focal:<N>+` (minimum) or `focal:<min>-<max>` (range)

**Description:** Filters by focal length in millimeters from variant source metadata.

**Examples:**

```
maki search "focal:50"          # exactly 50mm
maki search "focal:200+"        # 200mm or longer
maki search "focal:35-70"       # between 35mm and 70mm inclusive
```

**SQL behavior:** Same pattern as `iso` but on the `focal_length` metadata field. Triggers a JOIN to the variants table.

---

## f (aperture)

**Syntax:** `f:<N>` (exact) or `f:<N>+` (minimum) or `f:<min>-<max>` (range)

**Description:** Filters by f-number (aperture) from variant source metadata. Supports decimal values.

**Examples:**

```
maki search "f:2.8"             # exactly f/2.8
maki search "f:1.4+"           # f/1.4 or wider (numerically >= 1.4)
maki search "f:1.4-2.8"        # between f/1.4 and f/2.8 inclusive
```

**SQL behavior:** Same pattern as `iso` but on the `f_number` metadata field with floating-point comparison. Triggers a JOIN to the variants table.

---

## width

**Syntax:** `width:<N>` (exact/minimum) or `width:<N>+` (minimum, explicit)

**Description:** Filters by image width in pixels from variant source metadata.

**Examples:**

```
maki search "width:4000+"       # 4000 pixels wide or more
maki search "width:3840"        # exactly 3840 pixels (4K UHD width)
maki search "width:4000+ height:2000+"
```

**SQL behavior:** `WHERE CAST(json_extract(v.source_metadata, '$.width') AS INTEGER) >= ?`. Triggers a JOIN to the variants table. Both bare numbers and `+`-suffixed numbers set a minimum threshold.

---

## height

**Syntax:** `height:<N>` (exact/minimum) or `height:<N>+` (minimum, explicit)

**Description:** Filters by image height in pixels from variant source metadata.

**Examples:**

```
maki search "height:2000+"      # 2000 pixels tall or more
maki search "height:2160"       # exactly 2160 pixels (4K UHD height)
```

**SQL behavior:** Same pattern as `width` but on the `height` metadata field. Triggers a JOIN to the variants table.

---

## meta

**Syntax:** `meta:<key>=<value>`

**Description:** Generic key-value match against variant source metadata. The key and value are matched against the JSON source_metadata blob. Useful for filtering on metadata fields that don't have a dedicated prefix filter.

**Examples:**

```
maki search "meta:software=CaptureOne"
maki search "meta:camera_make=SONY"
maki search "meta:color_space=sRGB"
```

**SQL behavior:** `WHERE v.source_metadata LIKE '%key%value%'`. Triggers a JOIN to the variants table. Multiple `meta:` filters can be specified and all must match (AND).

---

## path

**Syntax:** `path:<prefix>` or `path:"<prefix with spaces>"`

**Description:** Restricts results to assets that have at least one file location whose volume-relative path starts with the given prefix. Useful for finding everything imported from a specific directory.

**Automatic normalization** (CLI only, not in web UI or saved searches):

| Input | Behavior |
|---|---|
| `path:Capture/2026` | Used as-is (volume-relative prefix match) |
| `path:~/Photos/2026` | ~ expanded to `$HOME`, then matched against volumes |
| `path:./subdir` | Resolved relative to current working directory |
| `path:../other` | Resolved relative to current working directory |
| `path:/Volumes/Photos/Capture/2026` | Stripped to `Capture/2026` with implicit `volume:` filter for the Photos volume |

An explicit `volume:` filter in the same query takes precedence over the auto-detected volume from path normalization.

**Examples:**

```
maki search "path:Capture/2026-02-22"
maki search 'path:"Photos/Family Trip"'
maki search "path:Capture/2026 rating:3+ tag:landscape"
maki search "path:/Volumes/Photos/Capture/2026"    # auto-normalized
```

**SQL behavior:** `WHERE fl.relative_path LIKE 'prefix%'`. Triggers a JOIN to the file_locations table.

---

## date

**Syntax:** `date:<prefix>`

**Values:** `YYYY-MM-DD` (day), `YYYY-MM` (month), or `YYYY` (year)

**Description:** Prefix match on the asset's creation date. Matches any asset whose `created_at` timestamp starts with the given prefix. This gives you day, month, or year granularity in a single filter.

**Examples:**

```
maki search "date:2026-02-25"       # assets created on Feb 25, 2026
maki search "date:2026-02"          # assets created in February 2026
maki search "date:2026"             # assets created in 2026
maki search "date:2026-02 tag:landscape"
```

**SQL behavior:** `WHERE a.created_at LIKE 'prefix%'`. Pure assets-table filter, no JOIN required. Uses the index on `assets.created_at`.

---

## dateFrom

**Syntax:** `dateFrom:<date>`

**Values:** `YYYY-MM-DD` (or any date prefix)

**Description:** Inclusive lower bound on the asset's creation date. Commonly used with `dateUntil:` to define a date range.

**Examples:**

```
maki search "dateFrom:2026-01-01"                       # from Jan 1, 2026 onward
maki search "dateFrom:2026-01-01 dateUntil:2026-03-31"   # Q1 2026
maki search "dateFrom:2025-06-01 rating:4+"
```

**SQL behavior:** `WHERE a.created_at >= ?`. String comparison is correct for RFC 3339 timestamps. Pure assets-table filter, no JOIN required.

---

## dateUntil

**Syntax:** `dateUntil:<date>`

**Values:** `YYYY-MM-DD`, `YYYY-MM`, or `YYYY`

**Description:** Inclusive upper bound on the asset's creation date. Internally converted to an exclusive next-day/month/year boundary for correct range semantics.

**Examples:**

```
maki search "dateUntil:2026-02-28"                      # up to and including Feb 28
maki search "dateFrom:2026-01-01 dateUntil:2026-12-31"   # full year 2026
maki search "dateUntil:2025-12-31 type:video"            # all videos before 2026
```

**SQL behavior:** Converts the inclusive bound to exclusive: `"2026-02-28"` → `a.created_at < "2026-03-01"`. Pure assets-table filter, no JOIN required.

---

## collection

**Syntax:** `collection:<name>` or `collection:"<multi-word name>"`

**Description:** Restricts results to assets that belong to a specific static collection (album). The collection's member asset IDs are pre-loaded and passed as a filter set.

**Examples:**

```
maki search "collection:Favorites"
maki search 'collection:"My Favorites"'
maki search 'collection:"Travel 2026" rating:4+'
```

**SQL behavior:** Pre-computes the set of asset IDs in the named collection, then filters with `WHERE a.id IN (...)`. Pure in-memory filter after initial lookup.

---

## volume

**Syntax:** `volume:<label>`, `volume:<label1>,<label2>`, `-volume:<label>`, `volume:none`

**Description:** Restricts results to assets that have at least one file location on the specified volume. Volume labels are matched case-insensitively. Supports negation (`-volume:Archive`) and comma-separated OR (`volume:Photos,Working`). The special value `volume:none` finds assets with no file locations on any currently online volume.

**Examples:**

```
maki search "volume:Photos"                  # assets on the Photos volume
maki search "volume:ScreenSaver type:image"  # images on the ScreenSaver volume
maki search "volume:Photos,Working"          # assets on either volume
maki search "-volume:Archive"                # exclude assets on the Archive volume
maki search "volume:none"                    # assets not on any online volume
maki search "volume:none orphan:false"       # has locations, but all on offline volumes
maki search "volume:\"External SSD\""        # volume label with spaces (quoted)
```

In the web UI, the volume filter is also available as a dropdown control that passes the volume UUID directly to the search backend.

**SQL behavior:** - `volume:<label>`: Label is resolved to a volume UUID via DeviceRegistry. `WHERE fl.volume_id IN (...)`. Triggers a JOIN to the file_locations table. - `-volume:<label>`: Resolved to UUID, then `WHERE a.id NOT IN (SELECT DISTINCT fl2.asset_id FROM file_locations fl2 WHERE fl2.volume_id IN (...))`. - `volume:none`: Pre-computes online volume IDs from DeviceRegistry. Adds `WHERE NOT EXISTS (SELECT 1 FROM file_locations fl JOIN variants v ... WHERE fl.volume_id IN (...online_ids...))`.

---

## id

**Syntax:** `id:<prefix>`

**Description:** Matches assets whose UUID starts with the given prefix. Useful for quickly finding a specific asset by its ID (or the beginning of it) in both CLI and web UI.

**Examples:**

```
maki search "id:c654e"
maki search "id:c654efa4-4e55"
```

**SQL behavior:** `WHERE a.id LIKE 'prefix%'`. Pure SQL prefix match.

---

## orphan

**Syntax:** `orphan:true` | `orphan:false`

**Description:** Filters by whether an asset has any file location records.

- `orphan:true` — assets with zero file locations (files removed but metadata remains)
- `orphan:false` — assets with at least one file location (has files on disk)

**Examples:**

```
maki search "orphan:true"                  # assets with no files on disk
maki search "orphan:false type:image"      # images that have files
maki search "volume:none orphan:false"     # has files, but not on any online volume
```

**SQL behavior:** `orphan:true` uses `NOT EXISTS (...)`, `orphan:false` uses `EXISTS (...)` on the file_locations/variants subquery. Pure SQL, no disk I/O.

---

## missing

**Syntax:** `missing:true`

**Description:** Finds assets where at least one file location points to a file that no longer exists on disk (on an online volume). This requires checking the filesystem and may be slow on large catalogs.

**Examples:**

```
maki search "missing:true"
maki search "missing:true type:video"
```

**SQL behavior:** Pre-computes affected asset IDs by iterating all file locations on online volumes and checking `Path::exists()`. The resulting ID set is passed as `WHERE a.id IN (...)`. Offline volumes are skipped.

---

## stale

**Syntax:** `stale:<N>`

**Values:** Number of days (0 or more)

**Description:** Finds assets with at least one file location that has not been verified in the specified number of days, or that has never been verified. Useful for identifying files due for integrity checks.

**Examples:**

```
maki search "stale:30"          # not verified in 30+ days (or never)
maki search "stale:0"           # never verified at all
maki search "stale:90 type:image"
```

**SQL behavior:** `WHERE EXISTS (SELECT 1 FROM file_locations fl JOIN variants v ... WHERE (fl.verified_at IS NULL OR fl.verified_at < datetime('now', '-N days')))`. Pure SQL, no disk I/O.

---

## stacked

**Syntax:** `stacked:true` or `stacked:false`

**Description:** Filters by whether an asset belongs to a stack. `stacked:true` finds assets that are members of a stack; `stacked:false` finds assets that are not in any stack.

**Examples:**

```
maki search "stacked:true"                  # all stacked assets
maki search "stacked:false rating:5"        # 5-star assets not in a stack
maki search "stacked:true type:image"       # stacked images
```

**SQL behavior:** - `stacked:true`: `WHERE a.stack_id IS NOT NULL` - `stacked:false`: `WHERE a.stack_id IS NULL`

Pure assets-table filter, no JOIN required.

---

## copies

**Syntax:** `copies:<N>` (exact) or `copies:<N>+` (minimum)

**Values:** Non-negative integer

**Description:** Filters by the total number of file locations across all variants of an asset. This counts every stored copy of every variant — a file on 3 volumes has 3 copies. Useful for finding assets with insufficient backup coverage or identifying heavily-duplicated files.

Common patterns: - `copies:1` — assets with only a single copy on disk (no backup) - `copies:2+` — assets with at least two copies (backed up) - `copies:0` — equivalent to `orphan:true` (no file locations at all)

**Examples:**

```
maki search "copies:1"              # single-copy assets (backup risk)
maki search "copies:2"              # exactly 2 copies
maki search "copies:3+"             # 3 or more copies
maki search "copies:1 rating:4+"    # highly-rated assets with no backup
maki search "copies:2+ type:video"  # backed-up videos
```

**SQL behavior:** Scalar subquery: `(SELECT COUNT(*) FROM file_locations fl2 JOIN variants v2 ON fl2.content_hash = v2.content_hash WHERE v2.asset_id = a.id) = N` (or `>= N` for minimum). Self-contained, no outer JOIN flags needed.

---

## variants

**Syntax:** `variants:<N>` (exact) or `variants:<N>+` (minimum)

**Values:** Non-negative integer

**Description:** Filters by the number of variants belonging to an asset. This counts distinct content-addressed files (originals, alternates, exports, processed) grouped under one asset. Useful for auditing mis-grouped assets (too many variants) or finding single-file assets.

Common patterns: - `variants:1` — assets with exactly one variant (single file) - `variants:3+` — assets with 3 or more variants (potentially mis-grouped) - `variants:5+` — suspiciously large groups worth manual review

**Examples:**

```
maki search "variants:1"              # single-variant assets
maki search "variants:3+"             # assets with 3+ variants
maki search "variants:5+ type:image"  # images with many variants
```

**SQL behavior:** Direct filter on the denormalized `a.variant_count` column. No JOIN required.

---

## scattered

**Syntax:** `scattered:<N>`

**Values:** Positive integer (minimum number of distinct directories)

**Description:** Finds assets whose variant files are stored in multiple distinct directories. Counts distinct volume + top-level directory combinations across all file locations of an asset's variants. Useful for detecting mis-grouped assets — variants that truly belong together typically share the same directory, while incorrectly merged assets often have files in different locations.

Common patterns: - `scattered:2` — variants in 2+ distinct directories (likely mis-grouped) - `scattered:2 variants:3+` — combine with variant count for maximum precision

**Examples:**

```
maki search "scattered:2"                    # variants in 2+ directories
maki search "scattered:2 variants:3+"        # scattered + many variants
maki search "scattered:3 type:image"         # images across 3+ directories
```

**SQL behavior:** Scalar subquery counting `DISTINCT volume_id || ':' || first_path_component` across `file_locations` joined through `variants`. Self-contained, no outer JOIN flags needed.

---

## geo

**Syntax:** `geo:any` | `geo:none` | `geo:<lat>,<lng>,<radius_km>` | `geo:<south>,<west>,<north>,<east>`

**Description:** Filters by GPS geolocation. GPS coordinates are extracted from EXIF data during import and stored as denormalized `latitude`/`longitude` columns on the assets table.

**Modes:**

| Form | Description |
|---|---|
| `geo:any` | Assets that have GPS coordinates |
| `geo:none` | Assets without GPS coordinates |

| Form | Description |
|---|---|
| `geo:52.5,13.4,10` | Assets within 10km of latitude 52.5, longitude 13.4 (bounding circle approximated as a bounding box) |
| `geo:48.0,11.0,53.0,14.0` | Assets within the bounding box: south 48.0, west 11.0, north 53.0, east 14.0 |

The 3-parameter form (lat, lng, radius) is converted to a bounding box internally using the approximation `dlat = radius / 111` and `dlng = radius / (111 * cos(lat))`.

**Examples:**

```
maki search "geo:any"                          # all geotagged assets
maki search "geo:none"                         # assets without GPS data
maki search "geo:52.52,13.405,5"               # within 5km of Berlin center
maki search "geo:48.0,11.0,53.0,14.0"          # bounding box covering Germany
maki search "geo:any rating:4+ tag:landscape"  # geotagged 4+ star landscapes
```

**SQL behavior:** - `geo:any`: `WHERE a.latitude IS NOT NULL AND a.longitude IS NOT NULL` - `geo:none`: `WHERE (a.latitude IS NULL OR a.longitude IS NULL)` - Bounding box/circle: `WHERE a.latitude >= ? AND a.latitude <= ? AND a.longitude >= ? AND a.longitude <= ?`

Pure assets-table filter, no JOIN required. Uses the composite index on `(latitude, longitude)`.

---

## faces

**Syntax:** `faces:any` | `faces:none` | `faces:<N>` | `faces:<N>+`

**Description:** Filters by the number of detected faces on an asset. Requires the `ai` feature for face detection to have been run.

**Modes:**

| Form | Description |
|---|---|
| `faces:any` | Assets with at least one detected face |
| `faces:none` | Assets with no detected faces |
| `faces:3` | Assets with exactly 3 detected faces |
| `faces:2+` | Assets with 2 or more detected faces |

**Examples:**

```
maki search "faces:any"                    # all assets with faces
maki search "faces:none type:image"        # images without detected faces
maki search "faces:3"                      # group portraits (exactly 3 faces)
maki search "faces:2+ tag:portrait"        # portraits with multiple people
```

**SQL behavior:** Uses the denormalized `face_count` column on the assets table. Pure assets-table filter, no JOIN required.

---

## person

**Syntax:** `person:<name>` or `person:"<multi-word name>"`

**Description:** Filters to assets that contain at least one face assigned to the named person. Supports quoted values for multi-word names.

**Examples:**

```
maki search "person:Alice"
maki search 'person:"John Smith"'
maki search "person:Alice rating:4+"
maki search "-person:Alice"                # exclude assets with Alice
```

**SQL behavior:** Looks up the person ID by name, then filters via `WHERE EXISTS (SELECT 1 FROM faces WHERE faces.asset_id = a.id AND faces.person_id = ?)`.

---

## similar (AI feature)

**Syntax:** `similar:<asset-id>` or `similar:<asset-id>:<limit>`

**Description:** Finds visually similar assets using stored SigLIP embeddings. Returns the top N most similar assets (default 20). Requires the `ai` feature (`--features ai`) and embeddings to have been generated via `maki embed` or `maki import --embed`. The reference asset ID supports prefix matching (e.g. `similar:abc1` resolves like all other asset ID references).

**Examples:**

```
maki search "similar:72a0bb4b"                        # top 20 similar to this asset
maki search "similar:72a0bb4b:50"                      # top 50 similar
maki search "similar:72a0bb4b rating:3+ tag:landscape"  # similar AND 3+ stars AND landscape tag
maki search -q "similar:72a0bb4b"                      # just IDs, for scripting
```

**Behavior:** Looks up the stored embedding for the reference asset, loads all embeddings into an in-memory index, and performs a dot-product similarity search. If no embedding exists for the reference asset, exits with an error suggesting `maki embed --asset <id>`. The result set can be further filtered by all other search filters (AND logic).

---

## min_sim (AI feature)

**Syntax:** `min_sim:<percent>`

**Description:** Sets a minimum similarity threshold (0–100) when used with `similar:`. Only assets with similarity at or above this percentage are included. Without `min_sim:`, all results from the similarity search are returned.

**Examples:**

```
maki search "similar:72a0bb4b min_sim:90"          # only >= 90% similar
maki search "similar:72a0bb4b min_sim:80 type:image"  # >= 80% AND images only
```

**Behavior:** The percentage is converted to a 0.0–1.0 cosine similarity threshold internally. Values are clamped to the 0–100 range. Most useful for finding near-duplicates (min_sim:95) or visually very close images (min_sim:85).

---

## embed (AI feature)

**Syntax:** `embed:any` | `embed:true` | `embed:none` | `embed:false`

**Description:** Filters by whether an asset has a stored AI embedding (SigLIP image embedding). Requires the `ai` feature (`--features ai`) and embeddings to have been generated via `maki embed` or `maki import --embed`.

**Examples:**

```
maki search "embed:any"                    # assets with AI embeddings
maki search "embed:true"                   # same as embed:any
maki search "embed:none"                   # assets without AI embeddings
maki search "embed:false"                  # same as embed:none
maki search "embed:none type:image"        # images that still need embeddings
```

**SQL behavior:** Uses an `EXISTS` / `NOT EXISTS` subquery on the `embeddings` table: `WHERE EXISTS (SELECT 1 FROM embeddings e WHERE e.asset_id = a.id)`. Pure subquery, no JOIN required.

---

## text (AI feature)

**Syntax:** `text:<query>`, `text:"<multi-word query>"`, or `text:"<query>":<limit>`

**Description:** Natural language image search using SigLIP's text encoder. Encodes the query text into the same embedding space as image embeddings, then finds the most similar images via dot-product similarity. Requires the `ai` feature (`--features ai`) and embeddings to have been generated via `maki embed` or `maki import --embed`.

The result limit defaults to 50 and can be configured at three levels (highest priority wins):

1. **Inline syntax**: `text:"sunset":100` — per-query override
2. **`[ai] text_limit`** in `maki.toml` — catalog-wide default
3. **Hardcoded fallback**: 50

**Examples:**

```
maki search "text:sunset"                              # images matching "sunset" (default limit)
maki search "text:\"colorful flowers in a garden\""       # multi-word query
maki search "text:\"person on a beach\" rating:3+"        # combined with other filters
maki search "text:\"mountain landscape\":100"             # return top 100 matches
```

**Behavior:** Loads the SigLIP model (text encoder), encodes the query string into an embedding vector, loads the in-memory embedding index, and returns the top N assets by dot-product similarity (default 50, configurable). The result set can be further filtered by all other search filters (AND logic). Since SigLIP is a vision-language model trained on image-text pairs, queries describe visual content ("red car", "sunset over water", "portrait of a woman") rather than metadata. Results quality depends on how well the SigLIP model generalizes.

---

## Combining Filters

All filters are combined with AND logic. Every specified filter must match for an asset to appear in results. Free-text terms are also AND-combined with all prefix filters.

**Example combinations:**

```
# 5-star landscape images shot with a Nikon
maki search "rating:5 tag:landscape type:image camera:NIKON"

# High-ISO night shots in RAW format
maki search "iso:3200+ format:nef tag:night"

# Wide-angle portraits with shallow depth of field
maki search "focal:24-35 f:1.4-2.8 tag:portrait"

# Unverified images on the Photos volume, imported from a specific path
maki search "stale:30 type:image path:Capture/2026-01"

# 4K or larger images in a specific collection
maki search 'width:3840+ collection:"Best of 2026"'

# Orphaned video assets (no files on disk)
maki search "orphan:true type:video"

# Everything labeled Red with 4+ stars
maki search "label:Red rating:4+"

# Single-copy RAW files (backup risk)
maki search "copies:1 format:nef"

# Well-backed-up 5-star images
maki search "copies:2+ rating:5 type:image"

# Assets from a specific date range
maki search "dateFrom:2026-01-01 dateUntil:2026-03-31 tag:landscape"

# Everything shot in February 2026
maki search "date:2026-02"

# Unstacked 5-star images (candidates for stacking review)
maki search "stacked:false rating:5 type:image"

# Find stacked assets with a hierarchical tag
maki search "stacked:true tag:animals/birds"

# Visually similar assets, filtered to 4+ stars (requires ai feature + embeddings)
maki search "similar:72a0bb4b rating:4+"

# Geotagged photos within 5km of a location
maki search "geo:52.52,13.405,5 rating:4+"

# All geotagged landscape images
maki search "geo:any tag:landscape type:image"
```

---

## Quoted Values

Use double quotes around filter values that contain spaces. When typing at the shell, wrap the entire query in single quotes to prevent the shell from stripping the inner double quotes.

```
# Shell-safe quoting: single quotes outside, double quotes for values
maki search 'tag:"Fools Theater"'
maki search 'camera:"Canon EOS R5" lens:"RF 50mm f/1.2"'
maki search 'collection:"My Favorites" rating:4+'
maki search 'path:"Photos/Family Trip" type:image'
```

Unquoted single-word values continue to work without quotes:

```
maki search "tag:landscape"
maki search "camera:fuji"
```

---

## Filter Availability

All filters work in the CLI (`maki search`), the web UI search box, and in saved searches. The web UI additionally provides dedicated controls for the most common filters:

| Filter | Web UI control | Saved Searches |
|---|---|---|
| Free text | search box | yes |
| `type:` | dropdown + search box | yes |
| `tag:` | tag chips + search box | yes |
| `format:` | multi-select panel + search box | yes |
| `rating:` | star clicks + search box | yes |
| `label:` | color dots + search box | yes |
| `camera:` | search box | yes |
| `lens:` | search box | yes |
| `iso:` | search box | yes |
| `focal:` | search box | yes |
| `f:` | search box | yes |
| `width:` | search box | yes |
| `height:` | search box | yes |
| `meta:` | search box | yes |
| `path:` | text input + search box | yes |
| `collection:` | dropdown + search box | yes |
| `volume:` | dropdown + search box | yes |
| `copies:` | search box | yes |
| `variants:` | search box | yes |
| `scattered:` | search box | yes |
| `date:` | search box | yes |
| `dateFrom:` | search box | yes |
| `dateUntil:` | search box | yes |
| `id:` | search box | yes |
| `orphan:` | search box | yes |
| `missing:true` | search box | yes |
| `stale:` | search box | yes |
| `stacked:` | search box | yes |
| `geo:` | search box | yes |
| `faces:` | search box | yes |
| `person:` | dropdown + search box | yes |
| `similar:` (ai) | "Browse similar" button + search box | no |
| `min_sim:` (ai) | search box | no |
| `text:` (ai) | search box | yes |

| Filter | Web UI control | Saved Searches |
|---|---|---|
| `embed:` (ai) | search box | yes |

---

## Related Topics

- Browse & Search (User Guide) – practical search workflows and output formatting
- Format Templates Reference – controlling output with `--format` and custom templates
- Configuration Reference – `maki.toml` settings

# Format Templates Reference

The `--format` flag controls how `maki search` and `maki duplicates` display their results. It accepts preset names or custom template strings with placeholder substitution.

---

## Presets

### ids

One full UUID per line, with no header or decoration. Ideal for scripting and piping into other commands.

**Shorthand:** `-q` (equivalent to `--format=ids`)

```
$ maki search -q "tag:landscape"
a1b2c3d4-5678-9abc-def0-123456789abc
e5f6a7b8-1234-5678-9abc-def012345678
```

The result count line is suppressed.

### short

Default compact view. One line per result showing short ID, filename, type, format, and date. A result count is printed at the end.

```
$ maki search "sunset"
a1b2c3d4  IMG_1234.jpg [image] (JPEG) -- 2026-01-15T10:30:00
e5f6a7b8  DSC_5678.nef [image] (NEF) -- 2026-01-14T16:45:00

2 result(s)
```

This is the default when no `--format` is specified.

### full

Detailed view including tags and description on each line.

```
$ maki search "sunset" --format full
a1b2c3d4  IMG_1234.jpg [image] (JPEG) -- 2026-01-15T10:30:00 tags:sunset,landscape A golden sunset over the mountains
e5f6a7b8  DSC_5678.nef [image] (NEF) -- 2026-01-14T16:45:00 tags:sunset,nature
```

### json

JSON array containing all result objects. Equivalent to using the global `--json` flag.

```
$ maki search "sunset" --format json
[
```

```
  {
    "asset_id": "a1b2c3d4-5678-9abc-def0-123456789abc",
    "original_filename": "IMG_1234.jpg",
    "asset_type": "image",
    ...
  }
]
```

---

## Custom Templates

Build your own output format by passing a string containing `{placeholder}` tokens.

**Syntax:** `--format '{placeholder}\t{another}'`

### Rules

- Placeholders are delimited by `{` and `}`.
- Known placeholders are replaced with the corresponding value for each result row.
- Unknown placeholders are left as-is in the output (e.g., `{bogus}` prints literally as `{bogus}`).
- An unclosed brace (no matching `}`) is emitted literally: `{world` prints as `{world`.
- The result count line is suppressed when a custom template (or any explicit `--format`) is used.

---

## Placeholders (search)

Available in `maki search --format`:

| Placeholder | Description |
|---|---|
| `{id}` | Full asset UUID |
| `{short_id}` | First 8 characters of the UUID |
| `{name}` | Asset name if set, otherwise falls back to the original filename |
| `{filename}` | Original filename (always the file's actual name, ignoring any custom asset name) |
| `{type}` | Asset type: `image`, `video`, `audio`, `document`, or `other` |
| `{format}` | Primary variant format/extension (e.g., `JPEG`, `NEF`, `MOV`) |
| `{date}` | Asset creation timestamp (ISO 8601 format) |
| `{tags}` | Comma-separated list of tags (empty string if no tags) |
| `{description}` | Description text (empty string if no description) |
| `{hash}` | Content hash of the best display variant |
| `{label}` | Color label name (e.g., `Red`, `Blue`; empty string if no label) |

## Placeholders (duplicates)

Available in `maki duplicates --format`. Includes all search placeholders above, plus:

| Placeholder | Description |
|---|---|
| `{locations}` | Comma-separated list of all file locations where the duplicate exists (format: `volume_label -> relative/path`) |

---

## Escape Sequences

Three escape sequences are recognized in template strings:

| Sequence | Output |
|---|---|
| `\t` | Tab character |
| `\n` | Newline character |
| `\\` | Literal backslash |

A backslash followed by any other character emits a literal backslash (the backslash is not consumed).

---

## Examples

### Tab-separated values for a spreadsheet

```
maki search "type:image" --format '{name}\t{format}\t{date}\t{tags}'
```

Output:

```
IMG_1234.jpg    JPEG    2026-01-15T10:30:00 sunset,landscape
DSC_5678.nef    NEF 2026-01-14T16:45:00 sunset,nature
```

### One name per line with rating label

```
maki search "rating:4+" --format '{name} [{label}]'
```

Output:

```
Golden Sunset [Red]
Mountain Lake [Blue]
```

### Multi-line detail view

```
maki search "tag:portfolio" --format '{name}\n  Format: {format}\n  Tags: {tags}\n  {description}\n'
```

Output:

```
Golden Sunset
  Format: NEF
  Tags: sunset,landscape,portfolio
  A golden sunset over the mountain range

Mountain Lake
```

```
  Format: JPEG
  Tags: nature,lake,portfolio
  Crystal clear reflections at dawn
```

### Short IDs with filenames (compact scripting format)

```
maki search "format:nef" --format '{short_id} {filename}'
```

Output:

```
a1b2c3d4 DSC_1234.NEF
e5f6a7b8 DSC_5678.NEF
```

### Hash-based file listing

```
maki search "type:image" --format '{hash}\t{filename}'
```

Output:

```
sha256:abc123def456...  IMG_1234.jpg
sha256:def789abc012...  DSC_5678.NEF
```

### Duplicate locations as tab-separated

```
maki duplicates --format '{filename}\t{format}\t{locations}'
```

Output:

```
DSC_1234.NEF       NEF  Photos  ->  Capture/2026-01-15/DSC_1234.NEF,  Backup  ->  Capture/2026-01-
15/DSC_1234.NEF
```

---

## Format Flag Behavior

| Flag | Equivalent | Result count shown |
|---|---|---|
| (none) | `--format short` | yes |
| `--format short` | default | yes |
| `--format ids` | `-q` | no |
| `-q` | `--format ids` | no |
| `--format full` | – | no |
| `--format json` | `--json` | no |
| `--format '{...}'` | custom template | no |

When `--format` is explicitly provided (any value including `short`), the result count line is suppressed, keeping output clean for piping and parsing. The count is only shown when no `--format` flag is given at all.

---

## Related Topics

- Search Filters Reference – all available search filters
- Browse & Search (User Guide) – practical search and output examples
- CLI Conventions – global flags, scripting patterns, exit codes

# Configuration Reference (maki.toml)

The `maki.toml` file stores catalog-level configuration. It lives at the root of your catalog directory (the same directory that contains the `metadata/`, `previews/`, and `catalog.db` files).

---

## File Location

`maki.toml` is created automatically by `maki init`. maki locates it by searching the current directory and walking up through parent directories until it finds a directory containing `maki.toml`.

```
my-catalog/
  maki.toml              <-- configuration file
  catalog.db
  metadata/
  previews/
  searches.toml
  collections.yaml
```

All sections and fields are optional. A missing file or an empty file is equivalent to all-defaults. A comment-only file is also valid:

```
# maki catalog configuration
```

---

## Top-Level Fields

### default_volume

- **Type:** UUID string (optional)
- **Default:** none

Fallback volume for `maki import` when auto-detection from the file path is ambiguous or fails. When set, import uses this volume if it cannot determine the correct volume from the first path argument.

```
default_volume = "550e8400-e29b-41d4-a716-446655440000"
```

Find your volume UUIDs with `maki volume list --json`.

---

## preview Section

Controls how preview thumbnails are generated during import and by `maki generate-previews`.

### max_edge

- **Type:** unsigned integer
- **Default:** `800`
- **Validation:** must be greater than 0

Maximum pixel size of the longest edge for generated thumbnails. The shorter edge is scaled proportionally. Applies to all preview types (standard images, RAW conversions, video frames, and info cards).

### format

- **Type:** string (`"jpeg"` or `"webp"`)
- **Default:** `"jpeg"`

Output format for preview files. JPEG produces lossy compressed thumbnails controlled by the `quality` setting. WebP produces lossless thumbnails via the `image` crate (the `quality` setting is ignored for WebP).

### quality

- **Type:** unsigned integer (1–100)
- **Default:** `85`
- **Validation:** must be between 1 and 100

JPEG compression quality. Higher values produce larger files with better visual quality. Only applies when `format = "jpeg"`. Ignored for WebP.

### smart_max_edge

- **Type:** unsigned integer
- **Default:** `2560`
- **Validation:** must be greater than 0

Maximum pixel size of the longest edge for smart previews. Smart previews are a higher-resolution preview tier (separate from regular thumbnails) that enable zoom and pan in the web UI.

### smart_quality

- **Type:** unsigned integer (1–100)
- **Default:** `85`

JPEG compression quality for smart previews. Only applies when the preview format is JPEG.

### generate_on_demand

- **Type:** boolean
- **Default:** `false`

When `true`, the web server generates smart previews automatically on first request. The first load takes a few seconds while the preview is generated; subsequent requests are served from disk. A pulsing "HD" badge in the lightbox and detail page provides visual feedback during generation.

When `false`, smart previews must be generated explicitly via `maki import --smart`, the "Regenerate previews" button on the asset detail page, or a future batch command.

## Notes

Changing `max_edge` or `format` affects only newly generated previews. Existing previews are not automatically regenerated. Use `maki generate-previews --force` to regenerate all previews with the new settings.

Smart previews are stored in a separate directory (`smart_previews/`) and do not replace regular thumbnails.

```
[preview]
max_edge = 1200
format = "jpeg"
quality = 90
smart_max_edge = 2560
smart_quality = 85
generate_on_demand = true
```

---

# [serve] Section

Controls the built-in web UI server started by `maki serve`.

## port

- **Type:** unsigned 16-bit integer
- **Default:** `8080`

TCP port for the web server.

## bind

- **Type:** string (IP address)
- **Default:** `"127.0.0.1"`

Bind address for the web server. Use `"127.0.0.1"` to restrict access to the local machine. Use `"0.0.0.0"` to allow connections from other devices on the network.

## per_page

- **Type:** unsigned 32-bit integer
- **Default:** `60`

Number of results per page in the browse grid.

## stroll_neighbors

- **Type:** unsigned 32-bit integer
- **Default:** `12`

Initial number of neighbor thumbnails shown around the center image on the stroll page. Controls the default position of the neighbor count slider.

## stroll_neighbors_max

- **Type:** unsigned 32-bit integer
- **Default:** `25`

Maximum value for the neighbor count slider on the stroll page.

## stroll_discover_pool

- **Type:** unsigned 32-bit integer
- **Default:** `80`

Candidate pool size for Discover mode on the stroll page. In Discover mode, the server fetches this many nearest neighbors and then randomly samples from the pool to produce the displayed satellites. Larger values increase variety; smaller values keep results closer to the center image.

### stroll_fanout

- **Type:** unsigned 32-bit integer
- **Default:** `5`

Initial fan-out count for L2 transitive neighbors on the stroll page. Controls how many second-level neighbors fan out from a focused satellite. Set to `0` to disable fan-out by default.

### stroll_fanout_max

- **Type:** unsigned 32-bit integer
- **Default:** `10`

Maximum value for the fan-out slider on the stroll page.

### CLI Override

The `--port`, `--bind`, and `--per-page` flags on `maki serve` override the values from `maki.toml`:

```
maki serve --port 9090 --bind 0.0.0.0 --per-page 100
```

```
[serve]
port = 8080
bind = "127.0.0.1"
per_page = 100
stroll_neighbors = 12
stroll_neighbors_max = 25
stroll_fanout = 5
stroll_fanout_max = 10
stroll_discover_pool = 80
```

---

## [import] Section

Controls import behavior for `maki import`.

### exclude

- **Type:** array of glob pattern strings
- **Default:** `[]` (empty, nothing excluded)

Glob patterns matched against filenames (not full paths). Files matching any pattern are skipped during import. Useful for excluding OS-generated files, temporary files, and other non-media clutter.

Pattern matching uses the `glob-match` crate with standard glob syntax: `*` matches any sequence of characters, `?` matches a single character.

```
[import]
exclude = [
    "Thumbs.db",
    ".DS_Store",
    "*.tmp",
    "*.bak",
```

```
    "desktop.ini",
]
```

### auto_tags

- **Type:** array of tag strings
- **Default:** `[]` (empty, no auto-tags)

Tags automatically applied to every newly imported asset. These are merged with any tags extracted from XMP metadata, CLI `--add-tag` values, and deduplicated (no duplicate tags are created).

Useful for marking import batches or applying a default workflow status:

```
[import]
auto_tags = ["inbox", "unreviewed"]
```

### smart_previews

- **Type:** boolean
- **Default:** `false`

When `true`, import automatically generates smart previews (high-resolution, 2560px) alongside regular thumbnails. Equivalent to passing `--smart` on every `maki import` command. Smart preview dimensions are controlled by `[preview] smart_max_edge`.

```
[import]
smart_previews = true
```

### embeddings

> **Feature-gated**: requires building with `--features ai`.

- **Type:** boolean
- **Default:** `false`

When `true`, import automatically generates SigLIP image embeddings for visual similarity search alongside previews. Equivalent to passing `--embed` on every `maki import` command. Embeddings enable `maki auto-tag --similar` and the web UI "Find similar" button.

Uses the model configured in `[ai] model`. Silently skips if the model is not downloaded. Non-image assets are skipped.

```
[import]
embeddings = true
```

### descriptions

- **Type:** boolean
- **Default:** `false`

When `true`, import automatically generates VLM descriptions for newly imported assets as a post-import phase. Equivalent to passing `--describe` on every `maki import` command.

Uses the VLM configured in `[vlm]` (endpoint, model, prompt, mode, temperature). Silently skips if the VLM endpoint is not available. Assets that already have descriptions are skipped. Works with all `[vlm] mode` settings (describe, tags, both).

```
[import]
descriptions = true
```

---

# [dedup] Section

Controls dedup behavior for `maki dedup` and the web UI's auto-resolve action.

### prefer

- **Type:** string (optional)
- **Default:** none

Default path substring for the `--prefer` flag. When set, dedup prefers keeping file locations whose relative path contains this string. Useful for always keeping files in a curated directory (e.g. `Selects`) while removing copies elsewhere.

The CLI `--prefer` flag overrides this value. The web UI duplicates page pre-populates its "Prefer keeping" input from this setting.

```
[dedup]
prefer = "Selects"
```

---

# [verify] Section

Controls incremental verify behavior for `maki verify`.

### max_age_days

- **Type:** integer (optional)
- **Default:** none

Default value for the `--max-age` flag. When set, `maki verify` skips files verified within the given number of days. The CLI `--max-age` flag overrides this value. `--force` overrides both.

```
[verify]
max_age_days = 30
```

---

# [contact_sheet] Section

Default settings for `maki contact-sheet`. All fields are optional; CLI flags override these values.

| Key | Type | Default | Description |
|---|---|---|---|
| `layout` | string | `"standard"` | Layout preset: `dense`, `standard`, `large` |
| `paper` | string | `"a4"` | Paper size: `a4`, `letter`, `a3` |
| `fields` | string | `"filename,date,rating"` | Comma-separated metadata fields |
| `margin` | float | `15.0` | Page margin in mm |
| `quality` | integer | `92` | JPEG quality for page images (1–100) |

| Key | Type | Default | Description |
|---|---|---|---|
| `label_style` | string | `"border"` | Color label display: `border`, `dot`, `none` |
| `copyright` | string | `""` | Copyright text for page footer |

```
[contact_sheet]
layout = "dense"
paper = "a3"
label_style = "dot"
copyright = "© 2026 Thomas Herrmann"
```

---

# [ai] Section

> **Feature-gated**: these settings only take effect when maki is built with `--features ai`.

Controls AI auto-tagging behavior for `maki auto-tag`.

## model

- **Type:** string
- **Default:** `"siglip-vit-b16-256"`

Which SigLIP model to use. Available models: `siglip-vit-b16-256` (768-dim, ~207 MB, good balance) and `siglip-vit-l16-256` (1024-dim, ~670 MB, higher accuracy). The CLI `--model` flag overrides this value. Embeddings are stored per model, so switching models doesn't corrupt existing data.

## threshold

- **Type:** float (0.0–1.0)
- **Default:** `0.1`
- **Validation:** must be between 0.0 and 1.0

Minimum confidence score for a tag to be suggested. Higher values produce fewer but more confident suggestions. Lower values produce more suggestions with more noise.

## labels

- **Type:** string (file path, optional)
- **Default:** none (uses built-in ~100 photography categories)

Path to a custom labels file (one label per line). When set, overrides the built-in default labels. The CLI `--labels` flag overrides this value.

## model_dir

- **Type:** string (directory path)
- **Default:** `"~/.maki/models"`

Where to cache downloaded model files. The ~ prefix is expanded to the user's home directory.

### prompt

- **Type:** string
- **Default:** `"a photograph of {}"`

Text encoder prompt template. The `{}` placeholder is replaced with each label name before encoding. Adjusting the prompt can improve classification accuracy for specific use cases (e.g., `"a photo of a {}"` or `"a professional photograph of {}"`).

### execution_provider

- **Type:** string (`"auto"`, `"cpu"`, `"coreml"`)
- **Default:** `"auto"`

  Requires building with `--features ai-gpu` for GPU providers. With `--features ai` only, all values fall back to CPU.

Selects the ONNX Runtime execution provider for AI inference (SigLIP, YuNet, ArcFace). `"auto"` uses CoreML when available on macOS (Neural Engine on Apple Silicon, Metal on Intel), falling back to CPU. `"cpu"` forces CPU-only inference. `"coreml"` explicitly requests CoreML (errors if unavailable).

### face_cluster_threshold

- **Type:** float (0.0–1.0)
- **Default:** `0.5`

Similarity threshold for face auto-clustering. Higher values require faces to be more similar to be grouped into the same person. Lower values produce larger groups with more potential false matches.

### face_min_confidence

- **Type:** float (0.0–1.0)
- **Default:** `0.5`

Minimum confidence score for a face detection to be stored. Faces below this threshold are discarded during detection.

### text_limit

- **Type:** unsigned integer
- **Default:** `50`

Maximum number of results returned by `text:` semantic search queries. This is the default limit used when no inline limit is specified in the query. Can be overridden per-query with the `text:"query":limit` syntax.

```
[ai]
threshold = 0.3
labels = "my-labels.txt"
model_dir = "~/.maki/models"
prompt = "a photograph of {}"
execution_provider = "auto"
face_cluster_threshold = 0.5
face_min_confidence = 0.5
text_limit = 50
```

---

# [vlm] Section

Controls the VLM (vision-language model) integration for `maki describe`. Unlike the `[ai]` section, this requires no special build features – it works with any maki binary because it uses HTTP calls to an external server.

## endpoint

- **Type:** string (URL)
- **Default:** `"http://localhost:11434"`

Base URL of the VLM server. Any server implementing the OpenAI-compatible `/v1/chat/completions` endpoint works. Ollama also supports its native `/api/generate` endpoint as an automatic fallback.

**Local servers:** - Ollama – `http://localhost:11434` (default) - LM Studio – `http://localhost:1234` - llama.cpp server – `http://localhost:8080`

**Cloud APIs** (OpenAI-compatible, require API key in environment): - OpenAI – `https://api.openai.com` (model: `gpt-4o`) - Groq – `https://api.groq.com/openai` (model: `llama-3.2-90b-vision-preview`) - Together AI – `https://api.together.xyz` (model: `meta-llama/Llama-3.2-90B-Vision-Instruct-Turbo`)

> **Note:** Cloud APIs charge per request. maki does not set authentication headers – if your endpoint requires an API key, you may need to use a local proxy or set the key via the server's own configuration.

## model

- **Type:** string
- **Default:** `"qwen2.5vl:3b"`

Model name passed to the VLM server. For Ollama, this is the model tag (e.g., `moondream`, `qwen2.5vl:3b`, `qwen2.5vl:7b`). For cloud APIs, this is the model identifier (e.g., `gpt-4o`).

**Recommended models for photography** (tested with Ollama on Apple Silicon):

| Model | Size | RAM | Speed (M3 Pro) | Quality |
|---|---|---|---|---|
| Moondream 2B | 1.7 GB | ~2 GB | ~3–5s | Good, fast for batch |
| Qwen2.5-VL 3B | 2.0 GB | ~3 GB | ~8–12s | Very good (default) |
| Qwen3-VL 4B | 2.8 GB | ~4 GB | ~10–15s | Very good |
| Gemma 3 4B | 3.3 GB | ~4 GB | ~10–15s | Very good |
| Qwen3-VL 8B | 5.2 GB | ~6 GB | ~15–20s | Excellent |
| Qwen2.5-VL 7B | 4.7 GB | ~6 GB | ~20–36s | Excellent |

See VLM Model Guide for a comprehensive comparison including Qwen3.5, backend setup instructions, and hardware recommendations.

## max_tokens

- **Type:** unsigned 32-bit integer
- **Default:** `500`

Maximum number of tokens in the VLM response. 500 tokens is typically 3–5 sentences, leaving headroom for models that use internal reasoning tokens.

### prompt

- **Type:** string (optional)
- **Default:** none (uses built-in photography-focused prompt)

Custom system prompt sent to the VLM. When not set, uses a built-in prompt appropriate for the mode (describe or tags). In `--mode both`, custom prompts are ignored because each of the two calls uses its specialized built-in prompt.

Override for specialized workflows:

```
[vlm]
prompt = "Describe the architectural style, materials, and notable design features."
```

### mode

- **Type:** string
- **Default:** `"describe"`

Default output mode for `maki describe`. One of: `describe` (natural language description), `tags` (JSON tag suggestions), `both` (two separate VLM calls: one for description, one for tags).

### temperature

- **Type:** float
- **Default:** `0.7`

Sampling temperature controlling randomness in VLM output. `0.0` = deterministic (same input always produces the same output), `0.7` = balanced (default), `1.0+` = more creative/varied. Lower values give more consistent results; higher values give more diverse but potentially less accurate output.

### timeout

- **Type:** unsigned 32-bit integer (seconds)
- **Default:** `300`

Maximum time to wait for a VLM response. Larger models on CPU may need higher timeouts. Assets that time out are reported as errors and skipped.

### concurrency

- **Type:** unsigned 32-bit integer
- **Default:** `1`

Number of concurrent VLM requests. When greater than 1, `maki describe`, `maki import --describe`, and web UI batch describe process multiple assets in parallel. Each batch of `concurrency` assets sends VLM HTTP calls concurrently using scoped threads; preparation (skip checks, image lookup) and result application (catalog writes) remain sequential. Set to the number of simultaneous requests your VLM server can handle efficiently — for local Ollama this depends on available VRAM and model size.

### models

- **Type:** array of strings (optional)
- **Default:** `[]` (empty — only the default model is shown)

List of VLM model names to offer in the web UI model selector dropdown on the asset detail page. When configured with two or more models (including the default `model`), a dropdown appears next to the "Describe" button letting you choose which model to use per request. The first entry is pre-selected. Models not in this

list are not offered in the web UI (useful when your Ollama server has many models loaded but only some are suitable for image description).

```
[vlm]
model = "moondream"
models = ["moondream", "qwen3-vl:4b"]
```

The CLI `maki describe --model` flag is unaffected by this setting — it accepts any model name regardless of the `models` list.

### num_ctx

- **Type:** unsigned 32-bit integer
- **Default:** `0` (not set — use server default)

Context window size passed to the VLM server. When non-zero, overrides the model's default context length. Useful for models that benefit from a larger context (e.g., `num_ctx = 4096` for Qwen models). A value of `0` means "not set" — the server uses its own default.

### top_p

- **Type:** float
- **Default:** `0.0` (not set — use server default)

Nucleus sampling parameter. When non-zero, only tokens whose cumulative probability exceeds `top_p` are considered. Lower values produce more focused output. A value of `0.0` means "not set" — the server uses its own default.

### top_k

- **Type:** unsigned 32-bit integer
- **Default:** `0` (not set — use server default)

Top-K sampling parameter. When non-zero, limits token selection to the K most probable candidates. A value of `0` means "not set" — the server uses its own default.

### repeat_penalty

- **Type:** float
- **Default:** `0.0` (not set — use server default)

Repetition penalty applied to tokens that have already appeared. Values above `1.0` discourage repetition; `1.0` disables the penalty. A value of `0.0` means "not set" — the server uses its own default.

## Per-Model Configuration

You can override any VLM setting for specific models using `[vlm.model_config."model-name"]` sections. When `maki describe` runs with a given model (via `--model` or `[vlm] model`), any matching per-model section is merged on top of the global `[vlm]` settings. CLI flags always win over both.

This is useful when different models need different timeouts, context sizes, or sampling parameters:

```
[vlm]
model = "qwen2.5vl:3b"
timeout = 300

[vlm.model_config."qwen3-vl:4b"]
max_image_edge = 384
```

```toml
timeout = 300
num_ctx = 4096

[vlm.model_config."moondream:latest"]
max_tokens = 200
temperature = 0.1
```

When running `maki describe --model qwen3-vl:4b`, the per-model overrides apply: `timeout` becomes 300, `num_ctx` becomes 4096, and `max_image_edge` becomes 384. All other settings fall through from the global `[vlm]` section.

Per-model sections support all the same fields as the global `[vlm]` section (except `model`, `models`, and `model_config` itself).

### CLI Override

The `--endpoint`, `--model`, `--prompt`, `--max-tokens`, `--timeout`, `--temperature`, `--mode`, `--num-ctx`, `--top-p`, `--top-k`, and `--repeat-penalty` flags on `maki describe` override the values from `maki.toml` (including per-model overrides).

```toml
[vlm]
endpoint = "http://localhost:11434"
model = "qwen2.5vl:3b"
max_tokens = 500
timeout = 300
temperature = 0.7
mode = "describe"
concurrency = 1
num_ctx = 0
top_p = 0.0
top_k = 0
repeat_penalty = 0.0
# models = ["qwen2.5vl:3b", "moondream"]
# prompt = "Custom prompt here."

# Per-model overrides (optional)
# [vlm.model_config."qwen3-vl:4b"]
# timeout = 300
# num_ctx = 4096
```

---

## [browse] Section

Controls default browsing behavior for both the CLI `maki search` and the web UI.

| Field | Type | Default | Description |
|---|---|---|---|
| `default_filter` | string | *(none)* | Search filter applied to all browse, search, and stroll views. Uses the same syntax as the search bar. |

The default filter is AND'd with whatever the user types — it acts as a persistent base filter. In the web UI, a toggle next to the search bar lets you temporarily disable it.

**Scope**: Applied to browse, search results, stroll, analytics, and map views. NOT applied to operational commands (`export`, `describe`, `contact-sheet`, `auto-tag`, etc.) where you pass your own explicit query.

### Examples

```
# Only show rated assets (hide unreviewed and unrated)
[browse]
default_filter = "rating:1+"

# Hide assets tagged "rest" (recommended for most users)
[browse]
default_filter = "-tag:rest"

# Combine multiple conditions
[browse]
default_filter = "-tag:rest -tag:rejected"

# Show only images
[browse]
default_filter = "type:image"
```

---

## Full Example

A complete `maki.toml` with all options set and annotated:

```
# Default volume for import when auto-detection is ambiguous.
# Find volume UUIDs with: maki volume list --json
default_volume = "550e8400-e29b-41d4-a716-446655440000"

[preview]
# Maximum pixel size of the longest edge for thumbnails.
max_edge = 1200
# Output format: "jpeg" (lossy) or "webp" (lossless).
format = "jpeg"
# JPEG quality (1-100). Ignored for WebP.
quality = 90
# Smart preview: maximum pixel size of the longest edge.
smart_max_edge = 2560
# Smart preview: JPEG quality (1-100).
smart_quality = 85
# Generate smart previews on first web request.
generate_on_demand = true

[serve]
# Web UI port. Override with: maki serve --port 9090
port = 8080
# Bind address. Use "0.0.0.0" to allow network access.
bind = "127.0.0.1"
# Stroll page: initial neighbor count and slider maximum.
stroll_neighbors = 12
stroll_neighbors_max = 25
# Stroll page: initial fan-out count and slider maximum.
```

```
stroll_fanout = 5
stroll_fanout_max = 10
# Stroll page: candidate pool size for Discover mode.
stroll_discover_pool = 80

[import]
# Glob patterns to exclude during import (matched against filenames).
exclude = [
    "Thumbs.db",
    ".DS_Store",
    "*.tmp",
    "*.bak",
    "desktop.ini",
]
# Tags automatically applied to every new asset.
auto_tags = ["inbox", "unreviewed"]
# Generate smart previews during import.
smart_previews = true
# Generate embeddings for visual similarity search during import (--features ai).
embeddings = true
# Generate VLM descriptions during import (requires running Ollama or compatible endpoint).
descriptions = true

[dedup]
# Default path substring for --prefer (keep files whose path contains this).
prefer = "Selects"

[verify]
# Skip files verified within this many days (incremental verify).
max_age_days = 30

[contact_sheet]
# Layout preset: "dense", "standard", "large".
layout = "standard"
# Paper size: "a4", "letter", "a3".
paper = "a4"
# Comma-separated metadata fields below each thumbnail.
fields = "filename,date,rating"
# Page margin in mm.
margin = 15.0
# JPEG quality for page images (1-100).
quality = 92
# Color label display: "border", "dot", "none".
label_style = "border"
# Copyright text for page footer.
copyright = ""

# AI auto-tagging settings (only with --features ai).
[ai]
model = "siglip-vit-b16-256"
threshold = 0.3
# labels = "my-labels.txt"
model_dir = "~/.maki/models"
```

```
prompt = "a photograph of {}"
# GPU acceleration (requires --features ai-gpu).
# execution_provider = "auto"

# VLM image description settings (works with any build).
[vlm]
endpoint = "http://localhost:11434"
model = "qwen2.5vl:3b"
max_tokens = 500
timeout = 300
temperature = 0.7
mode = "describe"
# Models offered in web UI dropdown (empty = only default model, no dropdown).
# models = ["qwen2.5vl:3b", "moondream"]
# prompt = "Describe this photograph concisely."
# Sampling parameters (0 = not set, use server default).
# num_ctx = 4096
# top_p = 0.9
# top_k = 40
# repeat_penalty = 1.1

# Per-model VLM overrides (optional).
# [vlm.model_config."moondream:latest"]
# max_tokens = 200
# temperature = 0.1
```

---

## Minimal Examples

### Preview-only: larger thumbnails in WebP

```
[preview]
max_edge = 1600
format = "webp"
```

### Import-only: exclude system files and auto-tag

```
[import]
exclude = [".DS_Store", "Thumbs.db", "*.tmp"]
auto_tags = ["2026-import"]
```

### Serve-only: custom port for LAN access

```
[serve]
port = 9090
bind = "0.0.0.0"
```

---

# Defaults Summary

When a field is absent from `maki.toml`, these defaults apply:

| Field | Default |
|---|---|
| default_volume | none |
| preview.max_edge | 800 |
| preview.format | "jpeg" |
| preview.quality | 85 |
| preview.smart_max_edge | 2560 |
| preview.smart_quality | 85 |
| preview.generate_on_demand | false |
| serve.port | 8080 |
| serve.bind | "127.0.0.1" |
| serve.per_page | 60 |
| serve.stroll_neighbors | 12 |
| serve.stroll_neighbors_max | 25 |
| serve.stroll_fanout | 5 |
| serve.stroll_fanout_max | 10 |
| serve.stroll_discover_pool | 80 |
| import.exclude | [] |
| import.auto_tags | [] |
| import.smart_previews | false |
| import.embeddings | false |
| import.descriptions | false |
| dedup.prefer | none |
| verify.max_age_days | none |
| contact_sheet.layout | "standard" |
| contact_sheet.paper | "a4" |
| contact_sheet.fields | "filename,date,rating" |
| contact_sheet.margin | 15.0 |
| contact_sheet.quality | 92 |
| contact_sheet.label_style | "border" |
| contact_sheet.copyright | "" |
| ai.model | "siglip-vit-b16-256" |
| ai.threshold | 0.1 |
| ai.labels | none |
| ai.model_dir | "~/.maki/models" |
| ai.prompt | "a photograph of {}" |
| ai.execution_provider | "auto" |
| ai.face_cluster_threshold | 0.5 |
| ai.face_min_confidence | 0.5 |
| ai.text_limit | 50 |
| vlm.endpoint | "http://localhost:11434" |
| vlm.model | "qwen2.5vl:3b" |
| vlm.max_tokens | 500 |
| vlm.prompt | none (built-in) |
| vlm.mode | "describe" |
| vlm.temperature | 0.7 |
| vlm.timeout | 300 |
| vlm.concurrency | 1 |
| vlm.models | [] (empty) |
| vlm.num_ctx | 0 (server default) |

| Field | Default |
|---|---|
| `vlm.top_p` | `0.0` (server default) |
| `vlm.top_k` | `0` (server default) |
| `vlm.repeat_penalty` | `0.0` (server default) |
| `browse.default_filter` | none |

---

## Related Topics

- Setup (User Guide) – creating a catalog with `maki init`
- CLI Conventions – global flags and catalog discovery
- Search Filters Reference – search query syntax

# Data Model

This page documents every entity in the maki data model, their fields, relationships, and storage mechanisms.

---

## Entities

### Asset

The top-level entity. An Asset represents a single logical media item – "photo of sunset at the beach" – regardless of how many physical files exist for it.

| Field | Type | Description |
|---|---|---|
| `id` | UUID | Primary key. Deterministic UUID v5 derived from the content hash of the first variant. Same content always produces the same asset ID. |
| `name` | Option<String> | User-assigned display name. When absent, the UI shows `original_filename` as a fallback. |
| `original_filename` | String | Filename of the primary variant at import time (e.g. `DSC_4521.NEF`). Used as display fallback and for stem-based grouping. |
| `asset_type` | AssetType | One of: `image`, `video`, `audio`, `document`, `other`. Inferred from file extension at import. |
| `description` | Option<String> | Free-text description. Extracted from XMP `dc:description` during import, or set manually. |
| `tags` | Vec<String> | Keyword list. Merged from XMP `dc:subject`, embedded XMP, and manual tagging. Deduplicated. |
| `rating` | Option<u8> | Star rating, 1–5. Extracted from XMP `xmp:Rating` during import, or set manually. |
| `color_label` | Option<String> | One of 7 canonical colors: Red, Orange, Yellow, Green, Blue, Pink, Purple. Extracted from XMP `xmp:Label`, or set manually. Stored as title-case English name. |
| `created_at` | DateTime<Utc> | Creation timestamp. Preferentially from EXIF `DateTimeOriginal`, falling back to filesystem modification time. |
| `variants` | Vec<Variant> | The physical files belonging to this asset (in YAML sidecar). |
| `recipes` | Vec<Recipe> | Processing sidecars attached to this asset's variants (in YAML sidecar). |

**Denormalized columns** (SQLite only, computed at write time to avoid expensive JOINs):

| Column | Type | Description |
|---|---|---|
| `best_variant_hash` | String | Content hash of the best display variant (see Display Priority). Used for the browse grid JOIN. |
| `primary_variant_format` | String | Identity format of the asset. Prefers Original+RAW, then Original+any, then best variant's format. Shown on browse cards (e.g. "NEF"). |
| `variant_count` | Integer | Number of variants. Shown as a badge on browse cards (e.g. "3v"). |
| `face_count` | Integer | Number of detected faces. Shown as a badge on browse cards. Only present with `--features ai`. |
| `stack_id` | Option<UUID> | Foreign key to the Stack this asset belongs to. `None` if unstacked. |
| `stack_position` | Option<Integer> | Position within the stack (0 = pick). `None` if unstacked. |

The variant-related columns are updated by `insert_asset()`, `update_denormalized_variant_columns()`, and `fix_roles`. The stack columns are updated by `StackStore` operations. All are backfilled during schema migration and rebuilt by `rebuild-catalog`.

## Variant

A concrete file belonging to an Asset. A RAW file, its JPEG conversion, and a high-res TIFF export are three Variants of the same Asset.

| Field | Type | Description |
|---|---|---|
| `content_hash` | String | Primary key. SHA-256 hash of the file contents. The same file always produces the same hash regardless of where it is stored. |
| `asset_id` | UUID | Foreign key to the parent Asset. |
| `role` | VariantRole | Purpose within the asset group: `original`, `alternate`, `processed`, `export`, or `sidecar`. See Variant Roles. |
| `format` | String | Lowercase file extension without dot (e.g. `nef`, `jpg`, `tif`, `mp4`). |
| `file_size` | u64 | File size in bytes. |
| `original_filename` | String | Filename at import time (e.g. `DSC_4521.NEF`). |
| `source_metadata` | HashMap<String, String> | Key-value pairs from EXIF and XMP extraction (camera model, lens, GPS coordinates, creator, rights, etc.). |
| `locations` | Vec<FileLocation> | Where this file physically exists on disk (in YAML sidecar; stored in a separate `file_locations` table in SQLite). |

**Indexed metadata columns** (SQLite only, extracted from `source_metadata` for fast filtering):

| Column | Type | Description |
|---|---|---|
| `camera_model` | String | Camera body (e.g. “NIKON Z 9”) |
| `lens_model` | String | Lens (e.g. “NIKKOR Z 50mm f/1.2 S”) |
| `focal_length_mm` | Real | Focal length in millimeters |
| `f_number` | Real | Aperture f-number |
| `iso` | Integer | ISO sensitivity |
| `image_width` | Integer | Image width in pixels |
| `image_height` | Integer | Image height in pixels |

## FileLocation

A pointer to where a Variant physically lives on disk. A single Variant can have multiple FileLocations – copies on different drives, backups, archives.

| Field | Type | Description |
|---|---|---|
| `id` | Integer | Primary key (auto-increment, SQLite only). |
| `content_hash` | String | Foreign key to the parent Variant. |
| `volume_id` | UUID | Foreign key to the Volume where the file resides. |
| `relative_path` | String | Path relative to the volume’s mount point (e.g. `Capture/2026-02-22/DSC_4521.NEF`). |
| `verified_at` | Option<DateTime<Utc» | Timestamp of the last successful integrity check via `maki verify`. `None` if never verified. |

## Recipe

A processing sidecar file attached to a Variant. Unlike Variants, Recipes are identified by **location** (volume + path) rather than content hash, because external tools like CaptureOne routinely modify them in place.

| Field | Type | Description |
|---|---|---|
| `id` | UUID | Primary key. |
| `variant_hash` | String | Foreign key to the parent Variant. |
| `software` | String | Processing tool identifier: `XMP`, `CaptureOne`, `RawTherapee`, `DxO`, `ON1`. |
| `recipe_type` | RecipeType | Either `sidecar` (external file) or `embedded_export`. |
| `content_hash` | String | SHA-256 hash of the recipe file. Updated when the file changes on disk. |
| `volume_id` | UUID | Foreign key to the Volume where the recipe file resides. |
| `relative_path` | String | Path relative to the volume’s mount point. |
| `verified_at` | Option<DateTime<Utc» | Last verification timestamp. |

**Supported recipe file extensions**:

| Extension | Software |
|---|---|
| `.xmp` | XMP (Lightroom, CaptureOne, Adobe) |
| `.cos` | CaptureOne settings |
| `.cot` | CaptureOne output |
| `.cop` | CaptureOne process |
| `.pp3` | RawTherapee |

| Extension | Software |
|---|---|
| `.dop` | DxO PhotoLab |
| `.on1` | ON1 Photo RAW |

## Volume

A registered storage device. Volumes give maki a stable reference to storage that may come and go (external drives, network shares).

| Field | Type | Description |
|---|---|---|
| `id` | UUID | Primary key (random UUID v4). |
| `label` | String | Human-readable name (e.g. "Photos SSD", "Archive NAS"). |
| `mount_point` | PathBuf | Filesystem path where the volume is mounted (e.g. `/Volumes/Photos`). |
| `volume_type` | VolumeType | One of: `local`, `external`, `network`. |
| `purpose` | Option<VolumePurpose> | Logical role: `working` (active editing), `archive` (long-term primary), `backup` (redundancy), `cloud` (sync folder). Optional — unclassified if not set. Used by duplicate analysis and backup coverage commands. |
| `is_online` | bool | Computed at runtime – `true` if `mount_point` exists on disk. Not persisted. |

## Collection

A manually curated list of assets (static album). Backed by both SQLite (for fast queries) and `collections.yaml` (for persistence across catalog rebuilds).

| Field | Type | Description |
|---|---|---|
| `id` | UUID | Primary key. |
| `name` | String | Unique human-readable name (e.g. "Portfolio", "Client Deliverables"). |
| `description` | Option<String> | Optional description text. |
| `created_at` | DateTime<Utc> | When the collection was created. |
| `asset_ids` | Vec<String> | Ordered list of asset UUIDs (in YAML). In SQLite, this is a separate `collection_assets` join table with `(collection_id, asset_id, added_at)`. |

## Stack

A lightweight anonymous group of assets for visually related images (burst shots, bracketing sequences, similar scenes). In the browse grid, stacked assets are collapsed to show only the "pick" image with a count badge.

| Field | Type | Description |
|---|---|---|
| `id` | UUID | Primary key (random UUID v4). |
| `created_at` | DateTime<Utc> | When the stack was created. |

| Field | Type | Description |
|---|---|---|
| `asset_ids` | Vec<String> | Ordered list of asset UUIDs. Index 0 is the pick (displayed in browse grid). |

**Constraints**: - Each asset can belong to at most one stack. - A stack must have at least 2 members. Removing members that would leave fewer than 2 causes automatic dissolution. - Stack membership is denormalized onto the `assets` table as `stack_id` (FK to the stack) and `stack_position` (integer, 0 = pick). These columns enable efficient filtering (`stacked:true/false`) and stack collapsing in browse queries without joining a separate table.

**Storage**: Stacks are persisted in `stacks.yaml` at the catalog root (alongside `collections.yaml` and `searches.toml`). The SQLite `stacks` table and the `stack_id`/`stack_position` columns on `assets` are derived from this file and rebuilt by `rebuild-catalog`.

## SavedSearch

A named query (smart album) stored in `searches.toml`. Re-evaluated every time it is run, so results update automatically as the catalog changes.

| Field | Type | Description |
|---|---|---|
| `name` | String | Unique identifier. |
| `query` | String | Search filter string in the same syntax as `maki search` (e.g. `type:image tag:landscape rating:4+`). |
| `sort` | Option<String> | Sort order (e.g. `date_desc`, `name_asc`). Omitted means default (`date_desc`). |

## Embedding

> Only present when built with `--features ai`.

A stored image embedding vector for an asset, used by `maki auto-tag` for classification, `--similar` for visual similarity search, and `maki embed` for batch generation.

| Field | Type | Description |
|---|---|---|
| `asset_id` | String | Primary key. Foreign key to the parent Asset. |
| `embedding` | Blob | 768-dimensional float32 vector (3072 bytes), stored as little-endian binary. |
| `model` | String | Model identifier (default: `siglip-vit-b16-256`). Ensures embeddings from different models are not compared. |

Storage overhead: ~3 KB per asset. For 100,000 assets: ~300 MB in SQLite.

**In-memory index**: For fast similarity search, the web server loads all embeddings into an `EmbeddingIndex` — a contiguous `Vec<f32>` buffer — on first query. Search uses dot product (SigLIP embeddings are L2-normalized) with a min-heap for top-K selection. At 100k assets, search completes in <10ms. The index is updated in-place when new embeddings are stored.

**Opportunistic storage**: Embeddings are stored not only by `maki auto-tag` and `maki embed`, but also opportunistically by the web UI "Suggest tags" and batch "Auto-tag" endpoints. This means using AI features in the web UI gradually builds up the similarity search index.

### Face

Only present when built with `--features ai`.

A detected face within an asset image, with bounding box, confidence, recognition embedding, and optional person assignment.

| Field | Type | Description |
|---|---|---|
| `id` | UUID | Primary key. |
| `asset_id` | UUID | Foreign key to the parent Asset. |
| `person_id` | Option<UUID> | Foreign key to the assigned Person. `None` if unassigned. |
| `bbox_x` | f32 | Bounding box X position (normalized 0–1). |
| `bbox_y` | f32 | Bounding box Y position (normalized 0–1). |
| `bbox_w` | f32 | Bounding box width (normalized 0–1). |
| `bbox_h` | f32 | Bounding box height (normalized 0–1). |
| `confidence` | f32 | Detection confidence score (0–1). |
| `embedding` | Blob | 512-dimensional float32 ArcFace vector (2048 bytes), stored as little-endian binary. |
| `crop_path` | Option<String> | Path to 150×150 JPEG face crop thumbnail (relative to catalog root). |
| `created_at` | DateTime<Utc> | When the face was detected. |

Storage overhead: ~2 KB per face (embedding + metadata). Face crops: ~5–15 KB each as JPEG.

### Person

Only present when built with `--features ai`.

A named or unnamed person group linking detected faces across assets.

| Field | Type | Description |
|---|---|---|
| `id` | UUID | Primary key. |
| `name` | Option<String> | User-assigned name. `None` for unnamed clusters. |
| `representative_face_id` | Option<UUID> | Foreign key to the face used as the person's thumbnail. |
| `created_at` | DateTime<Utc> | When the person record was created. |

---

## Variant Roles

Each Variant carries a `role` that describes its purpose within the asset group.

| Role | Meaning | Examples |
|---|---|---|
| **Original** | Camera source file. Each asset should have exactly one. | NEF, ARW, CR3, in-camera JPEG |
| **Alternate** | Secondary variant from grouping (e.g., JPEG paired with RAW original). | In-camera JPEG grouped with RAW |
| **Processed** | An edited or intermediate version, not straight from camera. | PSD, layered TIFF, edited DNG |

| Role | Meaning | Examples |
|---|---|---|
| **Export** | A derivative output produced by an editing tool. | Resized JPEG, web TIFF, final deliverable |
| **Sidecar** | A non-media sidecar file imported as a variant. | Embedded metadata files |

When assets are merged via `maki group` or `maki auto-group`, donor variants with the `original` role are automatically re-roled to `alternate` to avoid having multiple originals in one asset.

---

## Display Priority

The preview selection algorithm determines which variant is shown in the browse grid and asset detail page. The scoring follows this priority:

1. **Role** (highest weight): Export (300) > Processed (200) > Original (100) > Alternate (50) > Sidecar (0)
2. **Format bonus** (+50): Standard image formats (`jpg`, `jpeg`, `png`, `tiff`, `tif`, `webp`) are preferred over RAW
3. **File size tiebreak**: Larger files score slightly higher (up to +49 points)

This means a JPEG export is preferred over a RAW original for display, showing your best-quality deliverable in the browse grid rather than the camera file.

The `best_variant_hash` denormalized column caches this computation so the browse grid can join directly to the best variant without evaluating all variants per asset.

---

## Storage

maki uses a dual-storage architecture. Neither tier alone is sufficient; together they provide both robustness and performance.

### YAML Sidecar Files (source of truth)

One `.yml` file per Asset, stored at `metadata/<id-prefix>/<id>.yaml` within the catalog directory. Contains the complete Asset record: metadata, all Variants (with their FileLocations and source_metadata), and all Recipes. Human-readable, diffable, and version-control friendly.

```
catalog/
  metadata/
    3a/
      3a7b1e02-4fd2-4a6b-9c1d-e75a0bf3284c.yaml
    f1/
      f1c8d9e0-...yaml
```

### SQLite Catalog (derived cache)

A single `catalog.db` file providing fast indexed queries. Contains denormalized columns for efficient browse-grid rendering. The catalog is always rebuildable from the YAML sidecars via `maki rebuild-catalog` – it is a performance optimization, not a source of truth.

**Tables**: `assets`, `variants`, `file_locations`, `volumes`, `recipes`, `collections`, `collection_assets`, `stacks`, `embeddings`, `faces`, `people` (last three with `--features ai`)

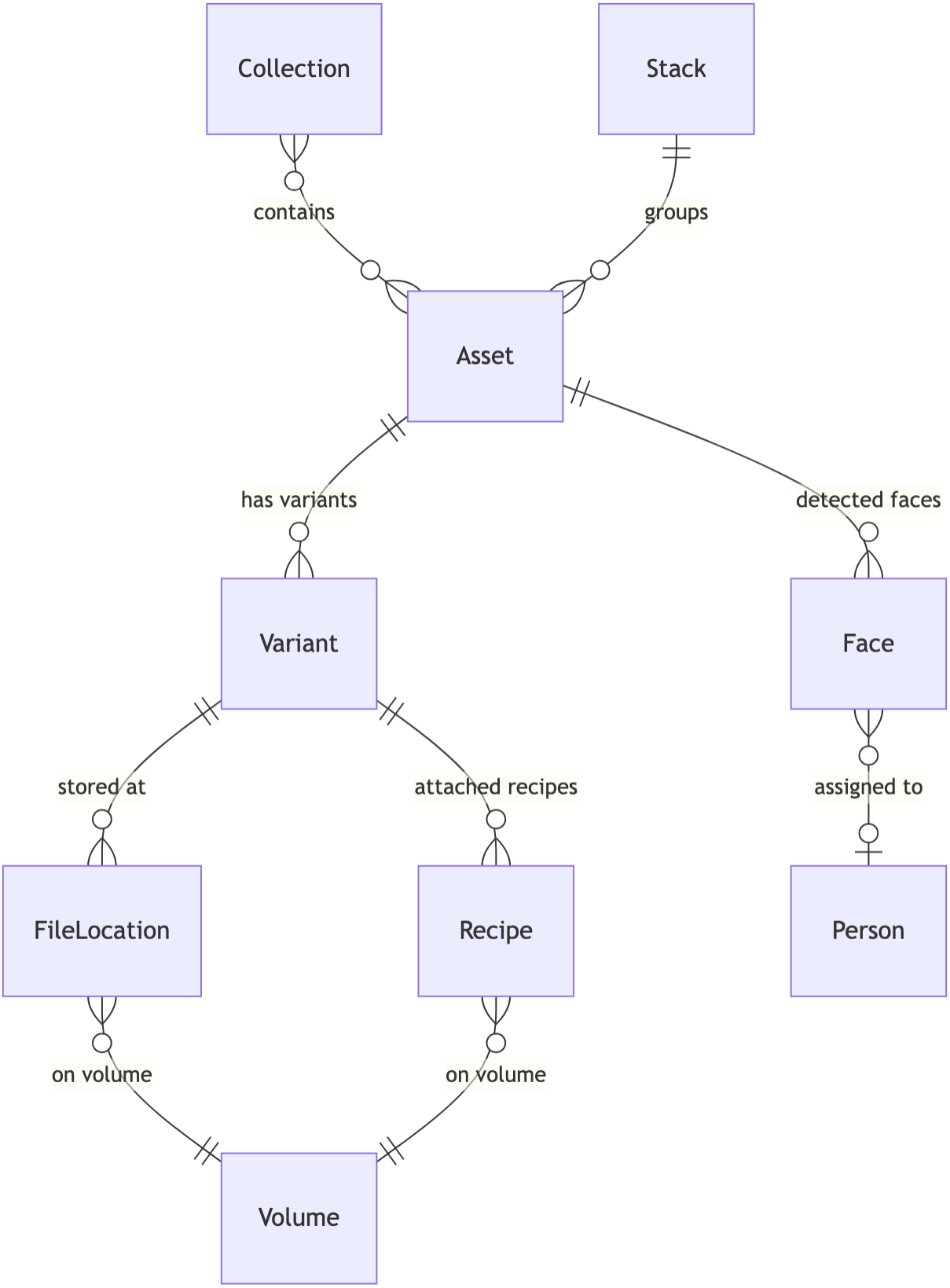
**Performance indexes** (created automatically via schema migrations):

- `variants(asset_id)`, `variants(format)`, `variants(camera_model)`, `variants(lens_model)`, `variants(iso)`, `variants(focal_length_mm)` — variant lookups and filter queries
- `file_locations(content_hash)`, `file_locations(volume_id)` — join and volume filter performance
- `assets(created_at)`, `assets(best_variant_hash)` — sort-by-date and best-variant join
- `recipes(variant_hash)` — recipe lookups by variant
- `collection_assets(asset_id)` — collection membership queries

## Other Files

| File | Format | Contents |
|---|---|---|
| `volumes.yaml` | YAML | Registered volume definitions (id, label, mount_point, type) |
| `searches.toml` | TOML | Saved search definitions (name, query, sort) |
| `collections.yaml` | YAML | Collection definitions with ordered asset ID lists |
| `stacks.yaml` | YAML | Stack definitions with ordered asset ID lists |
| `maki.toml` | TOML | User configuration (preview settings, serve settings, import settings) |
| `previews/<prefix>/<hash>.jpg` | JPEG | Preview thumbnails keyed by variant content hash |
| `faces/<prefix>/<face_id>.jpg` | JPEG | Face crop thumbnails (150×150, with `--features ai`) |

### Entity Relationships



## Content-Addressable Identity

Every file imported into maki is hashed with SHA-256. This hash is the file's identity:

- **Deduplication**: Importing the same file twice (even from different paths or drives) recognizes it as the same content and adds the new location to the existing Variant.
- **Integrity verification**: `maki verify` re-hashes files and compares against stored hashes to detect corruption or bit rot.
- **Transparent relocation**: Moving a file to a different drive does not change its identity. `maki relocate` and `maki update-location` update the catalog path.

Originals (RAW files, camera JPEGs) are immutable – their hash is stable forever. Recipe files are the exception: they are modified by external tools, so maki tracks them by location and updates their stored hash when changes are detected.

---

Previous: Configuration

# VLM Model Guide

This document covers vision-language models (VLMs) compatible with `maki describe`. It lists tested models, hardware requirements, quality notes, and setup instructions for each inference backend.

maki tries the **Ollama native API** (`/api/generate`) first, falling back to the **OpenAI-compatible** `/v1/chat/completions` endpoint. Both use base64-encoded images. Any server that implements either API works — Ollama, llama.cpp, vLLM, LM Studio, SGLang, or cloud providers.

---

## Quick Start

1. Install Ollama (or another backend — see Backends below).
2. Pull a model: `ollama pull qwen2.5vl:3b`
3. Test connectivity: `maki describe --check`
4. Describe assets: `maki describe "description:none" --apply --log`

To switch models, either set `[vlm] model` in `maki.toml` or pass `--model` on the command line.

---

## Tested Models

All timings measured on Apple M3 Pro (18 GB) with Ollama, using preview images (~800px). Your results will vary with hardware, image size, and model quantization.

### Recommended for Photography

| Model | RAM Download | Speed | Notes |
|---|---|---|---|
| **Qwen2.5-VL 3B** `qwen2.5vl:3b` | **3 GB** (2.0 GB) | 8–12s | **Default.** Best balance of speed, quality, and resource use. Very good quality. |
| **Qwen3-VL 8B** `qwen3-vl:8b` | **6 GB** (5.2 GB) | 15–20s | **Recommended upgrade.** Excellent quality, noticeably better descriptions. |
| **Qwen3-VL 4B** `qwen3-vl:4b` | **4 GB** (2.8 GB) | 10–15s | Good step up from Qwen2.5-VL 3B without much extra RAM. Very good quality. |
| **Qwen2.5-VL 7B** `qwen2.5vl:7b` | **6 GB** (4.7 GB) | 20–36s | Proven, widely tested. Excellent quality. |
| **Gemma 3 4B** `gemma3:4b` | **4 GB** (3.3 GB) | 10–15s | Strong reasoning, good at scene understanding. Very good quality. |

### Budget / Batch Processing

| Model | RAM Download | Speed | Notes |
|---|---|---|---|
| **Moondream 2B** `moondream` | **2 GB** (1.7 GB) | 3–5s | Fastest option. Good for bulk first passes before refining with a larger model. |
| **SmolVLM 2.2B** `smolvlm` | **2 GB** (1.5 GB) | 4–8s | HuggingFace, very compact. Similar niche to Moondream. |

### Large / High Quality

| Model | RAM Download | Speed | Notes |
|---|---|---|---|
| **Qwen3-VL 32B** `qwen3-vl:32b` | **24 GB** (20 GB) | 60–90s | Best quality via Ollama. Needs 32 GB+ system RAM. Outstanding quality. |
| **LLaVA 1.6 7B** `llava:7b` | **6 GB** (4.7 GB) | 15–25s | Well-established, wide compatibility. Good quality. |

### Qwen3.5 (Next Generation)

Qwen3.5 models use **early fusion** — vision and text are processed jointly from the earliest layers, giving better visual reasoning than the separate-encoder approach of older models. All Qwen3.5 models are natively multimodal (no separate "-VL" variant).

| Model | RAM Download | Backend | Notes |
|---|---|---|---|
| **Qwen3.5 4B** | **4 GB** (3 GB) | llama.cpp, vLLM | Comparable to Qwen3-VL 8B in some benchmarks. Very good quality. |
| **Qwen3.5 9B** | **8 GB** (6 GB) | llama.cpp, vLLM | Best quality-per-GB. Strong upgrade path. Excellent quality. |
| **Qwen3.5 27B** | **20 GB** (16 GB) | llama.cpp, vLLM | Needs significant RAM; best local quality. Outstanding quality. |

**Ollama caveat (as of March 2026):** Ollama cannot handle Qwen3.5 vision — the model's `mmproj` vision files are not supported yet. Text-only works, but image input silently fails. Use llama.cpp or vLLM for Qwen3.5 multimodal. This will likely be resolved in a future Ollama release.

---

## Backends

### Ollama (Recommended)

The simplest setup. Ollama manages model downloads, quantization, and serves an OpenAI-compatible API.

**Install:** https://ollama.com

```
# Pull a model
ollama pull qwen2.5vl:3b
```

```
# The server starts automatically; default endpoint is http://localhost:11434
maki describe --check
```

```
# maki.toml
[vlm]
endpoint = "http://localhost:11434"
model = "qwen2.5vl:3b"
```

**Supported models:** All models with Ollama vision support — Qwen2.5-VL, Qwen3-VL, LLaVA, Moondream, Gemma 3, SmolVLM. Not Qwen3.5 (see caveat above).

**Concurrency:** Ollama handles one request at a time by default. For `concurrency > 1`, set `OLLAMA_NUM_PARALLEL` environment variable or increase `num_parallel` in Ollama's config.

### llama.cpp

Direct inference with GGUF model files. Supports Qwen3.5 multimodal via the `--mmproj` flag.

**Install:** https://github.com/ggerganov/llama.cpp

```
# Download model files (e.g. from HuggingFace)
# You need both the main model and the vision projector (mmproj) file

# Start the server with vision support
llama-server \
  -m Qwen3.5-9B-Q4_K_M.gguf \
  --mmproj mmproj-BF16.gguf \
  --host 0.0.0.0 \
  --port 8080
```

```
# maki.toml
[vlm]
endpoint = "http://localhost:8080"
model = "Qwen3.5-9B"
```

**Key points:** - Vision requires the `--mmproj` flag with the separate vision projector file - Quantized GGUF files (Q4_K_M, Q5_K_M) reduce RAM needs significantly - No automatic model management — you download and manage files yourself - Serves the OpenAI-compatible API at `/v1/chat/completions`

### vLLM

High-throughput inference server, best for GPU machines and batch processing. Supports Qwen3.5 natively.

**Install:** https://docs.vllm.ai

```
# Requires vLLM 0.17.0+ for Qwen3.5
pip install vllm

# Start the server
vllm serve Qwen/Qwen3.5-9B --host 0.0.0.0 --port 8000
```

```
# maki.toml
[vlm]
endpoint = "http://localhost:8000"
model = "Qwen/Qwen3.5-9B"
concurrency = 4   # vLLM handles parallel requests well
```

**Key points:** - Best throughput for batch processing (set `concurrency` higher) - GPU recommended (CUDA, ROCm); CPU inference is slow - Automatic model download from HuggingFace - Full OpenAI-compatible

API

### LM Studio

Desktop application with a GUI for model management and a built-in server.

**Install:** https://lmstudio.ai

```
# maki.toml
[vlm]
endpoint = "http://localhost:1234"
model = "qwen2.5-vl-3b"   # Use the model name shown in LM Studio
```

### Cloud APIs

Any OpenAI-compatible cloud API works. Note that cloud APIs charge per request and maki does not set authentication headers — use a local proxy if your endpoint requires an API key.

```
# maki.toml — example with a self-hosted proxy that adds auth
[vlm]
endpoint = "https://api.openai.com"
model = "gpt-4o"
```

---

## Choosing a Model

### By Use Case

| Scenario | Recommended Model | Why |
|---|---|---|
| **Daily use, modest hardware** | Qwen2.5-VL 3B | Fast, 3 GB RAM, good quality |
| **Best quality via Ollama** | Qwen3-VL 8B | Excellent descriptions, reasonable speed |
| **Bulk first pass** | Moondream 2B | 3–5s per image, good-enough descriptions |
| **Best local quality** | Qwen3.5 9B (llama.cpp) | Early fusion, strong reasoning |
| **32 GB+ Mac or GPU server** | Qwen3-VL 32B or Qwen3.5 27B | Near-cloud quality |
| **Tag suggestions** | Qwen3-VL 8B | Structured output reliability |
| **Architectural / technical** | Gemma 3 4B or Qwen3-VL 8B | Good at detail and materials |
| **Multilingual descriptions** | Qwen3.5 (any size) | 201 languages |

### By Hardware

| System RAM | GPU VRAM | Suggested Models |
|---|---|---|
| 8 GB | — | Moondream 2B, SmolVLM 2.2B |
| 16 GB | — | Qwen2.5-VL 3B (default), Gemma 3 4B, Qwen3-VL 4B |
| 24–32 GB | — | Qwen3-VL 8B, Qwen2.5-VL 7B, Qwen3.5 9B |
| 32 GB+ | — | Qwen3-VL 32B, Qwen3.5 27B |
| — | 8 GB | Qwen3-VL 8B (FP16), Qwen3.5 9B (Q4) |
| — | 16 GB+ | Qwen3.5 27B (Q4), Qwen3-VL 32B (Q4) |

---

# Model Comparison

## Qwen Model Generations

| | Qwen2.5-VL | Qwen3-VL | Qwen3.5 |
|---|---|---|---|
| **Architecture** | Late fusion (separate vision encoder) | Late fusion (improved encoder) | Early fusion (native multimodal) |
| **Vision quality** | Good | Very good | Best |
| **Document understanding** | Good | Better | Best (90.8 OmniDocBench) |
| **Context length** | 128K | 128K | 256K |
| **Languages** | ~29 | ~119 | 201 |
| **Ollama vision** | Yes | Yes | Not yet |
| **llama.cpp vision** | Yes | Yes | Yes (with mmproj) |

For most users, **Qwen3-VL via Ollama** is the practical sweet spot right now. When Ollama adds Qwen3.5 vision support, it will become the clear best choice.

## Description Quality (Subjective)

Based on describing the same set of 50 photographs (landscapes, portraits, architecture, street):

- **Moondream 2B**: Short, accurate, occasionally generic. Good at identifying subjects, weaker on mood/lighting.
- **Qwen2.5-VL 3B**: Solid all-rounder. Good detail, reasonable sentence structure. Occasionally repetitive across similar images.
- **Qwen3-VL 4B**: Noticeable step up in vocabulary and scene understanding over Qwen2.5-VL 3B.
- **Gemma 3 4B**: Strong on materials, textures, and spatial relationships. Slightly less natural prose.
- **Qwen3-VL 8B**: Rich, natural descriptions. Good at lighting, mood, composition. Rarely generic.
- **Qwen2.5-VL 7B**: Similar quality to Qwen3-VL 8B but slower.
- **Qwen3.5 9B**: Best visual reasoning. Catches subtle details (reflections, depth of field, lens characteristics). Needs llama.cpp or vLLM.

---

# Thinking Models

Some models (Qwen3-VL, Qwen3.5) support "extended thinking" — they wrap internal chain-of-thought reasoning in `<think>...</think>` tags before producing the actual answer. This can consume most of the `max_tokens` budget with reasoning text, leaving little room for the description itself.

maki handles this automatically:

- Sends `think: false` to disable extended thinking (works with Ollama's native API)
- Strips any `<think>...</think>` tags from the response as a fallback
- Tries the Ollama native API first (where `think: false` is honored), falling back to the OpenAI-compatible endpoint

No configuration is needed — thinking models work out of the box. If you see empty or truncated descriptions, try increasing `max_tokens` (default 500) to give the model more headroom.

---

## Tips

**Start small, upgrade later.** Run `maki describe` with Moondream for a quick first pass, then re-describe the best assets with a larger model using `--force`:

```
maki describe "description:none" --model moondream --apply --log
maki describe "rating:4+" --model qwen3-vl:8b --apply --force --log
```

**Use `--mode tags` for discovery.** VLM-suggested tags can surface patterns you haven't tagged manually:

```
maki describe --mode tags "date:2024-06" --apply --log
```

**Lower temperature for batch consistency.** When describing hundreds of images, `--temperature 0` gives more uniform output:

```
maki describe "type:image" --temperature 0 --apply --log
```

**Increase timeout for first load.** Ollama loads the model into memory on first request, which can take 30–60 seconds for larger models:

```
maki describe --model qwen3-vl:8b --timeout 300 --asset a1b2c3d4
```

**Use per-model config for multi-model workflows.** If you switch between models frequently, set per-model overrides so each model gets its optimal settings automatically:

```
[vlm]
model = "qwen2.5vl:3b"
max_image_edge = 512
timeout = 300

[vlm.model_config."qwen3-vl:8b"]
timeout = 300
num_ctx = 8192

[vlm.model_config."moondream:latest"]
max_tokens = 200
temperature = 0.1
```

Now `maki describe --model qwen3-vl:8b` automatically uses the longer timeout and larger context window, while `maki describe --model moondream:latest` uses lower temperature and fewer tokens. See Configuration Reference — per-model config for full details.

**Tune sampling parameters per model.** Some models benefit from non-default sampling. Use `--num-ctx`, `--top-p`, `--top-k`, and `--repeat-penalty` on the CLI, or set them in `maki.toml` (globally or per-model). A value of zero means "not set — use server default":

```
maki describe "rating:5" --model qwen3-vl:8b --num-ctx 8192 --top-p 0.9 --apply --log
```

**Use concurrency with capable servers.** If your server handles parallel requests (vLLM, GPU Ollama with `OLLAMA_NUM_PARALLEL`):

```
[vlm]
concurrency = 4
```

---

## See Also

- Configuration Reference — [vlm] section — all configuration options
- maki describe — command reference

- Ingesting Assets — auto-describe during import

# REST API Reference

The `maki serve` command starts an Axum-based web server (default `127.0.0.1:8080`) that exposes both HTML page routes and JSON/HTML-partial API endpoints. All endpoints use `spawn_blocking` for SQLite access and open a fresh catalog connection per request via `Catalog::open_fast()`.

Base URL: `http://127.0.0.1:8080` (configurable via `--bind` and `--port`)

---

## Page Routes

These routes return full HTML pages (for direct browser requests) or HTML partials (for htmx requests).

### `GET /` – Browse Page

Returns the asset browse grid with search, filtering, sorting, and pagination.

| Parameter | Type | Default | Description |
|---|---|---|---|
| `q` | string | `""` | Free-text search query (supports filter syntax) |
| `type` | string | `""` | Asset type filter (e.g. `image`, `video`) |
| `tag` | string | `""` | Tag filter |
| `format` | string | `""` | File format filter, comma-separated for OR (e.g. `nef,cr3`) |
| `volume` | string | `""` | Volume label filter |
| `rating` | string | `""` | Rating filter (`5` exact, `3+` minimum) |
| `label` | string | `""` | Color label filter (e.g. `Red`, `Green`) |
| `collection` | string | `""` | Collection name filter |
| `path` | string | `""` | Path prefix filter |
| `sort` | string | `date_desc` | Sort order (see below) |
| `page` | u32 | `1` | Page number (configurable results per page, default 60) |

**Sort values**: `date_desc`, `date_asc`, `name_desc`, `name_asc`, `size_desc`, `size_asc`

**Behavior**: Detects `HX-Request` header. htmx requests receive a `ResultsPartial` (just the results grid). Direct browser requests receive the full `BrowsePage` with layout, CSS, filter controls, saved search chips, and all dropdown options.

```
# Full page (browser)
curl http://localhost:8080/

# With filters
curl "http://localhost:8080/?q=sunset&rating=4%2B&sort=date_desc&page=1"
```

```
# htmx partial
curl -H "HX-Request: true" "http://localhost:8080/?tag=landscape&page=2"
```

### GET /asset/{id} – Asset Detail Page

Returns the detail page for a single asset, including preview image, metadata, editable tags, rating, label, name, description, variants, recipes, and collection memberships.

| Parameter | Type | Description |
|---|---|---|
| `id` | string | Asset UUID (path param) |

```
curl http://localhost:8080/asset/550e8400-e29b-41d4-a716-446655440000
```

**Errors**: Returns 404 with `<h1>Not Found</h1>` if the asset does not exist.

### GET /tags – Tags Page

Returns an HTML page listing all tags with counts. Supports sortable columns and live text filtering.

```
curl http://localhost:8080/tags
```

### GET /stats – Stats Page

Returns an HTML page showing catalog statistics: overview, type/format breakdown, per-volume details, tag frequencies, and verification health.

```
curl http://localhost:8080/stats
```

### GET /collections – Collections Page

Returns an HTML page listing all collections with a "New Collection" button.

```
curl http://localhost:8080/collections
```

### GET /people – People Page

*Requires `--features ai` compilation.*

Returns an HTML page showing all detected people with face crop thumbnails, names, face counts, and management controls (rename, merge, delete). Includes a "Cluster" button to run auto-clustering from the UI.

```
curl http://localhost:8080/people
```

### GET /backup – Backup Status Page

Returns an HTML page showing backup health: summary cards (total assets, at-risk count, min copies), volume distribution bar chart, coverage by purpose table, and volume gaps table.

```
curl http://localhost:8080/backup
```

### GET /stroll – Stroll Page

*Requires `--features ai` compilation.*

Returns an interactive visual similarity exploration page. Centers on a randomly selected asset (or a specific asset via query parameter) and displays its nearest neighbors by SigLIP embedding distance. Clicking a neighbor re-centers the view on that asset.

| Parameter | Type | Default | Description |
|---|---|---|---|
| `id` | string | (random) | Asset UUID to center on (optional) |

```
# Random starting point
curl http://localhost:8080/stroll

# Start from a specific asset
curl "http://localhost:8080/stroll?id=550e8400-e29b-41d4-a716-446655440000"
```

---

## Search API

### `GET /api/search`

Same query parameters as `GET /`. Intended for htmx consumption.

**Behavior**: Non-htmx requests (missing `HX-Request` header) are redirected to `/?{params}` so that direct browser loads, back-button navigations, and bookmarks render the full page. htmx requests receive a `ResultsPartial`.

```
# htmx request (returns HTML partial)
curl -H "HX-Request: true" "http://localhost:8080/api/search?q=sunset&page=1"

# Browser request (redirects to /?q=sunset&page=1)
curl -v "http://localhost:8080/api/search?q=sunset&page=1"
# < HTTP/1.1 303 See Other
# < location: /?q=sunset&page=1
```

---

## Asset Editing

### `POST /api/asset/{id}/tags` – Add Tags

Adds one or more tags to an asset. Triggers XMP write-back.

| Field | Type | Description |
|---|---|---|
| `tags` | string | Comma-separated tag names (form-encoded) |

**Content-Type**: `application/x-www-form-urlencoded`

**Response**: HTML partial – updated tag list fragment.

```
curl -X POST http://localhost:8080/api/asset/{id}/tags \
  -d "tags=landscape,nature,sunset"
```

### `DELETE /api/asset/{id}/tags/{tag}` – Remove Tag

Removes a single tag from an asset. Triggers XMP write-back.

| Parameter | Type | Description |
|---|---|---|
| `id` | string | Asset UUID (path param) |
| `tag` | string | Tag to remove (path param, URL-encoded) |

**Response**: HTML partial – updated tag list fragment.

```
curl -X DELETE http://localhost:8080/api/asset/{id}/tags/landscape
```

### `PUT /api/asset/{id}/rating` – Set Rating

Sets or clears the asset's star rating. A value of `0` or `null` clears the rating. Triggers XMP write-back.

| Field | Type | Description |
|---|---|---|
| `rating` | u8 or null | Rating value 1-5, or 0/null to clear |

**Content-Type**: `application/x-www-form-urlencoded`

**Response**: HTML partial – updated rating fragment.

```
# Set rating to 5 stars
curl -X PUT http://localhost:8080/api/asset/{id}/rating \
  -d "rating=5"

# Clear rating
curl -X PUT http://localhost:8080/api/asset/{id}/rating \
  -d "rating=0"
```

### `PUT /api/asset/{id}/description` – Set Description

Sets or clears the asset's description. An empty string clears the description. Triggers XMP write-back.

| Field | Type | Description |
|---|---|---|
| `description` | string or null | Description text, or empty to clear |

**Content-Type**: `application/x-www-form-urlencoded`

**Response**: HTML partial – updated description fragment.

```
# Set description
curl -X PUT http://localhost:8080/api/asset/{id}/description \
  -d "description=A%20beautiful%20sunset%20over%20the%20mountains"

# Clear description
curl -X PUT http://localhost:8080/api/asset/{id}/description \
  -d "description="
```

### `PUT /api/asset/{id}/name` – Set Name

Sets or clears the asset's display name. An empty string clears the name (reverts to filename fallback).

| Field | Type | Description |
|---|---|---|
| `name` | string or null | Display name, or empty to clear |

**Content-Type**: `application/x-www-form-urlencoded`

**Response**: HTML partial – updated name fragment (includes fallback filename display when name is cleared).

```
# Set name
curl -X PUT http://localhost:8080/api/asset/{id}/name \
  -d "name=Mountain%20Sunset"

# Clear name
curl -X PUT http://localhost:8080/api/asset/{id}/name \
  -d "name="
```

### `PUT /api/asset/{id}/label` – Set Color Label

Sets or clears the asset's color label. Accepts case-insensitive color names from the 7-color set: Red, Orange, Yellow, Green, Blue, Pink, Purple. An empty string clears the label. Triggers XMP write-back.

| Field | Type | Description |
|---|---|---|
| `label` | string | Color name (case-insensitive), or empty to clear |

**Content-Type**: `application/x-www-form-urlencoded`

**Response**: HTML partial – updated label fragment.

```
# Set label
curl -X PUT http://localhost:8080/api/asset/{id}/label \
  -d "label=Red"

# Clear label
curl -X PUT http://localhost:8080/api/asset/{id}/label \
  -d "label="
```

### `PUT /api/asset/{id}/date` – Set Date

Sets or clears the asset's creation date. Accepts an ISO date or datetime string. An empty string clears the date.

| Field | Type | Description |
|---|---|---|
| `date` | string or null | ISO date string (e.g. `2024-12-25`), or empty to clear |

**Content-Type**: `application/x-www-form-urlencoded`

**Response**: HTML partial – updated date fragment.

```
# Set date
curl -X PUT http://localhost:8080/api/asset/{id}/date \
  -d "date=2024-12-25"

# Clear date
curl -X PUT http://localhost:8080/api/asset/{id}/date \
  -d "date="
```

### `POST /api/open-location` – Reveal in File Manager

Opens the system file manager with the specified file selected. macOS uses `open -R` (Finder); Windows uses `explorer /select,` (Explorer); Linux uses `xdg-open` on the parent directory.

| Field | Type | Description |
|---|---|---|
| `volume_id` | string | Volume UUID |
| `relative_path` | string | File path relative to the volume mount point |

**Content-Type**: `application/json`

**Response**: `{"ok": true}` on success, or error message.

```
curl -X POST http://localhost:8080/api/open-location \
  -H "Content-Type: application/json" \
  -d '{"volume_id":"abc-123","relative_path":"Photos/DSC_001.nef"}'
```

### `POST /api/open-terminal` – Open Terminal

Opens a terminal window in the file's parent directory. macOS uses Terminal.app; Windows uses cmd; Linux tries common terminal emulators.

| Field | Type | Description |
|---|---|---|
| `volume_id` | string | Volume UUID |
| `relative_path` | string | File path relative to the volume mount point |

**Content-Type**: `application/json`

**Response**: `{"ok": true}` on success, or error message.

```
curl -X POST http://localhost:8080/api/open-terminal \
  -H "Content-Type: application/json" \
  -d '{"volume_id":"abc-123","relative_path":"Photos/DSC_001.nef"}'
```

### `POST /api/asset/{id}/preview` – Regenerate Previews

Regenerates both the regular preview thumbnail and the smart preview for the asset's best variant. Requires the source file to be on an online volume. Returns cache-busted URLs so the browser displays the newly generated images without requiring a page reload.

**Content-Type**: None (no body required)

**Response**: HTML partial – updated preview fragment with cache-busted URLs.

```
curl -X POST http://localhost:8080/api/asset/{id}/preview
```

### `POST /api/asset/{id}/rotate` – Rotate Preview

Cycles the preview rotation 90° clockwise (0° → 90° → 180° → 270° → 0°). Regenerates both regular and smart previews with EXIF auto-orientation applied. The rotation is persisted per asset.

**Content-Type**: None (no body required)

**Response**: HTML partial – updated preview fragment with cache-busted URLs.

```
curl -X POST http://localhost:8080/api/asset/{id}/rotate
```

### `POST /api/asset/{id}/reimport-metadata` – Re-import Metadata

Clears the asset's tags, description, rating, and color label, then re-extracts metadata from all variant source files (XMP recipe sidecars and embedded XMP in JPEG/TIFF media files). Offline volumes are silently skipped.

**Content-Type**: None (no body required)

**Response**: HTML partial – updated tags fragment.

```
curl -X POST http://localhost:8080/api/asset/{id}/reimport-metadata
```

### `PUT /api/asset/{id}/stack-pick` – Set Stack Pick

Sets this asset as the pick (position 0) of its stack. The previous pick swaps to this asset's former position. Persists to both SQLite and `stacks.yaml`.

| Parameter | Type | Description |
|---|---|---|
| `id` | string | Asset UUID (path param) |

**Response**:

```
{
  "status": "ok"
}
```

**Errors**: Returns 500 if the asset is not in a stack.

```
curl -X PUT http://localhost:8080/api/asset/{id}/stack-pick
```

### `DELETE /api/asset/{id}/stack` – Dissolve Stack

Dissolves the entire stack that this asset belongs to. All members are unstacked. Persists to both SQLite and `stacks.yaml`.

| Parameter | Type | Description |
|---|---|---|
| `id` | string | Asset UUID (path param) |

**Response**:

```
{
  "status": "ok"
}
```

**Errors**: Returns 500 if the asset is not in a stack.

```
curl -X DELETE http://localhost:8080/api/asset/{id}/stack
```

---

## Batch Operations

All batch endpoints accept JSON request bodies and return JSON responses. They loop over individual assets, collecting successes and failures independently.

### `POST /api/batch/tags` – Batch Add/Remove Tags

Adds or removes tags on multiple assets. Each individual operation triggers XMP write-back.

**Content-Type**: `application/json`

**Request body**:

```
{
  "asset_ids": ["uuid-1", "uuid-2", "uuid-3"],
  "tags": ["landscape", "nature"],
  "remove": false
}
```

| Field | Type | Description |
|---|---|---|
| `asset_ids` | string[] | Array of asset UUIDs |
| `tags` | string[] | Tag names to add or remove |
| `remove` | bool | `false` to add tags, `true` to remove |

**Response**:

```
{
  "succeeded": 3,
  "failed": 0,
  "errors": []
}
```

```
curl -X POST http://localhost:8080/api/batch/tags \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2"], "tags": ["landscape"], "remove": false}'
```

### `PUT /api/batch/rating` – Batch Set Rating

Sets or clears the rating on multiple assets. Each individual operation triggers XMP write-back.

**Content-Type**: `application/json`

**Request body**:

```
{
  "asset_ids": ["uuid-1", "uuid-2"],
  "rating": 5
}
```

| Field | Type | Description |
|---|---|---|
| `asset_ids` | string[] | Array of asset UUIDs |

| Field | Type | Description |
|---|---|---|
| `rating` | u8 or null | Rating 1-5, or null/0 to clear |

**Response**:

```
{
  "succeeded": 2,
  "failed": 0,
  "errors": []
}
```

```
# Set rating
curl -X PUT http://localhost:8080/api/batch/rating \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2"], "rating": 4}'

# Clear rating
curl -X PUT http://localhost:8080/api/batch/rating \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2"], "rating": null}'
```

### `PUT /api/batch/label` – Batch Set Label

Sets or clears the color label on multiple assets. The label is validated against the 7-color set before processing any assets. Each individual operation triggers XMP write-back.

**Content-Type**: `application/json`

**Request body**:

```
{
  "asset_ids": ["uuid-1", "uuid-2"],
  "label": "Red"
}
```

| Field | Type | Description |
|---|---|---|
| `asset_ids` | string[] | Array of asset UUIDs |
| `label` | string or null | Color name (case-insensitive), or empty/null to clear |

**Response**:

```
{
  "succeeded": 2,
  "failed": 0,
  "errors": []
}
```

```
curl -X PUT http://localhost:8080/api/batch/label \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2"], "label": "Green"}'
```

### `POST /api/batch/collection` – Add to Collection

Adds assets to a named collection. Persists to both SQLite and YAML.

**Content-Type**: `application/json`

**Request body**:

```
{
  "asset_ids": ["uuid-1", "uuid-2"],
  "collection": "Best of 2025"
}
```

| Field | Type | Description |
|---|---|---|
| `asset_ids` | string[] | Array of asset UUIDs |
| `collection` | string | Collection name |

**Response**:

```
{
  "added": 2,
  "collection": "Best of 2025"
}
```

```
curl -X POST http://localhost:8080/api/batch/collection \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2"], "collection": "Best of 2025"}'
```

### `DELETE /api/batch/collection` – Remove from Collection

Removes assets from a named collection. Persists to both SQLite and YAML.

**Content-Type**: `application/json`

**Request body**:

```
{
  "asset_ids": ["uuid-1", "uuid-2"],
  "collection": "Best of 2025"
}
```

| Field | Type | Description |
|---|---|---|
| `asset_ids` | string[] | Array of asset UUIDs |
| `collection` | string | Collection name |

**Response**:

```
{
  "removed": 2,
  "collection": "Best of 2025"
}
```

```
curl -X DELETE http://localhost:8080/api/batch/collection \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2"], "collection": "Best of 2025"}'
```

### `POST /api/batch/stack` – Create Stack

Creates a stack from the selected assets. The first asset becomes the pick (position 0). Assets already in a stack are rejected. Persists to both SQLite and `stacks.yaml`.

**Content-Type**: `application/json`

**Request body**:

```
{
  "asset_ids": ["uuid-1", "uuid-2", "uuid-3"]
}
```

| Field | Type | Description |
|---|---|---|
| `asset_ids` | string[] | Array of asset UUIDs (minimum 2) |

**Response**:

```
{
  "stack_id": "550e8400-e29b-41d4-a716-446655440000",
  "member_count": 3
}
```

**Errors**: Returns 500 if any asset is already in a stack, or fewer than 2 assets are provided.

```
curl -X POST http://localhost:8080/api/batch/stack \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2", "uuid-3"]}'
```

### `DELETE /api/batch/stack` – Unstack Assets

Removes assets from their stacks. If a stack is left with 1 or fewer members, it is automatically dissolved. Persists to both SQLite and `stacks.yaml`.

**Content-Type**: `application/json`

**Request body**:

```
{
  "asset_ids": ["uuid-1", "uuid-2"]
}
```

| Field | Type | Description |
|---|---|---|
| `asset_ids` | string[] | Array of asset UUIDs |

**Response**:

```
{
  "removed": 2
}
```

```
curl -X DELETE http://localhost:8080/api/batch/stack \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2"]}'
```

### `GET /api/stack/{id}/members` – Stack Members

Returns all members of a stack as a JSON array of card data, ordered by stack position. Used by the per-stack expand/collapse feature on the browse page.

**Response**: `application/json`

```
[
  {
    "asset_id": "uuid-1",
    "name": "DSC_001.NEF",
    "asset_type": "image",
    "format": "NEF",
    "date": "2026-01-15",
    "preview_url": "/previews/ab/abcdef1234.jpg",
    "rating": 4,
    "label": "Green",
    "variant_count": 2,
    "preview_rotation": 0,
    "stack_id": "stack-uuid"
  }
]
```

```
curl http://localhost:8080/api/stack/550e8400-e29b-41d4-a716-446655440000/members
```

### `GET /analytics` – Analytics Dashboard

Returns the analytics page with shooting frequency, camera/lens usage, rating distribution, format breakdown, monthly import volume, and storage per volume charts.

### `POST /api/asset/{id}/suggest-tags` – AI Tag Suggestions

*Requires `--features ai` compilation.*

Analyzes the asset's preview image with SigLIP and returns suggested tags with confidence scores. Tags already on the asset are included but marked with `existing: true`. The model is lazy-loaded on first request and cached in server memory.

| Parameter | Type | Description |
|---|---|---|
| `id` | string | Asset UUID (path param) |

**Response**: `application/json`

```
[
  {"tag": "landscape", "confidence": 0.85, "existing": false},
  {"tag": "mountain", "confidence": 0.42, "existing": false},
  {"tag": "nature", "confidence": 0.31, "existing": true}
]
```

| Field | Type | Description |
|---|---|---|
| `tag` | string | Suggested tag name |
| `confidence` | float | Confidence score (0.0–1.0) |
| `existing` | bool | Whether the tag is already on the asset |

```
curl -X POST http://localhost:8080/api/asset/550e8400-e29b-41d4-a716-446655440000/suggest-tags
```

### `POST /api/batch/auto-tag` – Batch AI Auto-Tag

*Requires `--features ai` compilation.*

Auto-tags selected assets using SigLIP. For each asset, encodes the preview image, classifies against the configured label vocabulary, and applies tags above the threshold. Existing tags are not duplicated.

**Content-Type**: `application/json`

**Request body**:

```
{
  "asset_ids": ["uuid-1", "uuid-2", "uuid-3"]
}
```

| Field | Type | Description |
|---|---|---|
| `asset_ids` | string[] | Array of asset UUIDs |

**Response**:

```
{
  "succeeded": 3,
  "failed": 0,
  "tags_applied": 12,
  "errors": []
}
```

| Field | Type | Description |
|---|---|---|
| `succeeded` | u32 | Assets successfully processed |
| `failed` | u32 | Assets that failed |
| `tags_applied` | u32 | Total new tags applied across all assets |
| `errors` | string[] | Error messages for failed assets |

```
curl -X POST http://localhost:8080/api/batch/auto-tag \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2", "uuid-3"]}'
```

### `POST /api/batch/detect-faces` – Batch Face Detection

*Requires `--features ai` compilation.*

Detects faces in the preview images of the selected assets using YuNet + ArcFace. Stores face records, embeddings, and crop thumbnails.

**Content-Type**: `application/json`

**Request body**:

```
{
  "asset_ids": ["uuid-1", "uuid-2", "uuid-3"]
}
```

| Field | Type | Description |
|---|---|---|
| `asset_ids` | string[] | Array of asset UUIDs |

**Response**:

```
{
  "succeeded": 3,
  "failed": 0,
  "faces_detected": 7,
  "errors": []
}
```

```
curl -X POST http://localhost:8080/api/batch/detect-faces \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2", "uuid-3"]}'
```

### `POST /api/asset/{id}/split` – Split Variants Out of Asset

Splits one or more variants out of a multi-variant asset into new standalone assets. Each specified variant becomes a new asset with its own sidecar YAML. Recipes attached to split variants move with them.

**Content-Type**: `application/json`

**Request body**:

```
{
  "variant_hashes": ["sha256:abcdef1234...", "sha256:567890abcd..."]
}
```

| Field | Type | Description |
|---|---|---|
| `variant_hashes` | string[] | Content hashes of variants to split out |

**Response**:

```
{
  "source_id": "550e8400-e29b-41d4-a716-446655440000",
  "new_assets": [
    {
      "asset_id": "660f9511-f30c-52e5-b827-557766551111",
      "variant_hash": "sha256:abcdef1234...",
      "original_filename": "DSC_001_edit.tif"
    }
  ]
}
```

| Field | Type | Description |
|---|---|---|
| `source_id` | string | UUID of the original (source) asset |
| `new_assets[].asset_id` | string | UUID of the newly created asset |
| `new_assets[].variant_hash` | string | Content hash of the split variant |
| `new_assets[].original_filename` | string | Original filename of the variant |

**Errors**: Returns 404 if the asset does not exist. Returns 400 if a variant hash is not found on the asset, or if splitting would leave the source asset with zero variants.

```
curl -X POST http://localhost:8080/api/asset/{id}/split \
  -H "Content-Type: application/json" \
  -d '{"variant_hashes": ["sha256:abcdef1234..."]}'
```

### `POST /api/batch/group` – Merge Assets

Merges selected assets into a single asset. Donor variants are moved to the target asset, donor assets are removed. Tags and recipes are merged.

**Content-Type**: `application/json`

**Request body**:

```
{
  "asset_ids": ["uuid-1", "uuid-2", "uuid-3"],
  "target_id": "uuid-1"
}
```

| Field | Type | Description |
|---|---|---|
| `asset_ids` | string[] | Array of asset UUIDs to merge |
| `target_id` | string or null | Optional target asset UUID (default: auto-selected) |

**Response**:

```
{
  "target_id": "uuid-1",
  "variants_moved": 2,
  "donors_removed": 2
}
```

| Field | Type | Description |
|---|---|---|
| `target_id` | string | UUID of the target (surviving) asset |
| `variants_moved` | u32 | Number of variants moved to target |
| `donors_removed` | u32 | Number of donor assets removed |

**Errors**: Returns 400 if fewer than 2 asset IDs are provided.

```
curl -X POST http://localhost:8080/api/batch/group \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2", "uuid-3"], "target_id": "uuid-1"}'
```

### `POST /api/batch/auto-group` – Auto-Group by Stem

Groups selected assets by filename stem using fuzzy prefix matching. Merges donor variants into target assets (shortest stem, preferring RAW originals).

**Content-Type**: `application/json`

**Request body**:

```
{
  "asset_ids": ["uuid-1", "uuid-2", "uuid-3"]
}
```

| Field | Type | Description |
|---|---|---|
| `asset_ids` | string[] | Array of asset UUIDs |

**Response**:

```
{
  "groups_merged": 1,
  "donors_removed": 2,
  "variants_moved": 2
}
```

```
curl -X POST http://localhost:8080/api/batch/auto-group \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2", "uuid-3"]}'
```

### `POST /api/batch/export` – Export as ZIP

Downloads matching assets as a ZIP archive. Provide either explicit asset IDs (for selected assets) or browse filter parameters (for "export all" results).

**Content-Type**: `application/json`

**Request body**:

```
{
  "asset_ids": ["uuid-1", "uuid-2"],
  "filters": {
    "q": "sunset",
    "type": "image",
    "tag": "landscape",
    "format": "jpg",
    "volume": "Photos",
    "rating": "3+",
    "label": "Green",
    "collection": "Best",
    "path": "2026/March",
    "person": "Alice"
  },
  "layout": "flat",
  "all_variants": false,
  "include_sidecars": false
}
```

| Field | Type | Default | Description |
|---|---|---|---|
| `asset_ids` | string[] | `[]` | Explicit asset IDs (used for selected-asset export) |
| `filters` | object | `null` | Browse filter params (used for "export all" results) |
| `filters.q` | string | `""` | Free-text search query |
| `filters.type` | string | `""` | Asset type filter (image, video, audio) |
| `filters.tag` | string | `""` | Tag filter |
| `filters.format` | string | `""` | Format filter |
| `filters.volume` | string | `""` | Volume label filter |
| `filters.rating` | string | `""` | Rating filter (e.g. "3+", "5") |
| `filters.label` | string | `""` | Color label filter |
| `filters.collection` | string | `""` | Collection name filter |
| `filters.path` | string | `""` | Path prefix filter |
| `filters.person` | string | `""` | Person name filter |
| `layout` | string | `"flat"` | `"flat"` (all files in root) or `"mirror"` (preserve dirs) |
| `all_variants` | bool | `false` | Include all variants (not just best) |
| `include_sidecars` | bool | `false` | Include recipe/sidecar files |

When `asset_ids` is non-empty, `filters` is ignored. When `asset_ids` is empty, the server resolves matching assets from the filters using the same search pipeline as the browse page. If both are empty/null, all catalog assets are exported.

**Response**: `application/zip` with `Content-Disposition: attachment; filename="maki-export-N-assets.zip"`.

The ZIP uses `Stored` compression (no deflation) since media files are already compressed.

```
# Export specific assets
curl -X POST http://localhost:8080/api/batch/export \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2"], "layout": "flat"}' \
  -o export.zip

# Export all results matching filters
curl -X POST http://localhost:8080/api/batch/export \
  -H "Content-Type: application/json" \
  -d '{"filters": {"tag": "landscape", "rating": "4+"}, "layout": "mirror"}' \
  -o export.zip
```

---

## Face & People APIs

*All face and people endpoints require `--features ai` compilation.*

### `GET /api/asset/{id}/faces` – List Faces for Asset

Returns all detected faces for an asset as a JSON array.

**Response**: `application/json`

```
[
  {
    "id": "face-uuid-1",
    "asset_id": "asset-uuid",
    "person_id": "person-uuid",
    "bbox_x": 0.25,
    "bbox_y": 0.15,
    "bbox_w": 0.12,
    "bbox_h": 0.18,
    "confidence": 0.95,
    "created_at": "2026-03-05T10:30:00Z"
  }
]
```

```
curl http://localhost:8080/api/asset/{id}/faces
```

### `POST /api/asset/{id}/detect-faces` – Detect Faces in Asset

Runs face detection on a single asset's preview image. Stores detected faces with embeddings and crop thumbnails.

**Response**: `application/json`

```
{
  "faces_detected": 3
}
```

```
curl -X POST http://localhost:8080/api/asset/{id}/detect-faces
```

### `GET /api/people` – List All People

Returns all people with face counts as a JSON array.

**Response**: `application/json`

```
[
  {
    "id": "person-uuid",
    "name": "Alice",
    "face_count": 42,
    "representative_face_id": "face-uuid"
  }
]
```

```
curl http://localhost:8080/api/people
```

### `PUT /api/people/{id}/name` – Name a Person

Sets or updates the name of a person.

**Content-Type**: `application/json`

**Request body**:

```
{
  "name": "Alice"
}
```

**Response**:

```
{
  "status": "ok"
}
```

```
curl -X PUT http://localhost:8080/api/people/{id}/name \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice"}'
```

### `POST /api/people/{id}/merge` – Merge People

Merges the source person into the target person. All faces from the source are reassigned to the target.

**Content-Type**: `application/json`

**Request body**:

```
{
  "source_id": "person-uuid-to-merge"
}
```

**Response**:

```
{
  "status": "ok",
  "faces_moved": 15
}
```

```
curl -X POST http://localhost:8080/api/people/{id}/merge \
  -H "Content-Type: application/json" \
  -d '{"source_id": "person-uuid-to-merge"}'
```

### `DELETE /api/people/{id}` – Delete a Person

Deletes a person. All faces assigned to this person become unassigned.

**Response**:

```
{
  "status": "ok"
}
```

```
curl -X DELETE http://localhost:8080/api/people/{id}
```

### `PUT /api/faces/{face_id}/assign` – Assign Face to Person

Assigns a face to a person.

**Content-Type**: `application/json`

**Request body**:

```
{
  "person_id": "person-uuid"
}
```

**Response**:

```
{
  "status": "ok"
}
```

```
curl -X PUT http://localhost:8080/api/faces/{face_id}/assign \
  -H "Content-Type: application/json" \
  -d '{"person_id": "person-uuid"}'
```

### `DELETE /api/faces/{face_id}/unassign` – Unassign Face from Person

Removes the person assignment from a face.

**Response**:

```
{
  "status": "ok"
}
```

```
curl -X DELETE http://localhost:8080/api/faces/{face_id}/unassign
```

### `POST /api/faces/cluster` – Auto-Cluster Faces

Runs auto-clustering on unassigned faces, grouping similar faces into new person records.

**Content-Type**: `application/json`

**Request body** (optional):

```
{
  "threshold": 0.5
}
```

| Field | Type | Default | Description |
|---|---|---|---|
| `threshold` | float | 0.5 | Similarity threshold for clustering |

**Response**:

```
{
  "people_created": 12,
  "faces_assigned": 87,
  "singletons_skipped": 5
}
```

```
curl -X POST http://localhost:8080/api/faces/cluster \
  -H "Content-Type: application/json" \
  -d '{"threshold": 0.5}'
```

---

# Data APIs

## `GET /api/calendar` – Calendar Heatmap Data

Returns per-day asset counts for a given year, respecting all search filters. Used by the browse page calendar heatmap view.

| Parameter | Type | Default | Description |
|---|---|---|---|
| `year` | i32 | current year | Year to aggregate |
| `q` | string | "" | Free-text search query (supports filter syntax) |
| `type` | string | "" | Asset type filter |
| `tag` | string | "" | Tag filter |
| `format` | string | "" | File format filter, comma-separated for OR |
| `volume` | string | "" | Volume ID filter |
| `rating` | string | "" | Rating filter |
| `label` | string | "" | Color label filter |
| `collection` | string | "" | Collection name filter |
| `path` | string | "" | Path prefix filter |

**Response**: `application/json`

```
{
  "year": 2026,
  "counts": {
    "2026-01-15": 23,
    "2026-01-16": 8,
    "2026-02-25": 14
  },
  "years": [2024, 2025, 2026]
}
```

| Field | Type | Description |
|---|---|---|
| `year` | i32 | The requested year |
| `counts` | object | Map of `"YYYY-MM-DD"` → asset count (days with 0 assets are omitted) |
| `years` | i32[] | All distinct years that have assets in the catalog |

```
# Get 2026 calendar data
curl http://localhost:8080/api/calendar?year=2026

# With filters
curl "http://localhost:8080/api/calendar?year=2026&tag=landscape&rating=4%2B"
```

### `GET /api/stroll/neighbors` – Embedding Neighbors

*Requires `--features ai` compilation.*

Returns the nearest neighbors of an asset by SigLIP embedding similarity. Used by the Stroll page for visual exploration and by the fan-out feature, which fetches transitive neighbors when a satellite is focused (L2 neighbors-of-neighbors).

| Parameter | Type | Default | Description |
|---|---|---|---|
| `id` | string | – | Asset UUID to find neighbors for (required) |
| `n` | u32 | `12` | Number of neighbors to return (range: 5–25) |
| `q` | string | `""` | Optional search query to scope neighbors |

**Response**: `application/json`

```
{
  "center": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "DSC_001",
    "preview_hash": "a1b2c3d4..."
  },
  "neighbors": [
    {
      "id": "660f9511-f30c-52e5-b827-557766551111",
      "name": "DSC_002",
      "preview_hash": "b2c3d4e5...",
      "similarity": 0.87
    }
  ]
}
```

| Field | Type | Description |
|---|---|---|
| `center.id` | string | Asset UUID of the center asset |
| `center.name` | string | Display name or filename |
| `center.preview_hash` | string | Content hash for preview URL |
| `neighbors[].id` | string | Neighbor asset UUID |
| `neighbors[].name` | string | Display name or filename |
| `neighbors[].preview_hash` | string | Content hash for preview URL |
| `neighbors[].similarity` | float | Cosine similarity score (0.0–1.0) |

```
# Get 12 nearest neighbors
curl "http://localhost:8080/api/stroll/neighbors?id=550e8400-e29b-41d4-a716-446655440000"

# Get 24 neighbors, scoped to landscapes
curl "http://localhost:8080/api/stroll/neighbors?id=550e8400-e29b-41d4-a716-446655440000&n=24&q=tag:landscape"
```

### `GET /api/tags` – List All Tags

Returns all tags with their usage counts as a JSON array of `[name, count]` tuples.

**Response**: `application/json`

```
[
  ["landscape", 42],
  ["nature", 38],
  ["portrait", 15]
]
```

```
curl http://localhost:8080/api/tags
```

### `GET /api/stats` – Catalog Stats

Returns a JSON object with catalog overview statistics.

**Response**: `application/json`

```
{
  "assets": 1250,
  "variants": 2100,
  "recipes": 800,
  "total_size": 524288000000
}
```

```
curl http://localhost:8080/api/stats
```

### `GET /api/collections` – List Collections

Returns all collections as a JSON array.

**Response**: `application/json`

```
curl http://localhost:8080/api/collections
```

### `POST /api/collections` – Create Collection

Creates a new collection. Persists to both SQLite and YAML.

**Content-Type**: `application/json`

**Request body**:

```
{
  "name": "Best of 2025",
  "description": "Top picks from this year"
}
```

| Field | Type | Description |
|---|---|---|
| `name` | string | Collection name (required, unique) |

| Field | Type | Description |
|---|---|---|
| `description` | string or null | Optional description |

**Response** (201 Created):

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Best of 2025",
  "description": "Top picks from this year"
}
```

**Errors**: Returns 409 Conflict if a collection with the same name already exists.

```
curl -X POST http://localhost:8080/api/collections \
  -H "Content-Type: application/json" \
  -d '{"name": "Best of 2025", "description": "Top picks from this year"}'
```

### `GET /api/saved-searches` – List Saved Searches

Returns all saved searches as a JSON array.

**Response**: `application/json`

```
[
  {
    "name": "5-star landscapes",
    "query": "tag:landscape rating:5",
    "sort": "date_desc"
  }
]
```

```
curl http://localhost:8080/api/saved-searches
```

### `POST /api/saved-searches` – Create/Update Saved Search

Creates a new saved search or updates an existing one with the same name. Persists to `searches.toml`.

**Content-Type**: `application/json`

**Request body**:

```
{
  "name": "5-star landscapes",
  "query": "tag:landscape rating:5",
  "sort": "date_desc"
}
```

| Field | Type | Description |
|---|---|---|
| `name` | string | Search name (required) |
| `query` | string | Search query string (required) |
| `sort` | string or null | Sort order (optional) |

**Response**:

```
{
  "status": "saved",
  "name": "5-star landscapes"
}
```

```
curl -X POST http://localhost:8080/api/saved-searches \
  -H "Content-Type: application/json" \
  -d '{"name": "5-star landscapes", "query": "tag:landscape rating:5", "sort": "date_desc"}'
```

### `DELETE /api/saved-searches/{name}` – Delete Saved Search

Deletes a saved search by name. Persists the change to `searches.toml`.

| Parameter | Type | Description |
|---|---|---|
| `name` | string | Saved search name (path param, URL-encoded) |

**Response**:

```
{
  "status": "deleted",
  "name": "5-star landscapes"
}
```

**Errors**: Returns 404 if no saved search with that name exists.

```
curl -X DELETE http://localhost:8080/api/saved-searches/5-star%20landscapes
```

---

## Static Assets

### `GET /static/htmx.min.js`

Serves the embedded htmx library. Compiled into the binary at build time.

**Response**: `application/javascript`

### `GET /static/style.css`

Serves the embedded stylesheet. Compiled into the binary at build time.

**Response**: `text/css`

### `GET /preview/*`

Serves preview images from the `previews/` directory within the catalog root. Handled by `tower-http::ServeDir`.

Preview files are stored at `previews/<hash-prefix>/<hash>.jpg` (or `.webp` depending on configuration).

```
curl http://localhost:8080/preview/a1/a1b2c3d4e5f6...abc.jpg -o preview.jpg
```

---

## Error Handling

All endpoints return standard HTTP status codes:

| Code | Meaning |
|---|---|
| 200 | Success |
| 201 | Created (collection creation) |
| 303 | See Other (non-htmx redirect from `/api/search`) |
| 404 | Not Found (missing asset or saved search) |
| 409 | Conflict (duplicate collection name) |
| 500 | Internal Server Error (database or processing error) |

Error responses are plain text with a descriptive message.

---

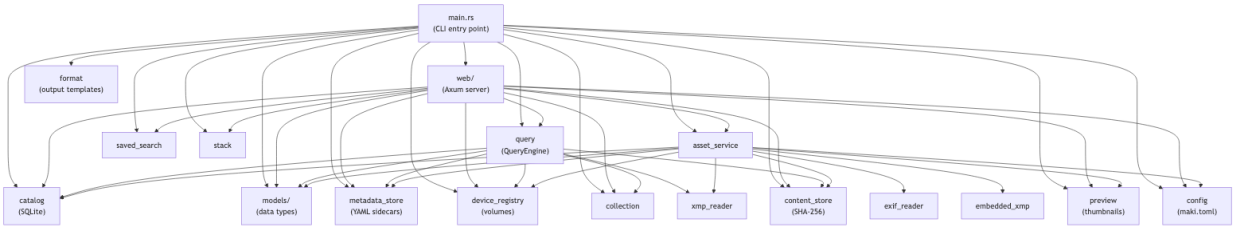
## Content Types Summary

| Endpoint Pattern | Request Content-Type | Response Content-Type |
|---|---|---|
| Page routes (`/`, `/asset/*`, etc.) | – | `text/html` |
| Single-asset editing (`PUT/POST/DELETE /api/asset/*`) | `application/x-www-form-urlencoded` | `text/html` (partial) |
| Batch operations (`/api/batch/*`) | `application/json` | `application/json` |
| Export (`POST /api/batch/export`) | `application/json` | `application/zip` |
| Data APIs (`GET /api/*`) | – | `application/json` |
| Mutation APIs (`POST/DELETE /api/*`) | `application/json` | `application/json` |

# Module Reference

This document describes the modules that make up the `maki` crate. All source code lives under `src/`.

## Module Dependency Graph




## Module Table

| Module | File(s) | Purpose |
|---|---|---|
| `ai` | `src/ai.rs` | *Feature-gated (`ai`).* Multi-model SigLIP wrapper with `ModelSpec` registry. Supports ViT-B/16-256 (768-dim) and ViT-L/16-256 (1024-dim). ONNX session management for vision and text encoders, image preprocessing (256×256 squash resize, normalize to [-1,1] CHW tensor), SentencePiece tokenization (pad to 64 tokens), sigmoid scoring (`logit_scale * dot + logit_bias`), and ~100 default photography labels. Provides `ModelSpec`, `get_model_spec()`, `SigLipModel` with `encode_image()`, `encode_texts()`, and `classify()`. |

| Module | File(s) | Purpose |
|---|---|---|
| `asset_service` | `src/asset_service.rs` | Orchestrates import, export, grouping, auto-grouping, auto-tagging, relocate, delete, and preview generation. Coordinates between content store, metadata store, catalog, EXIF/XMP readers, and preview generator. Entry point for all write operations that involve multiple subsystems. |
| `catalog` | `src/catalog.rs` | SQLite database operations: schema creation, migrations, CRUD for assets/variants/recipes/locations, search queries with pagination and filtering, statistics, and tag management. Provides `Catalog::open()` (with migrations) and `Catalog::open_fast()` (skip migrations, for per-request use in the web server). |
| `collection` | `src/collection.rs` | Collection (static album) persistence. Dual storage: SQLite tables (`collections`, `collection_assets`) for fast queries, plus `collections.yaml` at catalog root for rebuild resilience. Provides `CollectionStore` with create, list, add/remove assets, and export/import operations. |
| `config` | `src/config.rs` | Parses and validates `maki.toml` configuration file. Defines `CatalogConfig` with sub-structs `PreviewConfig` (max_edge, format, quality), `ServeConfig` (port, bind), `ImportConfig` (exclude globs, auto_tags), and `AiConfig` (threshold, labels, model_dir, prompt). All sections are optional with sensible defaults. |
| `content_store` | `src/content_store.rs` | SHA-256 content hashing for content-addressable storage. Provides `hash_file()` for computing file digests used as variant identity and integrity verification. |

| Module | File(s) | Purpose |
|---|---|---|
| `embedding_store` | `src/embedding_store.rs` | *Feature-gated (`ai`).* Dual persistence for image embeddings: SQLite table for queries + binary files for rebuild resilience. Provides `EmbeddingStore` with `store()`, `get()`, `find_similar()`, `has_embedding()`, `count()`, `remove()`, `list_models()`, and `all_embeddings_for_model()` — all scoped by model ID. `EmbeddingIndex` loads all embeddings for a model into a contiguous `Vec<f32>` buffer for fast in-memory similarity search (dot product with `BinaryHeap` min-heap for top-K). Index is cached in `AppState` and updated in-place when new embeddings are stored. Embeddings stored as little-endian f32 BLOBs in SQLite and as raw binary files under `embeddings/<model>/<prefix>/<asset_id>.bin`. Free functions: `write_embedding_binary()`, `read_embedding_binary()`, `delete_embedding_binary()`, `scan_embedding_binaries()`. Table: `embeddings(asset_id TEXT, embedding BLOB, model TEXT, PRIMARY KEY (asset_id, model))`. Migrates old single-PK schema automatically. |

| Module | File(s) | Purpose |
|---|---|---|
| `face` | `src/face.rs` | *Feature-gated (`ai`).* Face detection and recognition pipeline using ONNX models. `FaceDetector` wraps YuNet (face detection with bounding boxes, landmarks, confidence) and ArcFace (512-dim recognition embeddings). Multi-stride output decoder handles YuNet models with 12 separate tensors at strides 8/16/32. Provides `detect_faces()` (returns `Vec<DetectedFace>` with normalized bboxes), `compute_embedding()` (112×112 aligned face → 512-dim L2-normalized vector), and `generate_crop()` (150×150 JPEG thumbnail). Model specs: `DETECTION_MODEL` (YuNet), `RECOGNITION_MODEL` (ArcFace), `FACE_MODEL_SPECS` (for listing/downloading). |

| Module | File(s) | Purpose |
|---|---|---|
| face_store | src/face_store.rs | *Feature-gated (ai).* Dual persistence for face detections and named people: SQLite tables for queries + YAML/binary files for rebuild resilience. Tables: faces(id, asset_id, person_id, bbox_*, confidence, embedding, crop_path, created_at), people(id, name, representative_face_id, created_at). Files: faces.yaml, people.yaml, embeddings/arcface/<prefix>/<face_id>.bin. Provides FaceStore with insert_face(), faces_for_asset(), create_person(), name_person(), assign_face(), unassign_face(), merge_people(), delete_person(), auto_cluster(), list_people(), update_face_count(), save_all_yaml(), export_all_faces(), export_all_people(), import_faces_from_yaml(), import_people_from_yaml(), import_face_embedding(). Free functions: write_arcface_binary(), read_arcface_binary(), delete_arcface_binary(), scan_arcface_binaries(), load_faces_yaml(), save_faces_yaml(), load_people_yaml(), save_people_yaml(). Structs: StoredFace, Person, AutoClusterResult, FaceMatch, FaceRecord, PersonRecord, FacesFile, PeopleFile. |
| device_registry | src/device_registry.rs | Volume registration and online detection. Manages the volumes.yaml file at the catalog root. Detects whether volumes are online by checking mount point existence. Provides find_volume_for_path() to auto-detect which volume contains a given file path. |

| Module | File(s) | Purpose |
|---|---|---|
| `embedded_xmp` | `src/embedded_xmp.rs` | Extracts XMP metadata embedded in JPEG and TIFF binary data (APP1 marker for JPEG, IFD tag 700 for TIFF). Returns parsed XMP data for merging during import. Supported formats: `.jpg`/`.jpeg`/`.tif`/`.tiff`; all other extensions return empty immediately with zero I/O. |
| `exif_reader` | `src/exif_reader.rs` | EXIF metadata extraction from image files using the `kamadak-exif` crate. Extracts camera make/model, lens, focal length, aperture, shutter speed, ISO, date/time, GPS coordinates, dimensions, and orientation. Provides `apply_exif_orientation()` for auto-rotating images based on EXIF orientation tags, and `apply_rotation()` for manual rotation overrides (used by preview generation). |
| `format` | `src/format.rs` | Output format template engine for CLI commands. Supports preset formats (`ids`, `short`, `full`, `json`) and custom templates with `{placeholder}` substitution and escape sequences (`\t`, `\n`). Used by `search --format` and `duplicates --format`. |
| `model_manager` | `src/model_manager.rs` | *Feature-gated (`ai`)*. Download and cache management for SigLIP ONNX model files from HuggingFace. Model-aware: derives HuggingFace URL from `ModelSpec.hf_repo`. Downloads `vision_model_quantized.onnx`, `text_model_quantized.onnx`, and `tokenizer.json` via curl. Provides `ModelManager` with `ensure_model()`, `download_model()`, `remove_model()`, `list_files()`, `total_size()`, and `list_available_models()`. Default cache dir: `~/.maki/models/<model-id>/`. |

| Module | File(s) | Purpose |
|---|---|---|
| metadata_store | src/metadata_store.rs | YAML sidecar file read/write operations. Sidecars are the source of truth for asset metadata. Stored at assets/<id-prefix>/<id>.yaml within the catalog root. Handles serialization/deserialization of Asset structs with all nested variants, recipes, locations, and tags. |
| models | src/models/ | Core data structures shared across all modules. Split into sub-modules: |
| | src/models/mod.rs | Re-exports and shared types (Asset, FileLocation) |
| | src/models/asset.rs | Asset struct: id, name, description, asset_type, tags, rating, color_label, created_at, variants. Includes validate_color_label() for the 7-color set. |
| | src/models/variant.rs | Variant struct: content_hash, original_filename, format, role, file_size, source_metadata, file_locations, recipes. Includes best_preview_index() / best_preview_index_details() for display variant selection, and compute_best_variant_hash() / compute_primary_format() for denormalized column computation. |
| | src/models/recipe.rs | Recipe struct: recipe_type, relative_path, volume_id, content_hash. Represents processing sidecars (XMP, COS, pp3, etc.) attached to variants. |
| | src/models/volume.rs | Volume struct: id, label, mount_point, is_online. |
| preview | src/preview.rs | Preview/thumbnail generation. Standard images use the image crate (resized to configurable max edge, default 800px). RAW files use dcraw or dcraw_emu (LibRaw). Videos use ffmpeg. Non-visual formats get info cards rendered with imageproc/ab_glyph. Previews stored at previews/<hash-prefix>/<hash>.{jpg,webp}. |

| Module | File(s) | Purpose |
|---|---|---|
| query | src/query.rs | Search query parsing and QueryEngine orchestration. Parses filter syntax (tag:X, rating:N+, label:Red, collection:X, path:prefix, orphan:true, missing:true, stale:N, volume:none, similar:<id>[:<limit>]). QueryEngine wraps catalog + metadata store + device registry to provide high-level operations: search, show, tag, edit, set_rating, set_description, set_name, set_color_label, auto_group, fix_roles. Includes path normalization for volume-relative prefix matching. |
| saved_search | src/saved_search.rs | Saved search (smart album) persistence. Stores named queries in searches.toml at the catalog root. Provides load(), save(), and SavedSearch struct with to_url_params() for web UI chip rendering. |
| stack | src/stack.rs | Stack (scene grouping) persistence. Dual storage: SQLite stacks table for fast queries + stacks.yaml at catalog root for rebuild resilience. Provides StackStore with create, add, remove, set_pick, dissolve, list, and export/import operations. Assets in a stack have stack_id and stack_position columns on the assets table. Position 0 is the pick (displayed in browse grid when stacks are collapsed). |
| web | src/web/ | Axum web server. Split into sub-modules: |
| | src/web/mod.rs | Server setup, router construction, AppState, request logging middleware, graceful shutdown. |

| Module | File(s) | Purpose |
|---|---|---|
| | `src/web/routes.rs` | All HTTP route handlers: browse, asset detail, tags page, stats page, collections page, stroll page (AI feature-gated), search API, asset editing endpoints, batch operations, saved search CRUD, collection CRUD, stroll neighbors API. Filter bar markup extracted into shared Askama includes (`filter_bar.html`, `filter_bar_js.html`). The L2 transitive neighbor feature (fan-out slider on the stroll page) is handled entirely in the frontend JavaScript within `templates/stroll.html`, reusing the existing `/api/stroll/neighbors` endpoint with chained requests — no additional backend route is needed. |
| | `src/web/templates.rs` | Askama HTML templates and template helper structs (`BrowsePage`, `ResultsPartial`, `AssetPage`, `AssetCard`, `TagsPage`, `StatsPage`, `CollectionsPage`, `StrollPage`, `StrollCenter`, `StrollNeighbor`, and HTML fragment types for htmx partial updates). |
| | `src/web/static_assets.rs` | Compile-time embedded static files (`htmx.min.js`, `style.css`) served as in-memory responses. |
| `xmp_reader` | `src/xmp_reader.rs` | XMP sidecar parsing and write-back. Reads XMP files to extract `dc:subject` (tags), `dc:description`, `xmp:Rating`, `xmp:Label`, `dc:creator`, `dc:rights`. Write-back functions: `update_rating()`, `update_label()`, `update_tags()` (operation-level deltas), `update_description()`. Handles XML escaping, namespace injection, and `rdf:Description` format conversion. |

## Generated API Documentation

For detailed Rust API documentation with full type signatures, run:

```
cargo doc --no-deps --open
```

This generates HTML documentation from doc comments in the source code and opens it in your browser. The output is written to `target/doc/maki/`.

# Building & Testing

## Building

### Debug Build

```
cargo build
```

Produces an unoptimized binary at `target/debug/maki` with debug symbols and runtime checks enabled. Fast compilation, suitable for development.

### Release Build

```
cargo build --release
```

Produces an optimized binary at `target/release/maki`. Significantly faster runtime performance. Use this for production deployment.

### AI Feature Build

```
cargo build --features ai
cargo build --release --features ai
```

Enables the `maki auto-tag` command with SigLIP-based zero-shot image classification. Adds ONNX Runtime (`ort`), `ndarray`, and `tokenizers` as dependencies. The ONNX Runtime shared library is downloaded automatically during build. Without `--features ai`, these dependencies are not compiled and the `auto-tag` command is not available.

### Requirements

- **Rust edition**: 2021 (stable toolchain)
- **Platforms**: macOS, Linux, Windows
- **SQLite**: Bundled via `rusqlite` with the `bundled` feature (no system SQLite required)

## Testing

### Run All Tests

```
cargo test
```

Runs approximately 693 tests total: ~465 unit tests and ~228 integration tests. With `--features ai`, adds ~41 unit tests and ~13 integration tests.

### Unit Tests Only

```
cargo test --lib
```

Runs the ~465 unit tests embedded in library source files (`#[cfg(test)]` modules within `src/`).

### Integration Tests Only

```
cargo test --test integration
```

Runs the ~228 integration tests defined in `tests/cli.rs`. These tests exercise the full system through the CLI binary and library API, using temporary catalogs and volumes.

### Run a Specific Test

```
cargo test test_name_pattern
```

Runs only tests whose names match the given pattern.

### Test Helpers

The integration test suite provides helper functions for setting up test catalogs:

- **`setup_search_catalog()`** – Creates a catalog with pre-populated assets for search testing. Requires `asset.variants` to be populated before calling `catalog.insert_asset()` (because denormalized columns `best_variant_hash`, `primary_variant_format`, and `variant_count` are computed at insert time).
- **`setup_metadata_catalog()`** – Creates a catalog with assets that have metadata (tags, ratings, descriptions, recipes) for metadata operation testing. Same requirement: variants must be populated before insert.

## Documentation

### Rust API Docs

```
cargo doc --no-deps --open
```

Generates HTML documentation from doc comments and opens it in your browser. The `--no-deps` flag skips building docs for third-party dependencies, which speeds up the build considerably. Output is at `target/doc/maki/`.

### PDF Manual

```
bash doc/manual/build-pdf.sh
```

Generates `doc/manual/maki-manual.pdf` from the 21 Markdown source files. The script concatenates all sections in order, renders mermaid diagrams to PNG, and produces a PDF with table of contents, headers/footers, and syntax-highlighted code blocks. The version number is read from `Cargo.toml`.

**Prerequisites** (not required for building or running maki itself):

- **pandoc** – Document conversion. `brew install pandoc`
- **XeLaTeX** – PDF typesetting with Unicode support. `brew install --cask mactex-no-gui`
- **mermaid-cli** (`mmdc`) – Diagram rendering. `brew install mermaid-cli`

# Release Process

1. **Update documentation**: User manual, README, CHANGELOG, and any other docs affected by the release.
2. **Bump version** in `Cargo.toml`:

```toml
[package]
version = "X.Y.Z"
```

3. **Update lockfile**:

```
cargo build
```

   This regenerates `Cargo.lock` with the new version.

4. **Run all tests**:

```
cargo test
```

   All tests must pass before releasing.

5. **Commit**:

```
git add -A
git commit -m "Release vX.Y.Z -- brief description"
```

6. **Tag**:

```
git tag vX.Y.Z
```

7. **Push**:

```
git push && git push --tags
```

8. **Create GitHub release**:

```
gh release create vX.Y.Z --title "vX.Y.Z" --notes "changelog content here"
```

# Dependencies

## Rust Crates

| Crate | Purpose |
|---|---|
| `clap` | CLI argument parsing with derive macros |
| `sha2` | SHA-256 content hashing |
| `serde` / `serde_json` / `serde_yaml` | Serialization for JSON output, YAML sidecars |
| `rusqlite` | SQLite database access (bundled) |
| `kamadak-exif` | EXIF metadata extraction from images |
| `quick-xml` | XMP/XML parsing and manipulation |
| `regex` | Pattern matching in query parsing and XMP processing |
| `image` | Image decoding, resizing, and encoding for previews |
| `imageproc` | Text rendering on info card previews |
| `ab_glyph` | Font loading for info card text (embedded DejaVu Sans) |
| `lofty` | Audio metadata extraction (duration, bitrate) for info cards |
| `uuid` | UUID v4 generation (asset IDs) and v5 (deterministic IDs) |
| `axum` | HTTP web framework for the `serve` command |
| `askama` | Compile-time HTML template engine |
| `tokio` | Async runtime for the web server |

| Crate | Purpose |
|---|---|
| `tower-http` | Static file serving middleware (`ServeDir` for previews) |
| `toml` | Configuration file parsing (`maki.toml`, `searches.toml`) |
| `glob-match` | Filename glob matching for import exclusion patterns |
| `chrono` | Date/time handling with serde support |
| `anyhow` / `thiserror` | Error handling |
| `ort` | ONNX Runtime bindings for AI inference (optional, `ai` feature) |
| `ndarray` | N-dimensional array operations for tensor manipulation (optional, `ai` feature) |
| `tokenizers` | HuggingFace tokenizer for SentencePiece text encoding (optional, `ai` feature) |

## Dev Dependencies

| Crate | Purpose |
|---|---|
| `assert_cmd` | CLI binary testing (running `maki` as a subprocess) |
| `predicates` | Assertion helpers for CLI output matching |
| `tempfile` | Temporary directories for test isolation |

## External Tools (Highly Recommended)

These tools are not Rust dependencies but are invoked as subprocesses for specific preview generation tasks. Their absence does not prevent the application from running; missing tools result in info card fallback previews.

- **dcraw** or **LibRaw** (`dcraw_emu`) – RAW image preview extraction. Used to decode camera-native formats (NEF, ARW, CR2, CR3, etc.) into RGB data for thumbnail generation. LibRaw's `dcraw_emu` is preferred when available.
- **ffmpeg** – Video thumbnail extraction. Used to capture a frame from video files (MP4, MOV, AVI, etc.) for preview generation.
- **curl** – Model file download for AI auto-tagging (only with `--features ai`) and VLM image descriptions (`maki describe`). Used to download ONNX model files from HuggingFace. Available by default on macOS and most Linux distributions.

**Install on macOS** (Homebrew):

```
brew install libraw ffmpeg curl
```

**Install on Linux** (package manager):

```
# Debian/Ubuntu
sudo apt install libraw-bin ffmpeg curl

# Fedora
sudo dnf install LibRaw ffmpeg curl
```

**Install on Windows** (winget or scoop):

```
# winget
winget install LibRaw.LibRaw Gyan.FFmpeg cURL.cURL

# scoop
scoop install libraw ffmpeg curl
```

To check if these tools are available:

```
# macOS / Linux
which dcraw_emu || which dcraw
which ffmpeg
which curl
```

```
# Windows (PowerShell)
Get-Command dcraw_emu -ErrorAction SilentlyContinue
Get-Command ffmpeg -ErrorAction SilentlyContinue
Get-Command curl -ErrorAction SilentlyContinue
```

Preview generation silently falls back to info cards (metadata display images) when these tools are missing. Use `maki generate-previews --debug` to see external tool invocations and errors.